



How To Write A CUDA Program: **THE NINJA EDITION**

Stephen Jones, NVIDIA | GTC 2024

What This Is Not: A Programming Talk

These are some great programming talks



Multi GPU Programming Models for HPC and AI [S61339]

Jiri Kraus, Principal DevTech Compute, NVIDIA

Wednesday, Mar 20 @ 2:00 PM - 2:50 PM PDT



More Data, Faster: GPU Memory Management Best Practices in Python and C++ [S62550]

Mark Harris, Distinguished Engineer, NVIDIA

Wednesday, Mar 20 @ 2:00 PM - 2:50 PM PDT

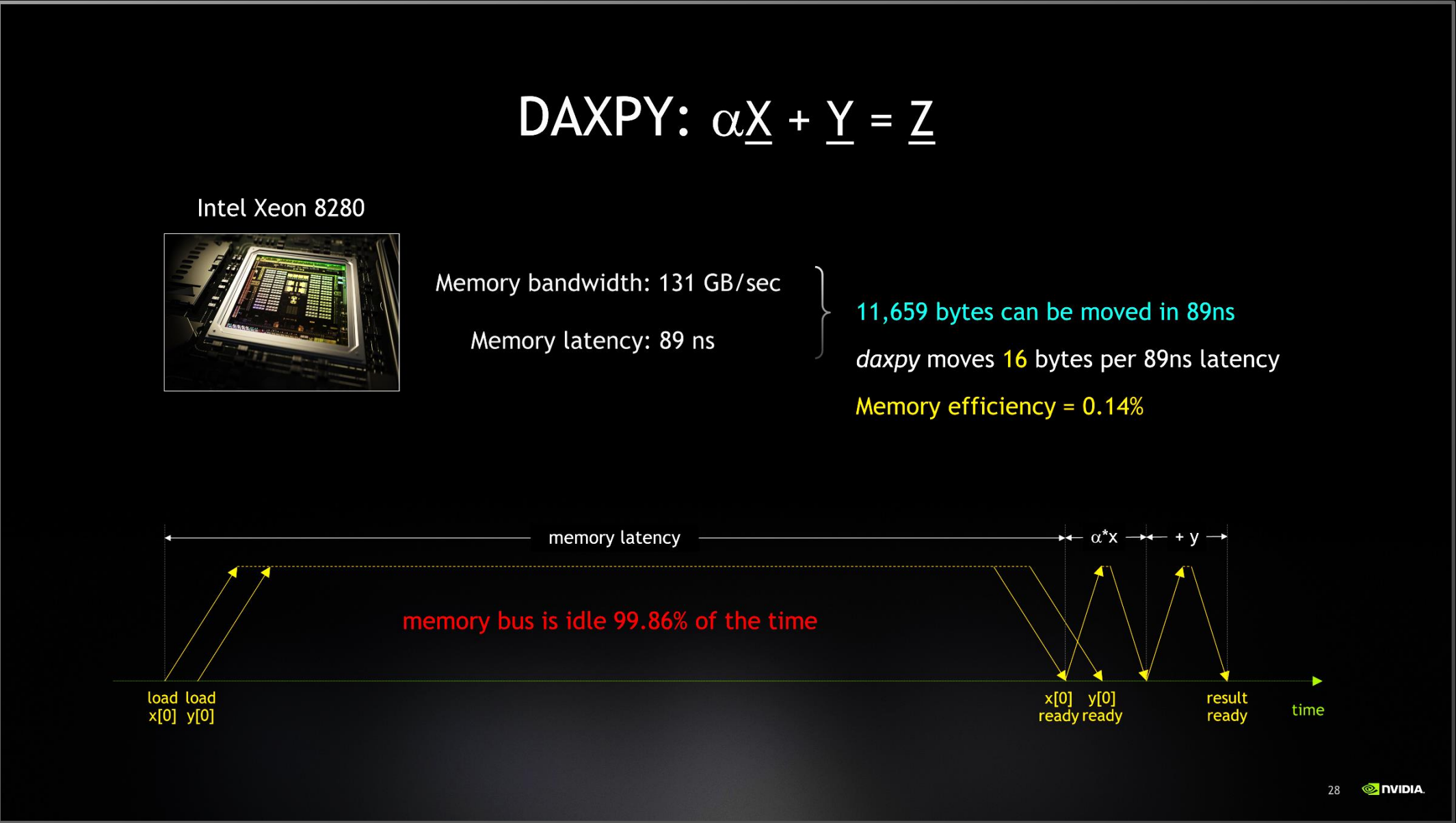


Mastering CUDA C++: Modern Best Practices with the CUDA C++ Core Libraries [S62175]

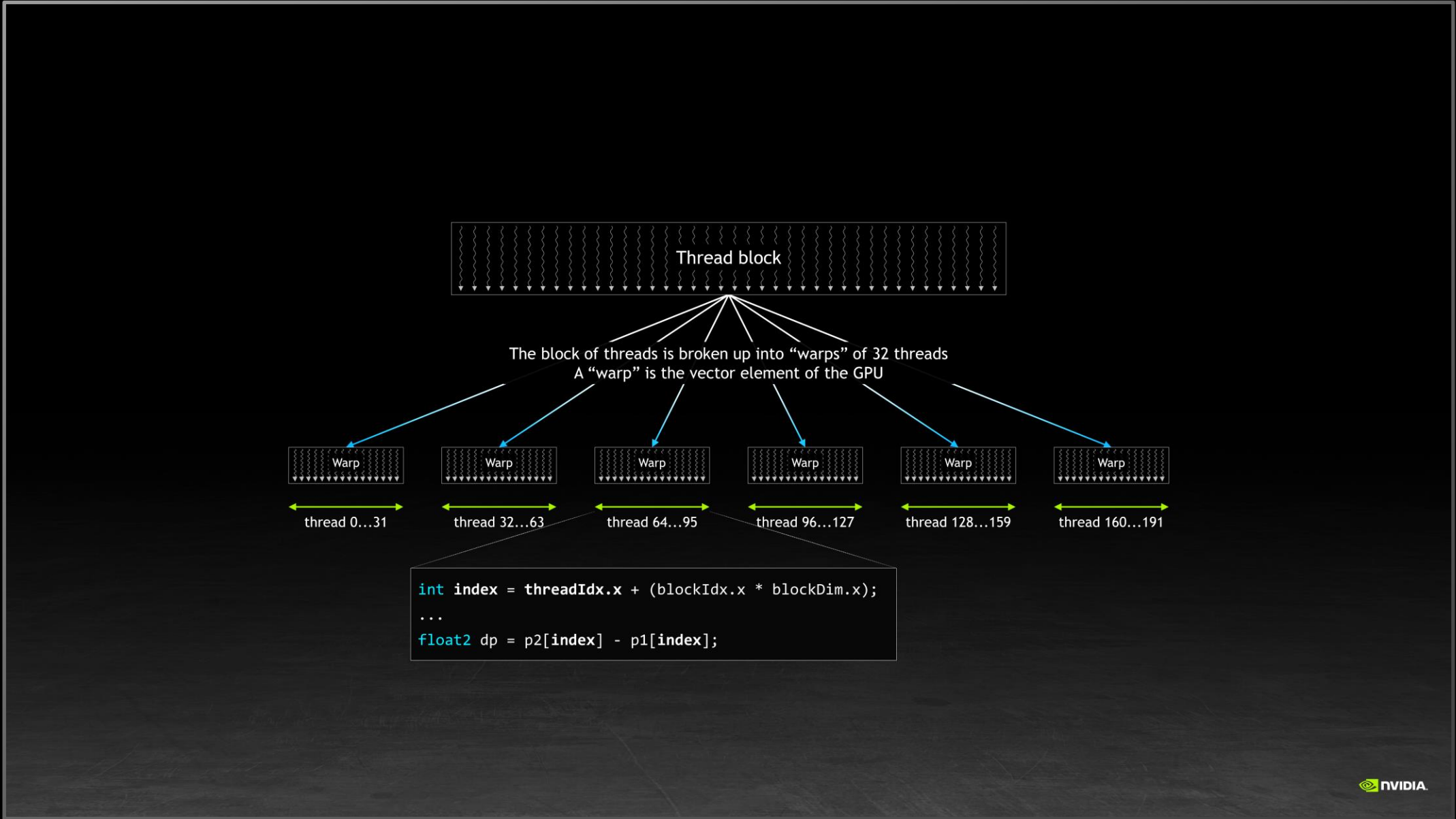
Jake Hemstad, Software Engineering Manager, NVIDIA

Georgii Evtushenko, Senior Software Engineer, NVIDIA

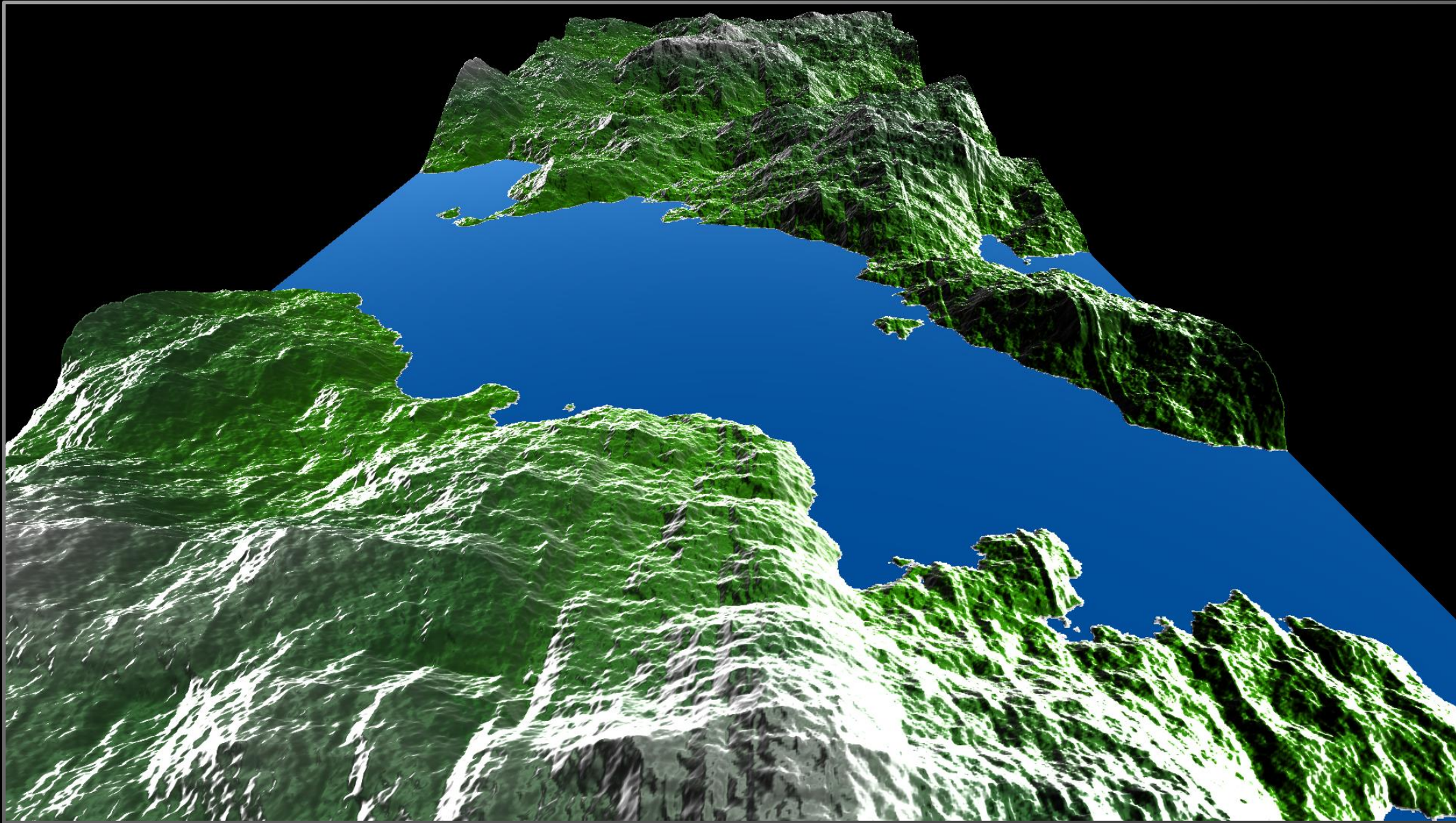
Wednesday, Mar 20 @ 8:00 AM - 8:50 AM PDT



2021: How GPU Computing Works



2022: How CUDA Programming Works



2023: How to Write a CUDA Program



NVIDIA

*Think About
Writing*

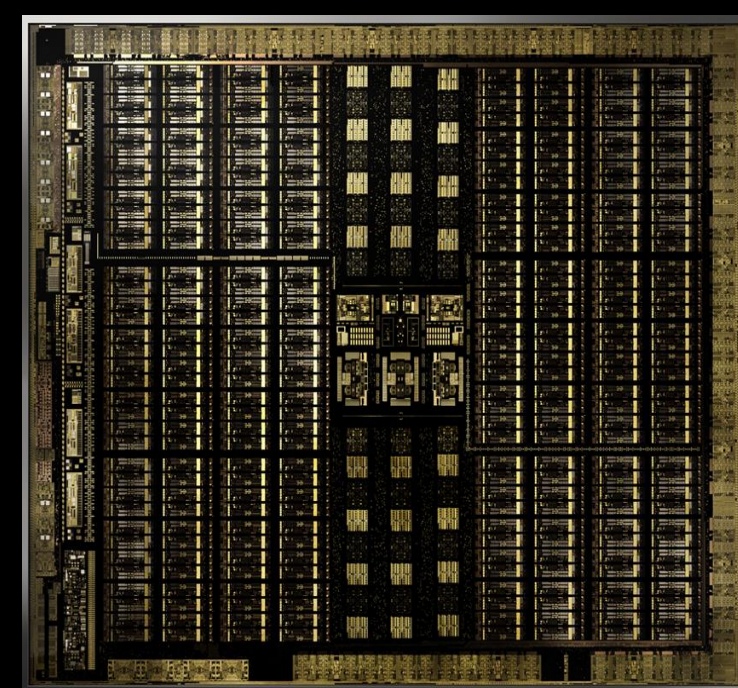
How To ~~Write~~ *Write* A CUDA Program: **THE NINJA EDITION**

Stephen Jones, NVIDIA | GTC 2024

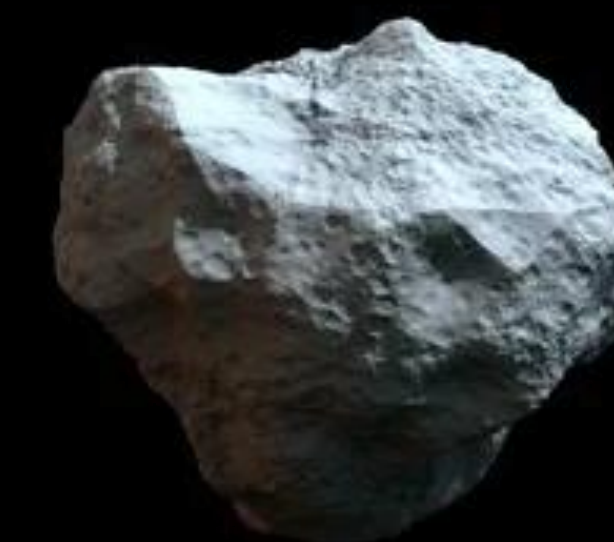
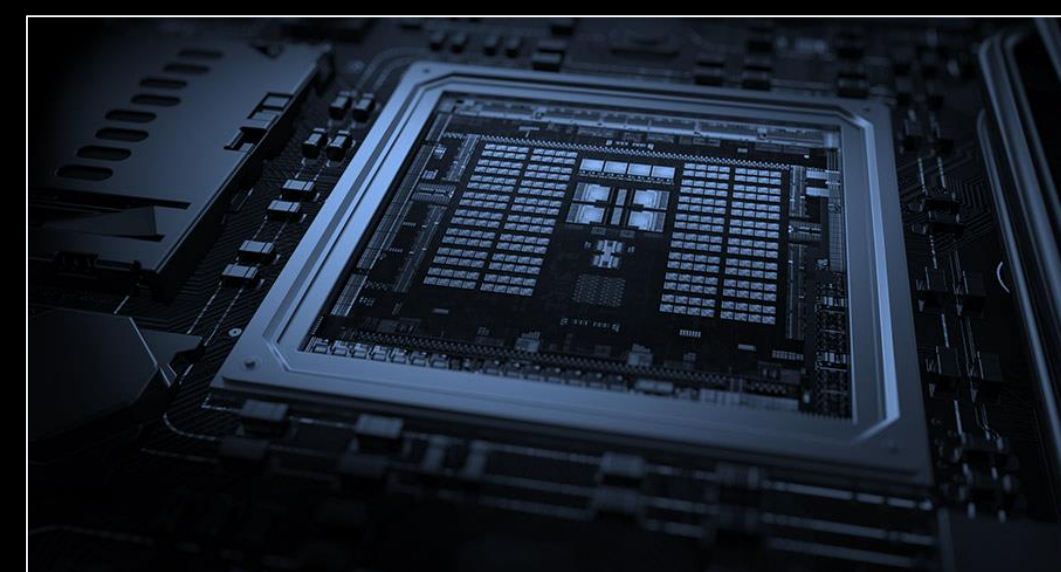
CPUs and GPUs Solve Problems In Very Different Ways



GPU

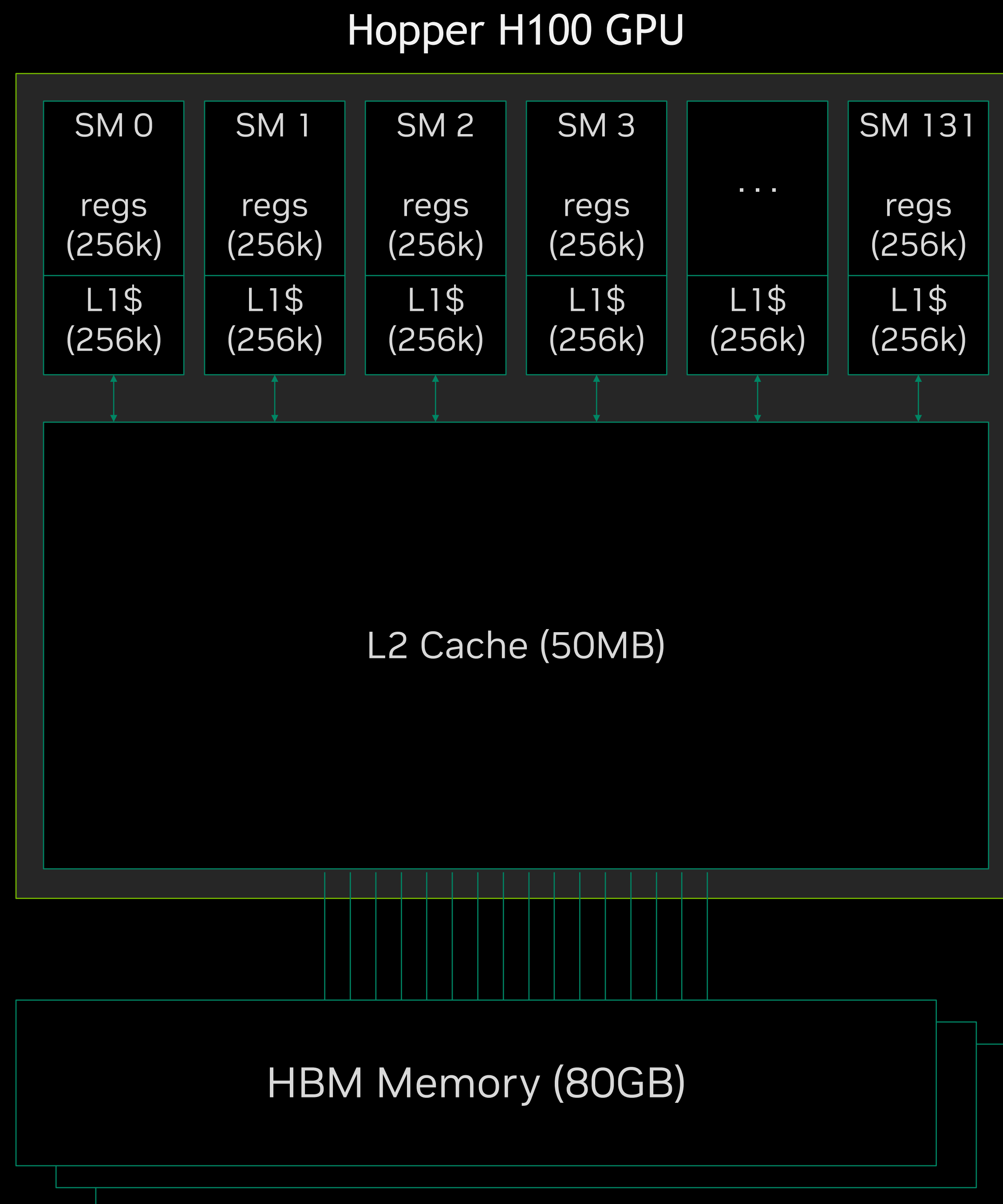


CPU



The H100 Architecture

It's very, very parallel



Hopper architecture

132 SMs

64 warps/SM = **8,448 warps total**

32 threads/warp = **270,336 threads total**

4-way superscalar

4-way * 132 SMs = **528 active warps**

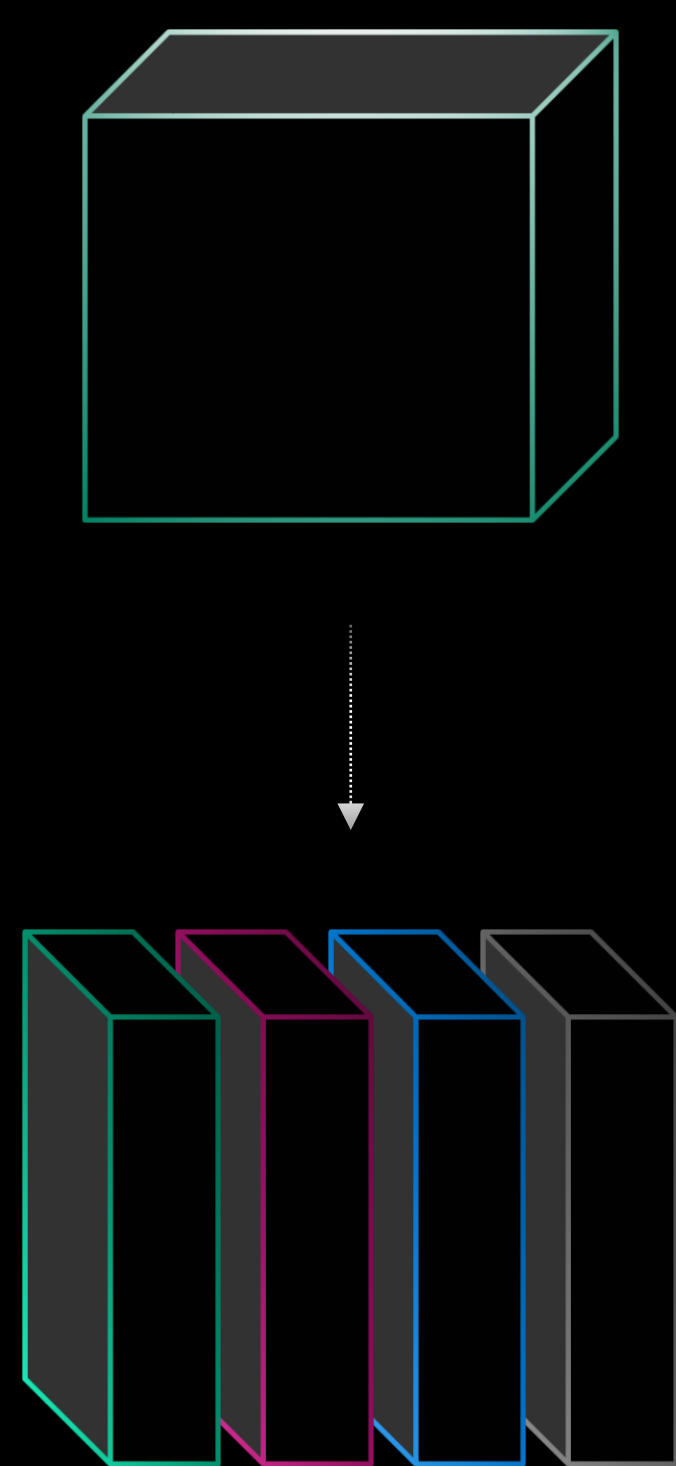
4 warps * 132 SMs = **33,792 live threads**

For typical-size block of 256 threads

256 threads = 8 warps → max 8 blocks / SM

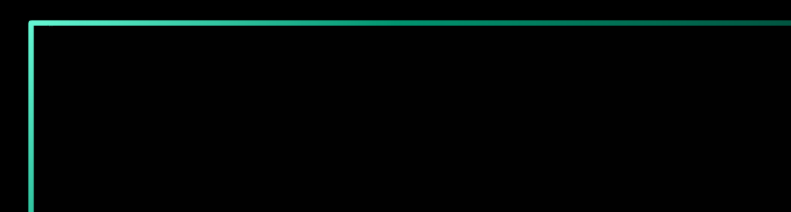
8 blocks * 132 SMs = **1,056 concurrent blocks**

There Are Many Types of Parallelism Patterns



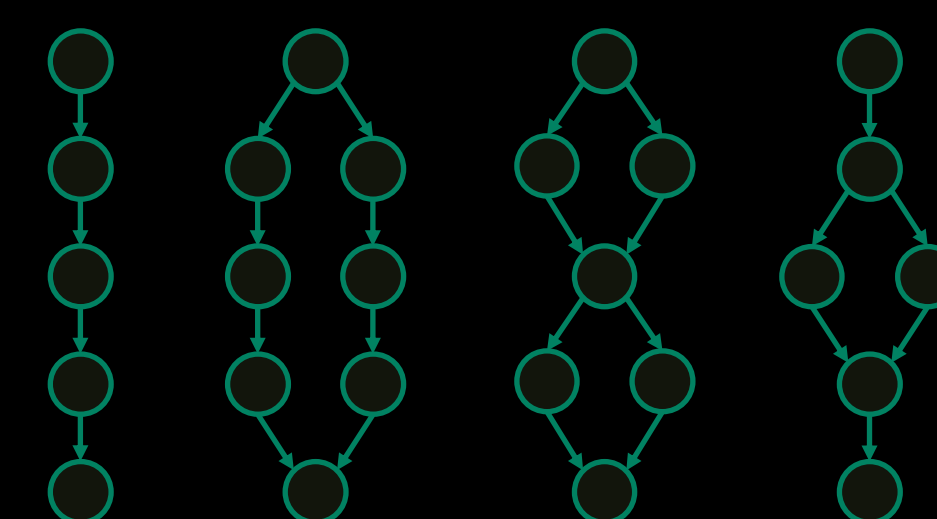
Tensor Parallelism

Divide data along **dimensions** across processors



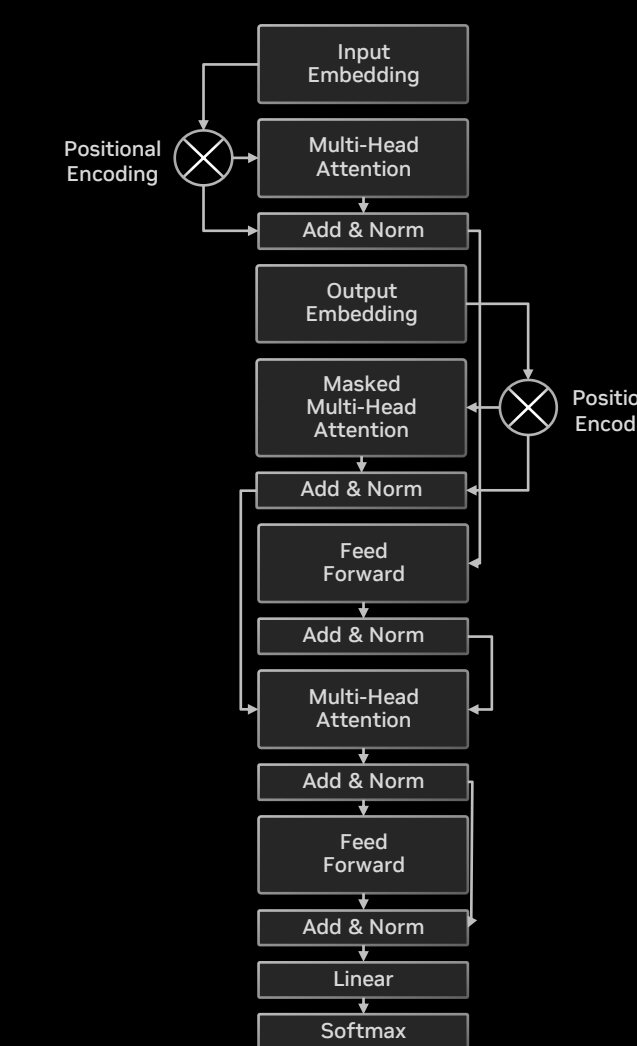
Data Parallelism

Divide individual **elements** across processors



Task Parallelism

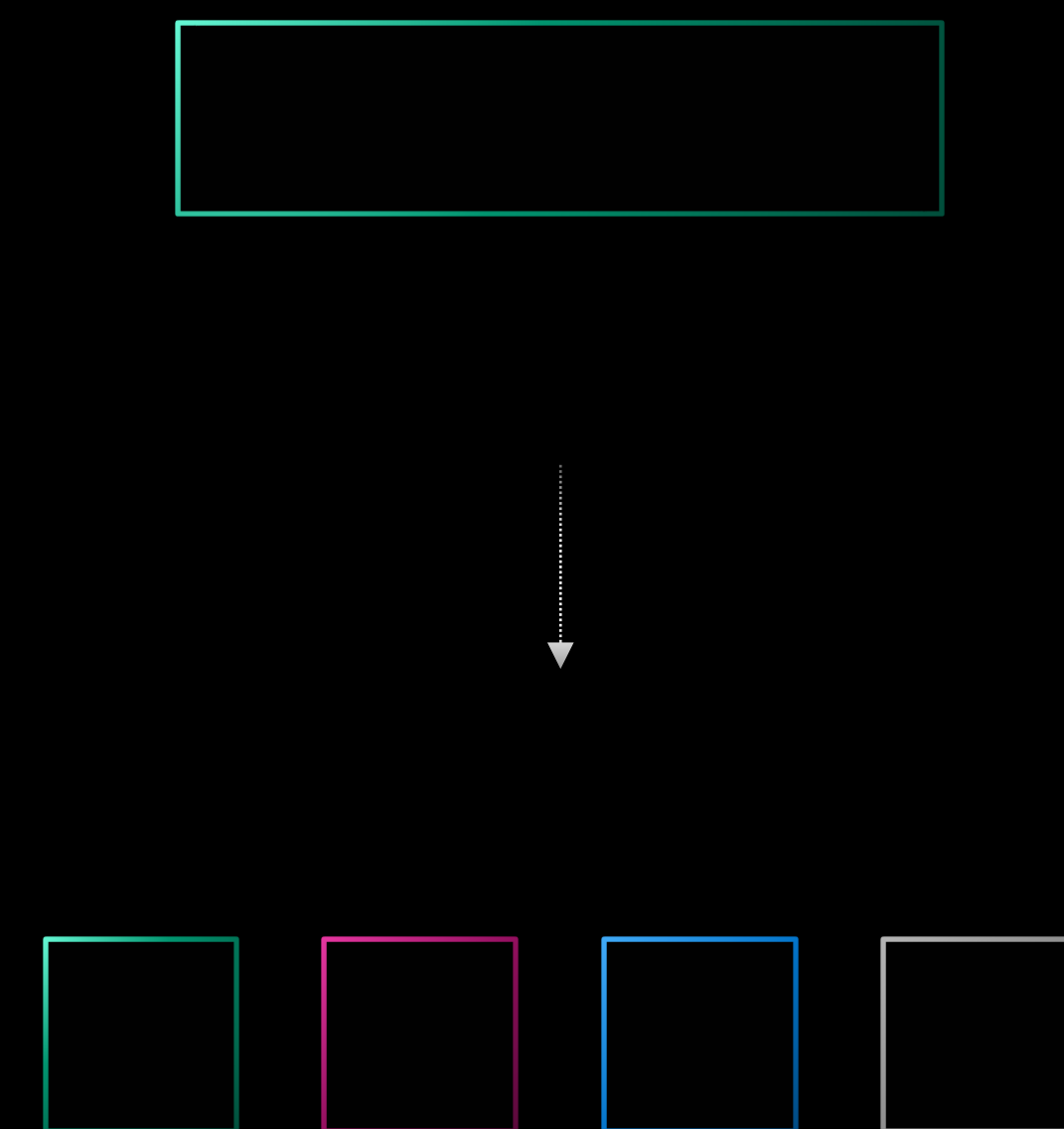
Divide independent **workflows** across processors



Model Parallelism

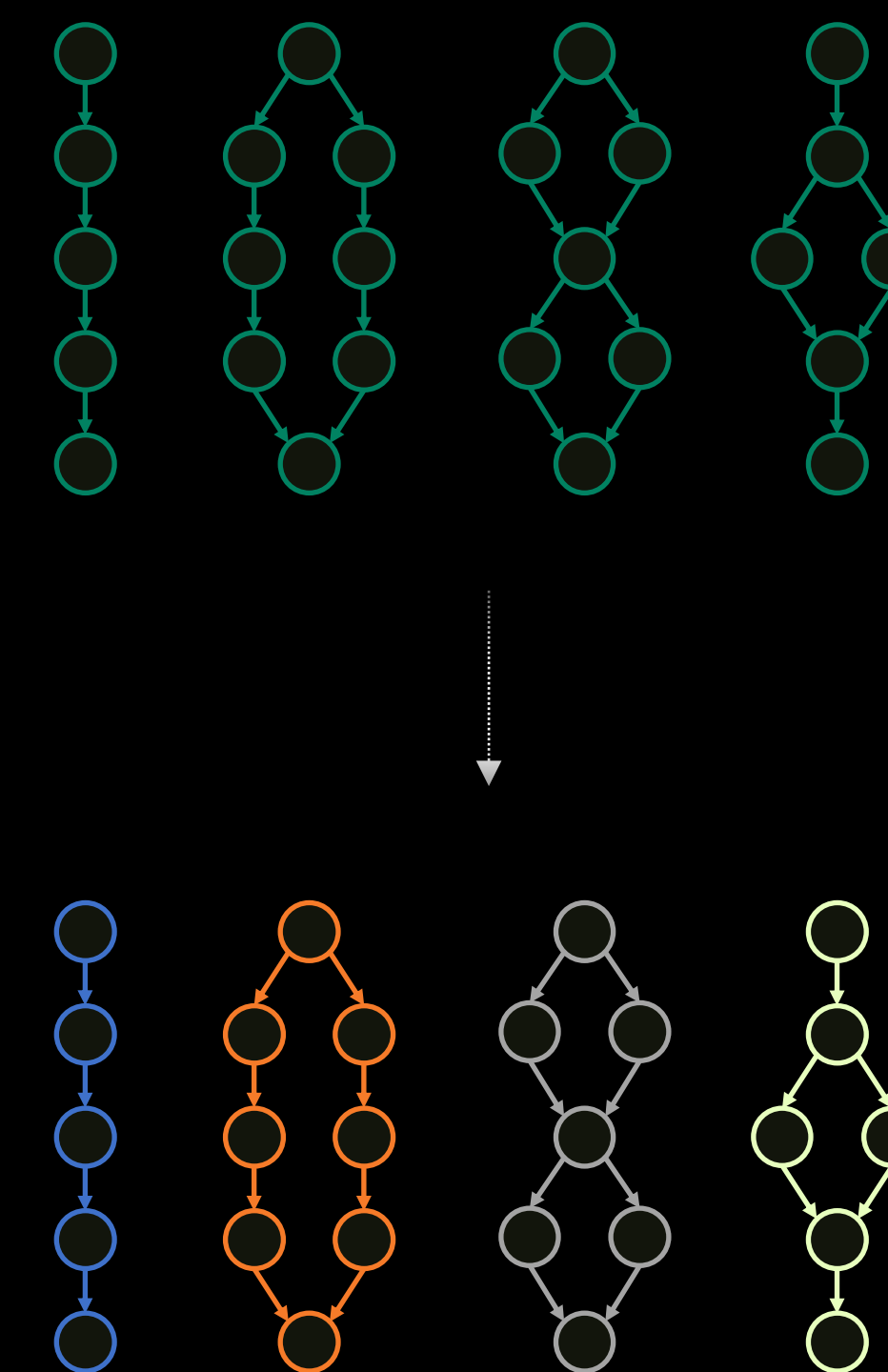
Divide sequences of **operations** across processors

But Really There Are Only Two Types of Parallelism Patterns



Data Parallelism

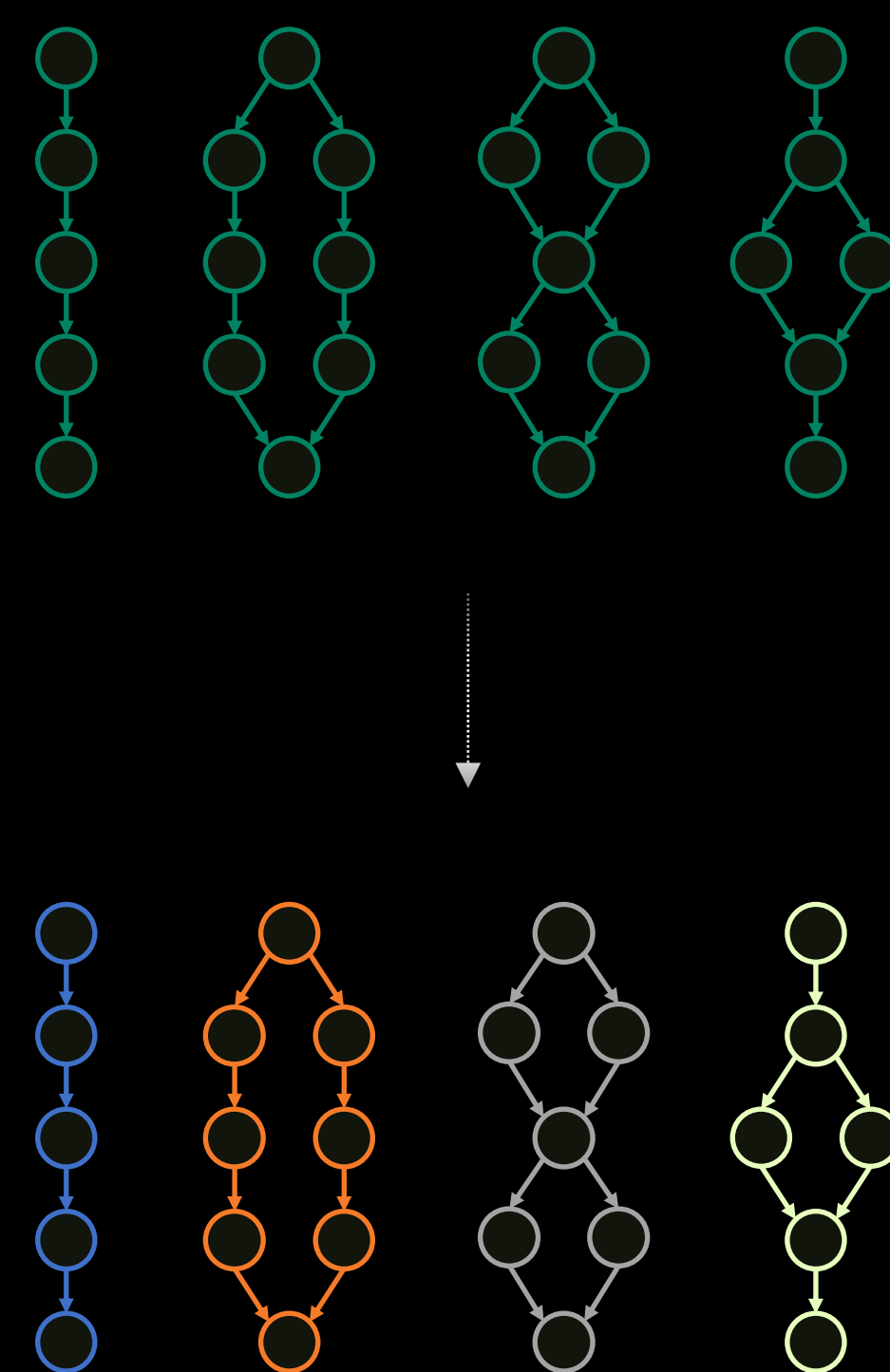
Divide individual
elements across
processors



Task Parallelism

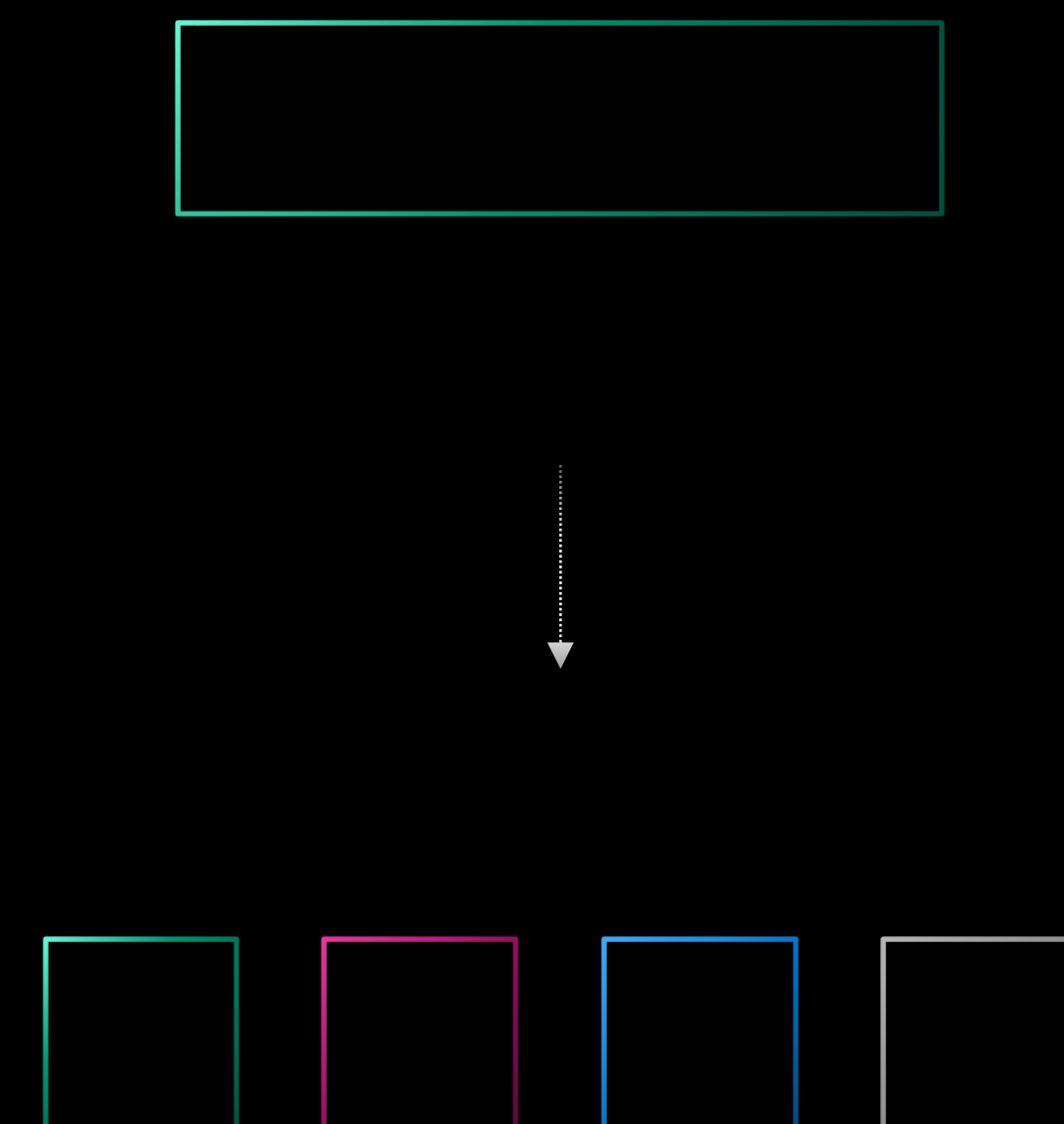
Divide independent
workflows across
processors

The CUDA Execution Model



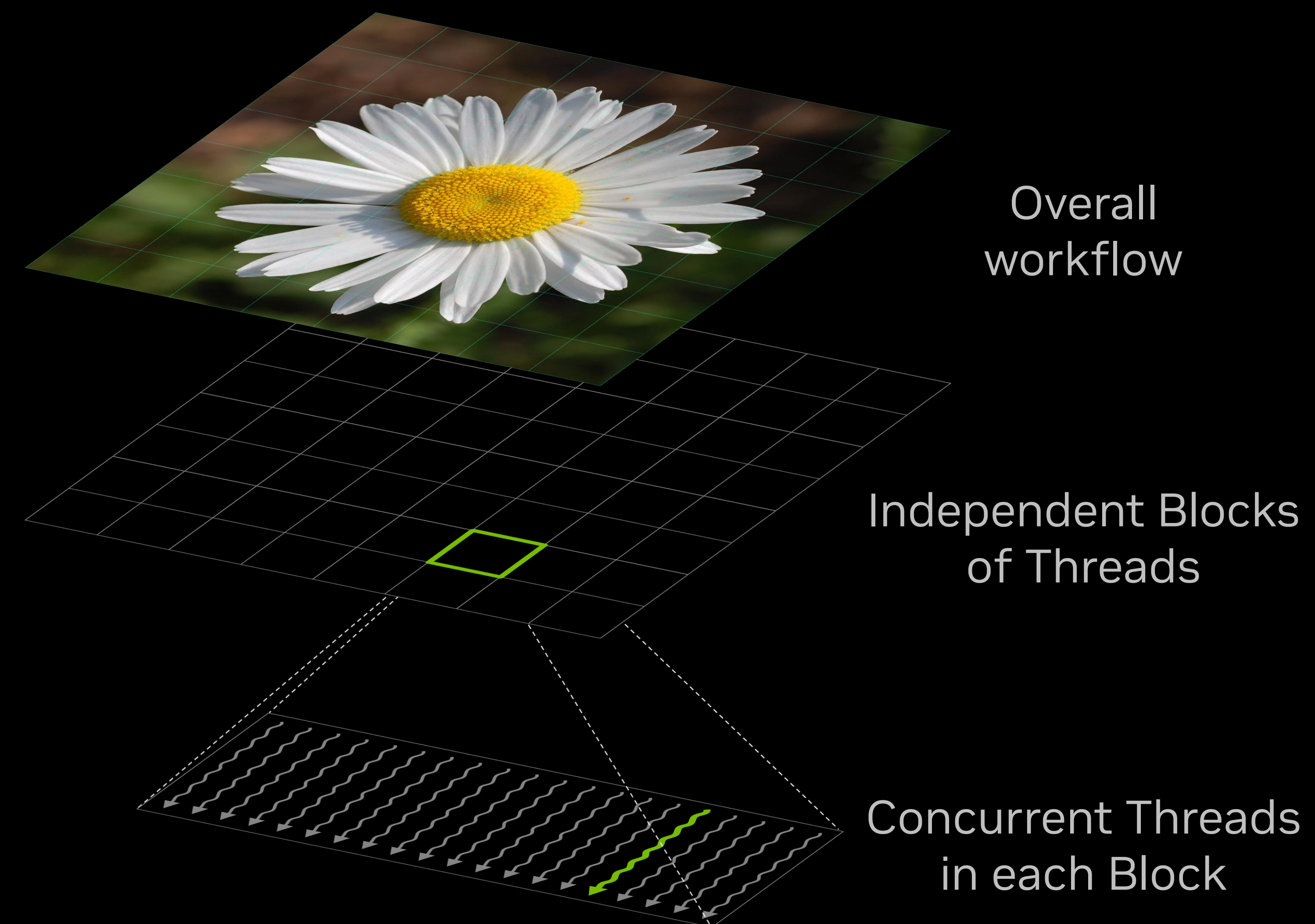
Task Parallelism

Divide independent **workflows** across processors

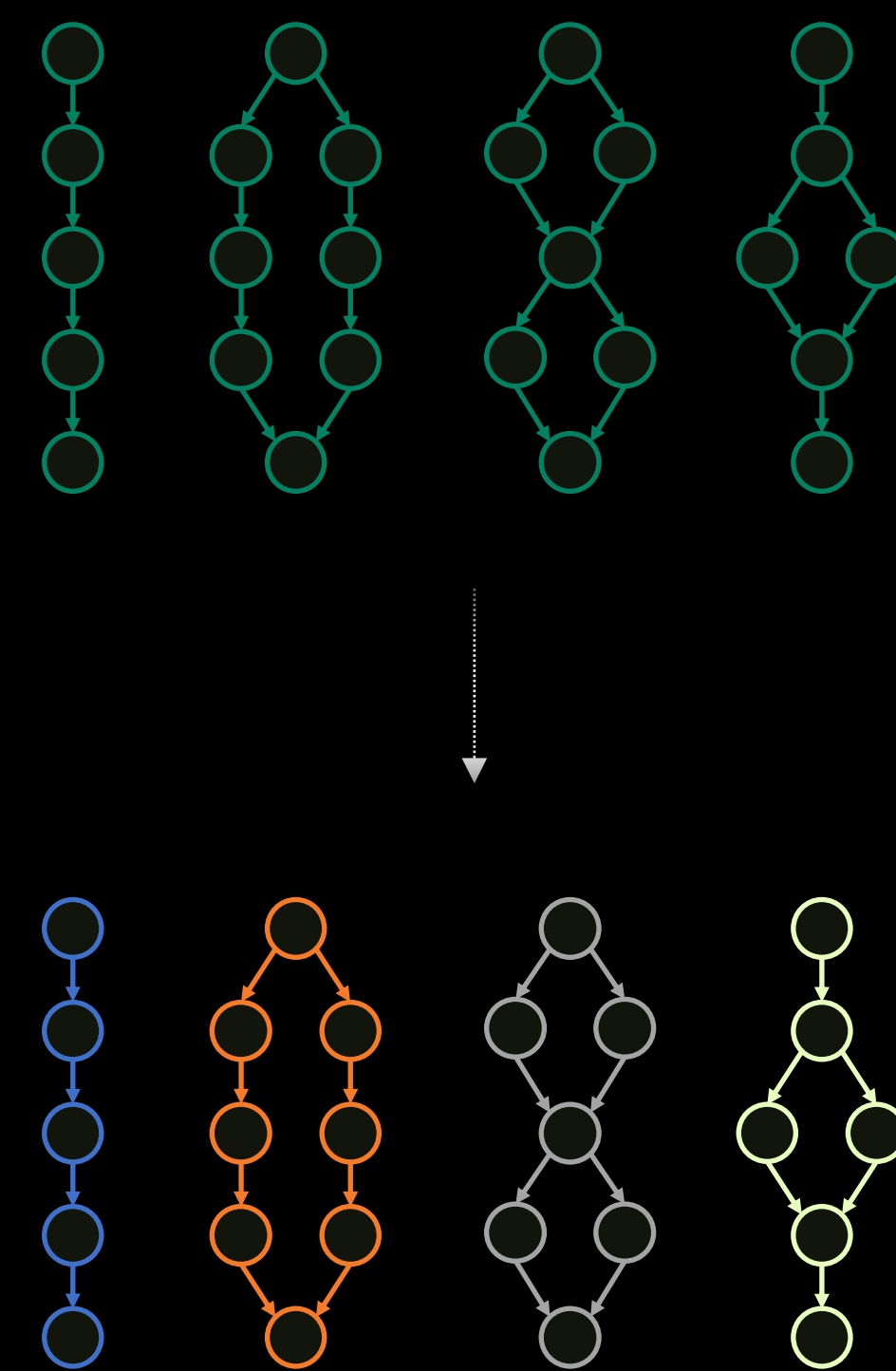


Data Parallelism

Divide individual **elements** across processors

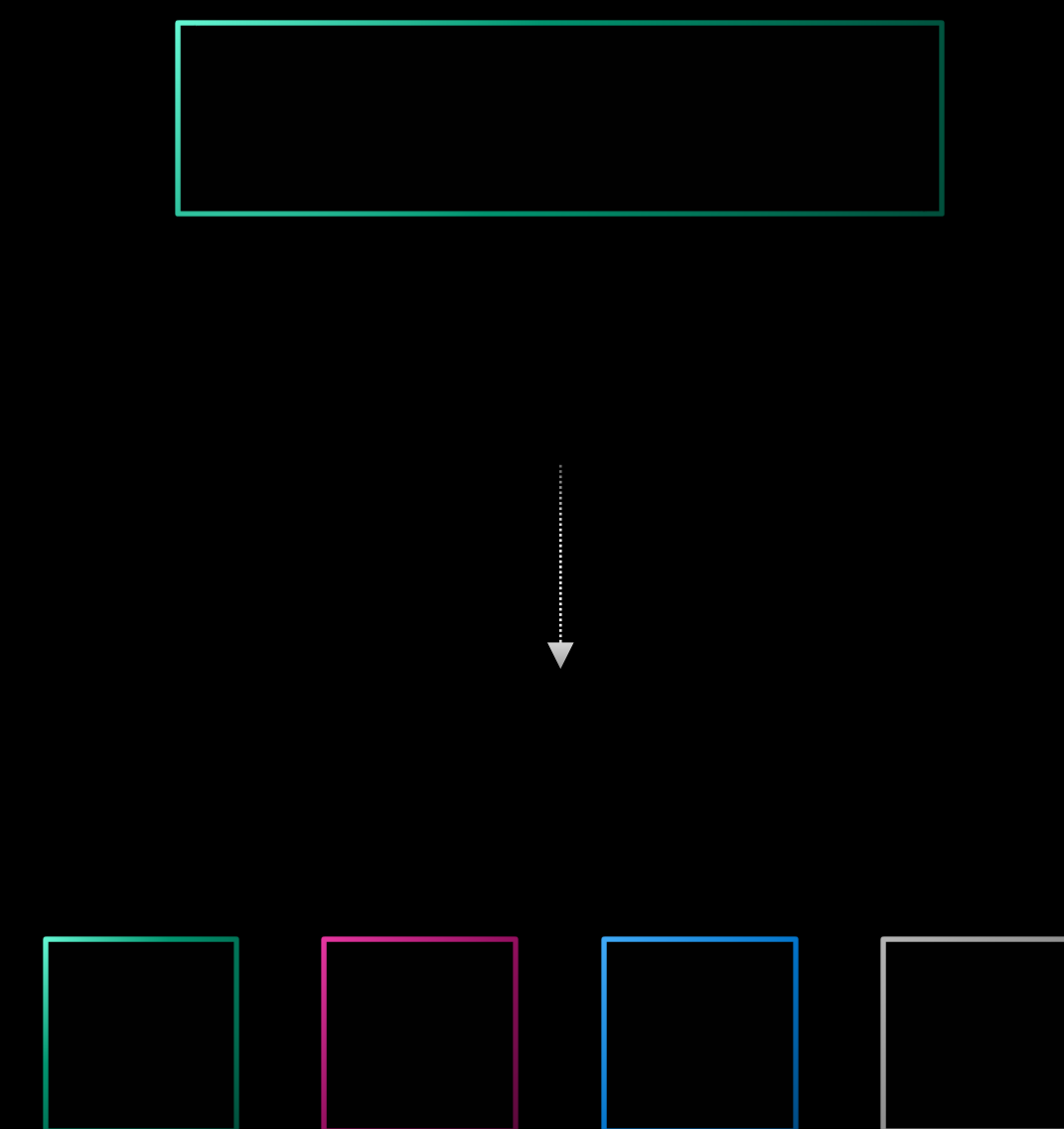


CUDA is Both: Data Parallelism Inside Task Parallelism



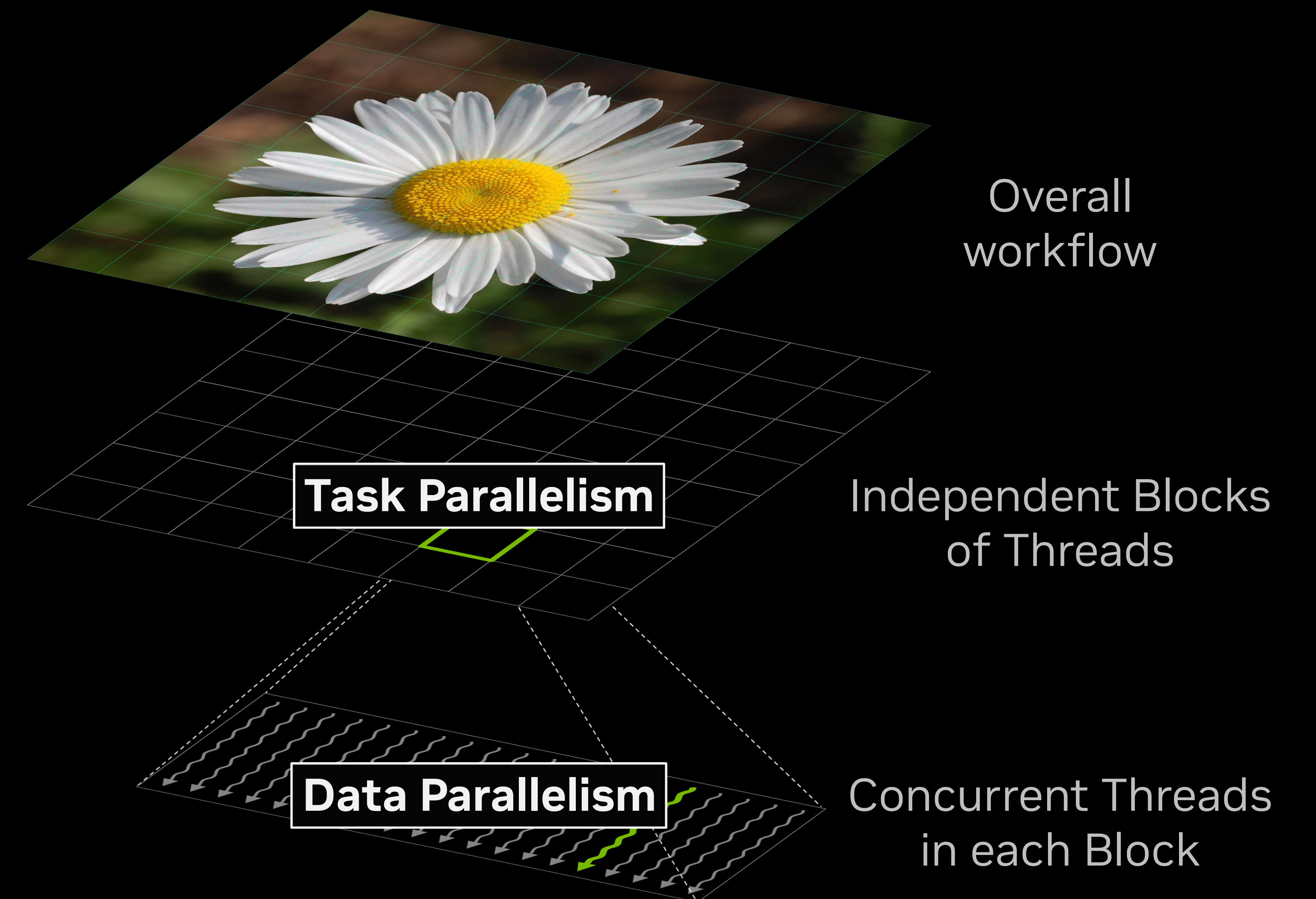
Task Parallelism

Divide independent **workflows** across processors



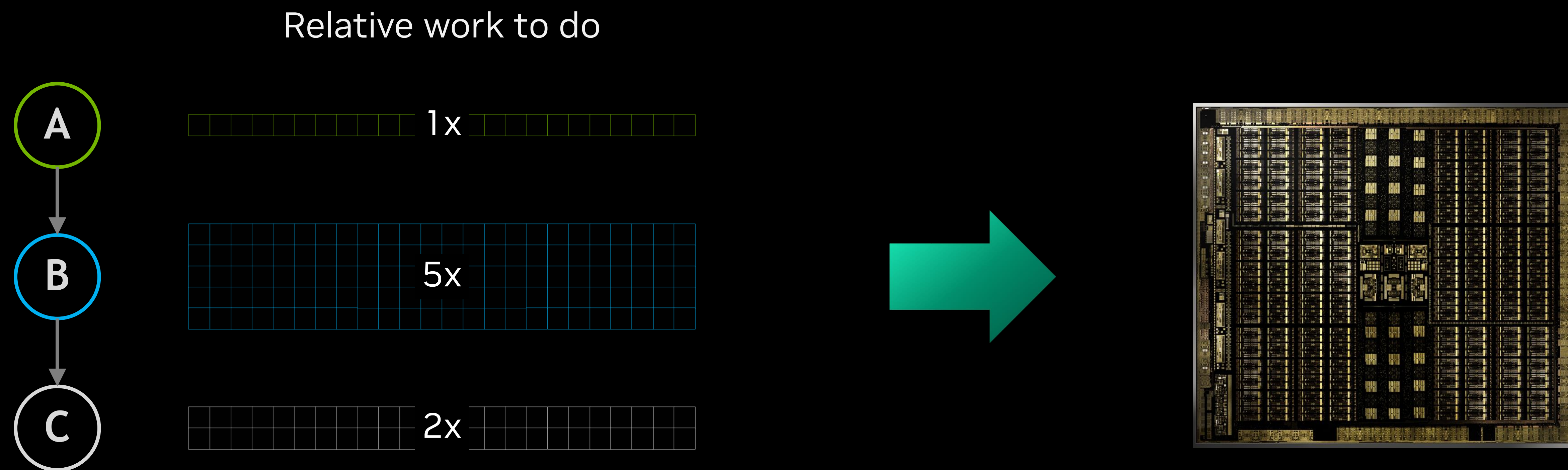
Data Parallelism

Divide individual **elements** across processors

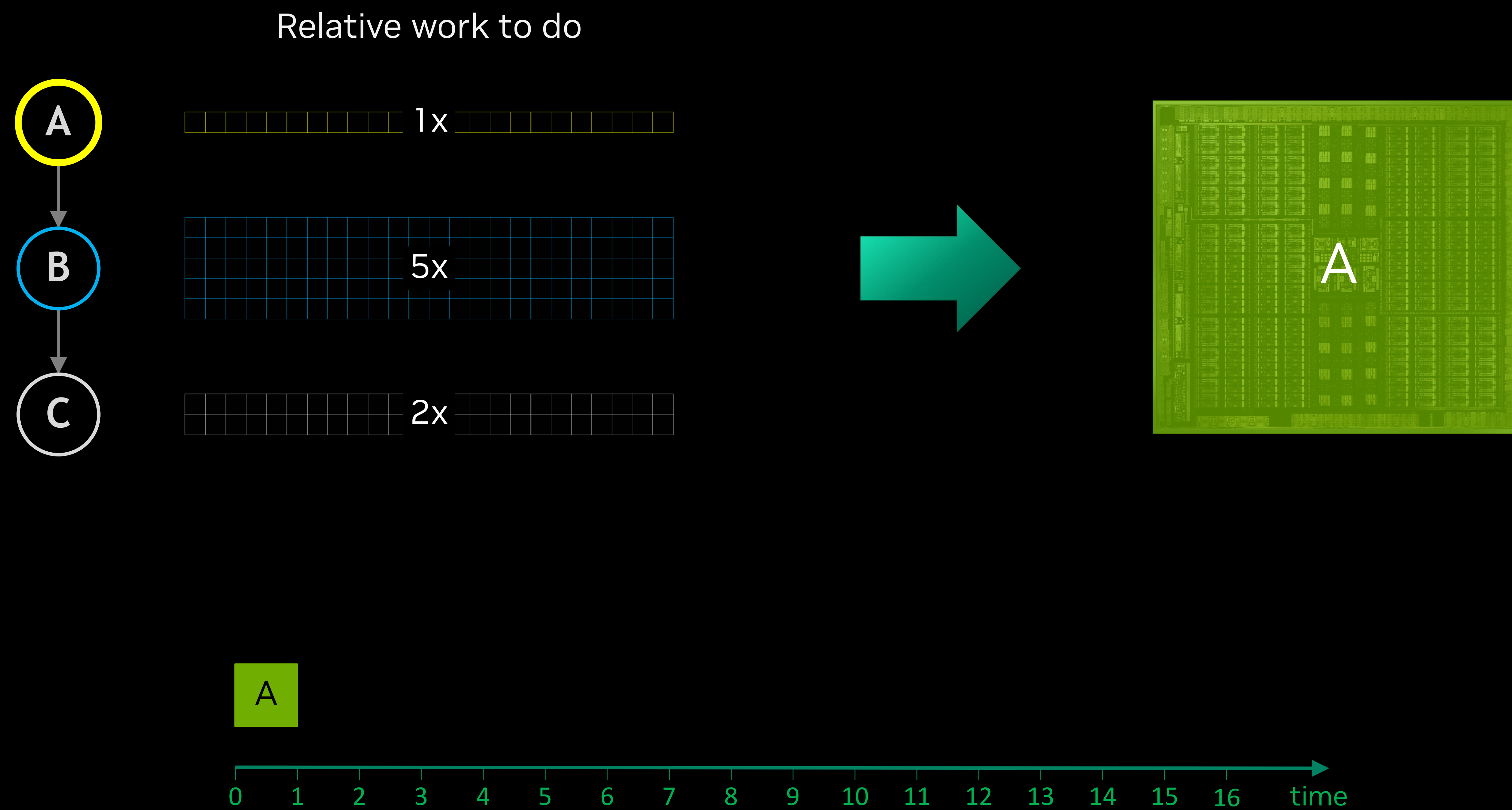


So to run **at peak performance**
we have to think about
how the GPU runs a program

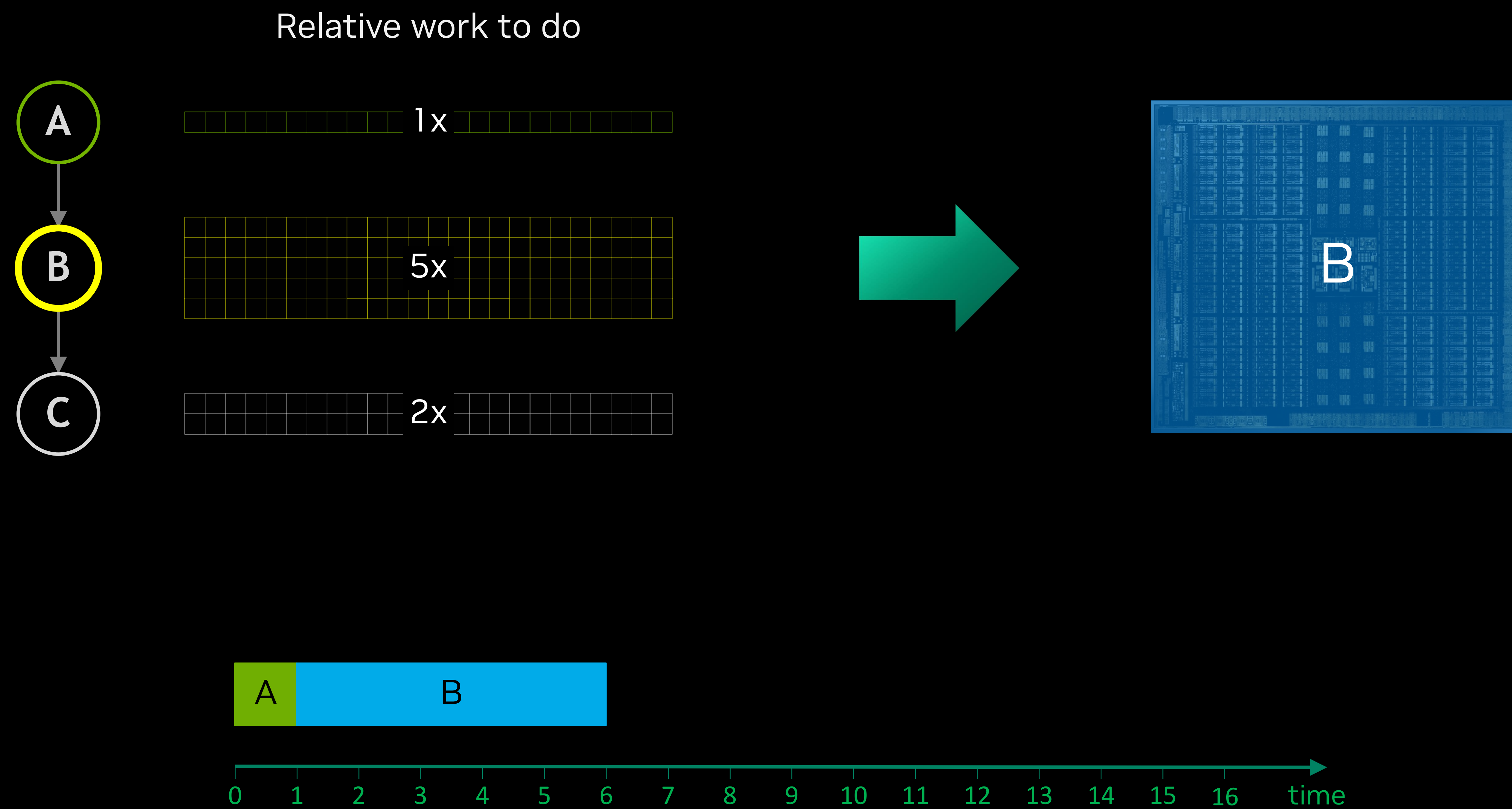
How Bulk Data Parallelism Runs on the GPU



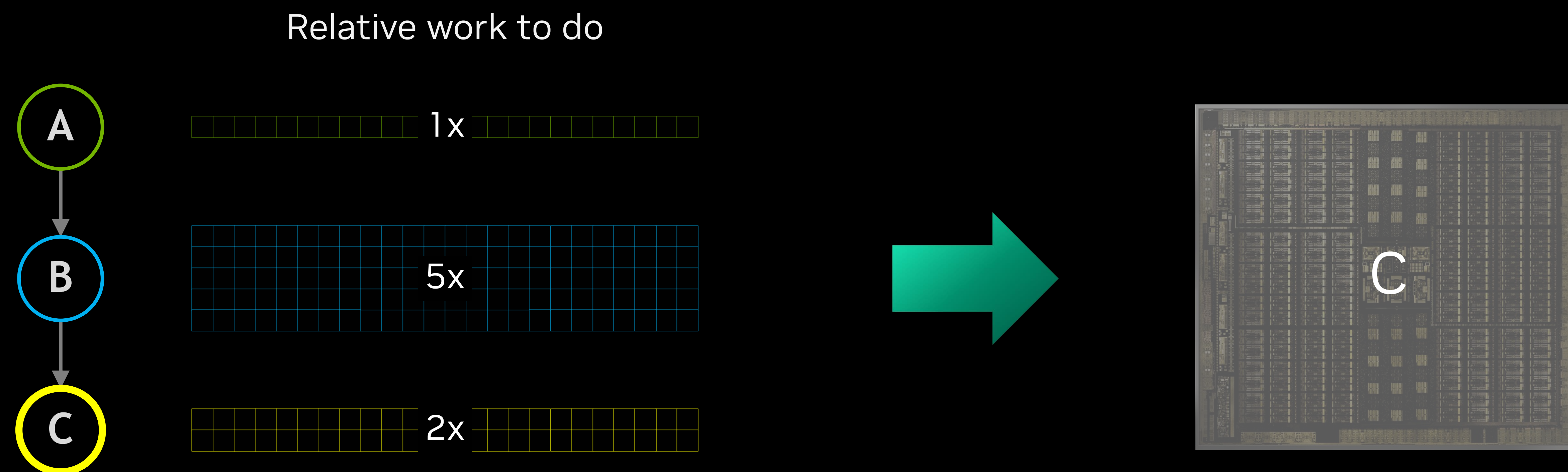
How Bulk Data Parallelism Runs on the GPU



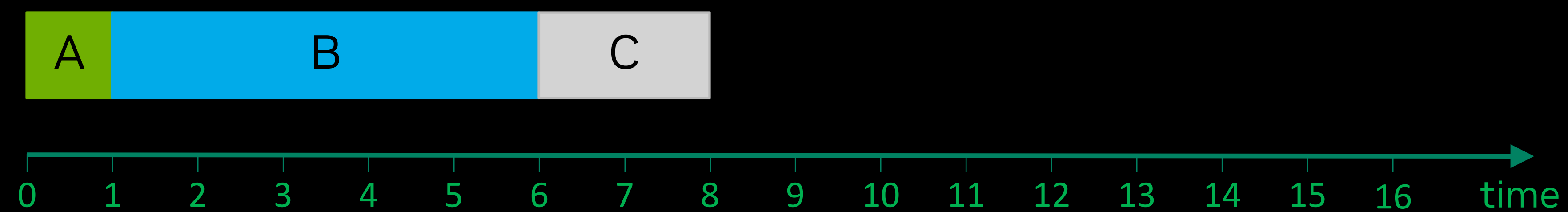
How Bulk Data Parallelism Runs on the GPU



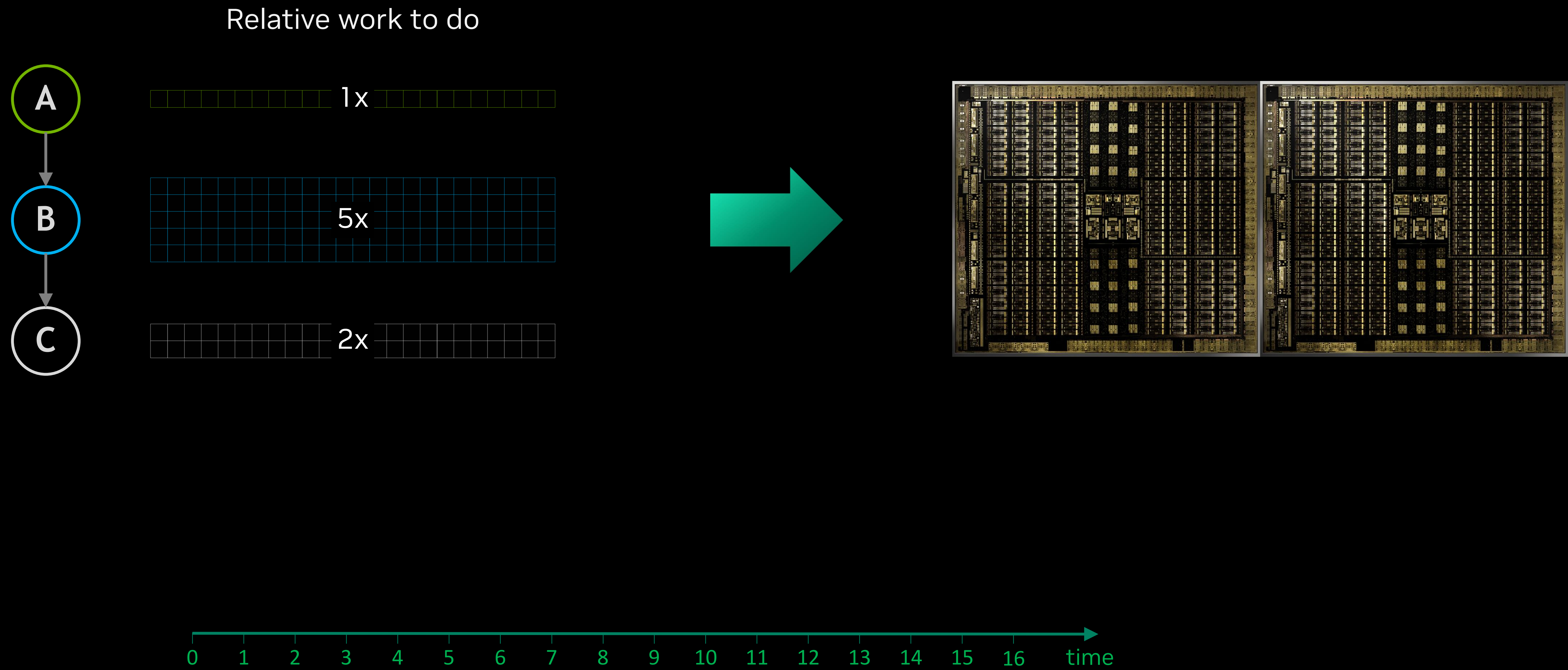
How Bulk Data Parallelism Runs on the GPU



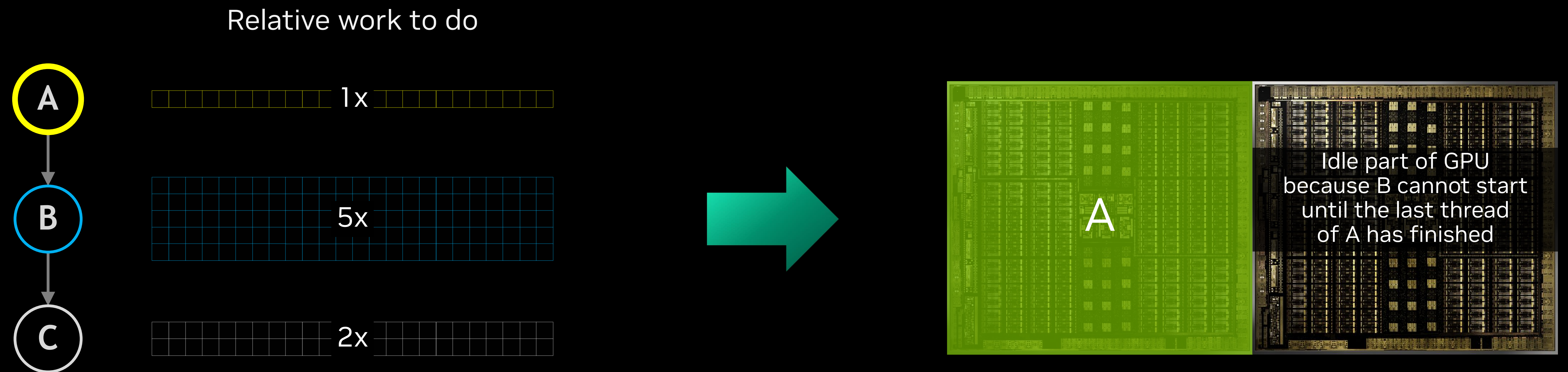
This is what you think is happening:



Doubling The Size of the GPU



Bulk Per-Kernel Data Parallelism



This is what you think **should be** happening:

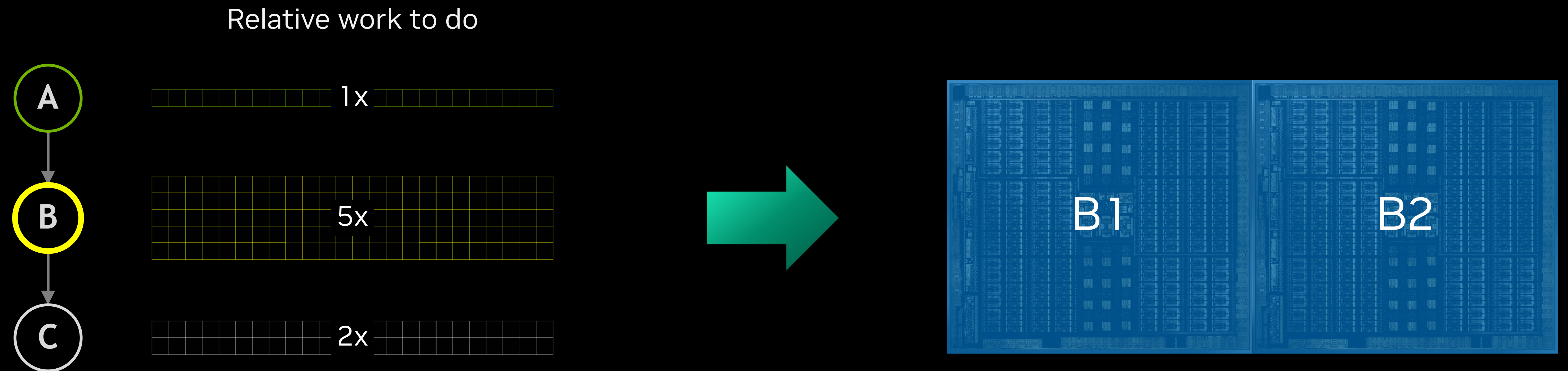
A

But this is what is **actually** happening:

A



Bulk Per-Kernel Data Parallelism



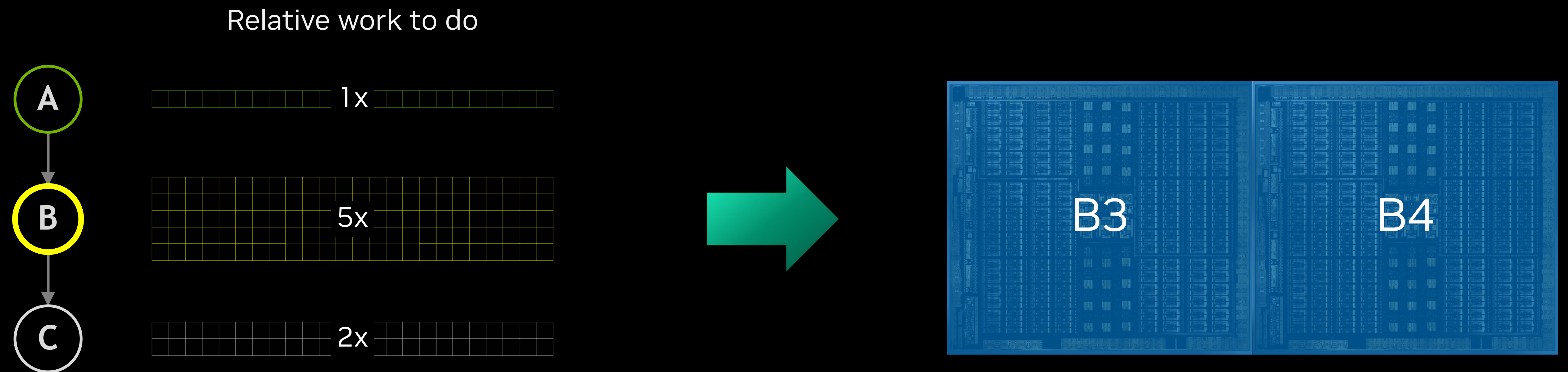
This is what you think **should be** happening:



But this is what is **actually** happening:



Bulk Per-Kernel Data Parallelism



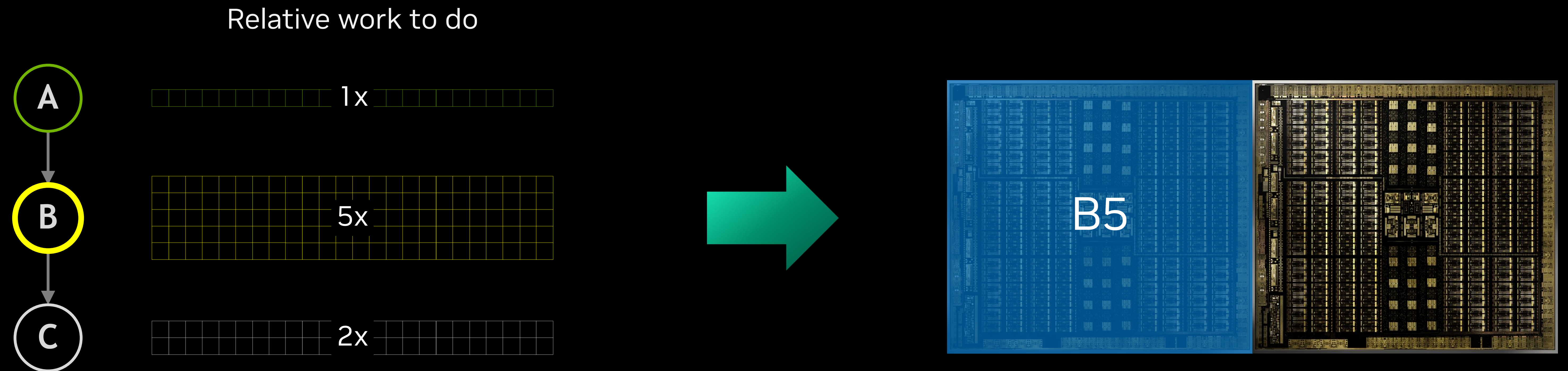
This is what you think **should be** happening:



But this is what is **actually** happening:



Bulk Per-Kernel Data Parallelism



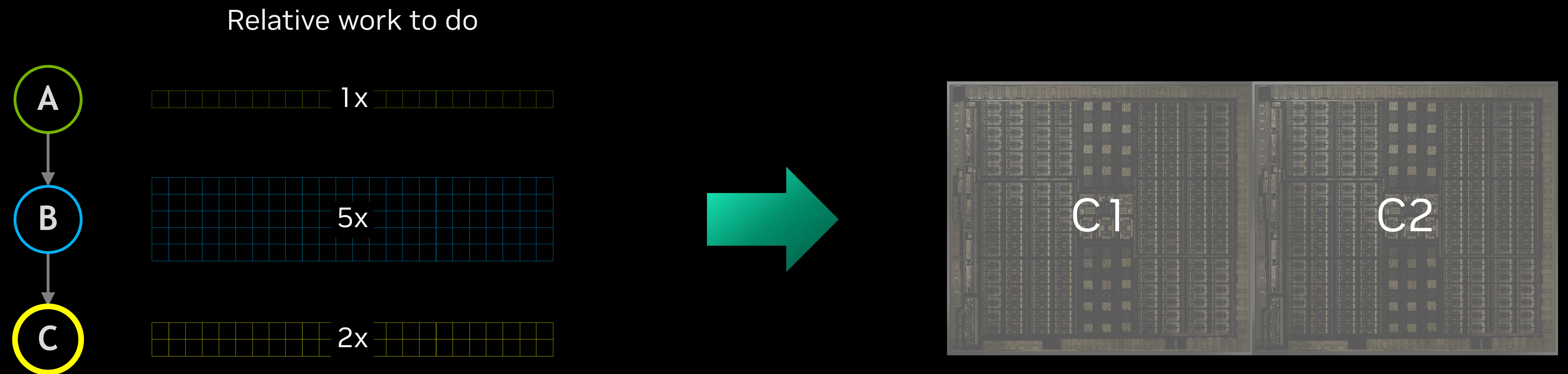
This is what you think **should be** happening:



But this is what is **actually** happening:



Bulk Per-Kernel Data Parallelism



This is what you think **should be** happening:



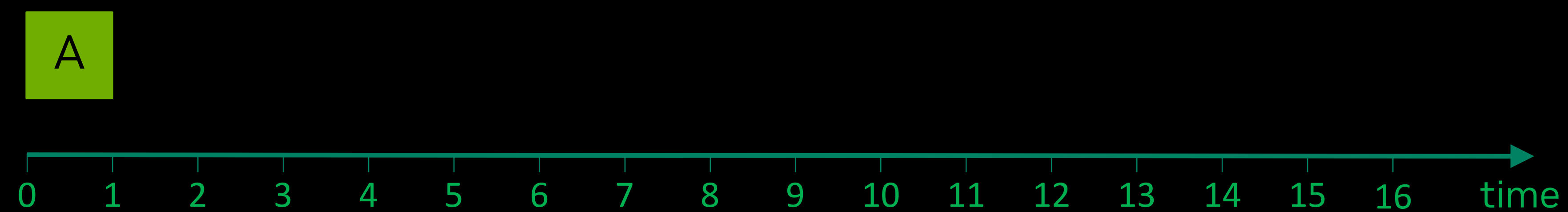
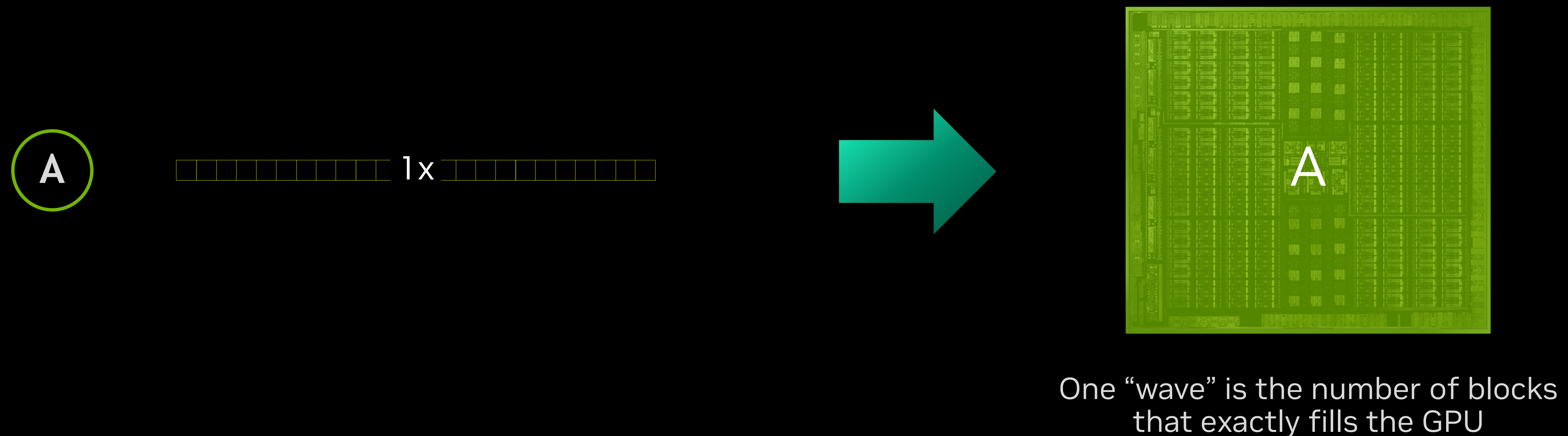
But this is what is **actually** happening:



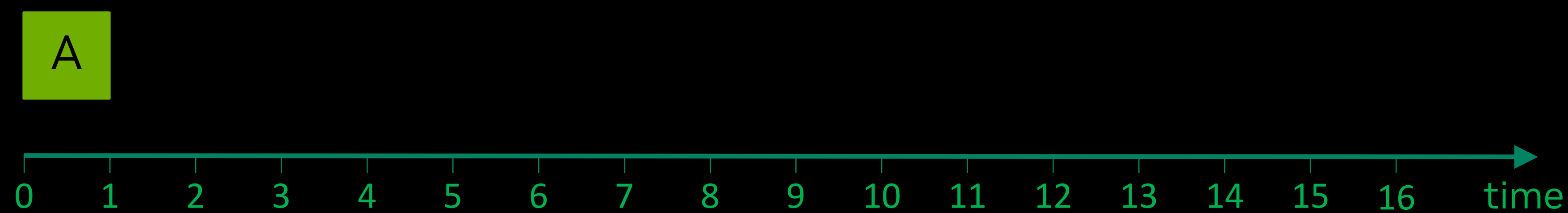
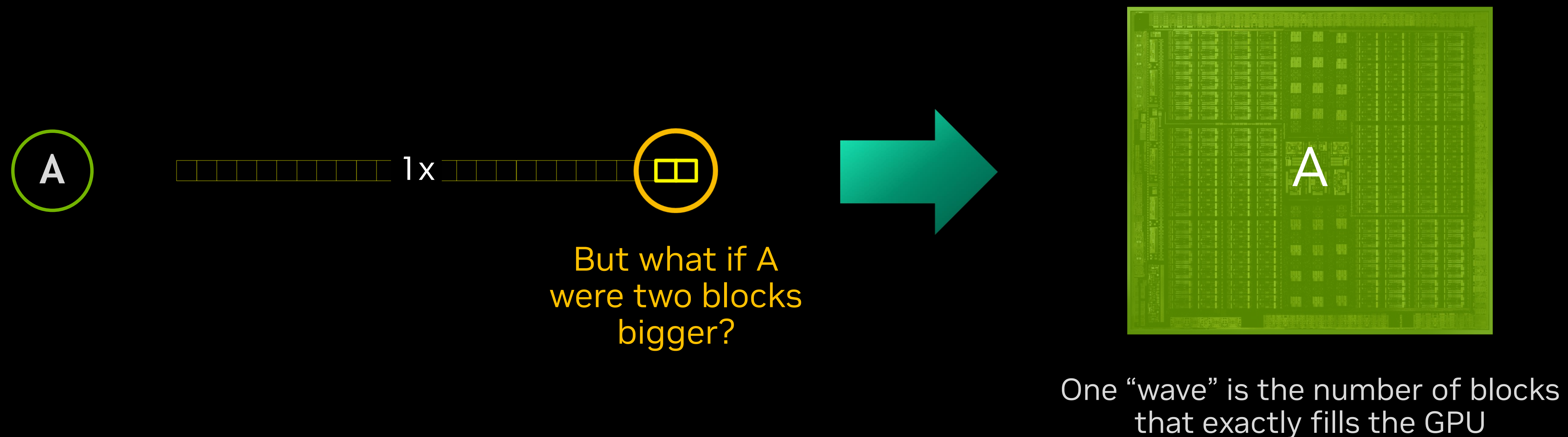
We have 2x the GPU but only went 1.75x faster!



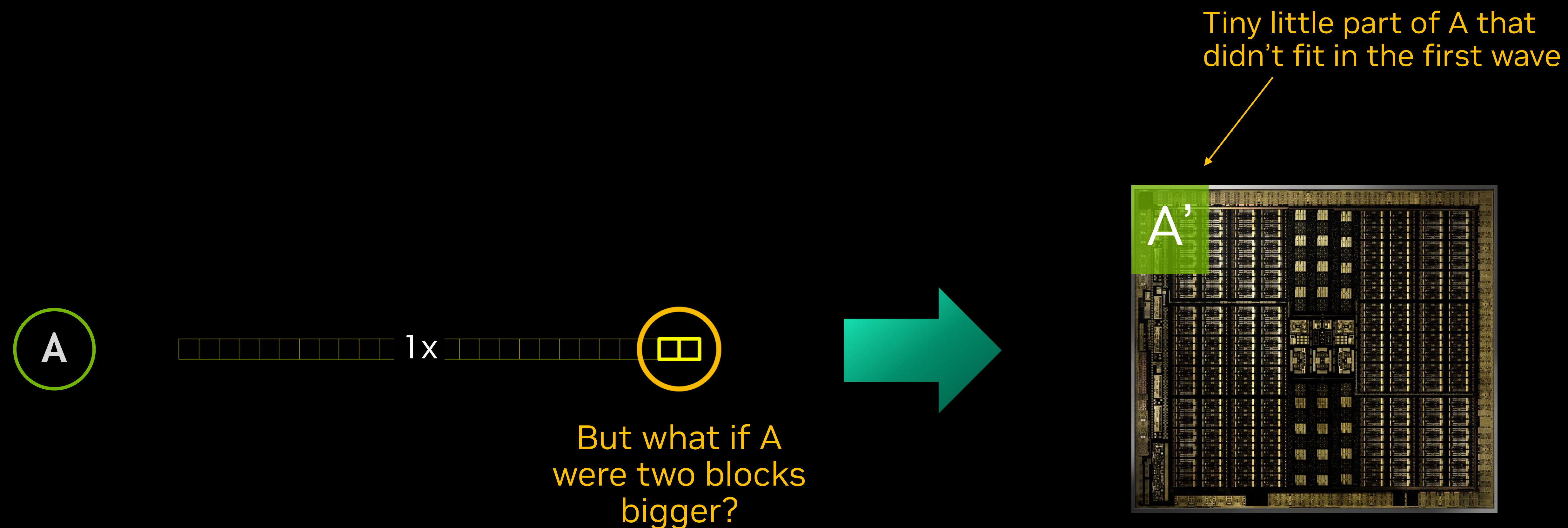
The Scourge of the Ninja: Wave Quantization



The Scourge of the Ninja: Wave Quantization

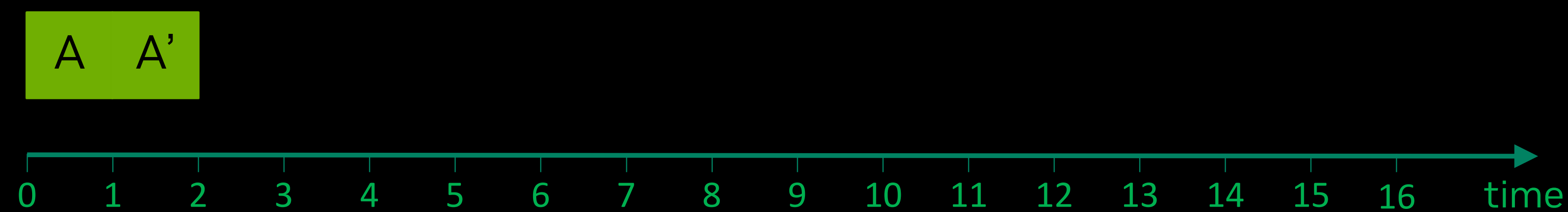


The Scourge of the Ninja: Wave Quantization

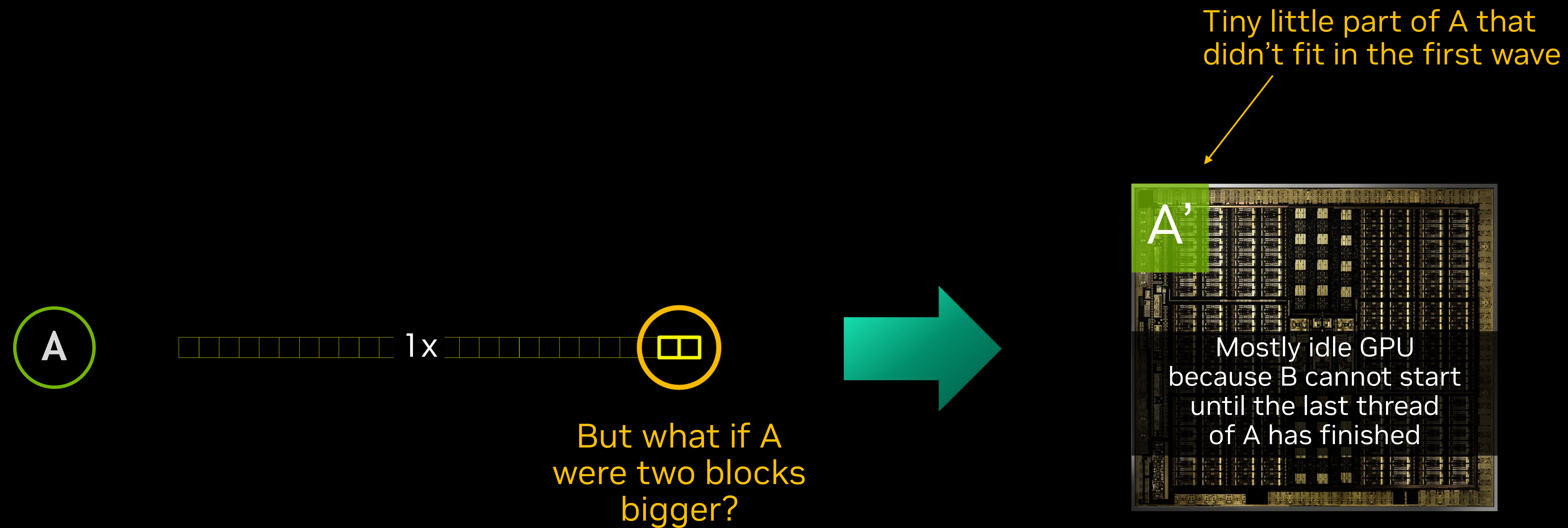


Extra wave of A can waste a lot of resources

There are **no partial waves**, hence “**wave quantization**”

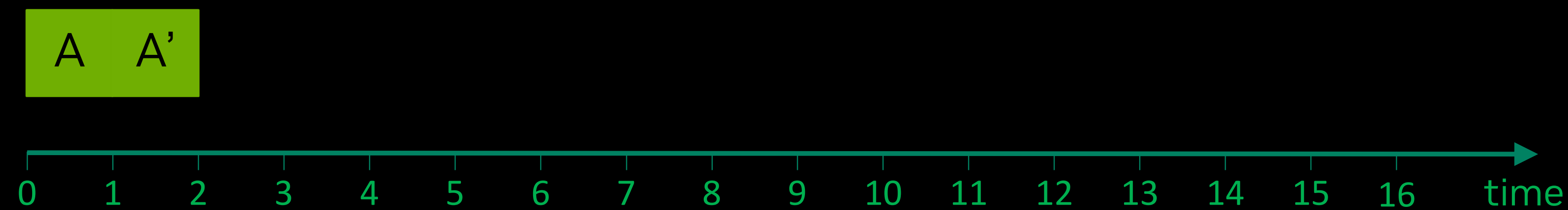


The Scourge of the Ninja: Wave Quantization

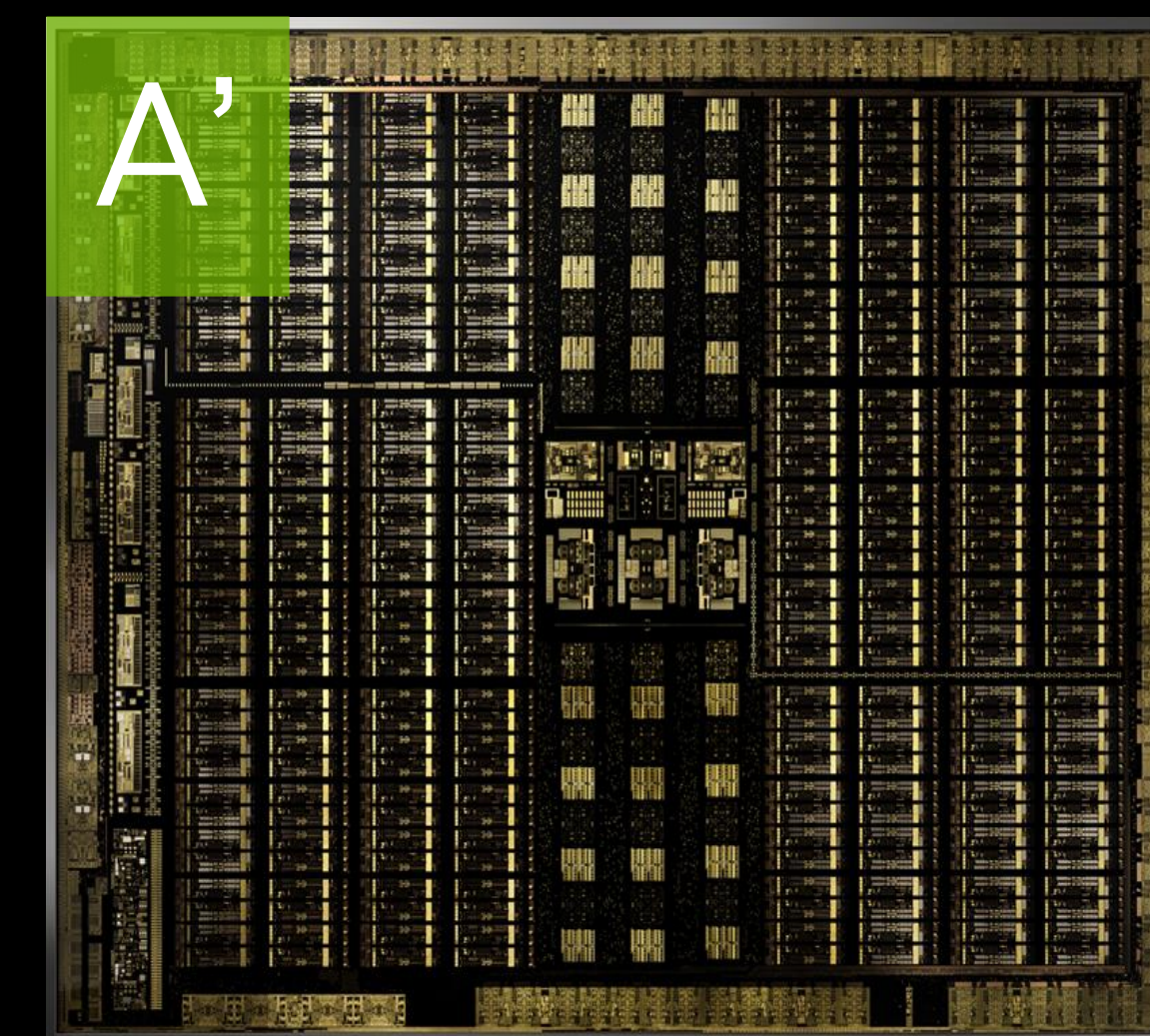
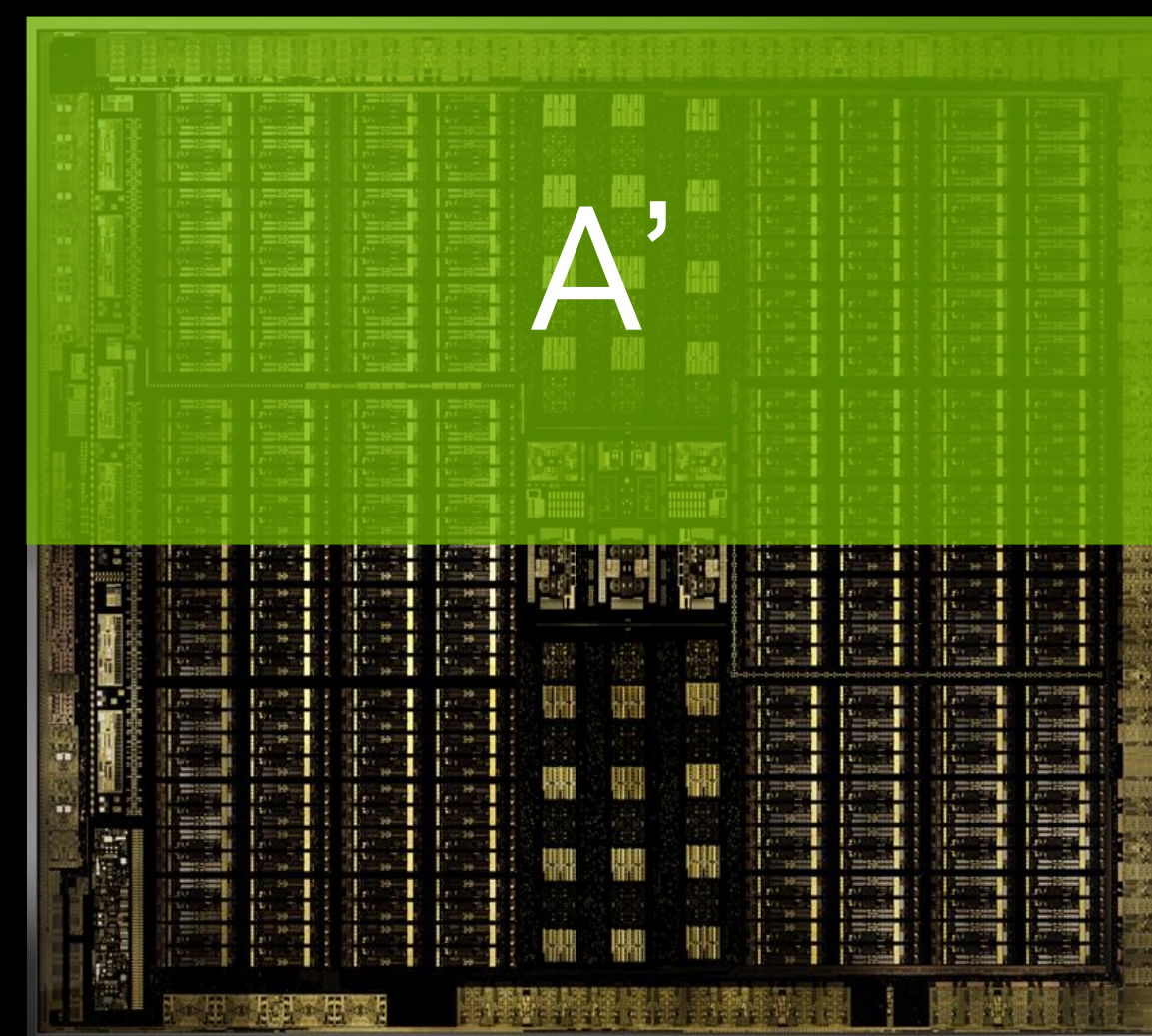
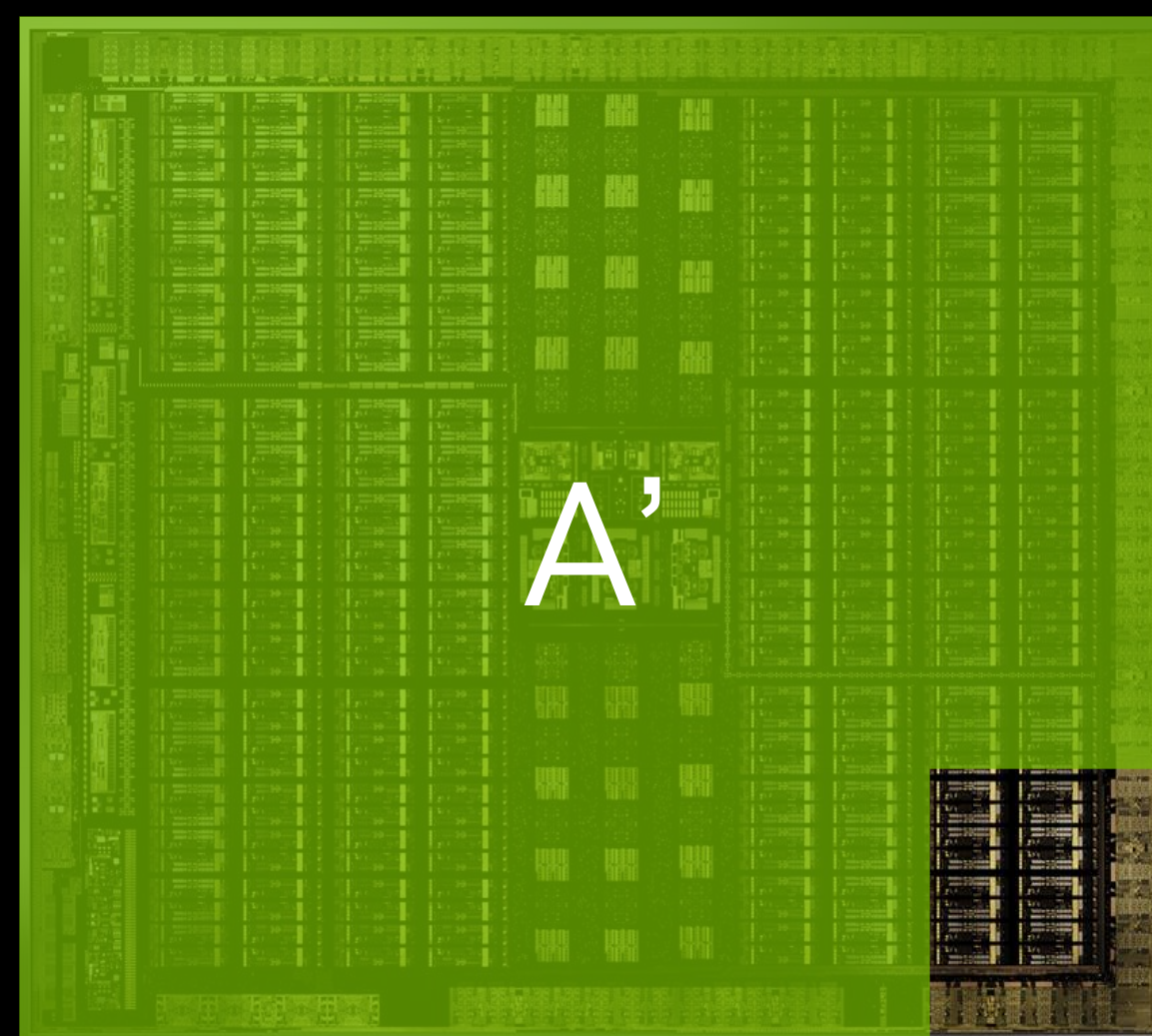


Extra wave of A can waste a lot of resources

There are **no partial waves**, hence “**wave quantization**”



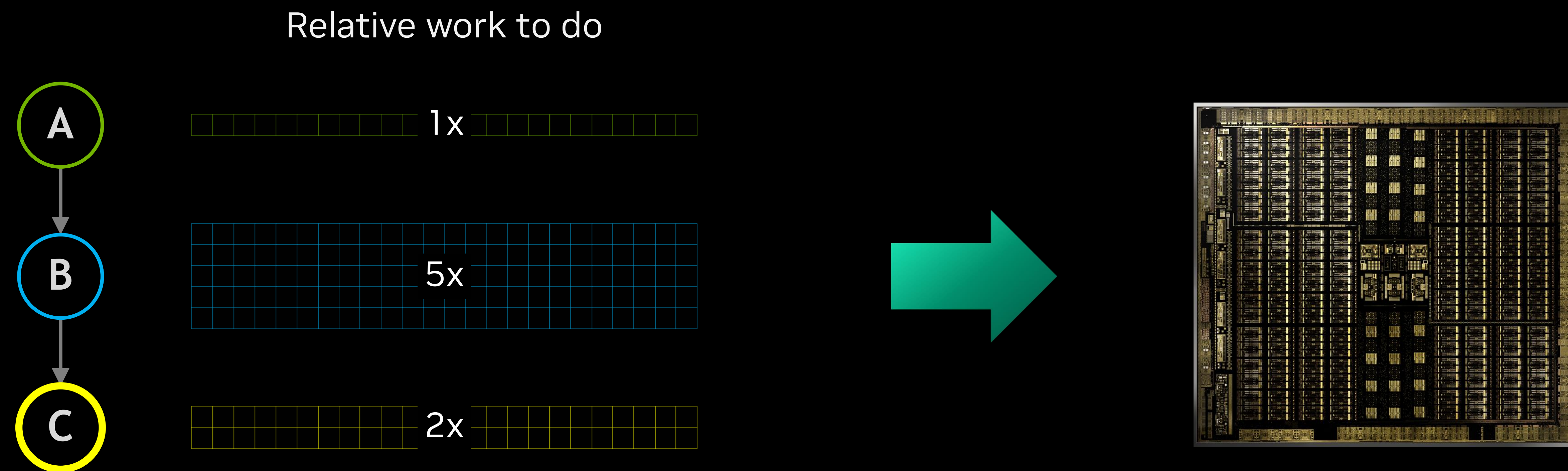
The Scourge of the Ninja: Wave Quantization



Unless you've intentionally sized your grid launch to be one wave,
the quantization effect will on average be random

Lose ~50% (+/- 50%) of efficiency on the final wave

The Scourge of the Ninja: Wave Quantization



So here's what really happens:



This is what you thought was happening:



Yes, More Waves Mitigates This, But Bigger GPUs = Fewer Waves



Hopper H100 Full Chip

132 SMs

Kepler GK110
Full Chip
15 SMs

2012: Kepler GK110 architecture = **15 SMs**

2022: Hopper H100 architecture = **132 SMs**

9x SM count increase in 10 years

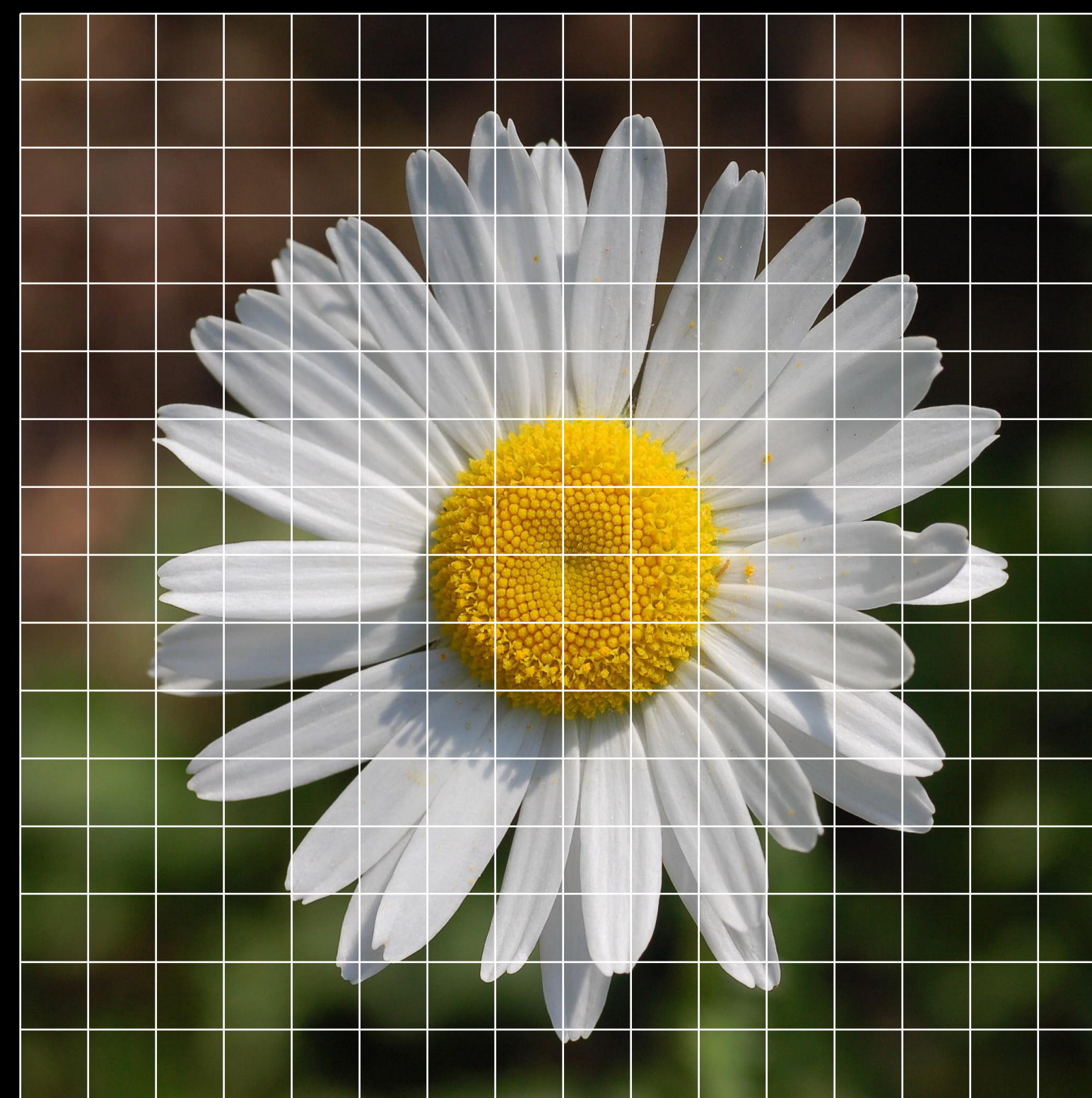
So a kernel which ran in 10 waves on GK110
now runs in 1.1 waves on H100

We went from **10% overhead** from wave
quantization **to 50% overhead**

That is: **you could be running twice as fast**

Ninjas Use Single-Wave Kernels

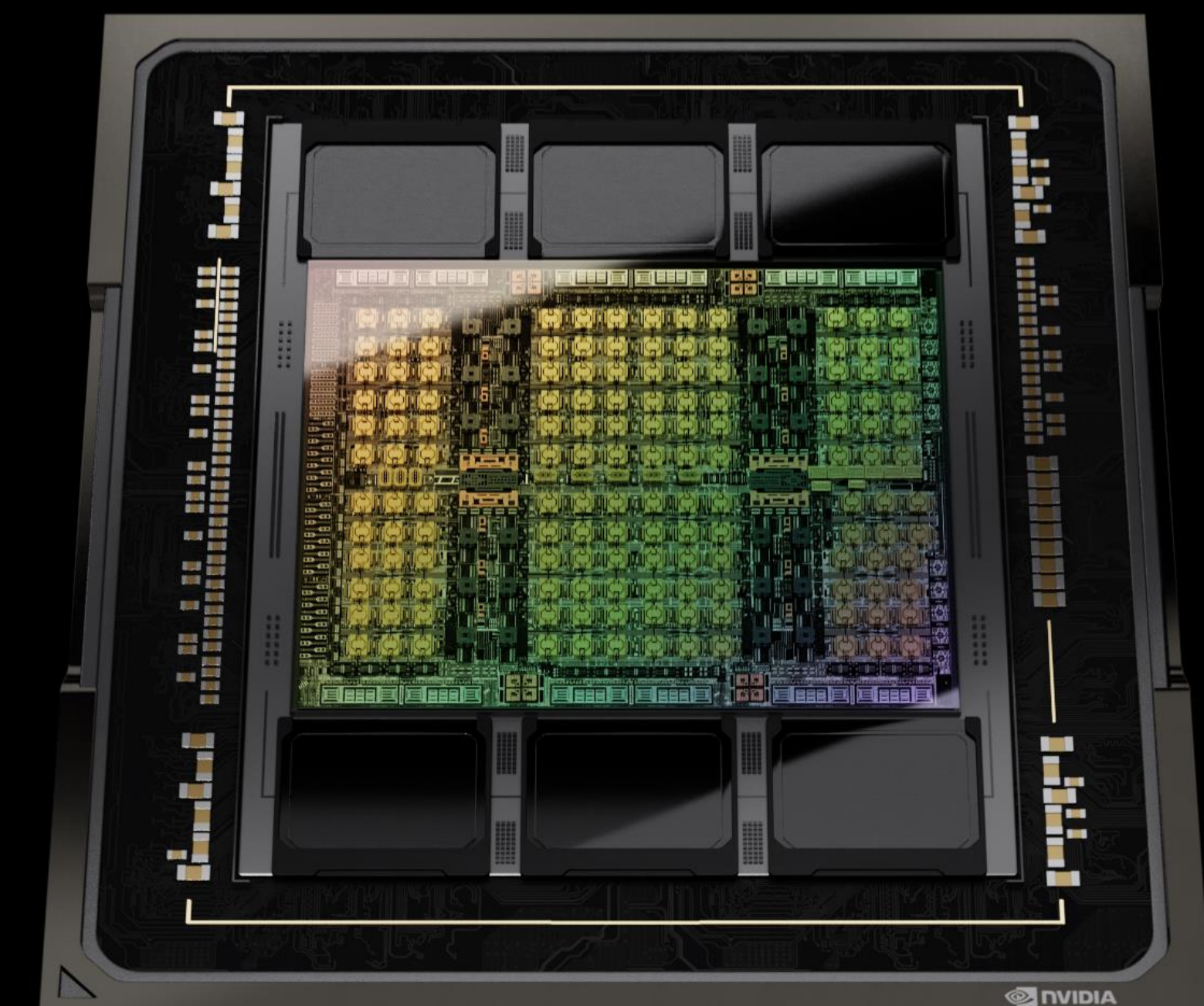
Don't map threads to data; map data to threads



1024x1024 image divided into 16x16 tiles

Image suggests 16x16 tiles = 256 blocks

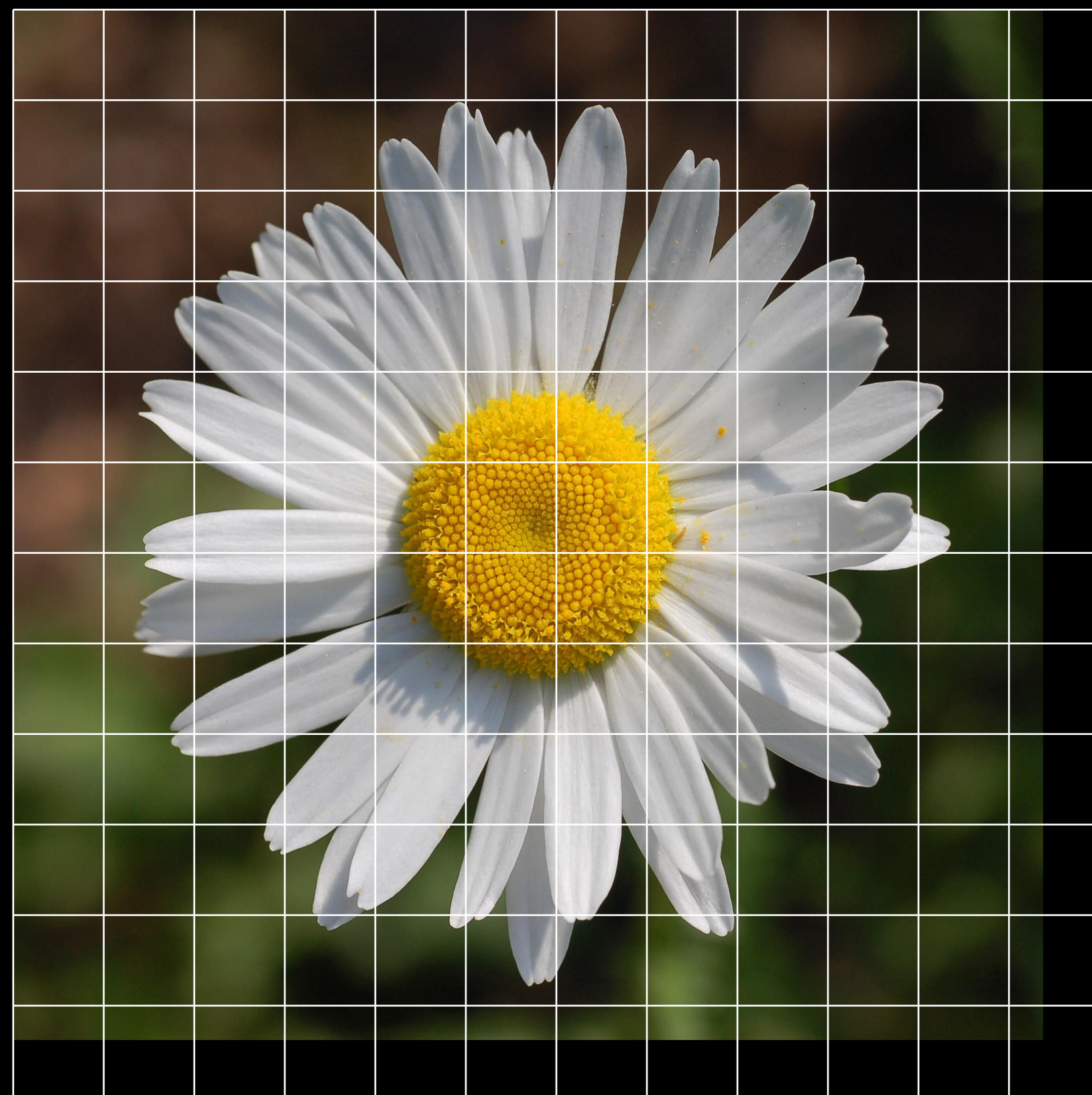
Hardware suggests $\sqrt{132}$ SMs = **11.5 x 11.5** tiles



H100 with 132 SMs

Ninjas Use Single-Wave Kernels

Don't map threads to data; map data to threads

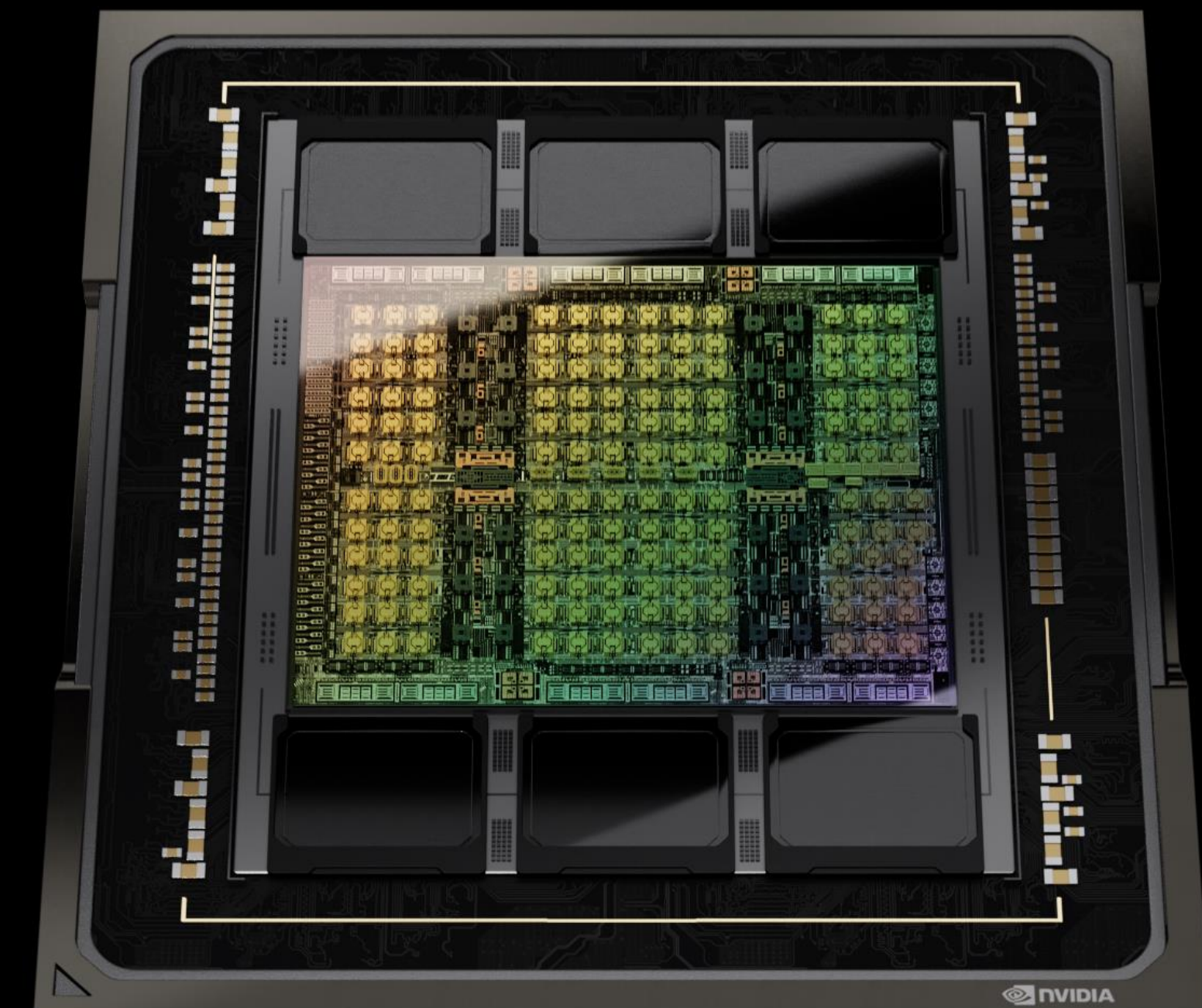


1024x1024 image divided into 16x16 tiles

Image suggests 16x16 tiles = 256 blocks

Hardware suggests $\sqrt{132}$ SMs = **11.5 x 11.5** tiles

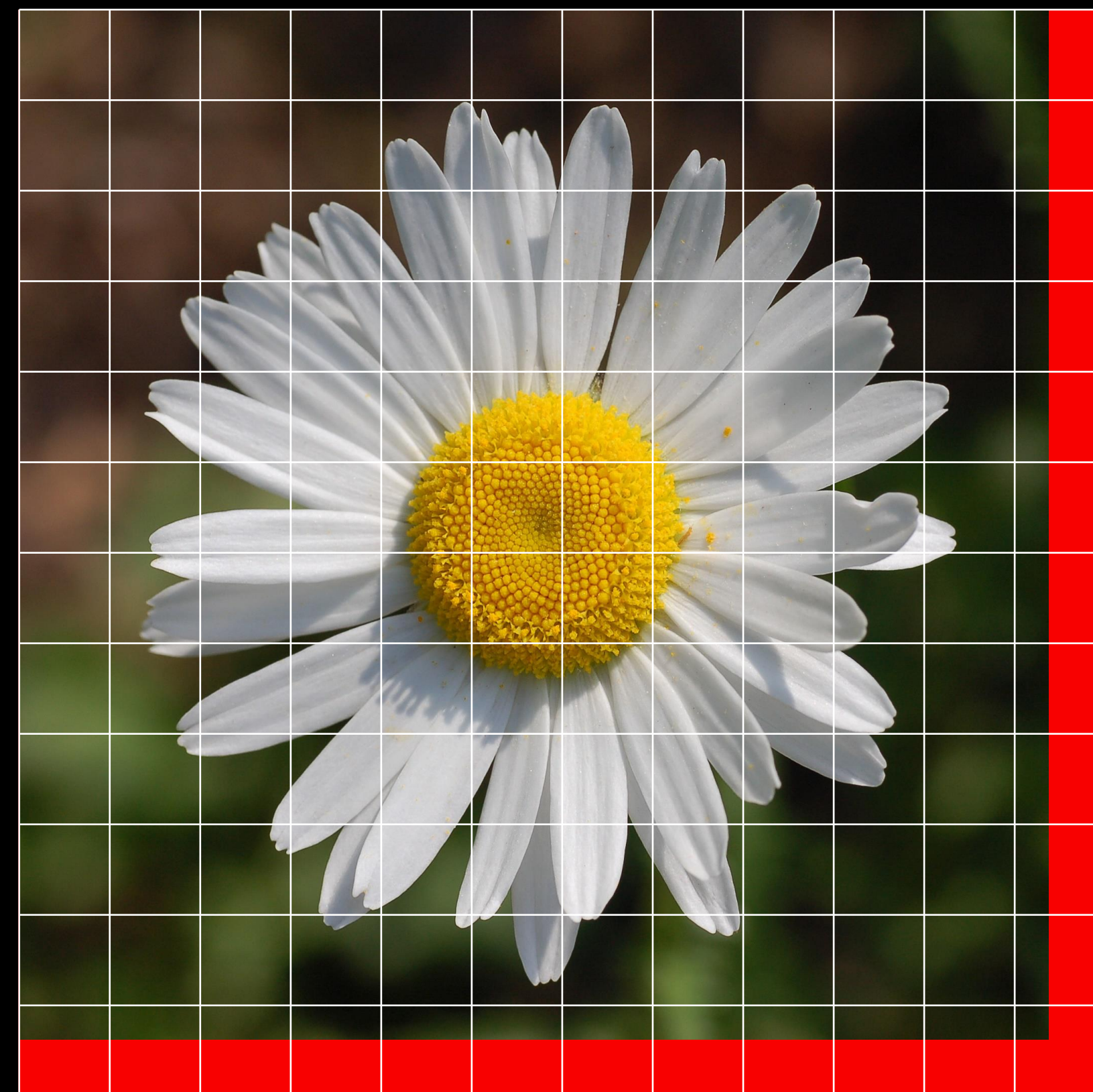
Where possible, always map data to threads



H100 with 132 SMs

Ninjas Use Single-Wave Kernels

Don't map threads to data; map data to threads



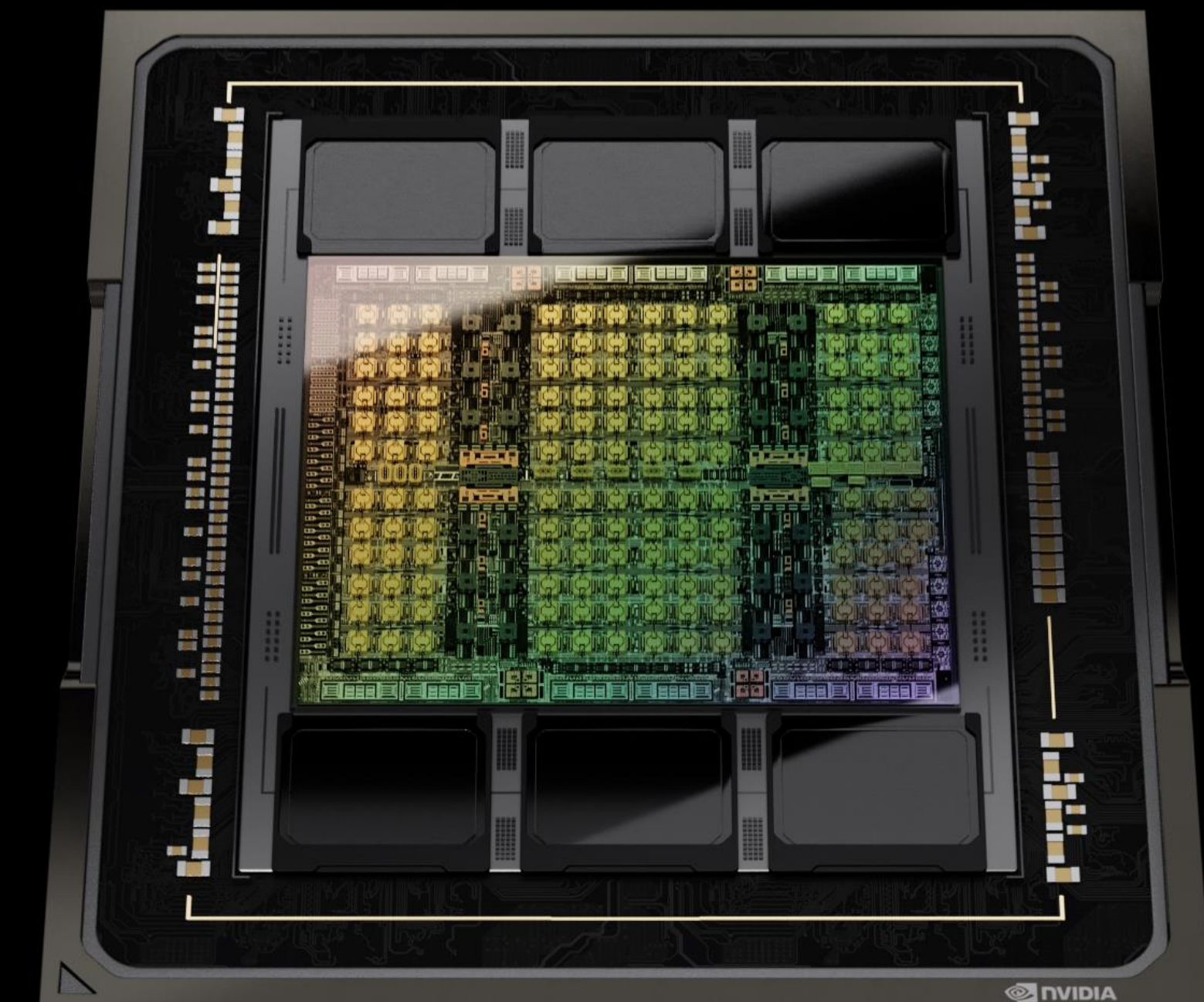
1024x1024 image divided into **12x12 tiles**

Red edge indicates where naïve rounding leaves imbalanced workload(*)

Image suggests 16x16 tiles = 256 blocks

Hardware suggests $\sqrt{132}$ SMs = 11.5 x 11.5 tiles

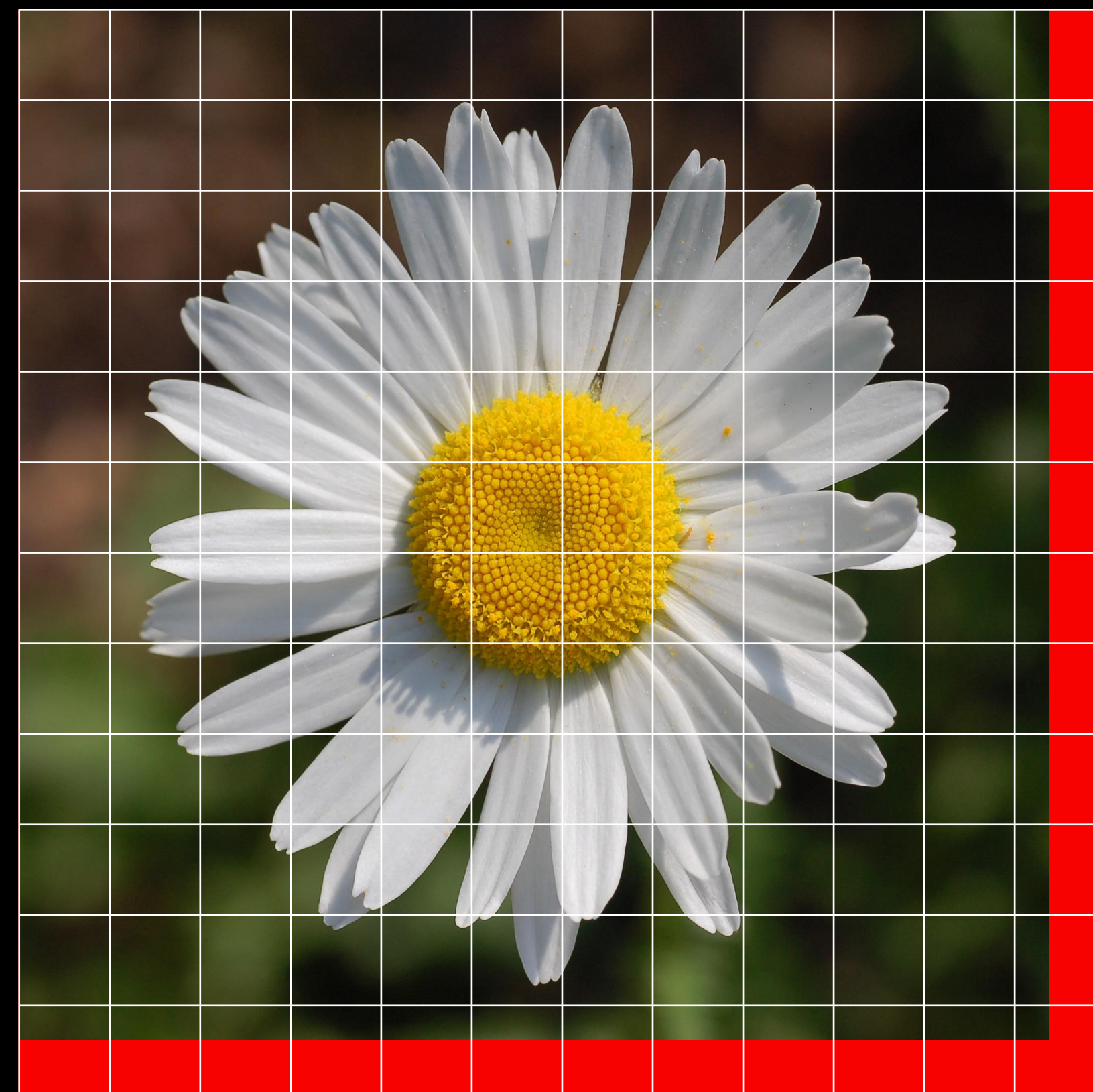
Where possible, always map data to threads



H100 with 132 SMs

(*) For better load balancing alternate tile sizes of 11 and 12, although this increases complexity and may be impossible depending on application

Not a Silver Bullet

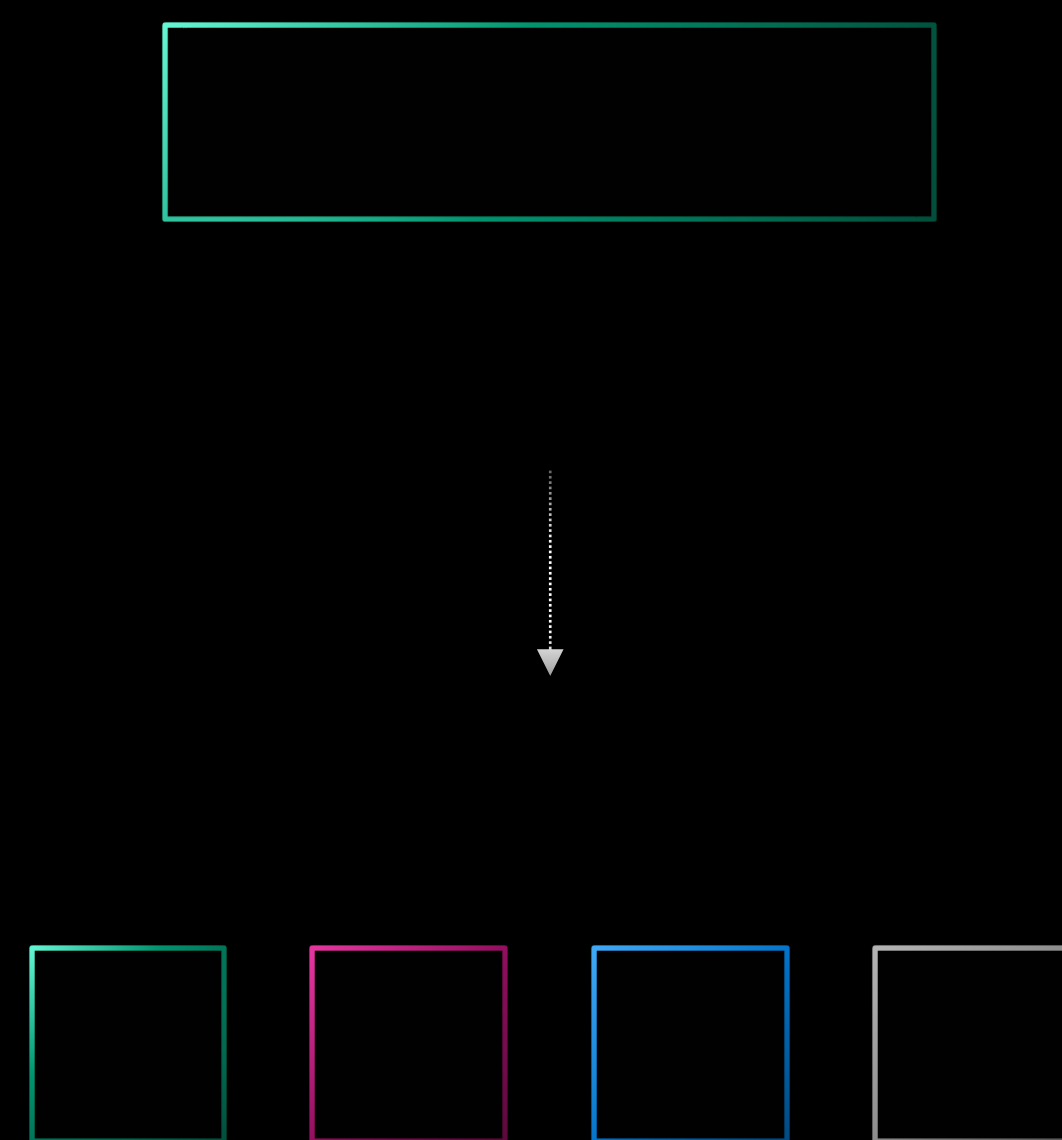


Single-wave kernels are better in almost all cases than non-integer-wave

Similar to “grid-stride loop” pattern frequently taught for CUDA

But there are a number of problems which may prevent use:

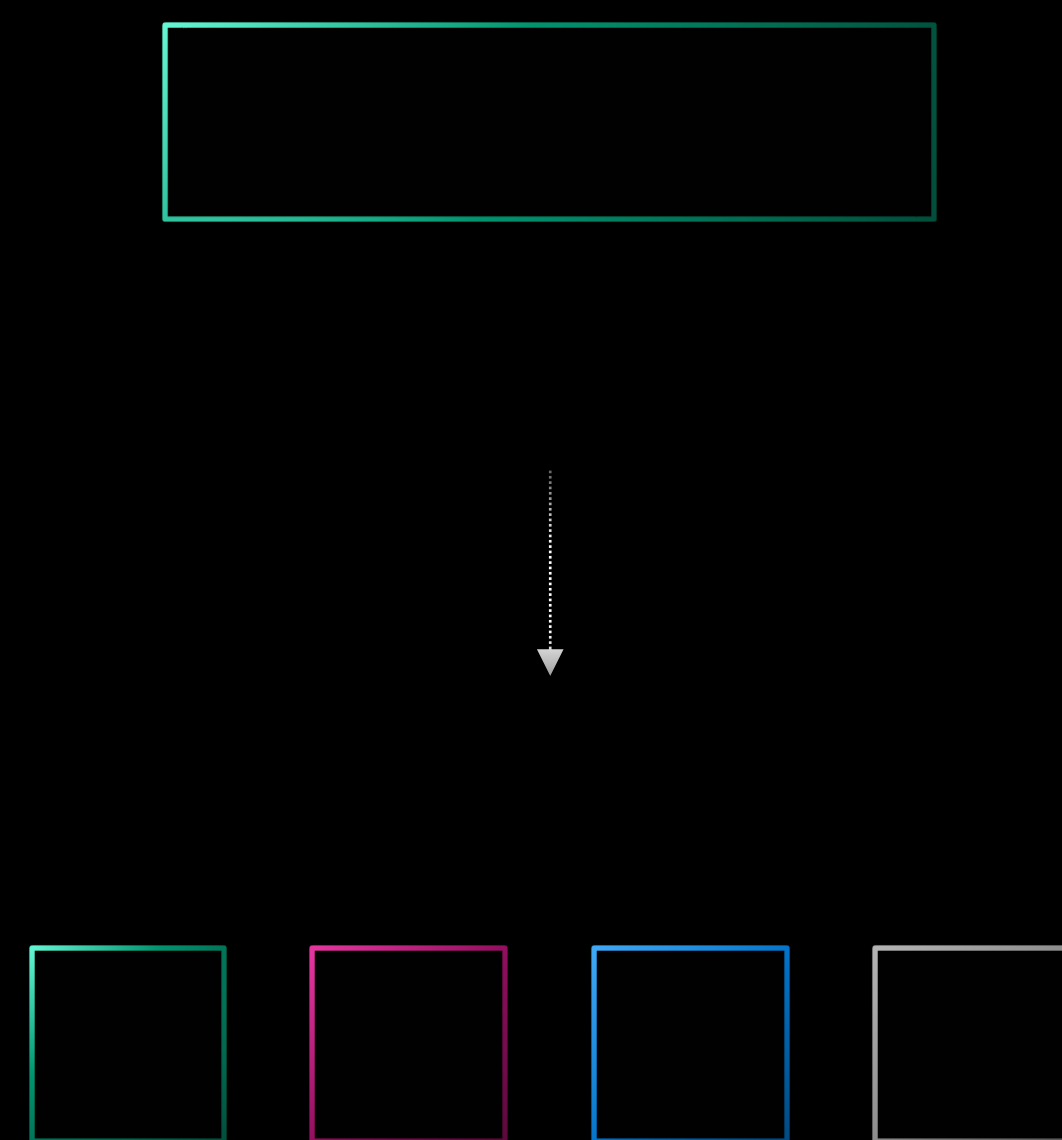
1. Some algorithms require specific size of tiling
2. Must account for GPUs of different sizes (e.g. RTX-3090/80/70/60)
3. Increase in code complexity by having non-constant tile size
4. Load imbalance remains; may be no better than an extra partial wave



Data Parallelism

Divide individual
elements across
processors

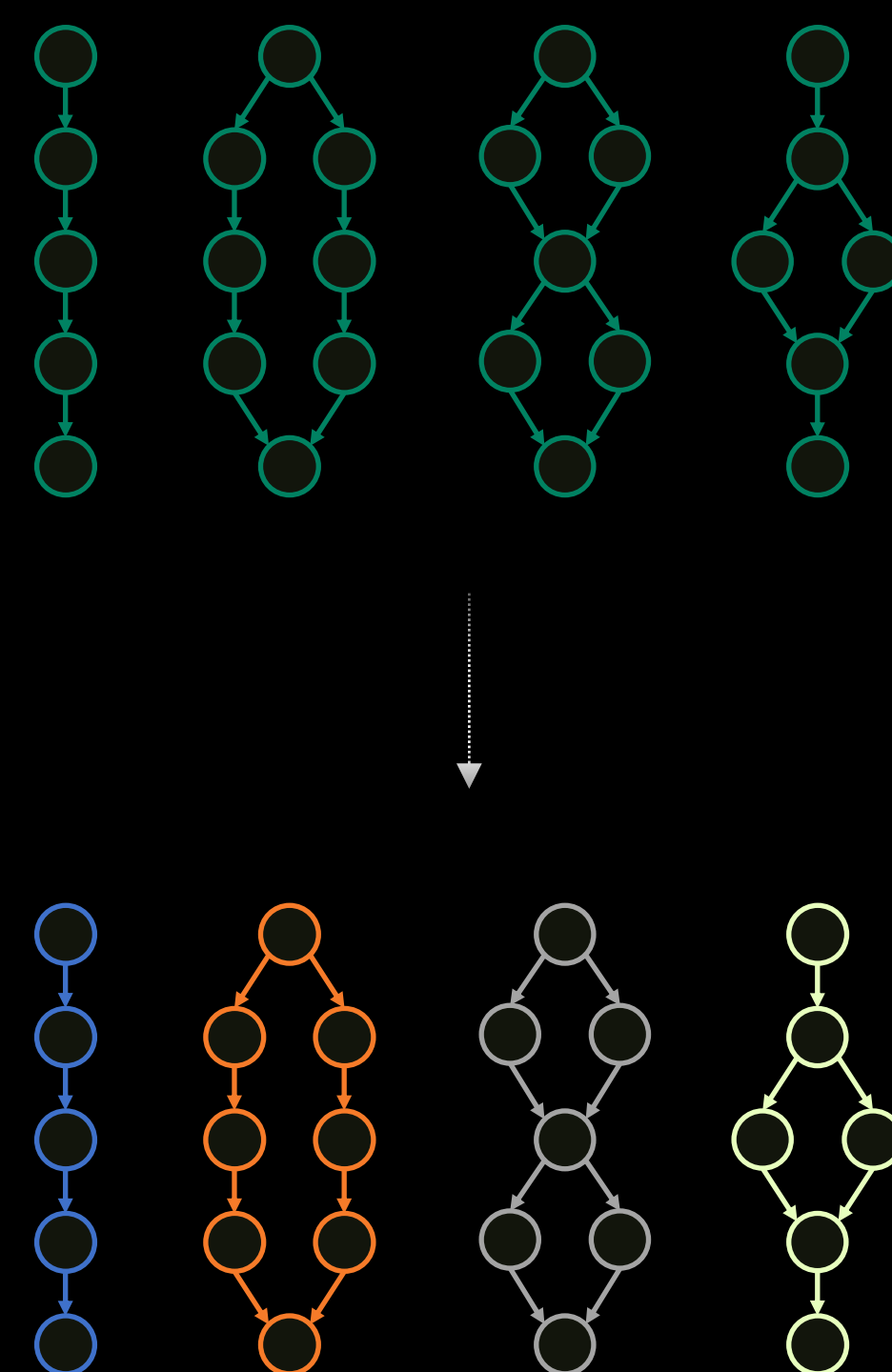
So bulk data parallelism won't
get us to 100% efficiency



Data Parallelism

Divide individual
elements across
processors

Thinking about things differently...

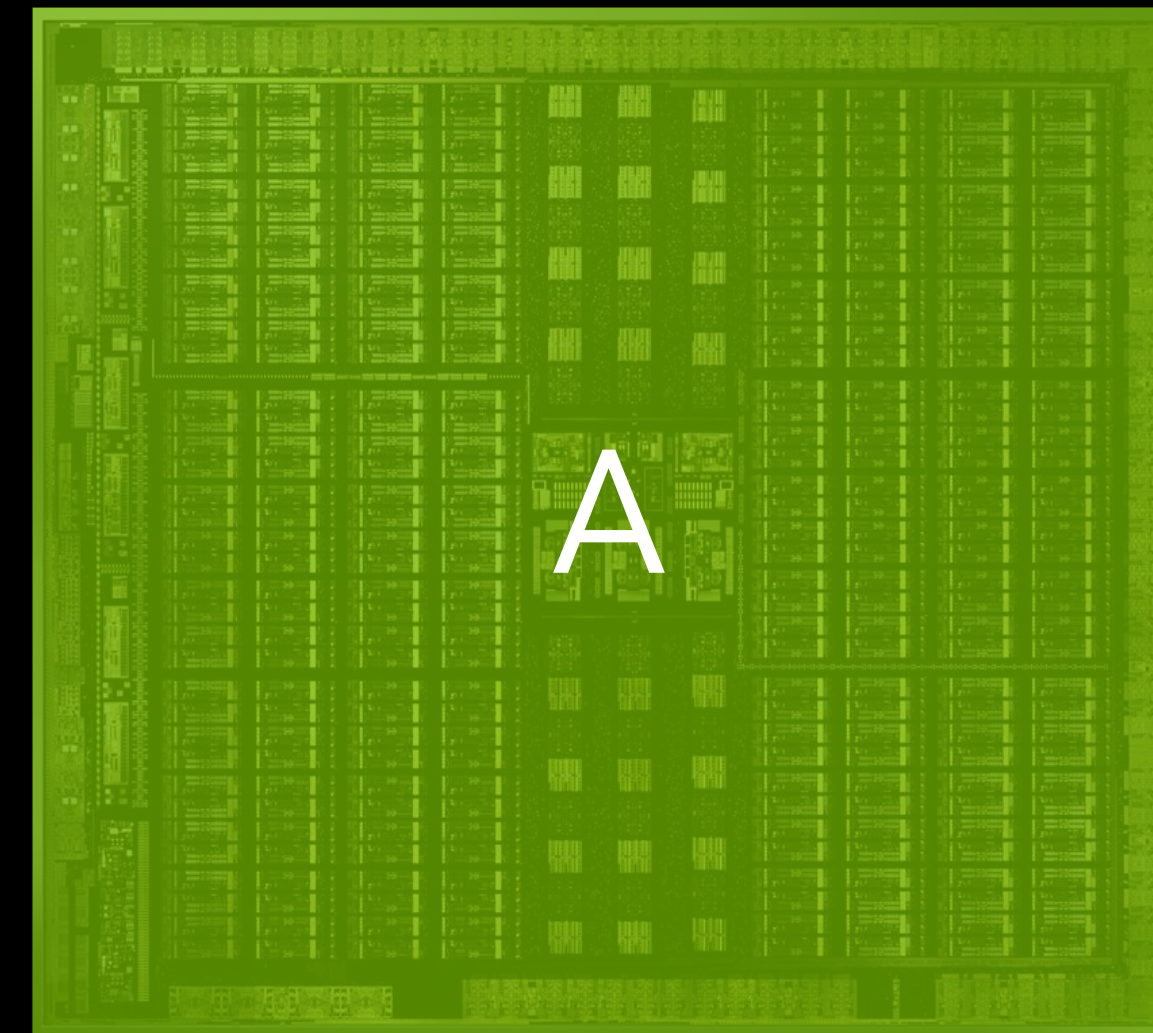
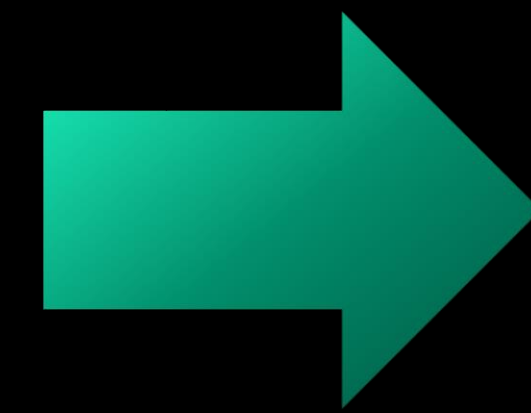
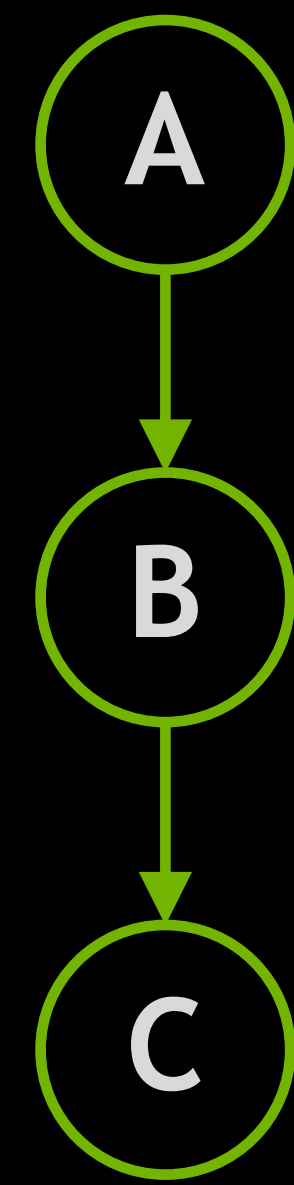


Task Parallelism

Divide independent
workflows across
processors

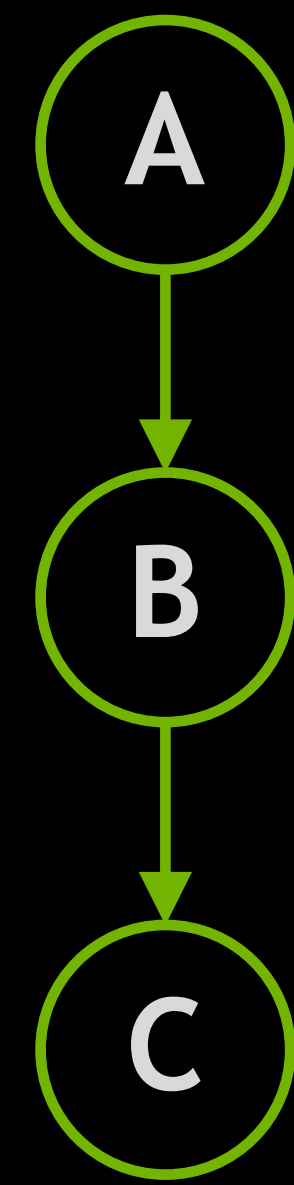
How about the other
kind of parallelism?

So Why Does Task Parallelism Help?

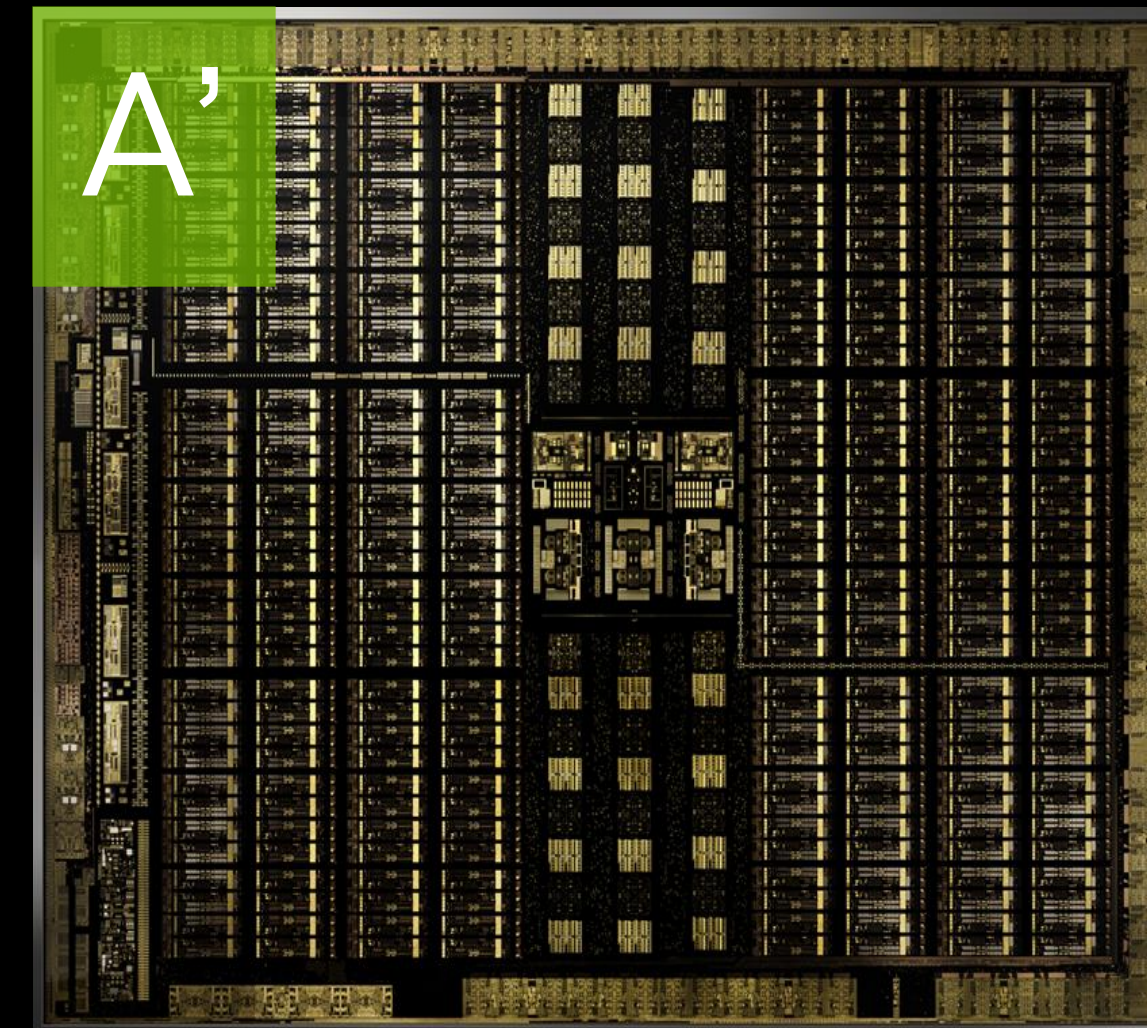
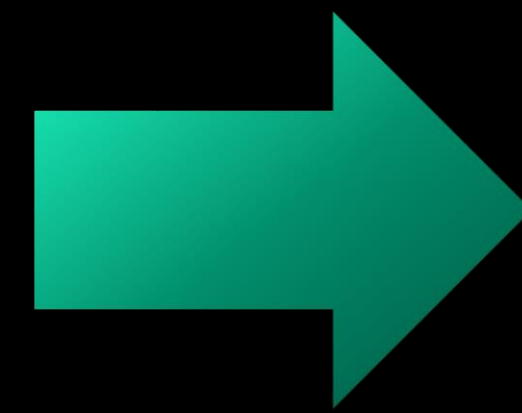


Task 1: $A \rightarrow B \rightarrow C$

So Why Does Task Parallelism Help?



Task 1: $A \rightarrow B \rightarrow C$

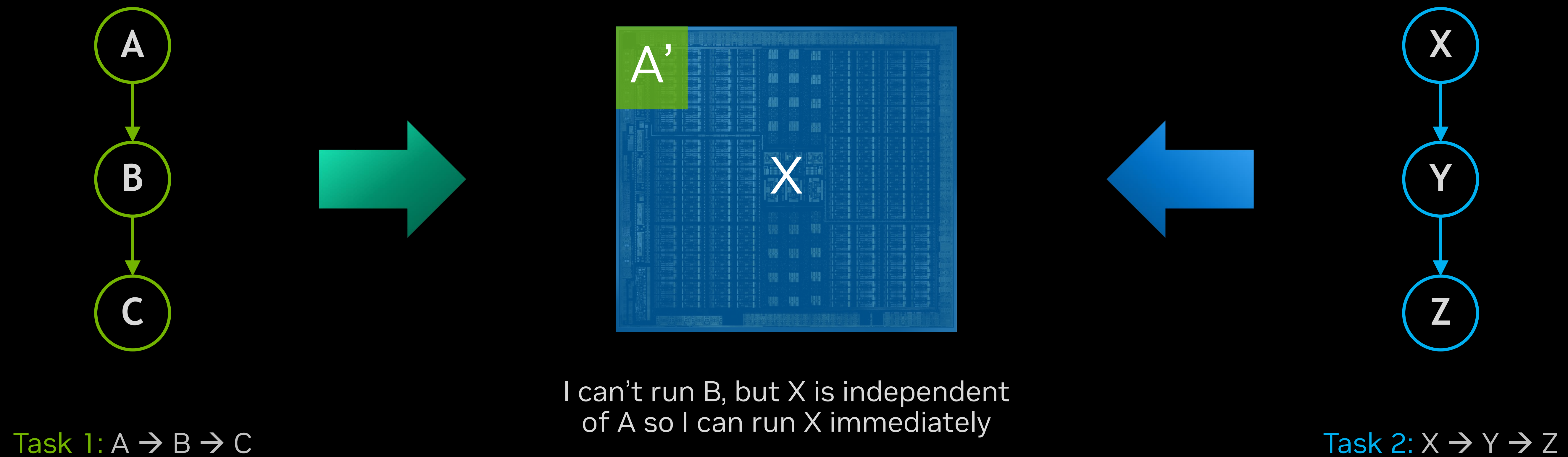


Mostly idle GPU because B cannot start until the last thread of A has finished

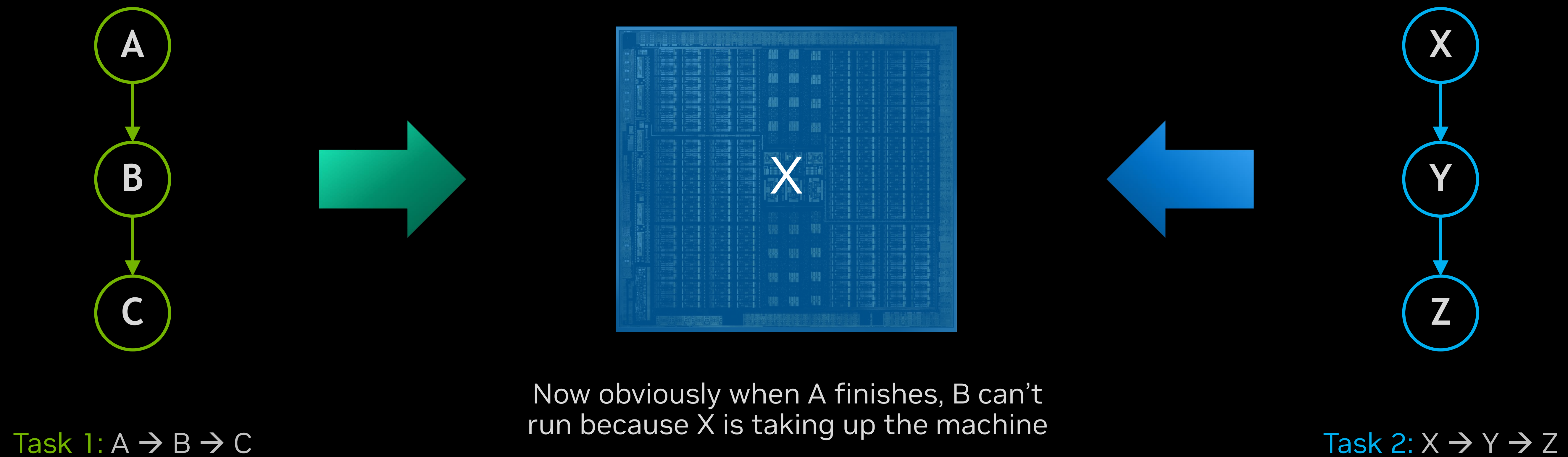
So Why Does Task Parallelism Help?



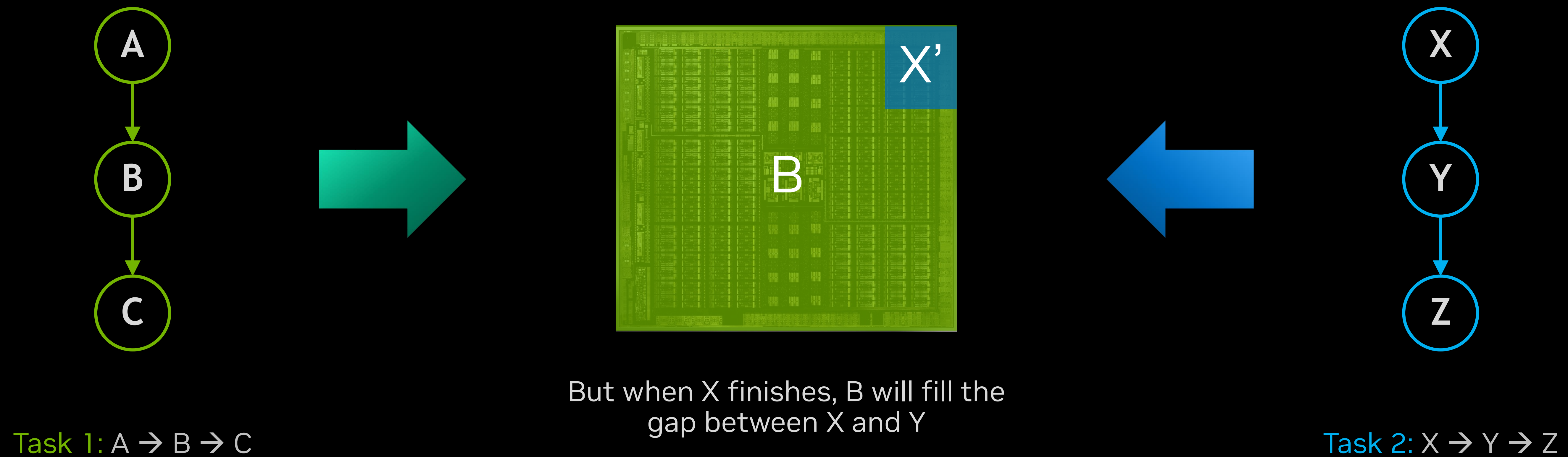
So Why Does Task Parallelism Help?



So Why Does Task Parallelism Help?

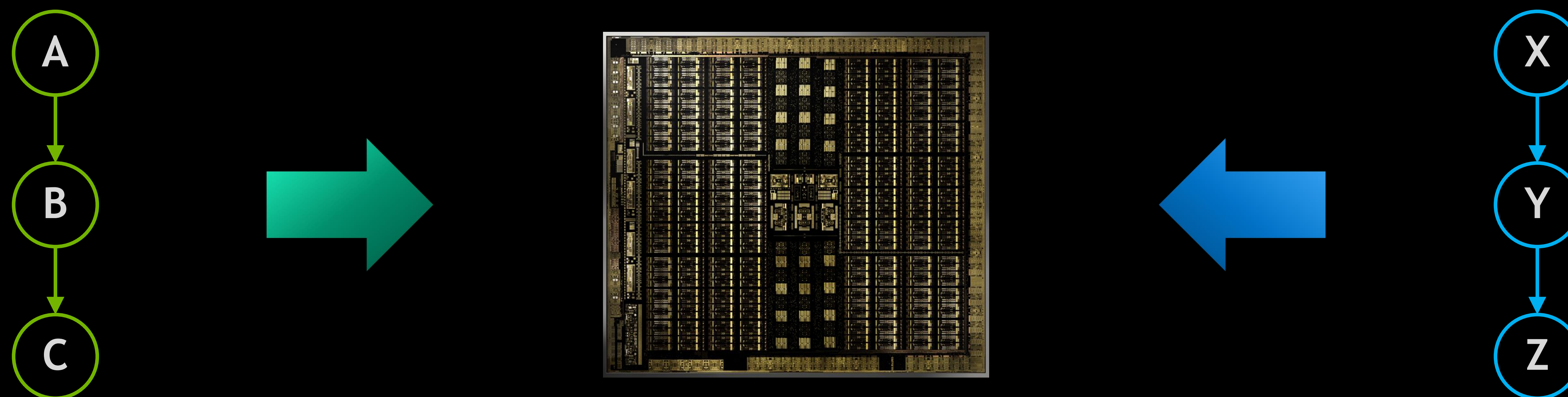


So Why Does Task Parallelism Help?

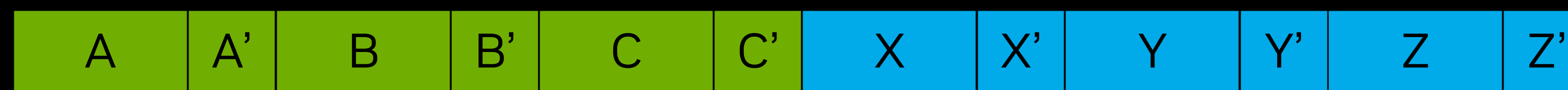


So Why Does Task Parallelism Help?

But ABC runs faster **in parallel** with XYZ than ABC+XYZ would run sequentially



Two independent tasks in series:

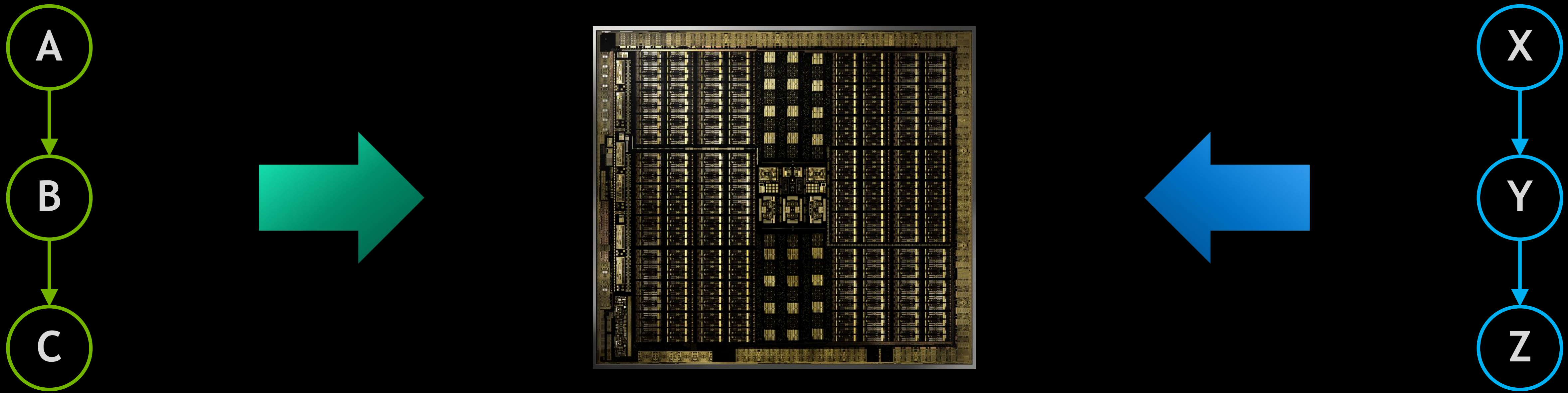


Are slower than two tasks in parallel:

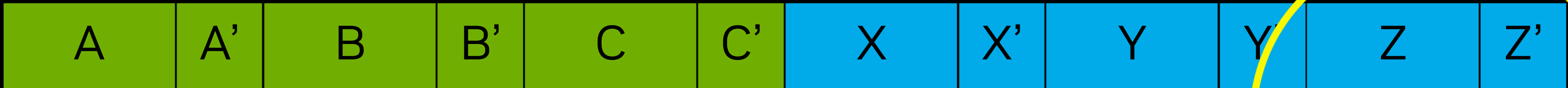


So Why Does Task Parallelism Help?

But ABC runs faster **in parallel** with XYZ than ABC+XYZ would run sequentially



Two independent tasks in series:



But tasks 1 & 2 complete sooner overall with higher **throughput**

Are slower than two tasks in parallel:

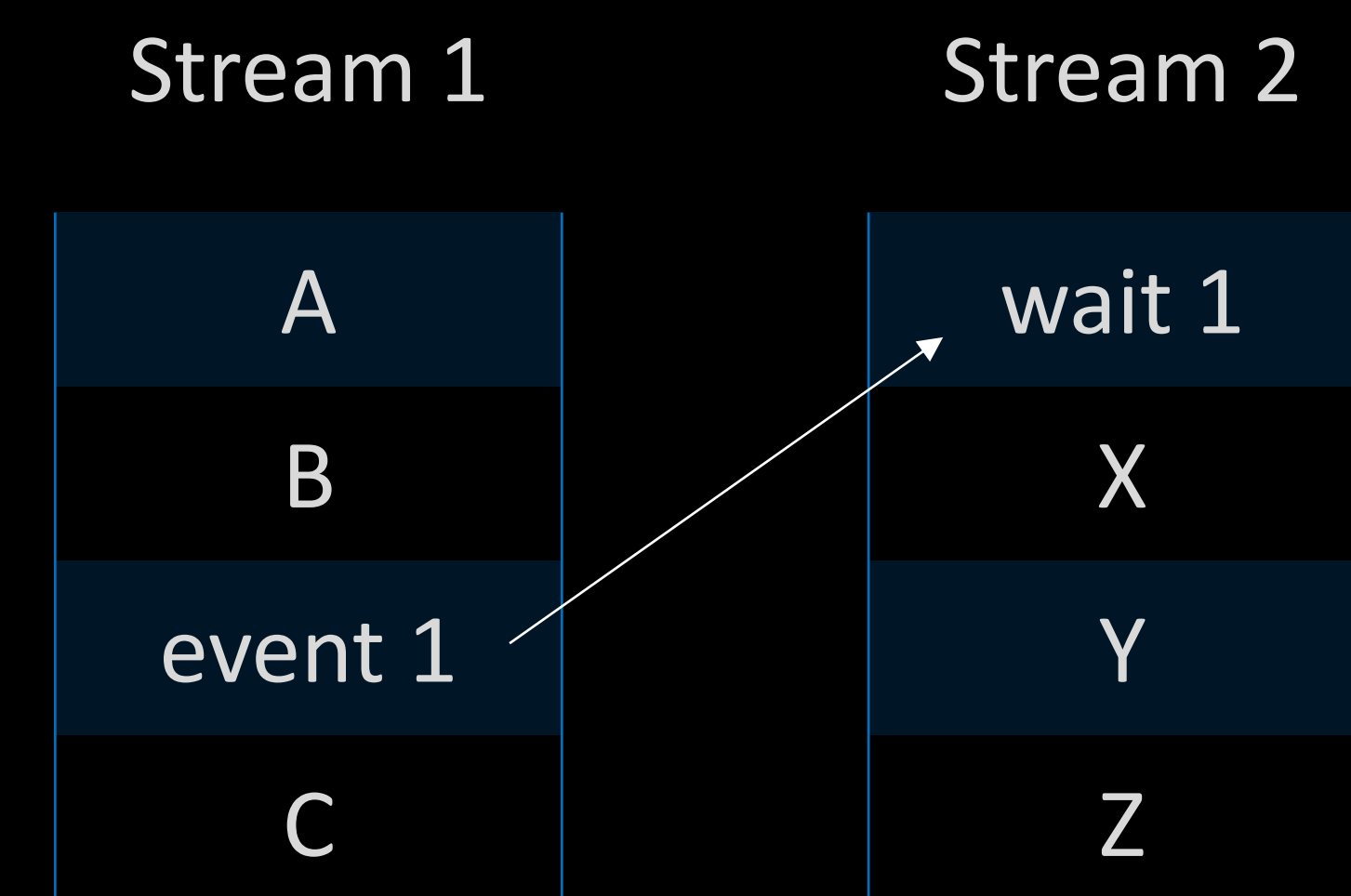


Task ABC experiences longer **latency**

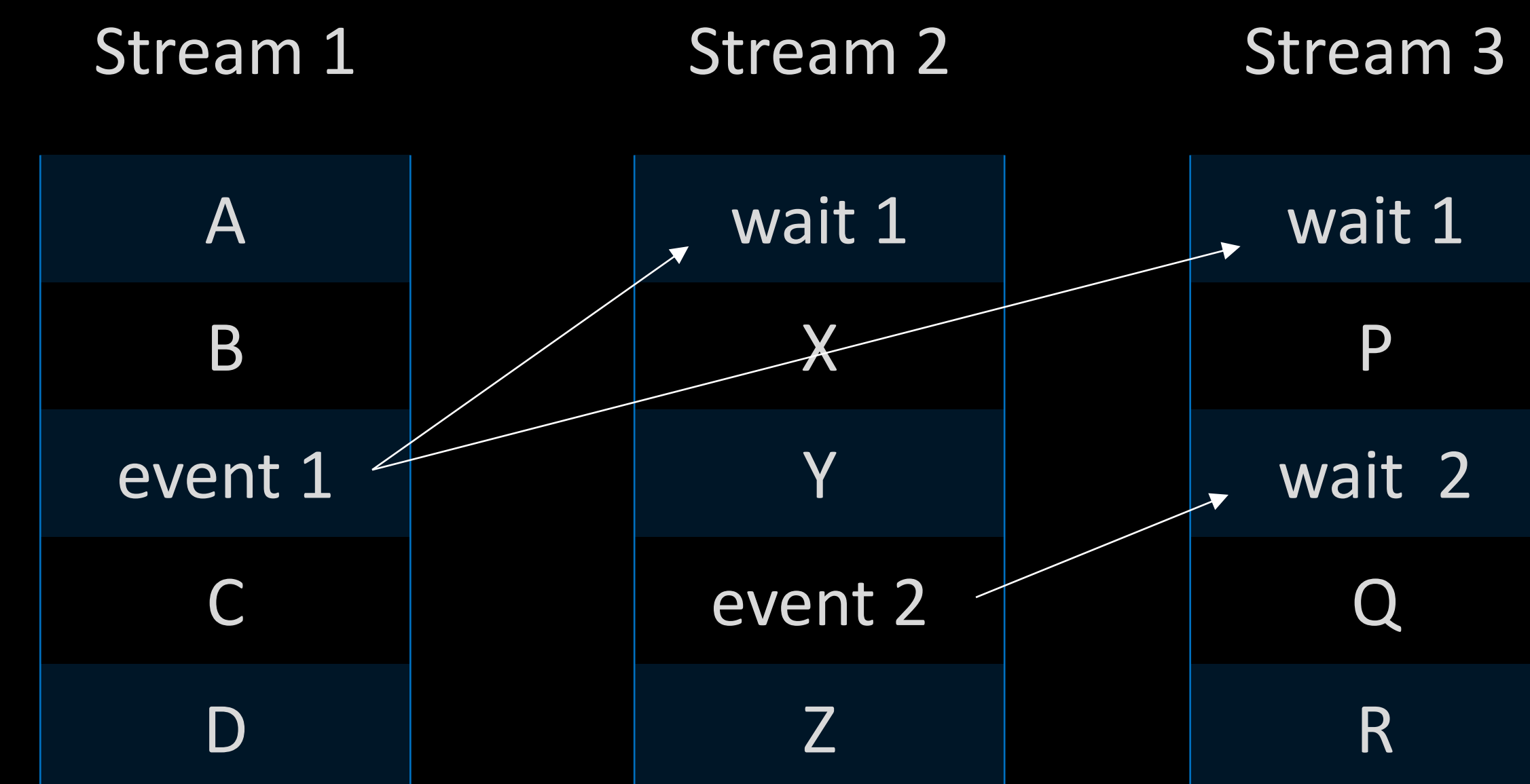
CUDA Makes Task Parallelism Easy To Do With Streams

Stream 1	Stream 2
A	X
B	Y
C	Z

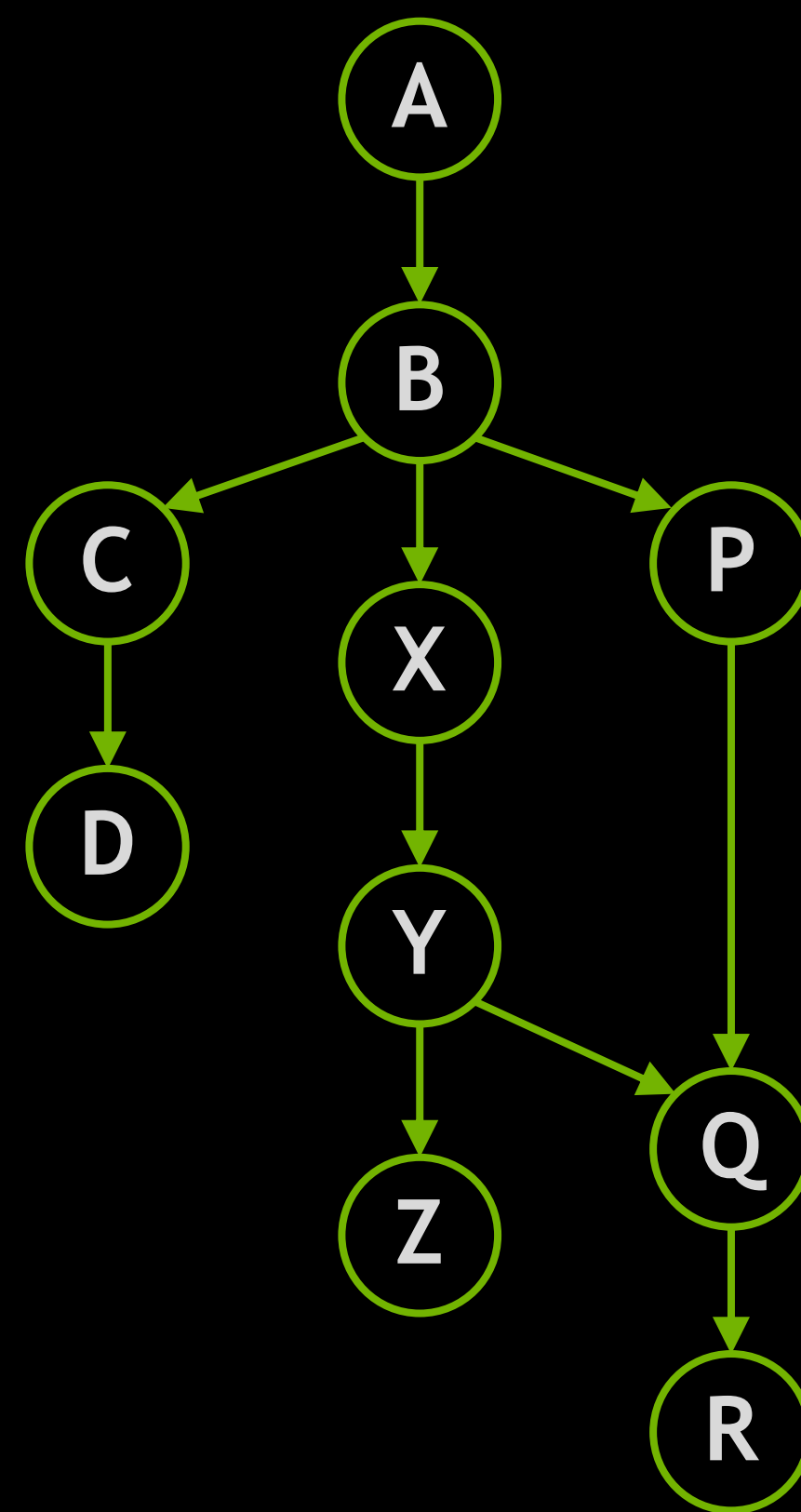
I Can Create Dependencies Between Streams



I Can Create Lots Of Streams With Lots Of Dependencies



I Can Turn It All Into A CUDA Graph



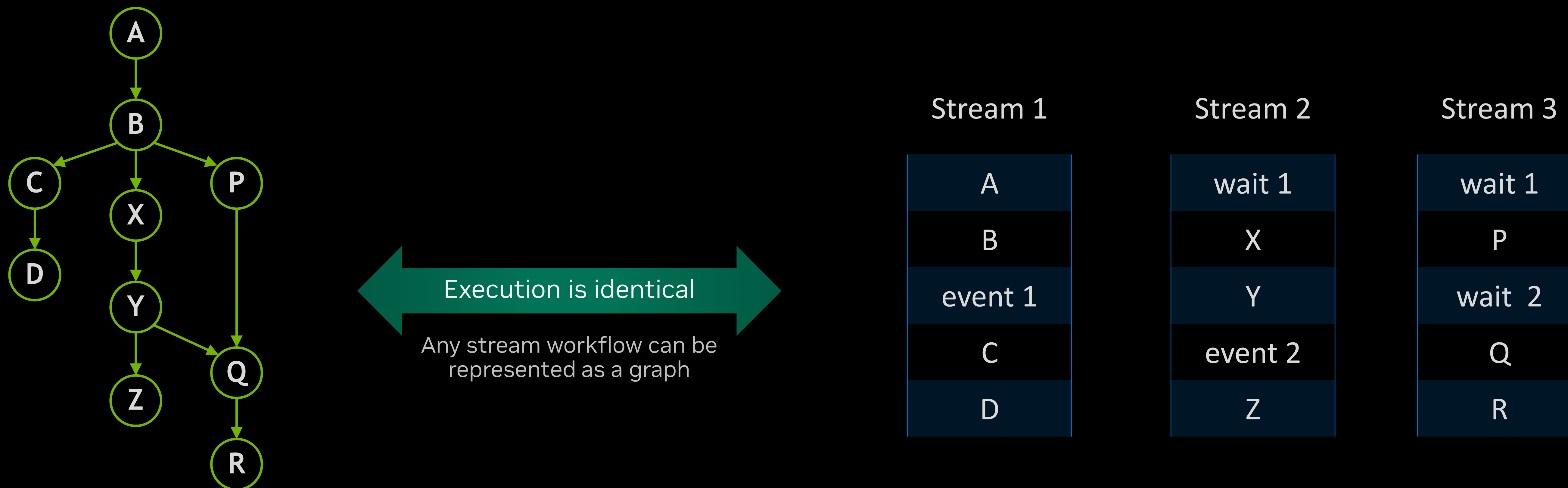
Execution is identical
Any stream workflow can be
represented as a graph



All this is great but it's just basic (or maybe advanced) CUDA

We're ninjas here – where is this going?

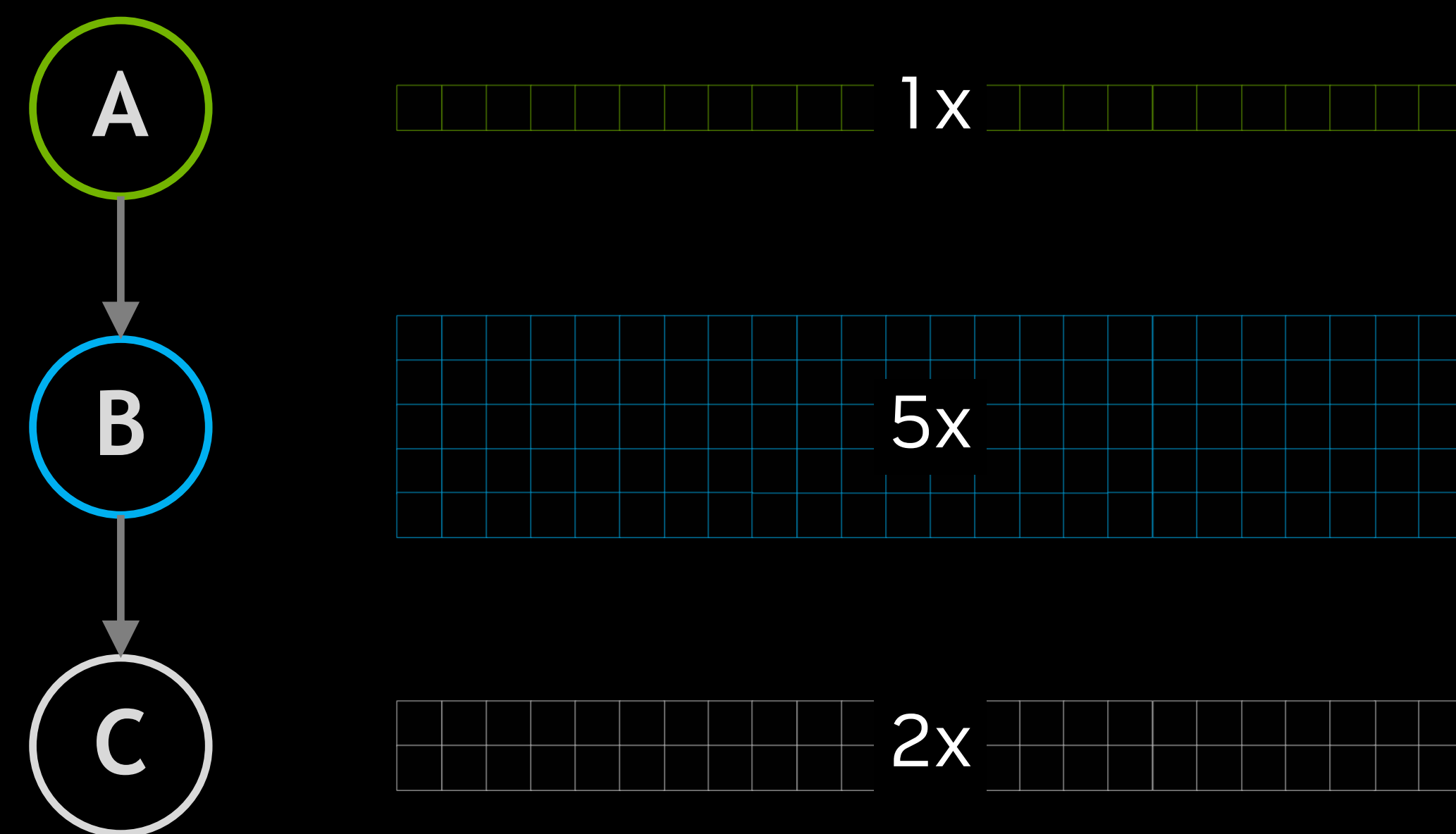
I Can Turn It Into A CUDA Graph



The problem is that to use these techniques I need lots of **independent** work in the first place

But What If I **DON'T** Conveniently Have Independent Work To Do?

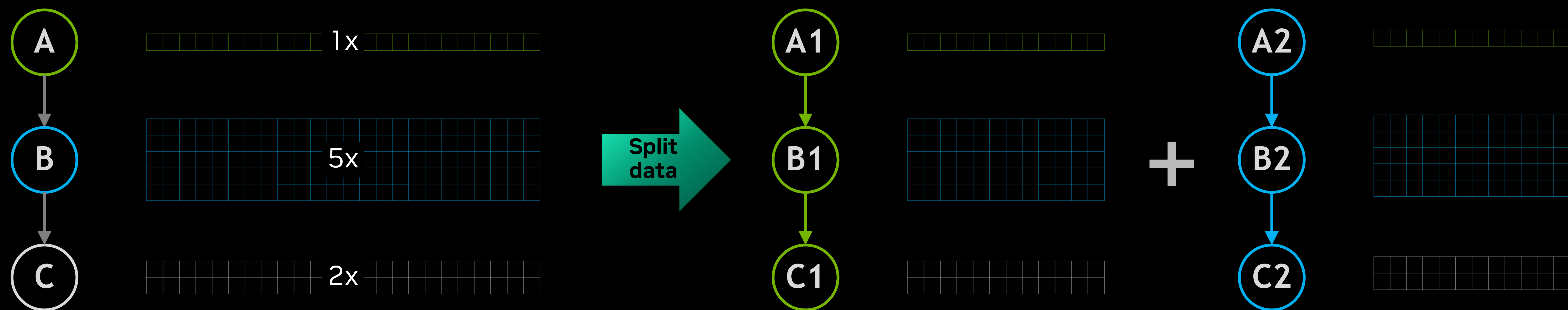
But What If I **DON'T** Conveniently Have Independent Work To Do?



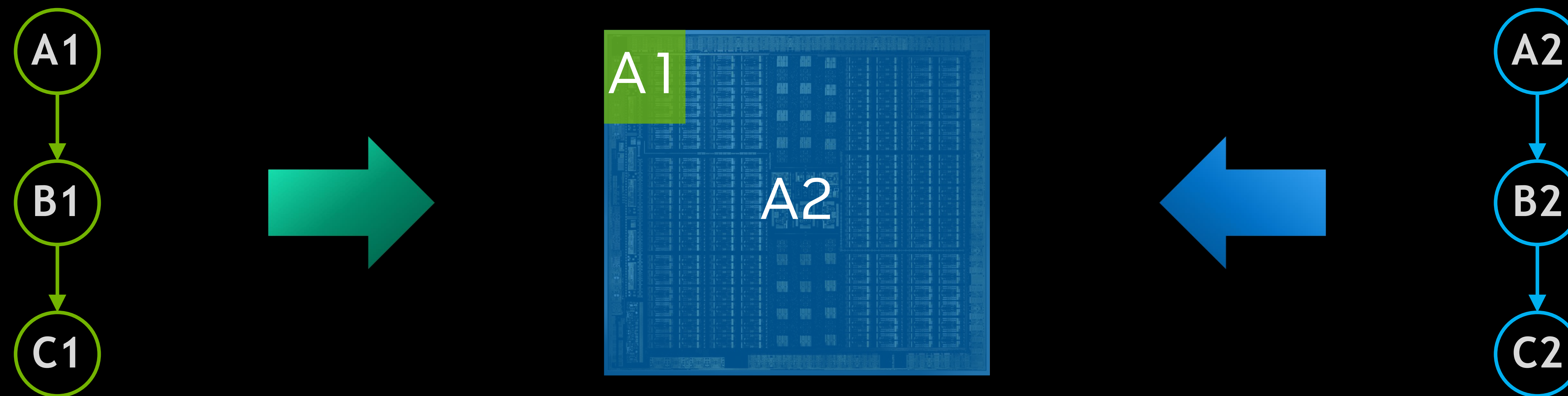
How do I create task parallelism when there's only one task?

The Obvious Approach: Split The Data In Two

Also known as “Pipeline Parallelism” – yet another type of parallelism which is really just task parallelism

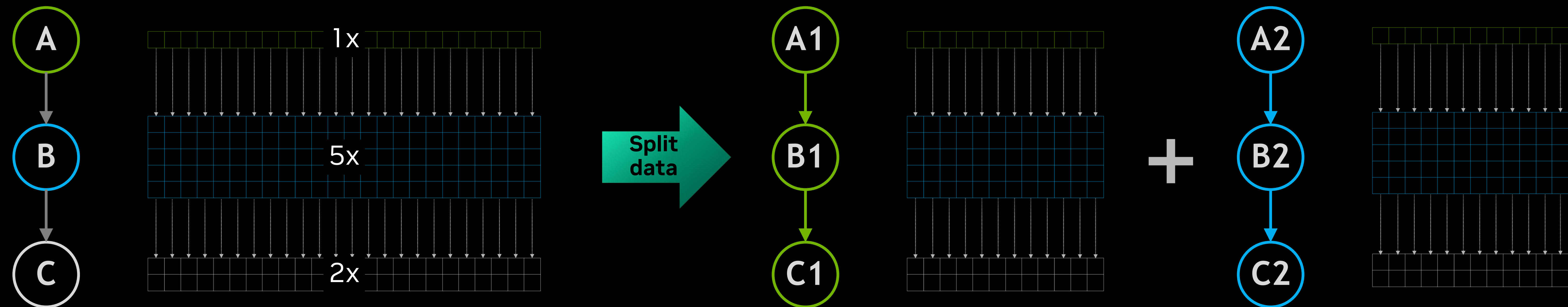


This Works Well If All Data In A1-B1-C1 Is Independent Of A2-B2-C2



Easy For Elementwise Programs

Programs never are entirely elementwise, but splitting the kernels which are will always win a little

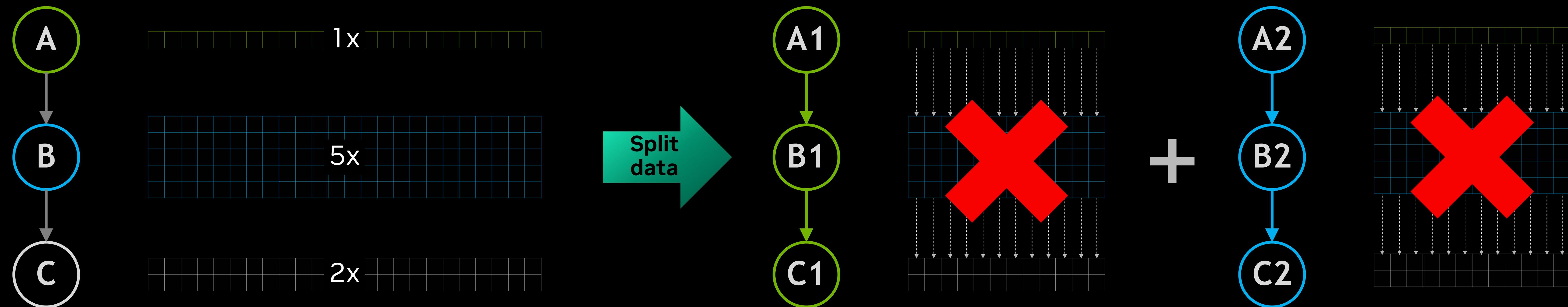


```
void daxpy(int n, double alpha, double *x, double *y)
{
    for( i = 0; i < n; i++ )
    {
        y[i] = alpha * x[i] + y[i];
    }
}
```

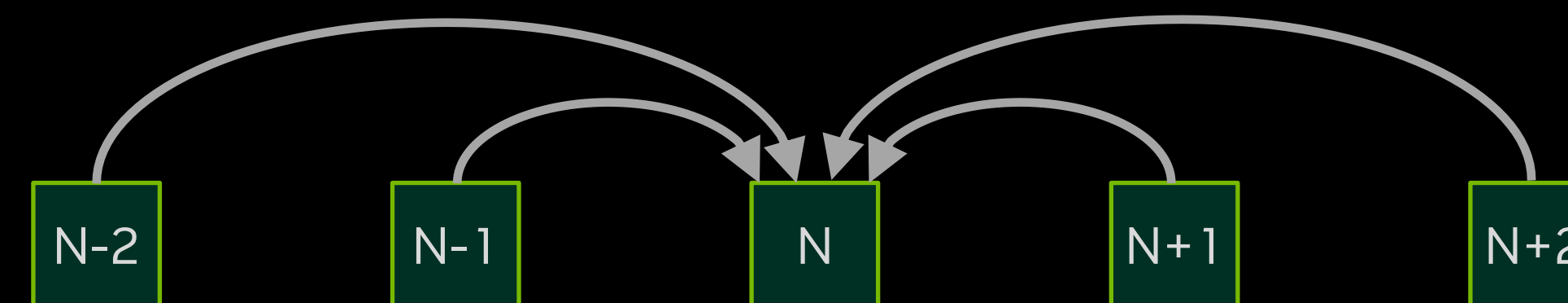
Elementwise “ax plus y” vector scale-and-addition

But The Real World Is Almost Never Elementwise

However, it is very often localised, like a stencil, instead of needing random access to all data

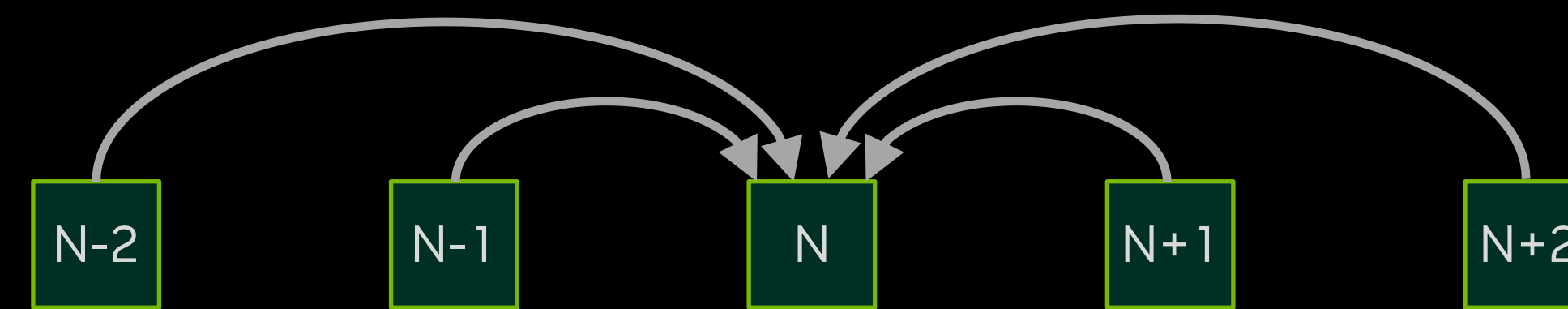


Simple split obviously does not work for stencils

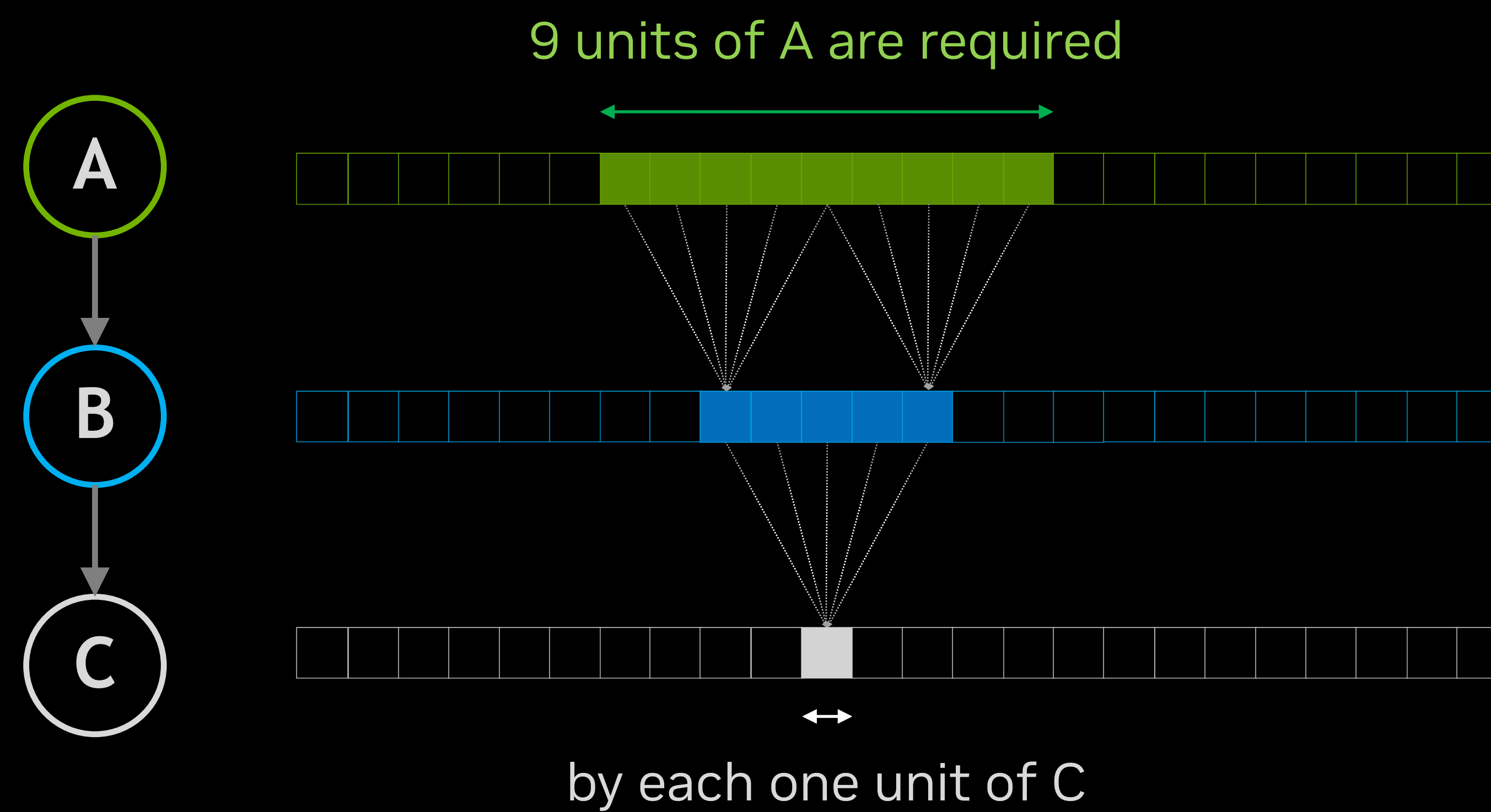


Example: 1D Stencil, radix-5

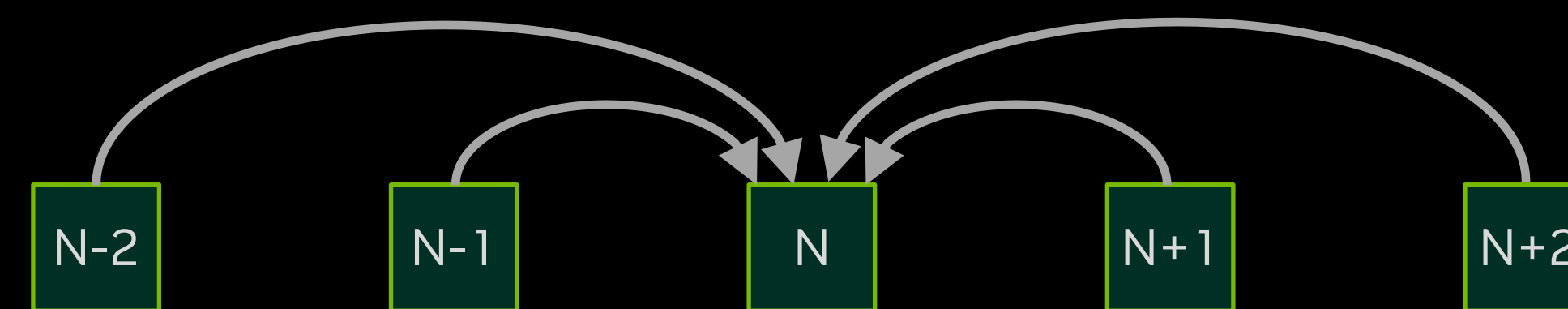
The Problem Is Obvious: Widening Dependency Trees



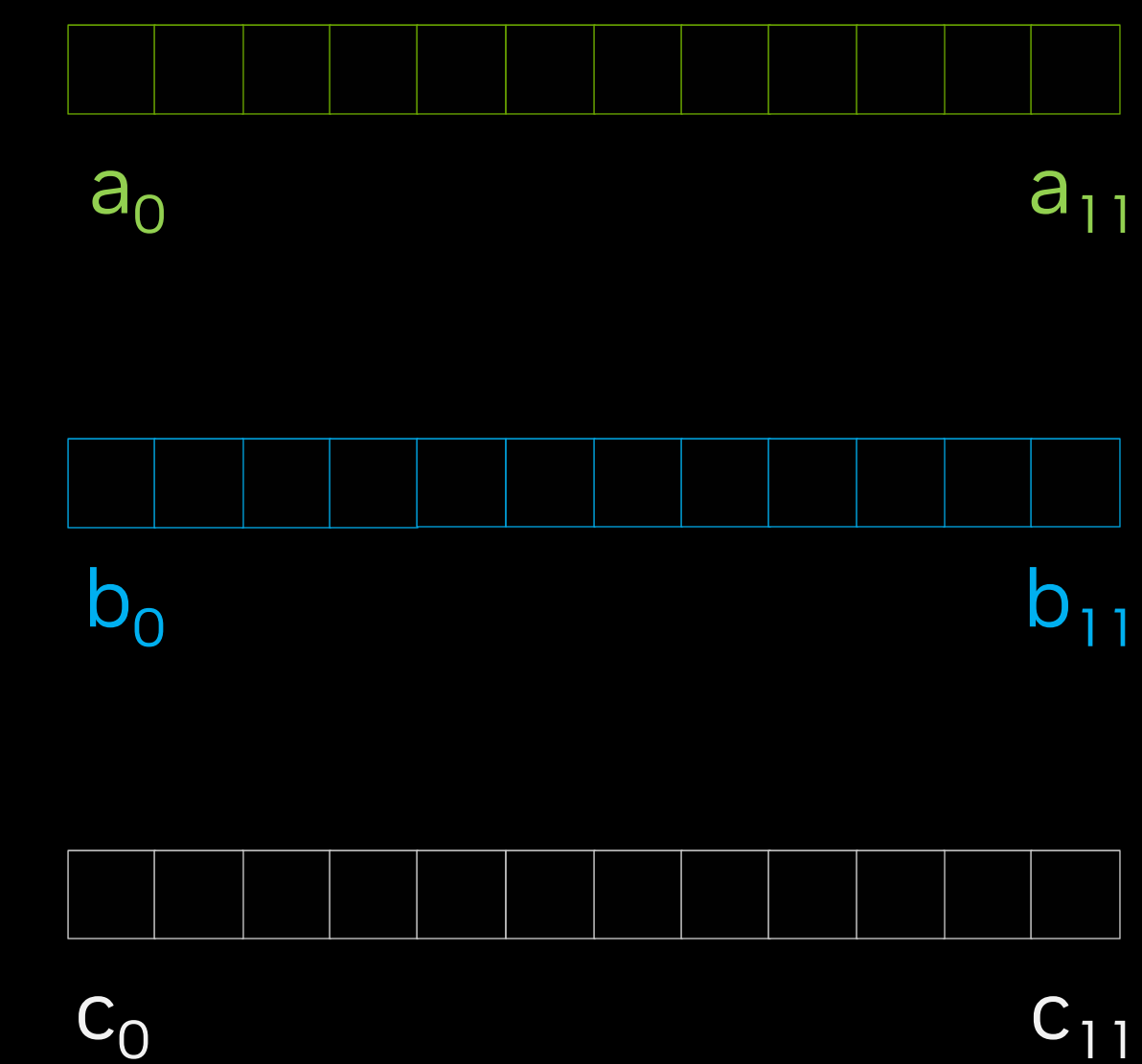
Example: 1D Stencil, radix-5



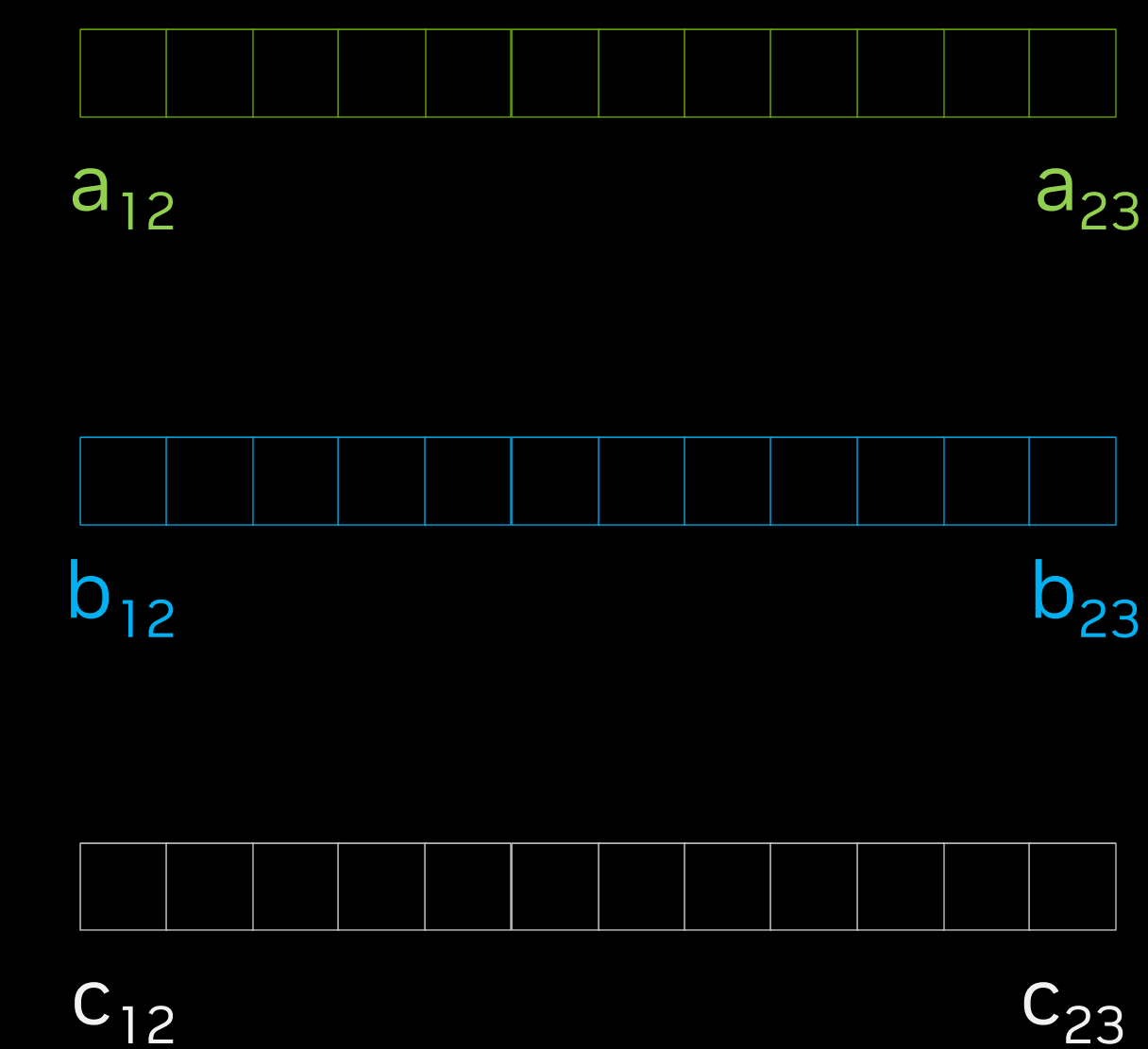
When Splitting Data Into Two



Example: 1D Stencil, radix-5



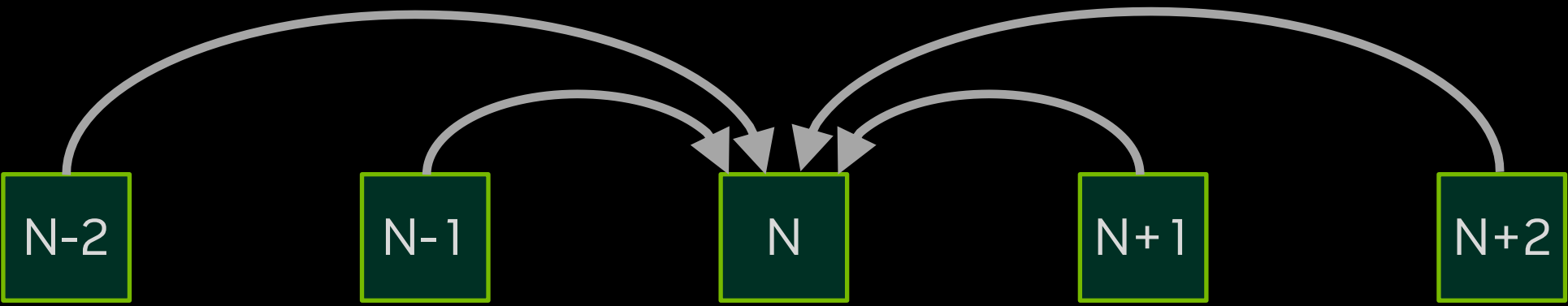
Task 1 gets elements 0-11



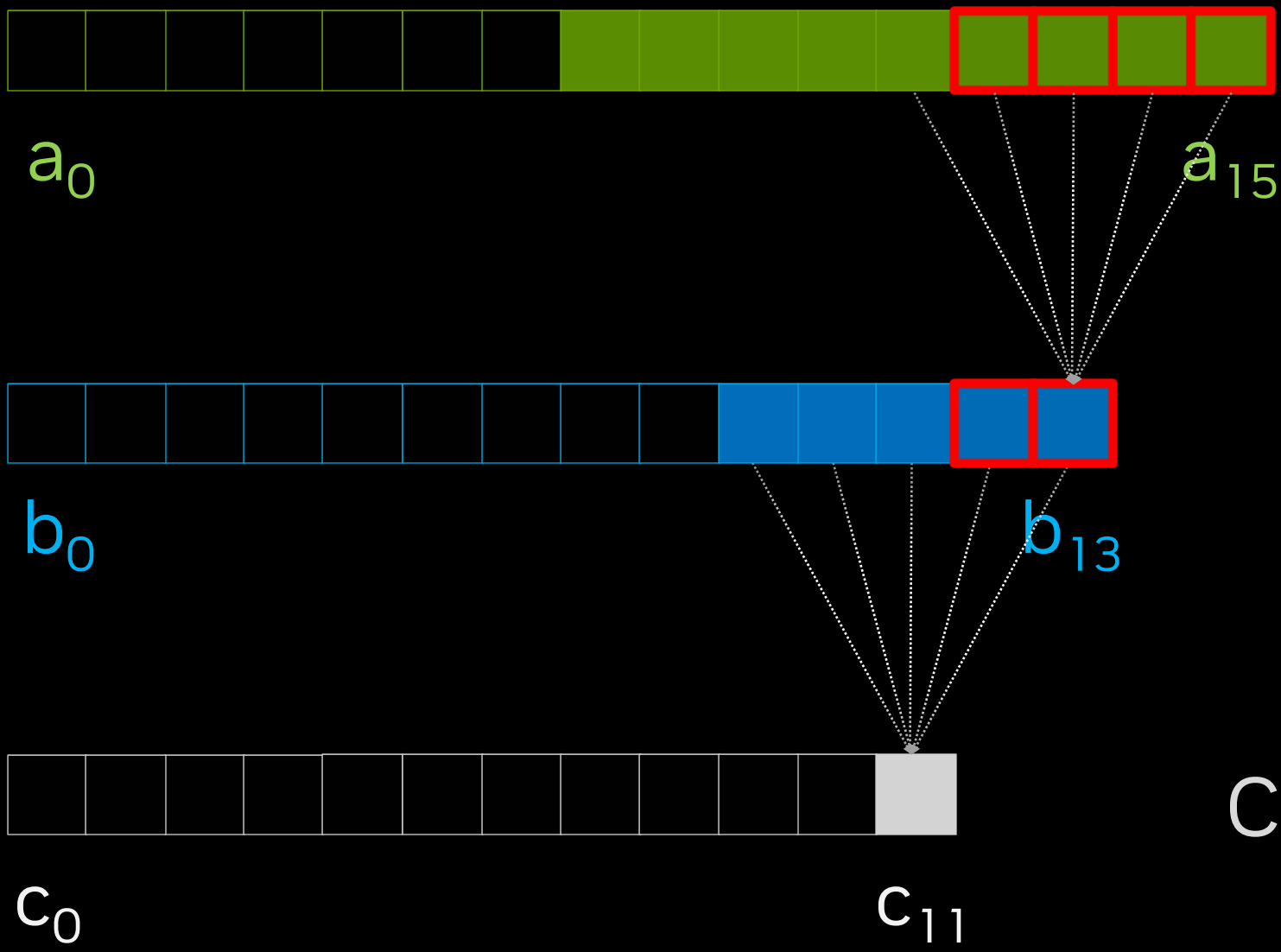
Task 2 gets elements 12-23



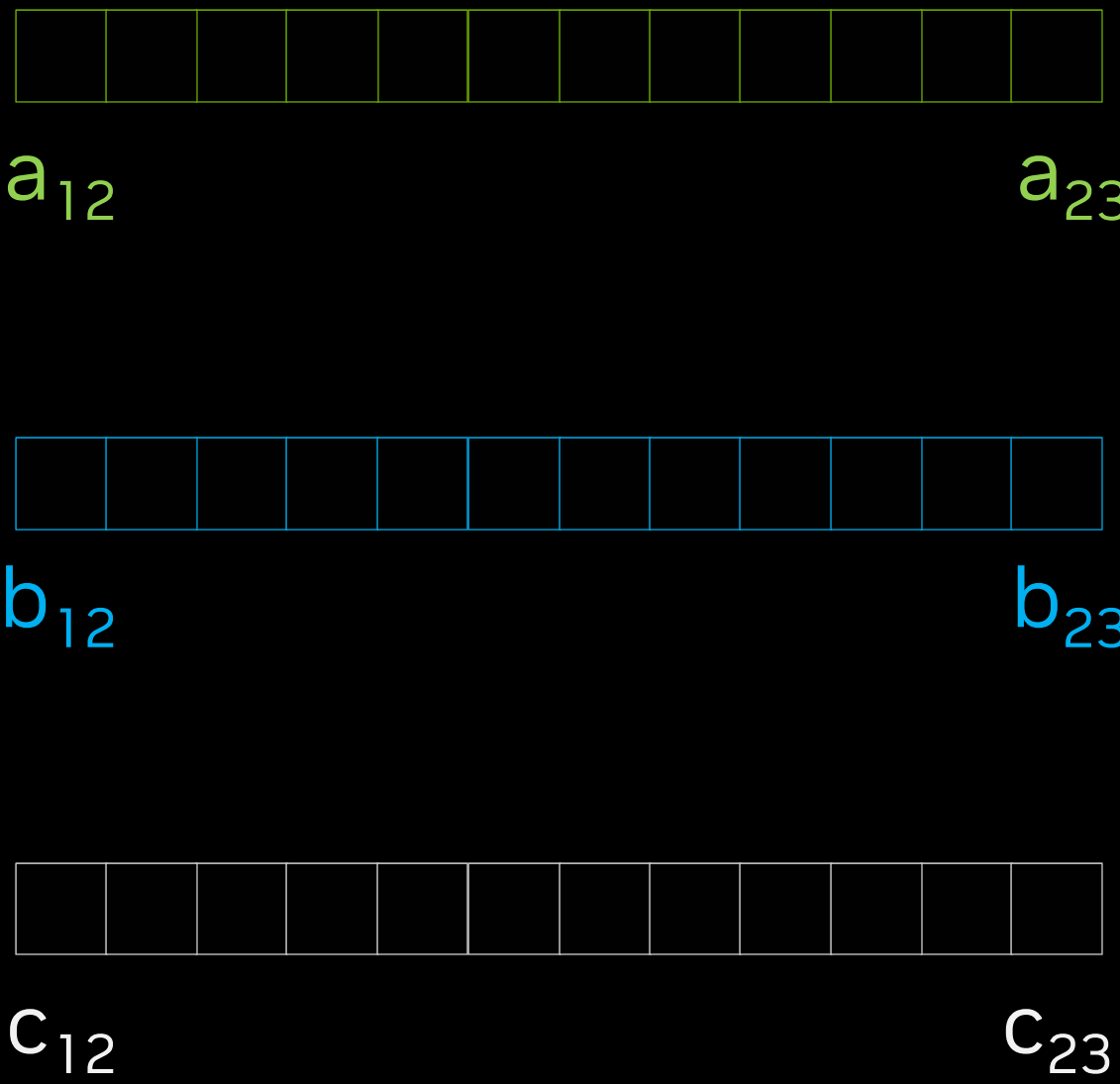
Create Dependencies Between Tasks 1 & 2



Example: 1D Stencil, radix-5

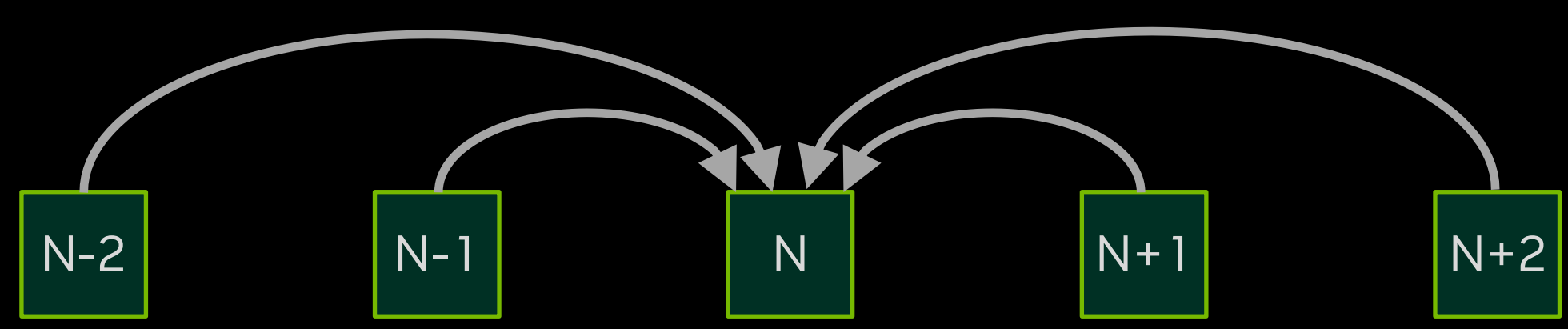


$C1$ needs access to b_{13} and a_{15}

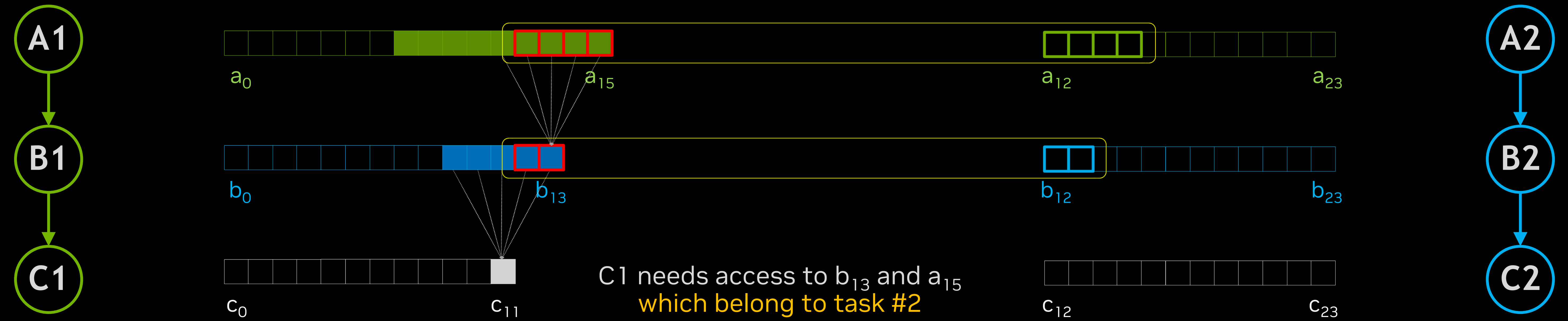


Create Dependencies Between Tasks 1 & 2

This makes it very hard to co-schedule sub-portions of A, B and C



Example: 1D Stencil, radix-5

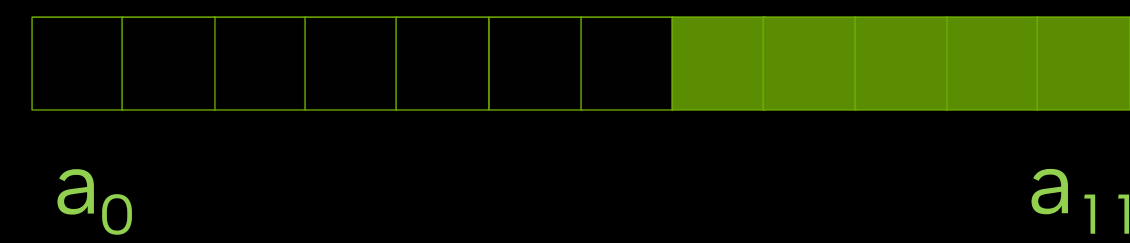


B1 needs to wait until A2 has done its work
C1 has to wait until B2 has done its work
But B2 has to wait until A1 has done its work
... and this is why pipeline parallelism is hard

So Instead, Reduce the Sizes of B & C

Keep all data completely contained within a given task by allowing for data fan-out

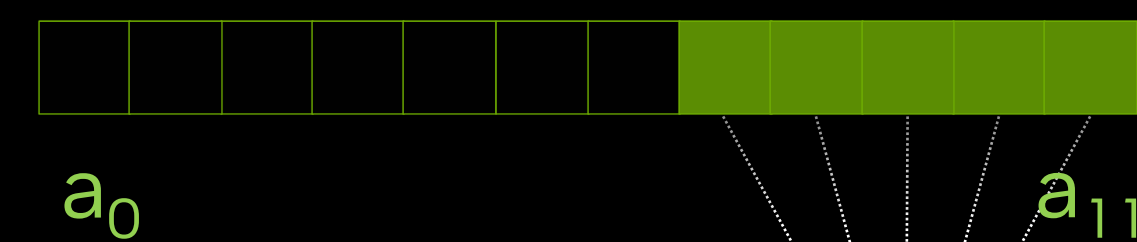
Task A1 retains the original
12-wide task split



So Instead, Reduce the Sizes of B & C

Keep all data completely contained within a given task by allowing for data fan-out

Task A1 retains the original
12-wide task split

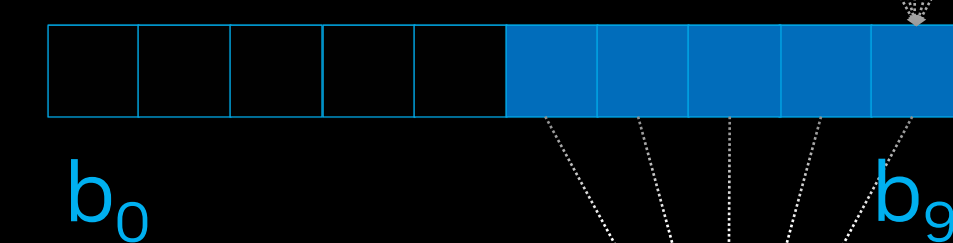
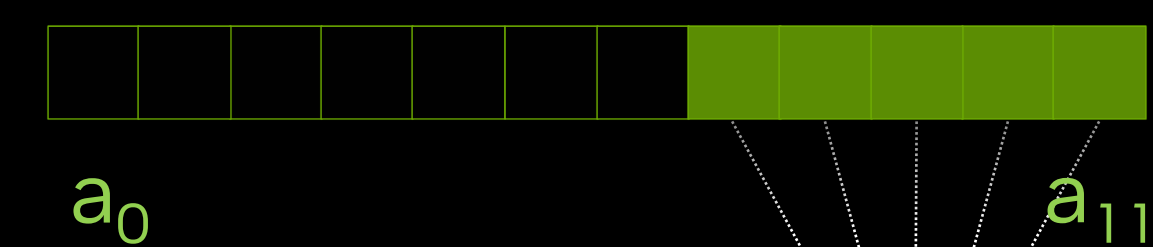


B can only compute its first 10 elements

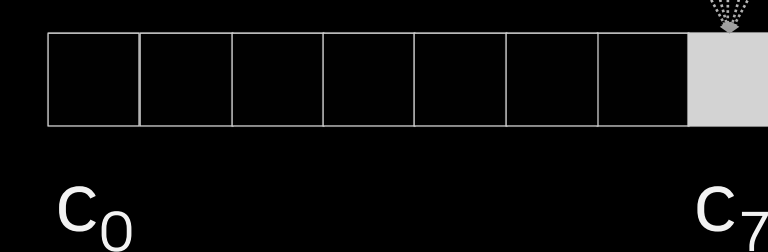
So Instead, Reduce the Sizes of B & C

Keep all data completely contained within a given task by allowing for data fan-out

Task A1 retains the original
12-wide task split



B can only compute its first 10 elements

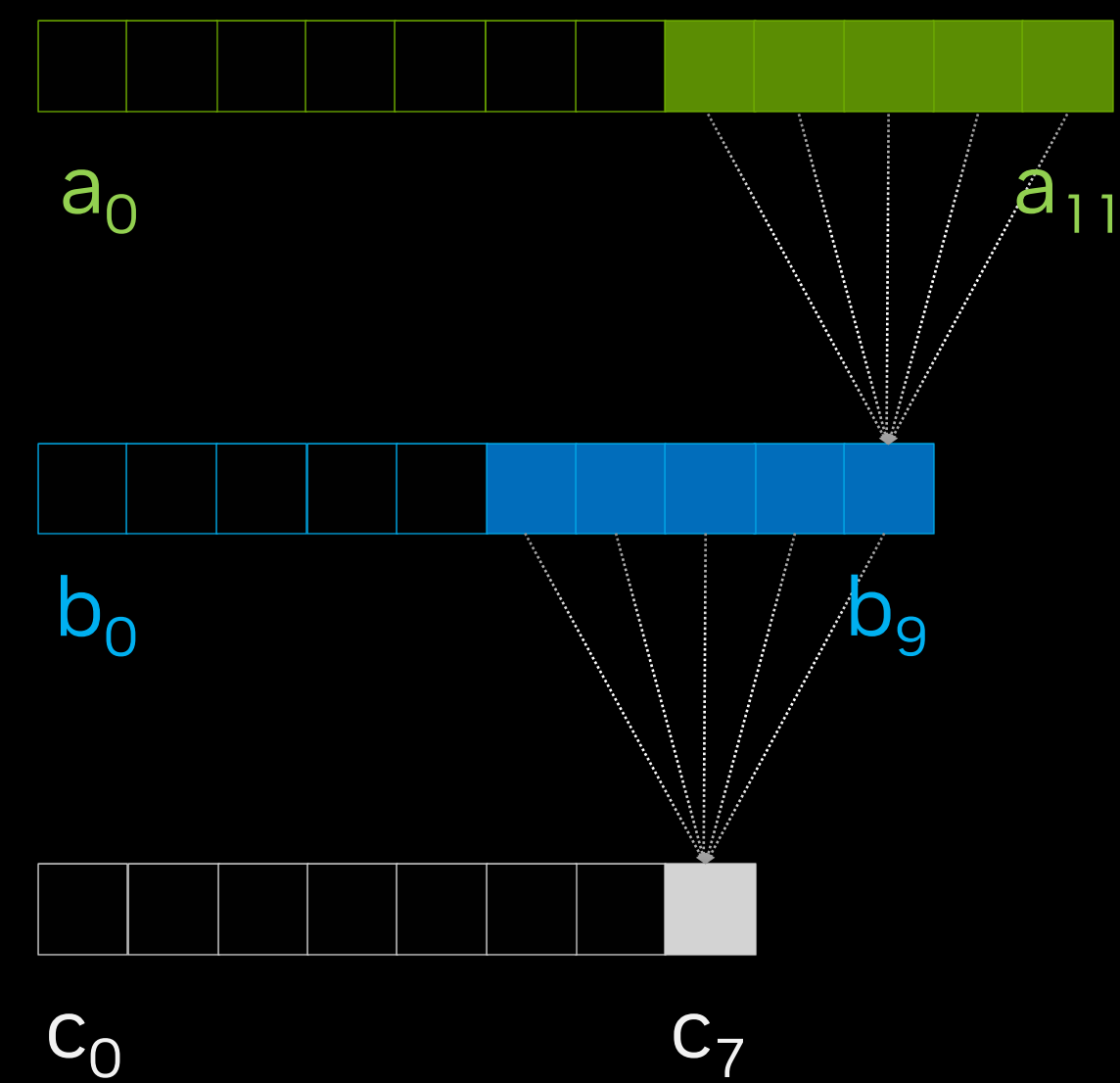


←→
C is now only 8-wide

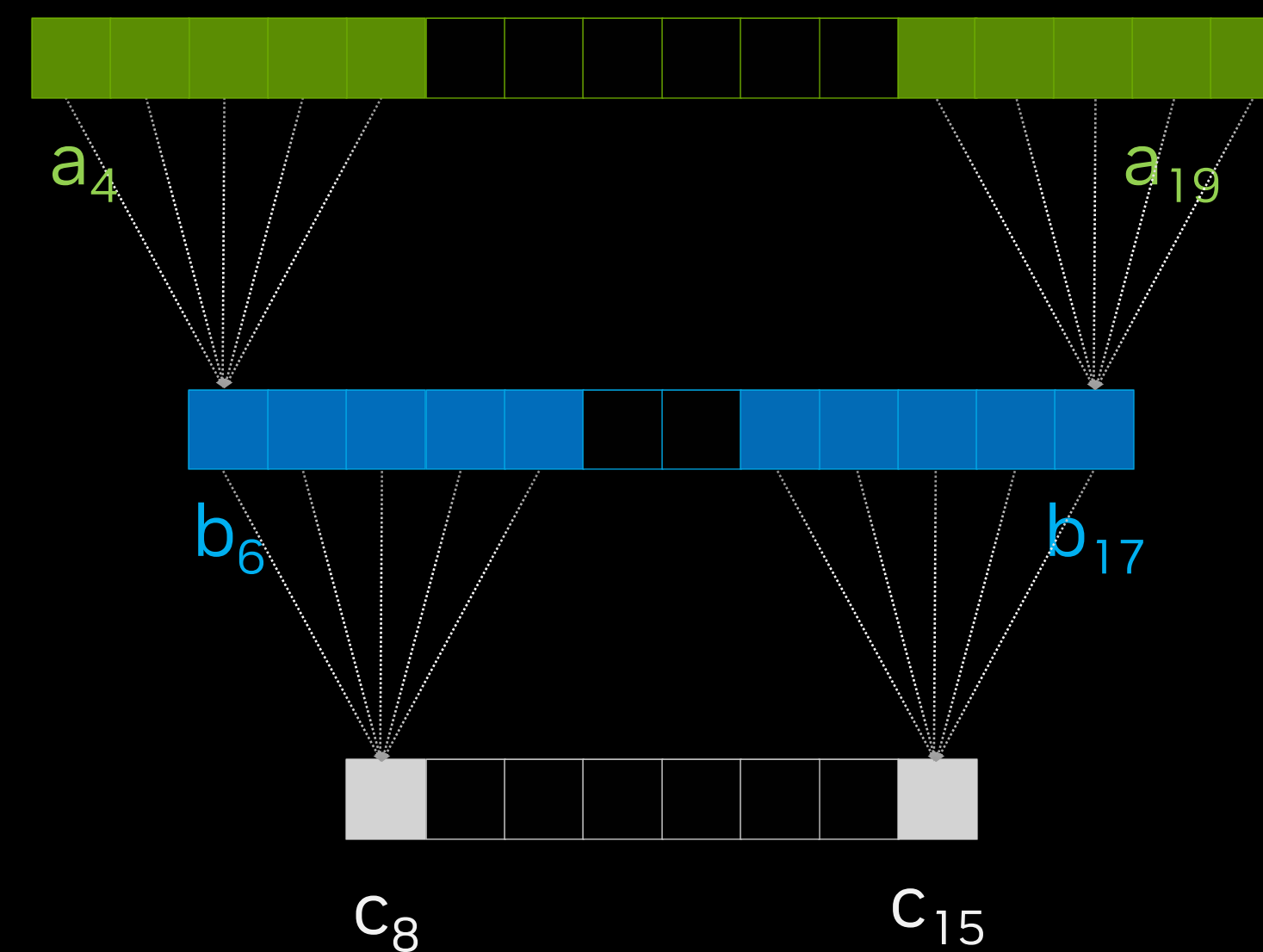
So Instead, Reduce the Sizes of B & C

Of course, now you need more than two tasks

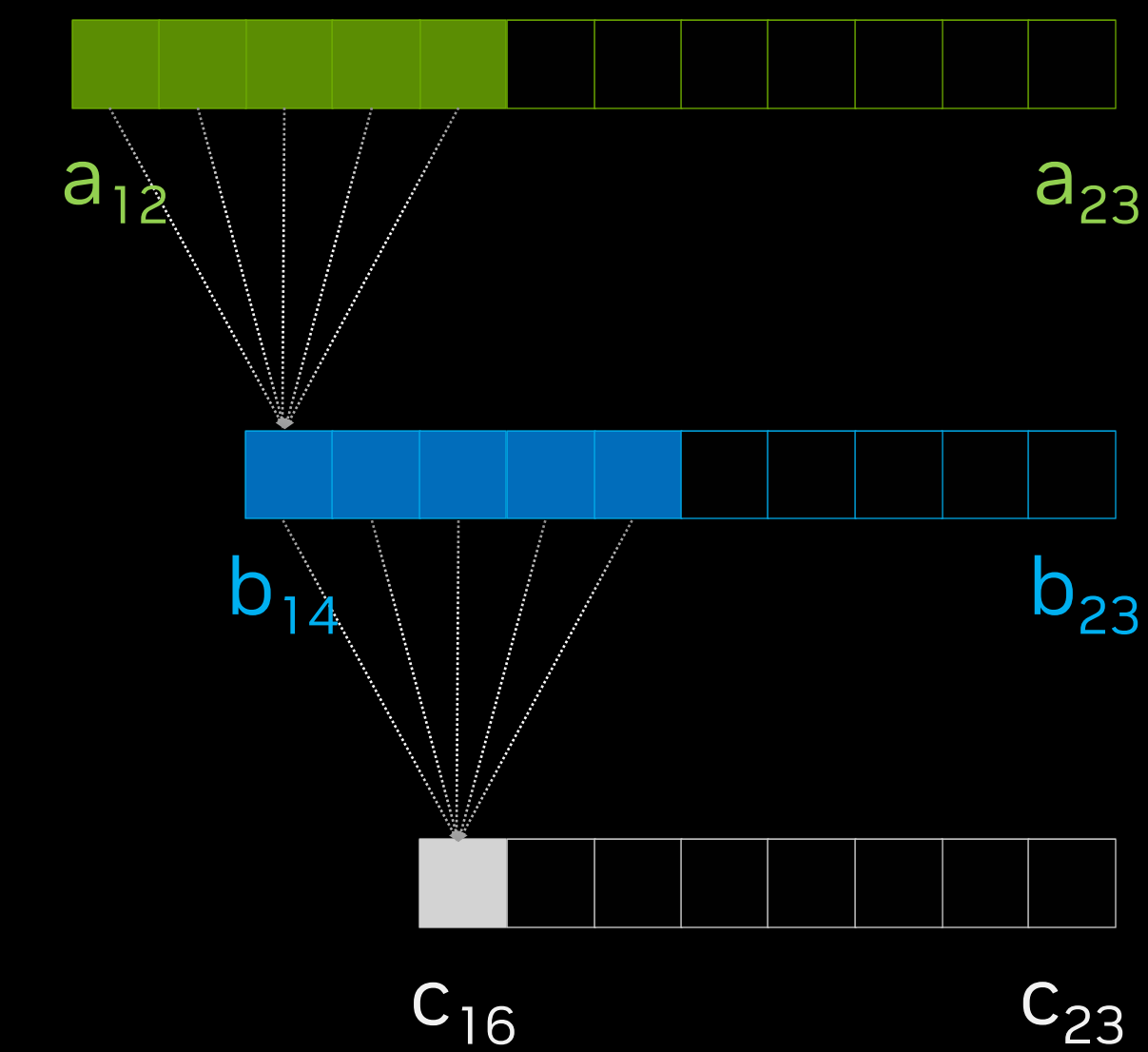
Task A1 retains the original
12-wide task split



Task A2 is now wider because
it extends on both sides



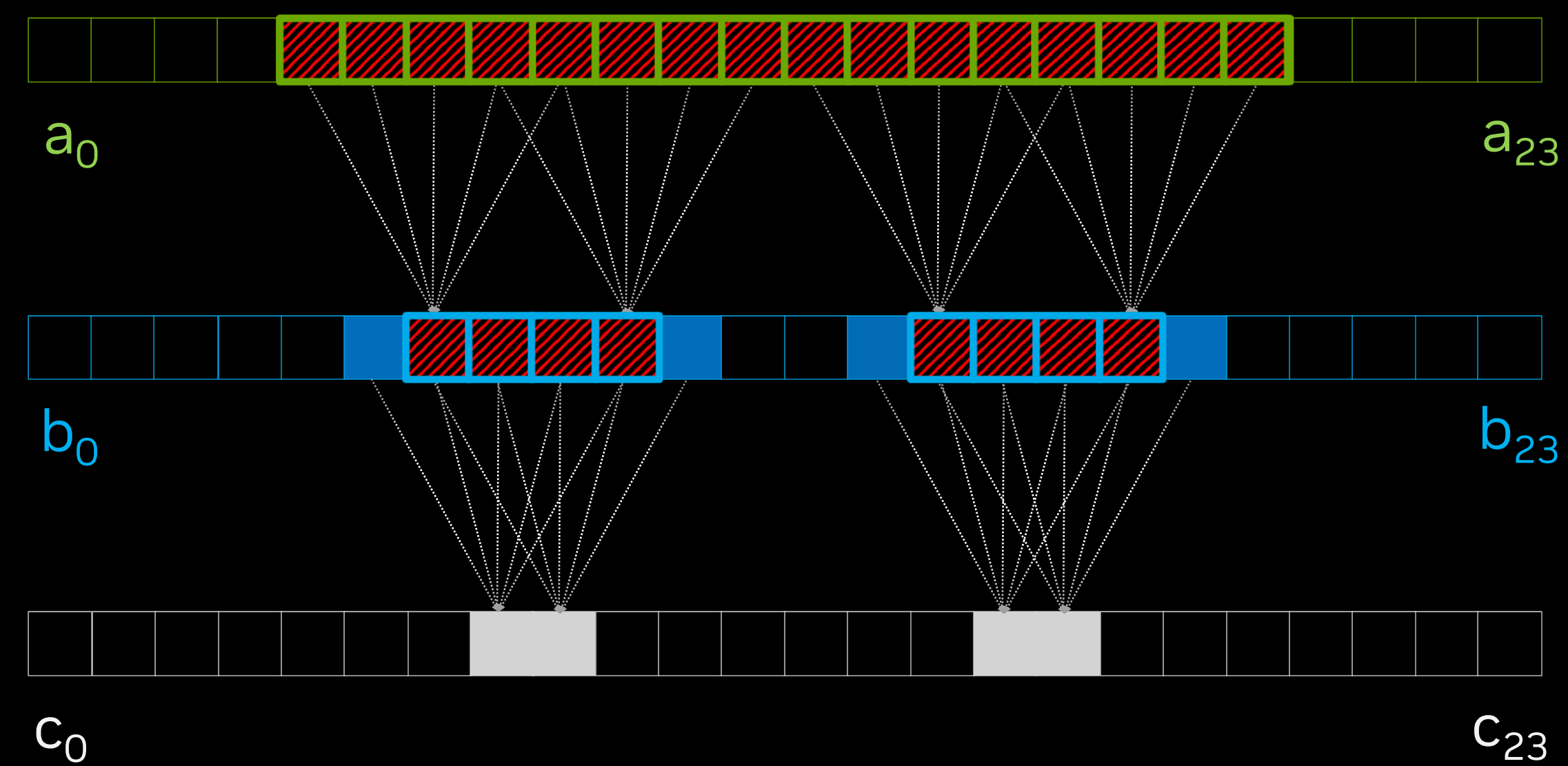
And now we have a Task A3



← We do this enough times that every value of **C** has been computed exactly once →

The Result Is Self-Contained Tasks But With **Redundant** Computation

This isn't as bad as it sounds...

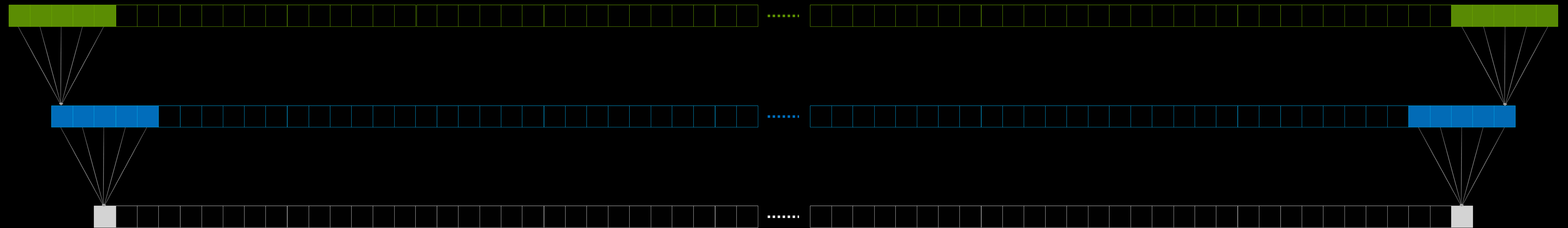


Cost Of Redundant Computation Depends On Data Size

because it's common that each sub-task A1, A2, A3 operates over many data elements

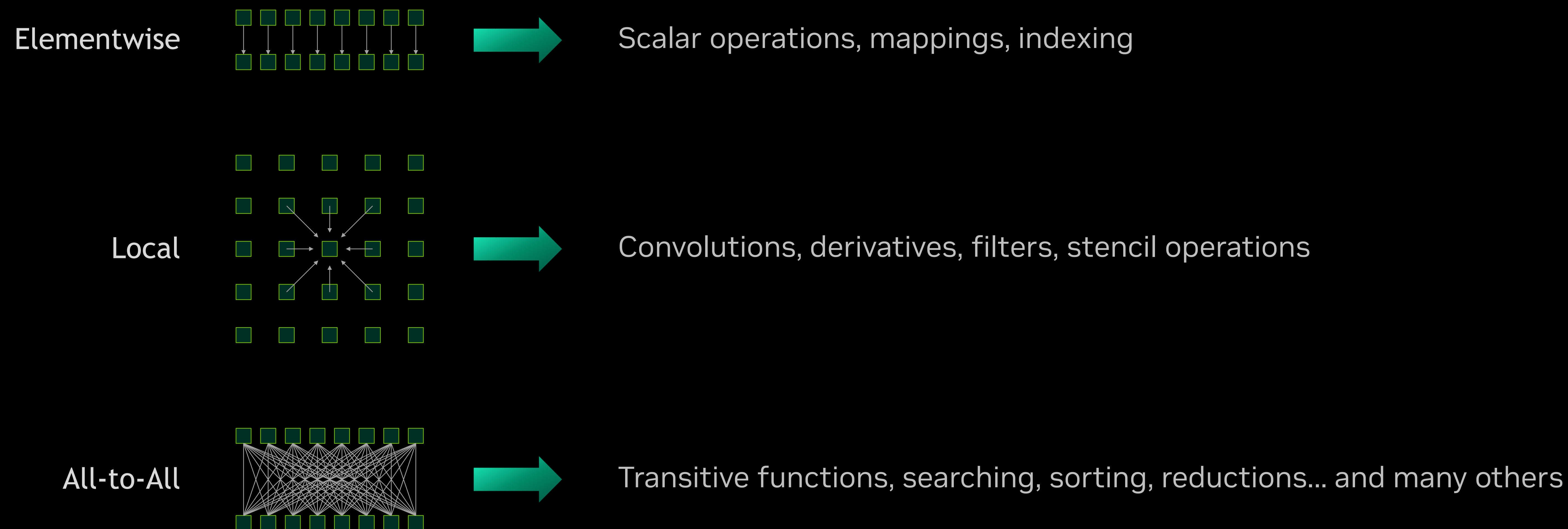
Often, recomputing something is faster than writing it out to memory and then reading it back in

Typically arrays are large so overlap is a small fraction,
but redundancy grows with each dependent kernel



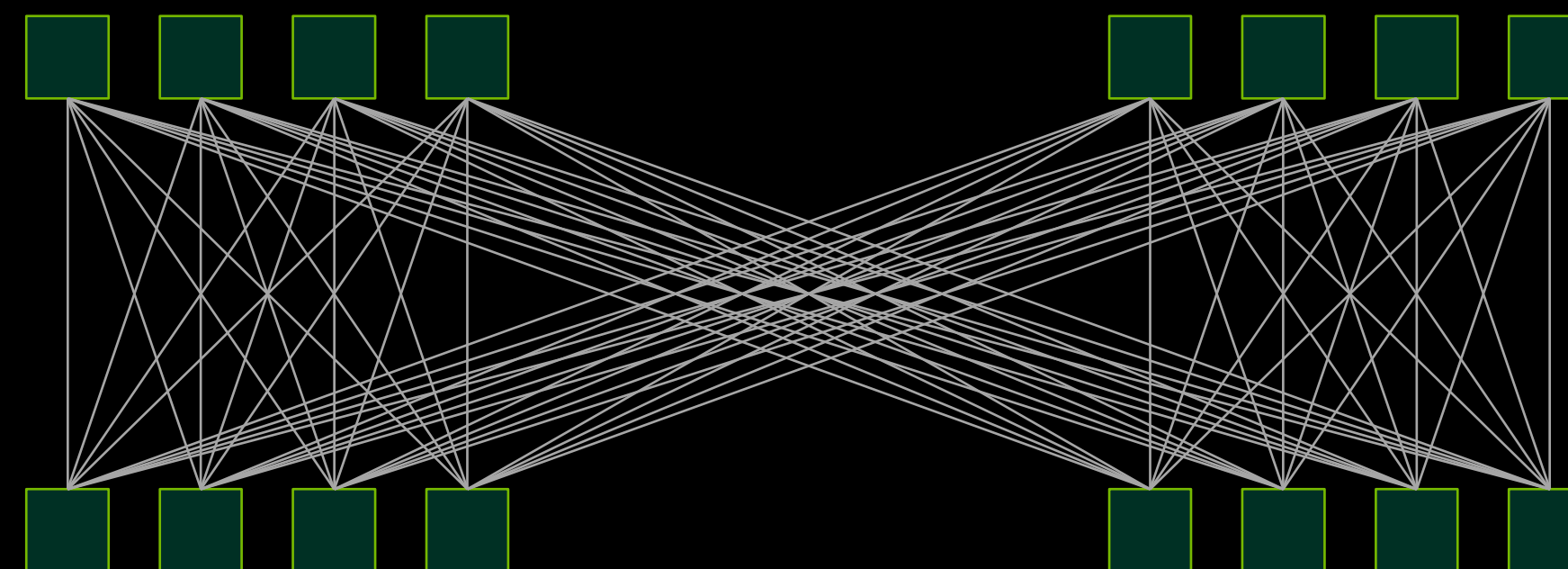
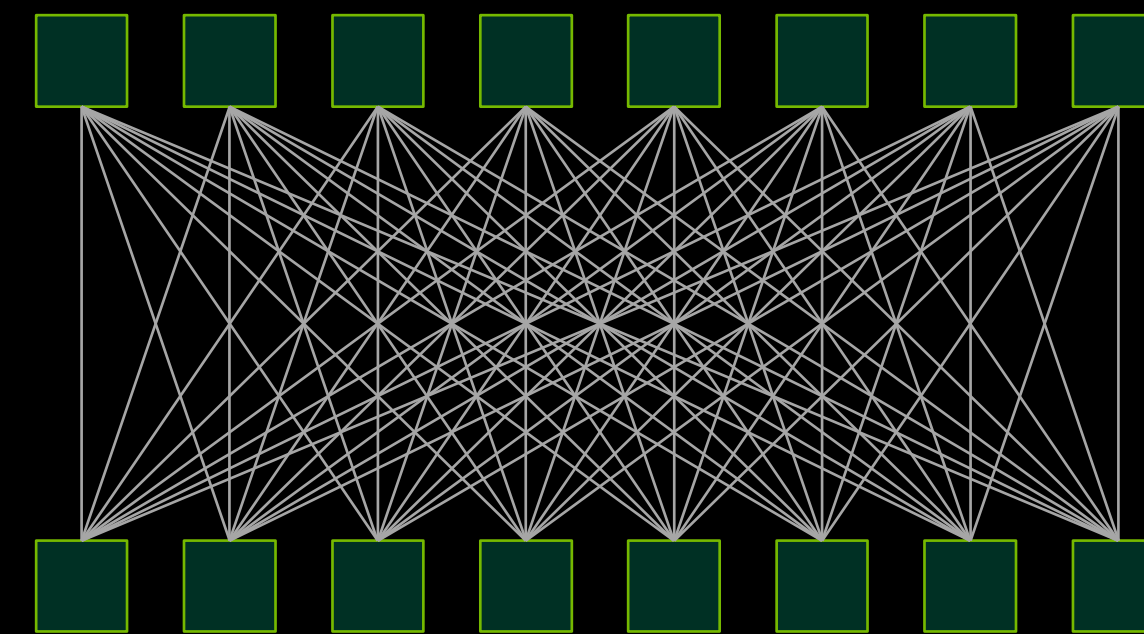
The Real Problem: All-to-All Algorithms

For example: sorting, fourier transforms, and unfortunately many other useful things



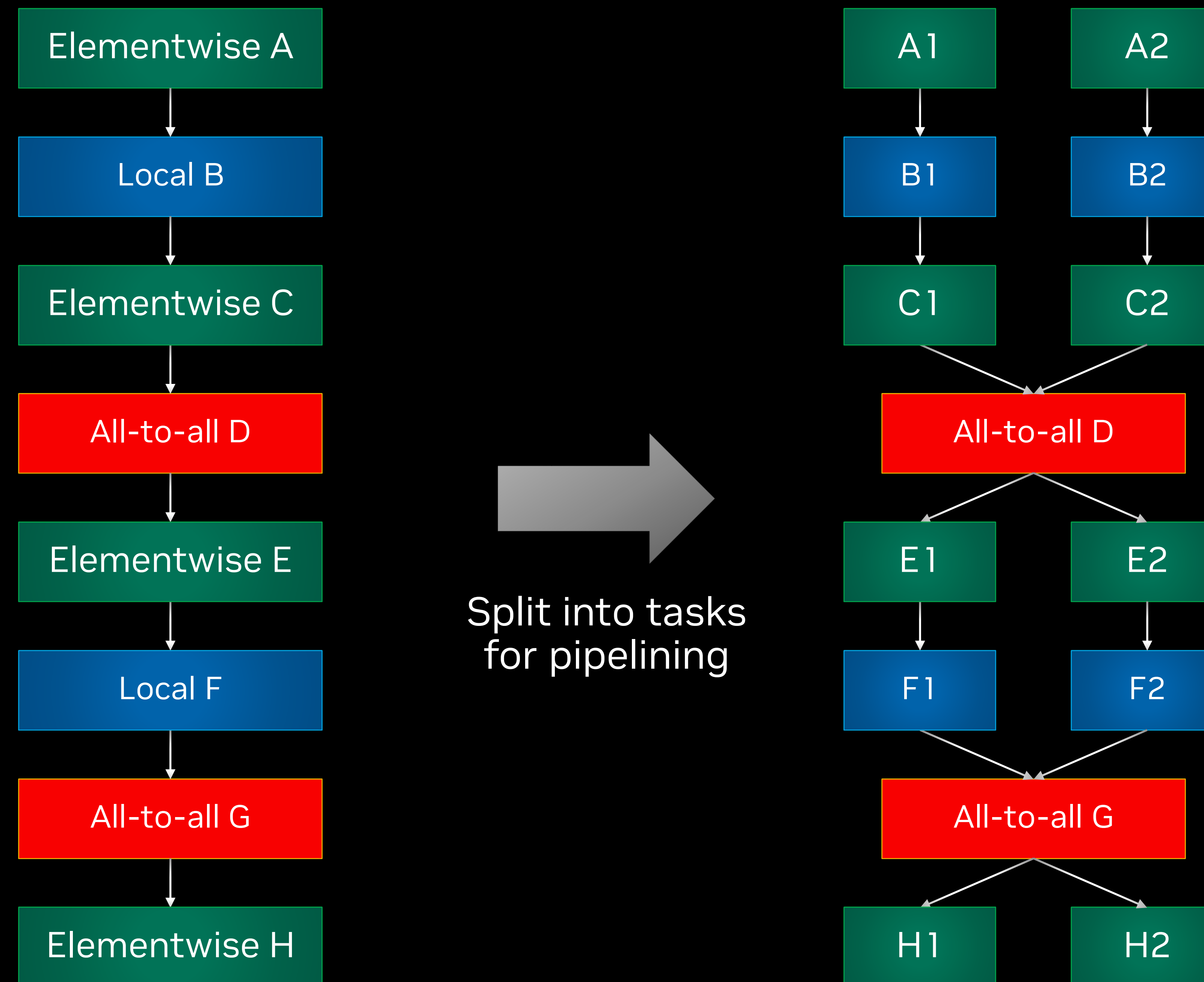
All-to-All Algorithms Break Pipelining

I always end up with 100% redundant computation, so there's no point splitting the operation



All-to-All Algorithms Break Pipelining

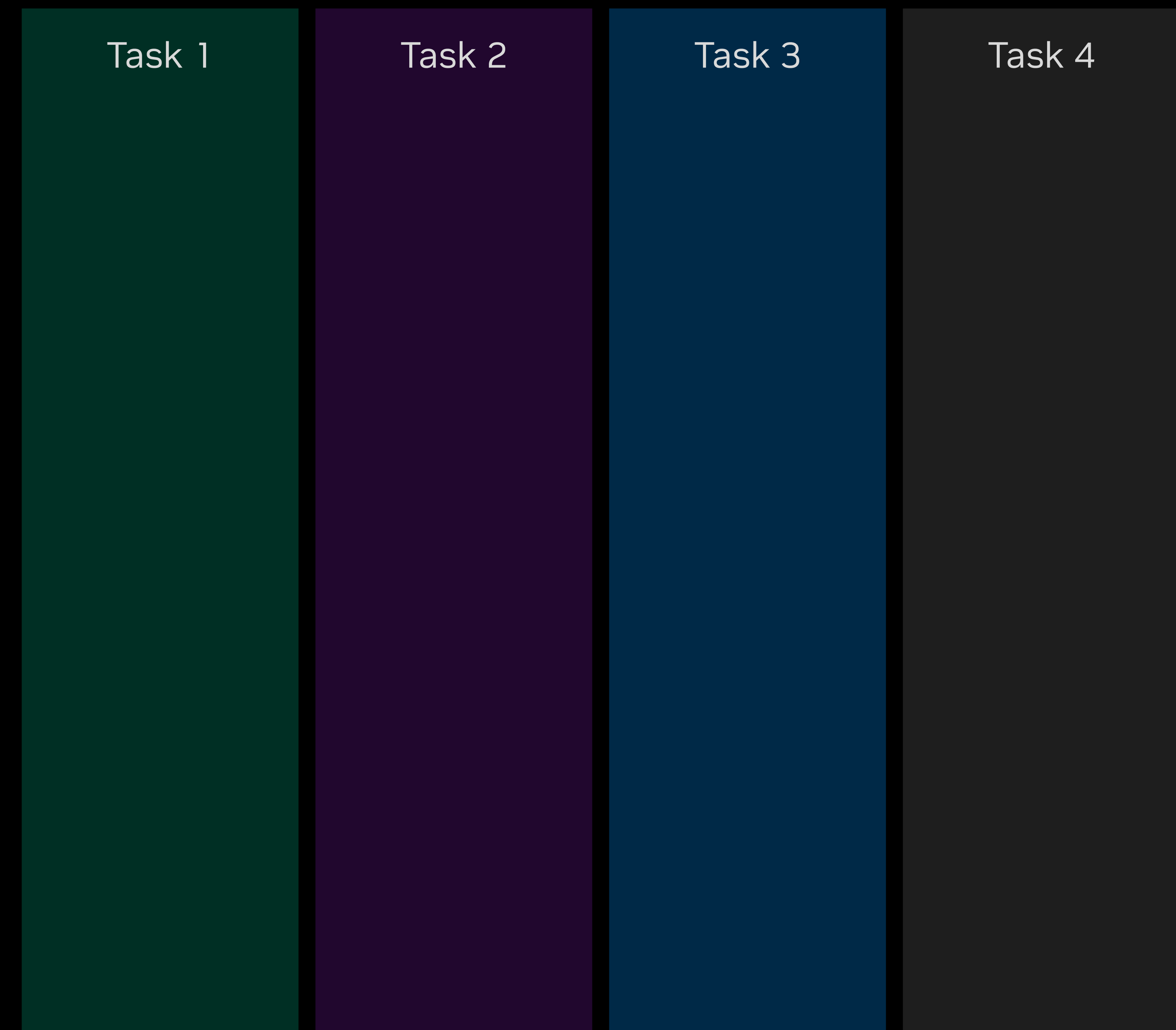
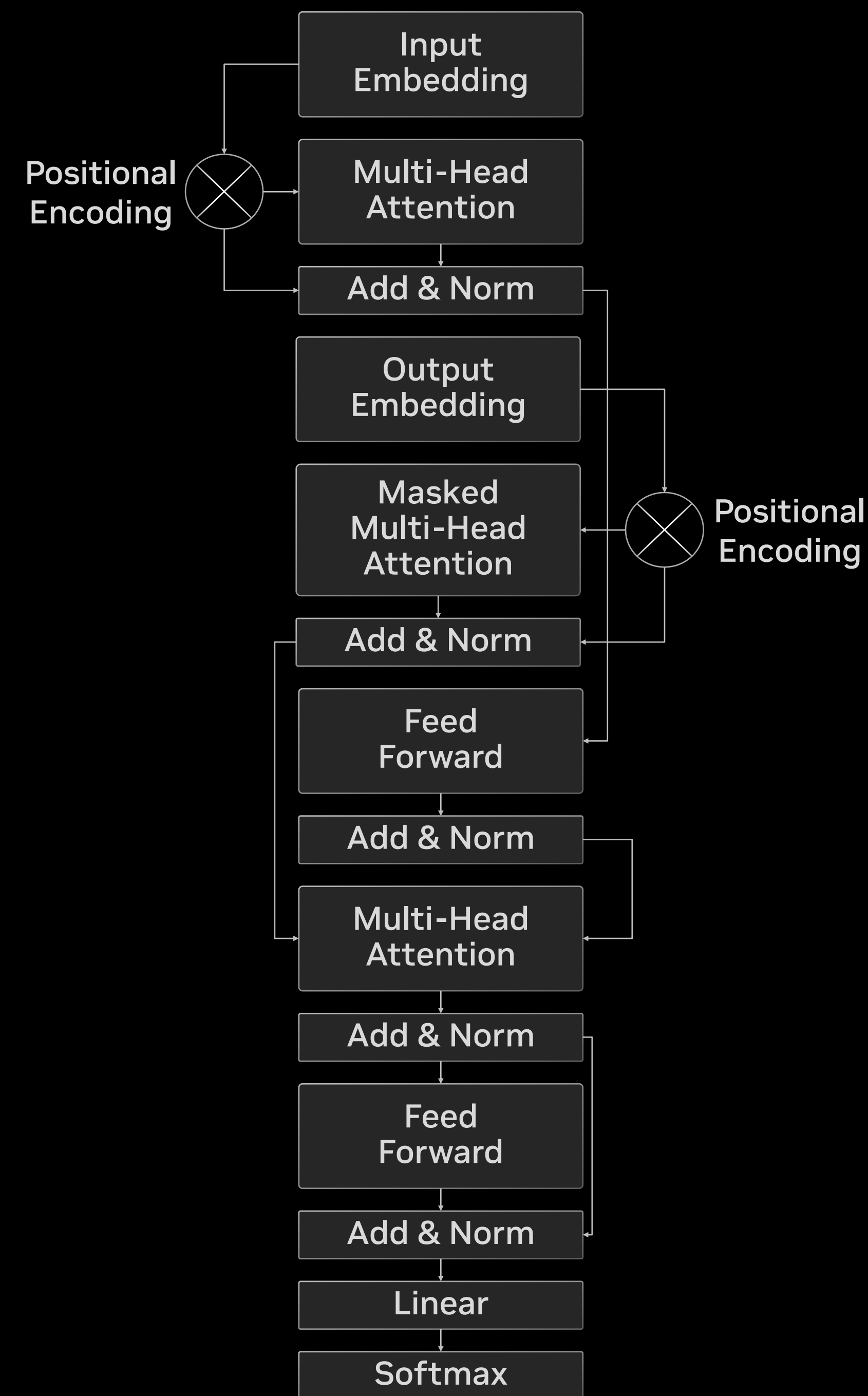
I can run until I hit an all-to-all operation, but then I need to sync across the whole workload



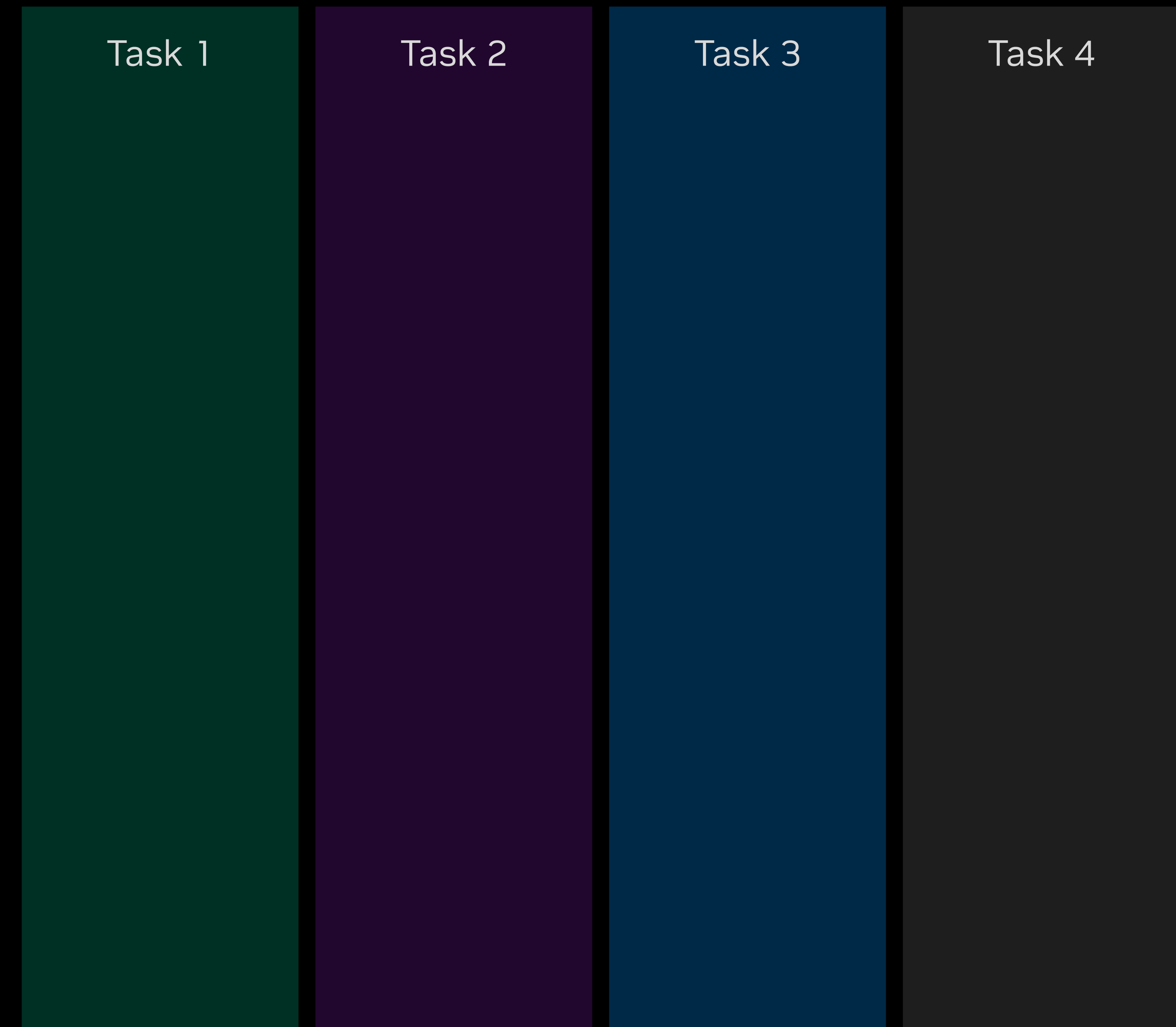
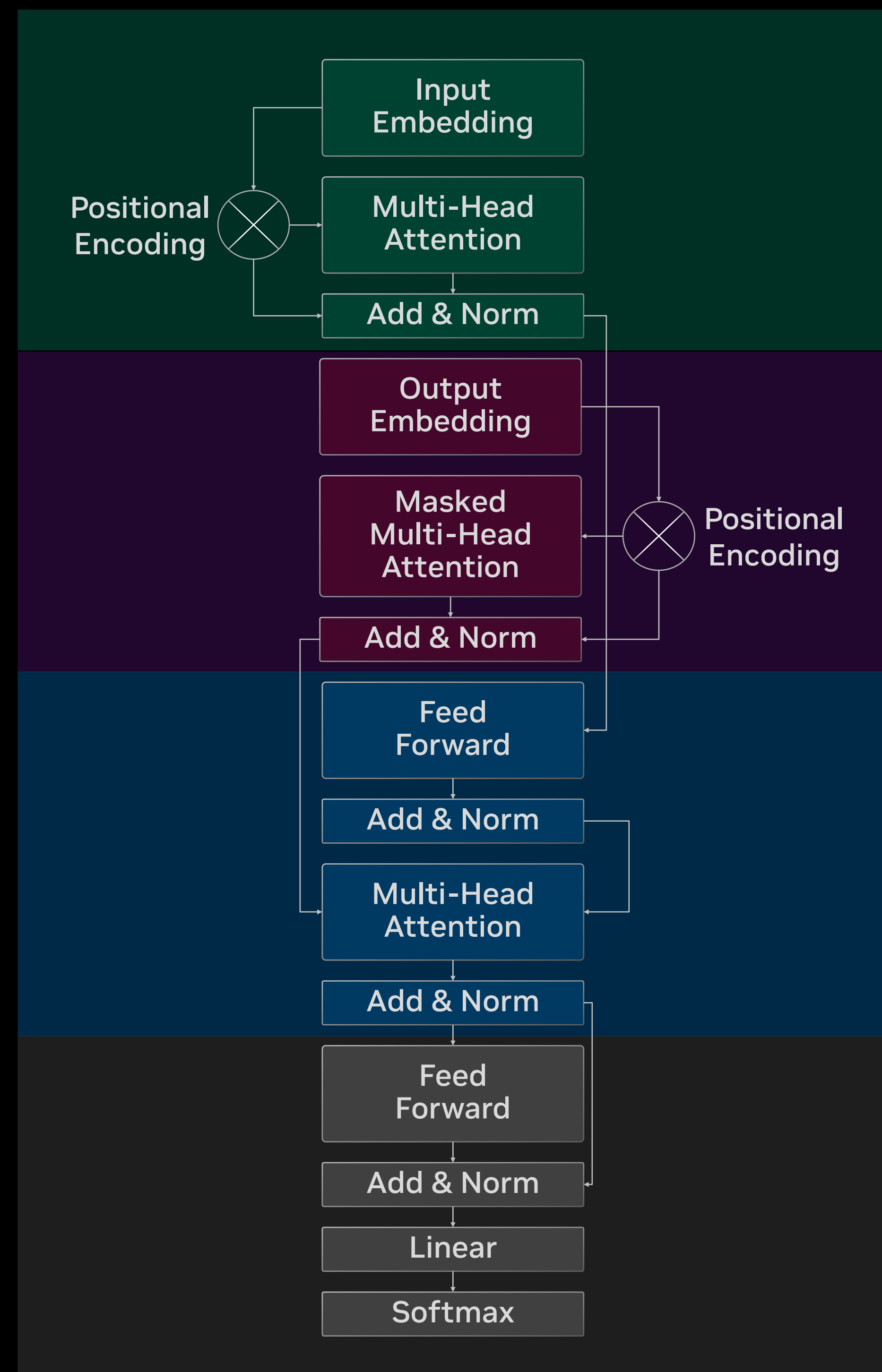
All-to-all operations **break** pipeline asynchrony

They are in effect a **synchronization point**

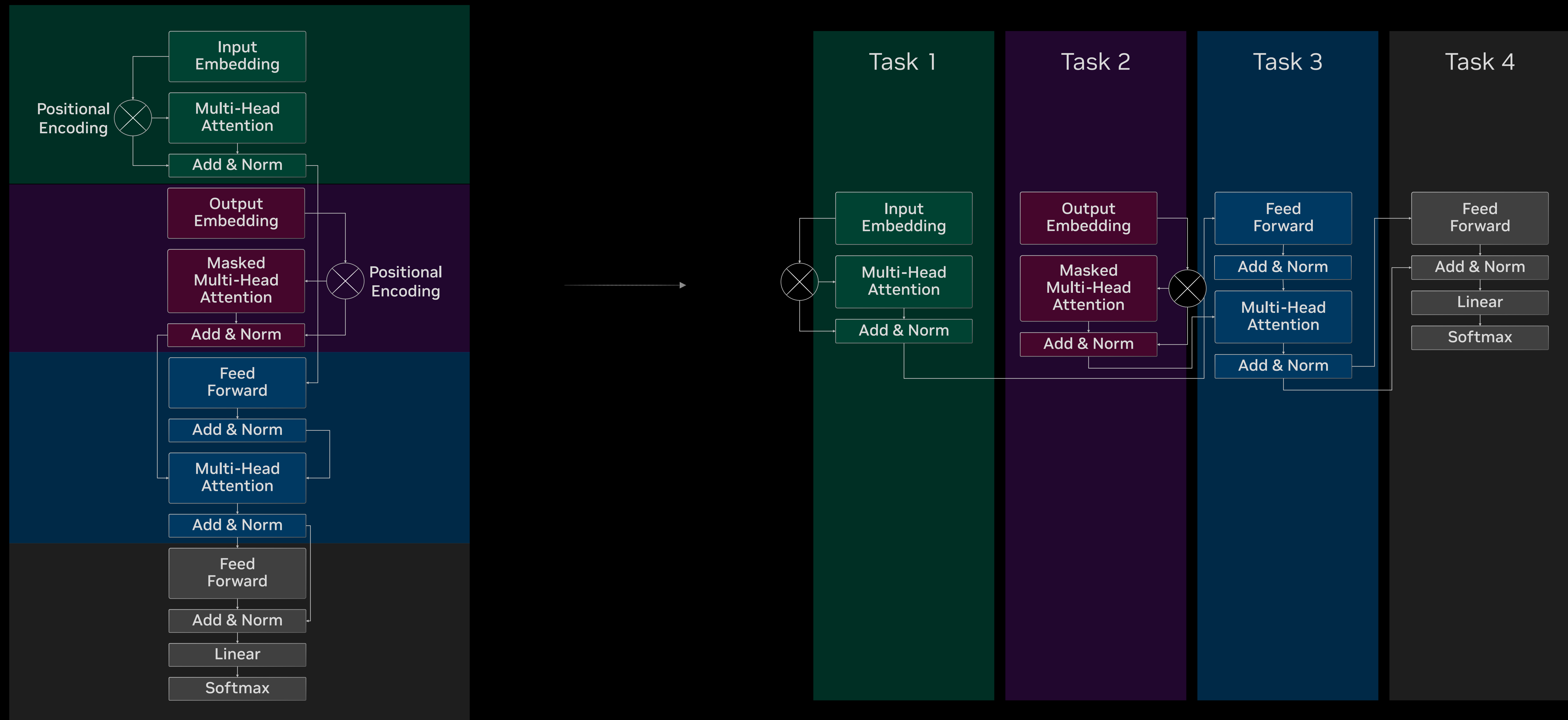
Model Parallelism: Task Parallelism For Complex Workflows



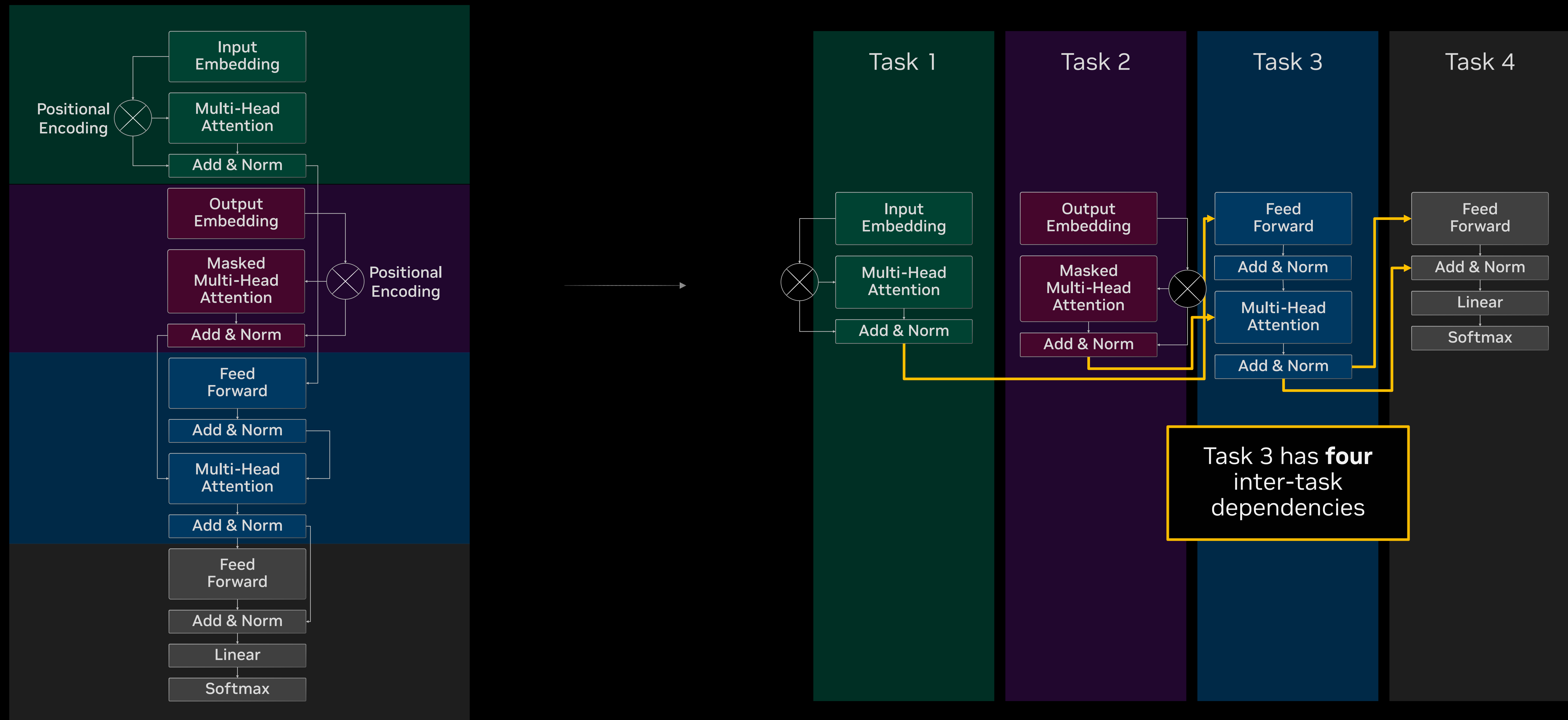
Naïve split of tasks to span GPUs



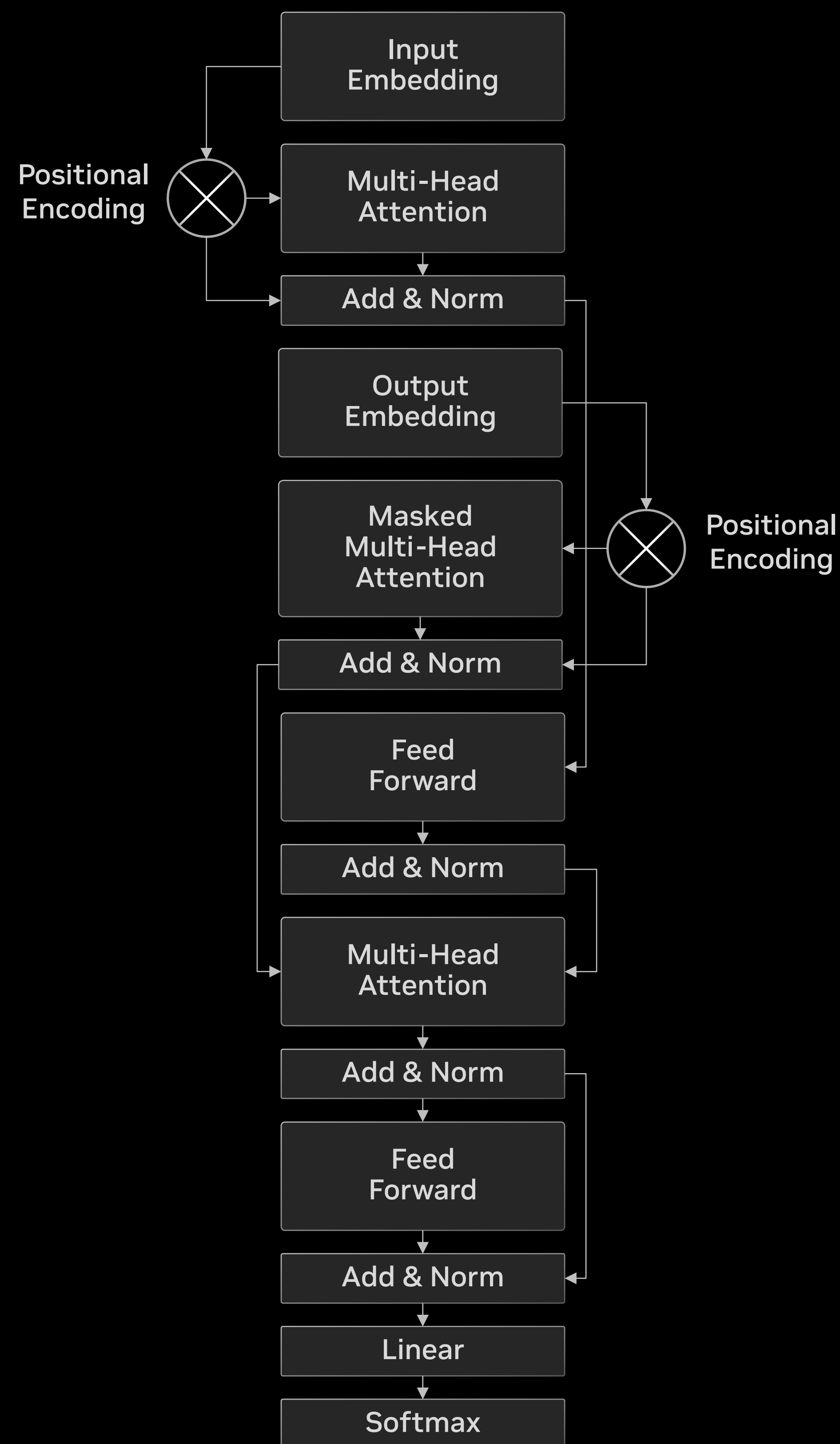
Naïve split of tasks to span GPUs

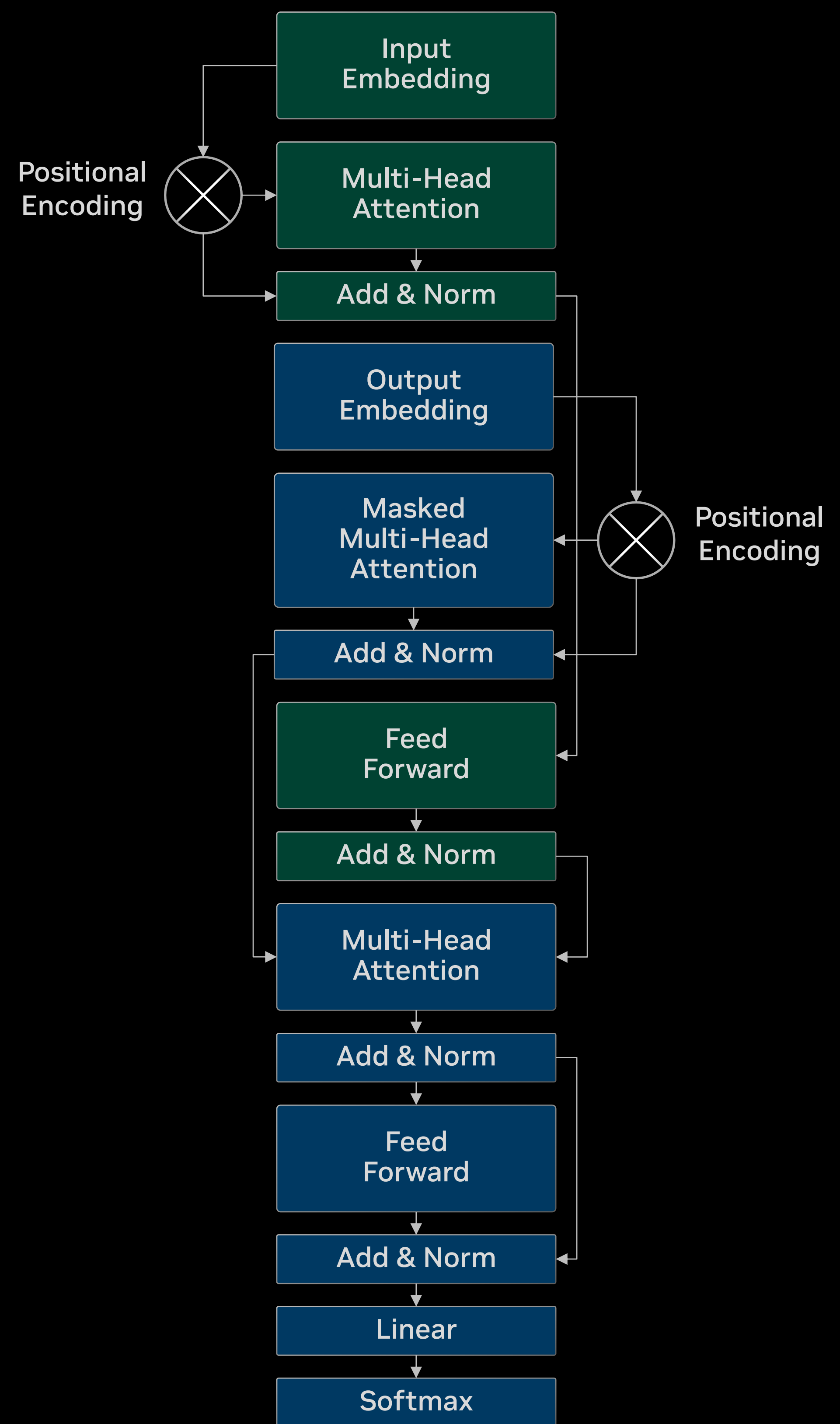


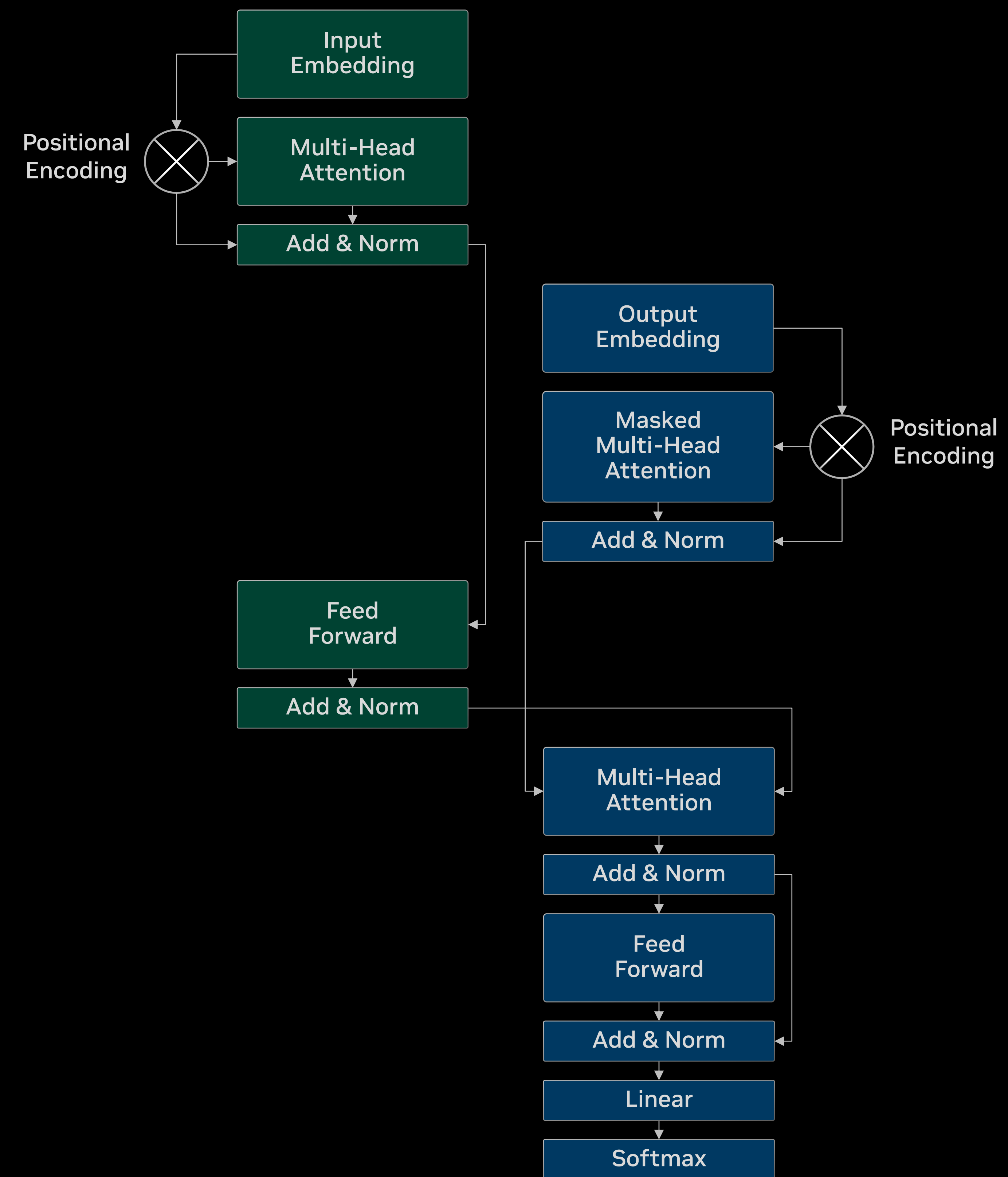
Naïve split of tasks to span GPUs

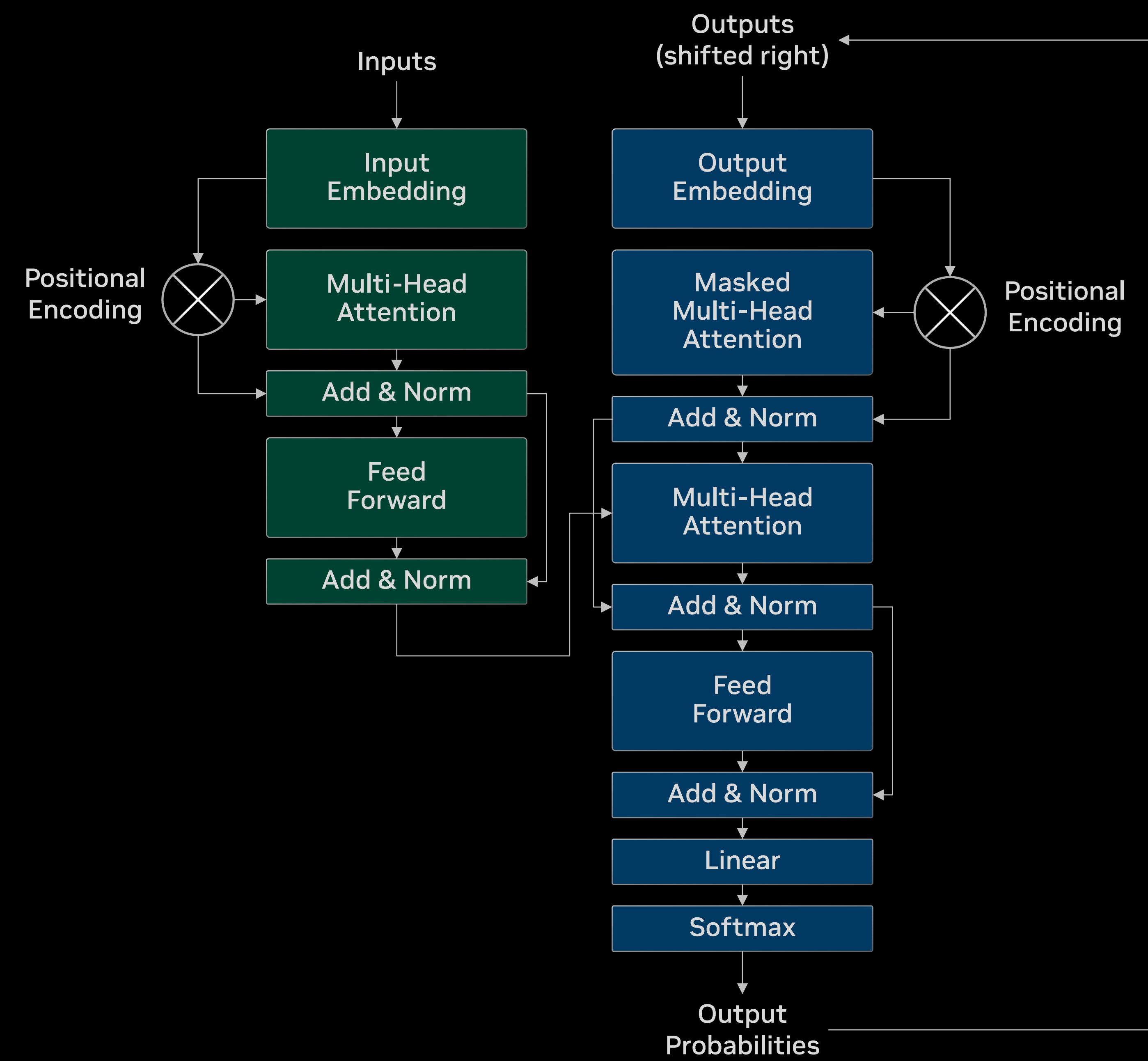


Don't Be Naïve: Minimise Inter-Task Dependencies

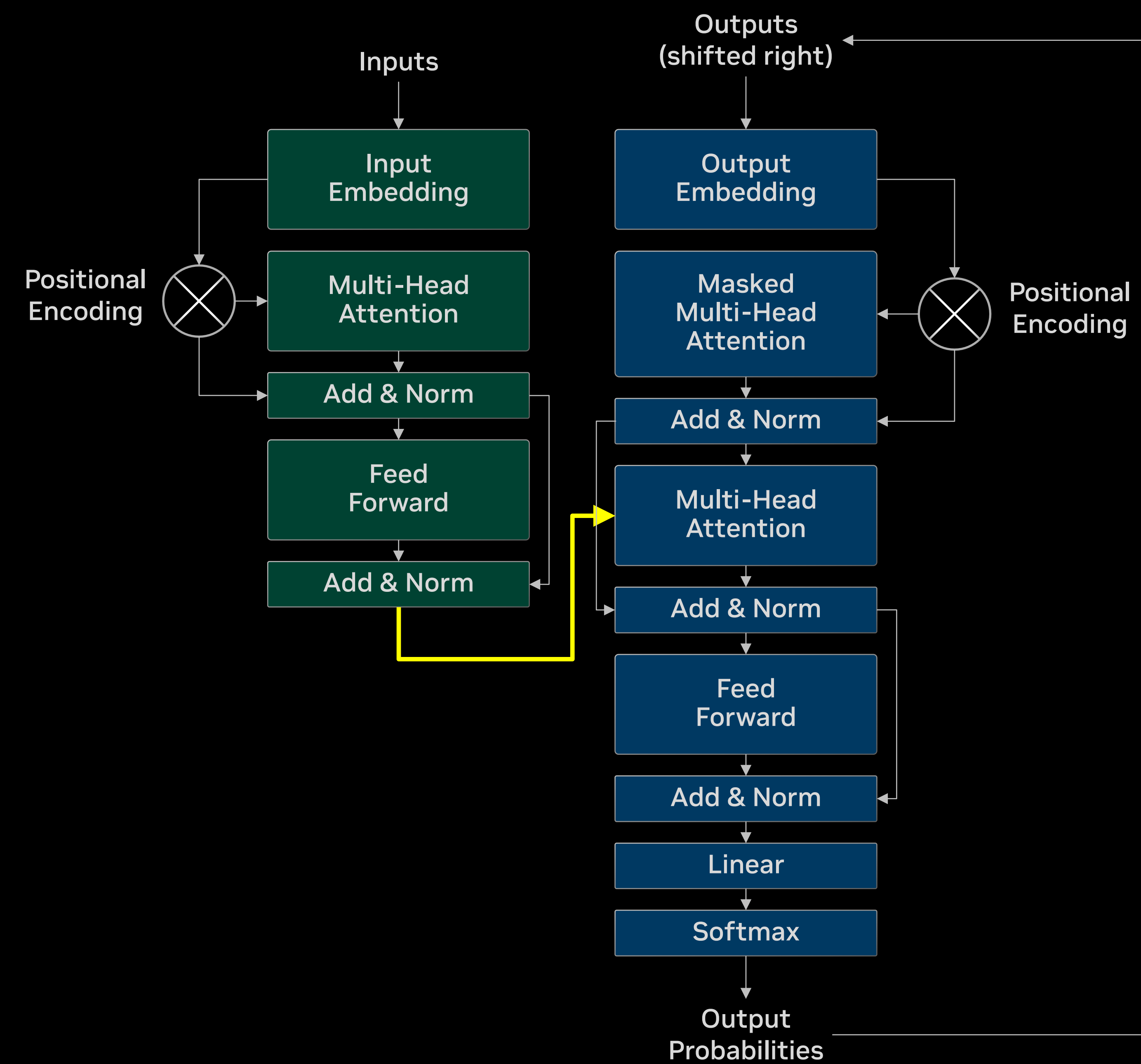




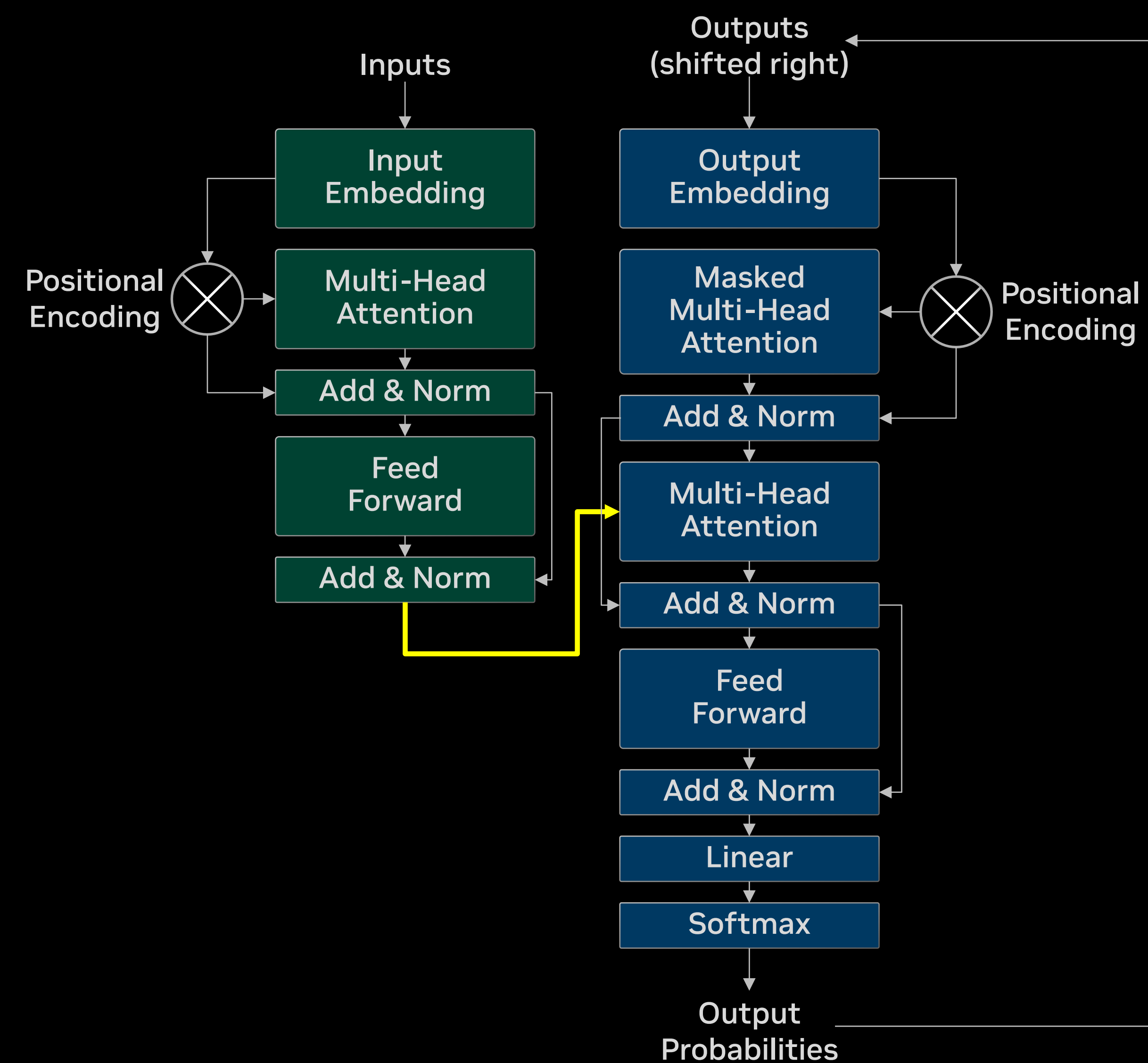




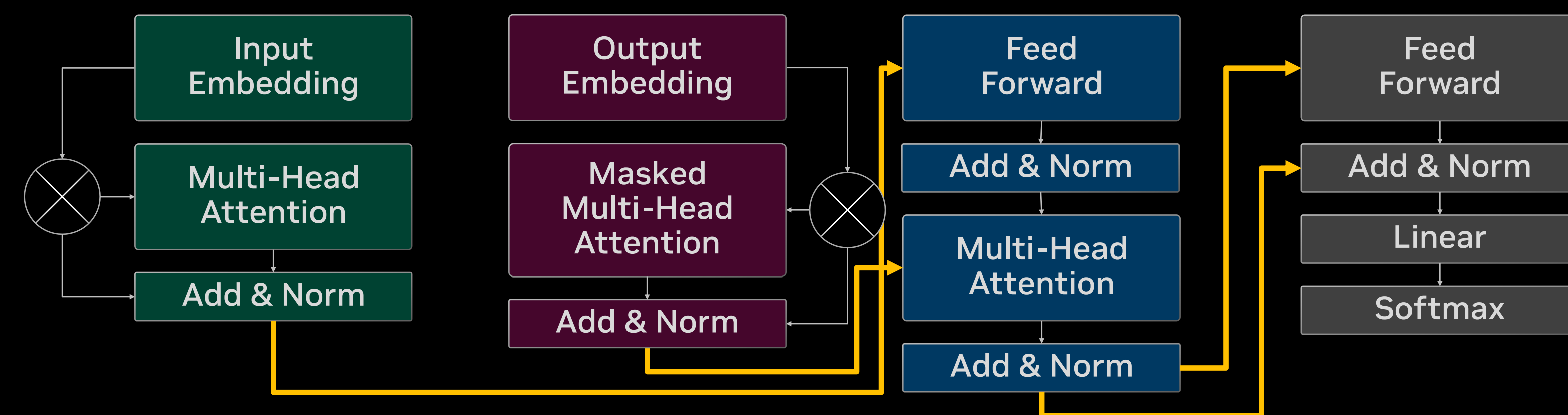
Unbalanced Tasks Are Often Much Cheaper Than Over-Synchronization



In Other Words, Split The Model To Minimise Inter-Task Sync

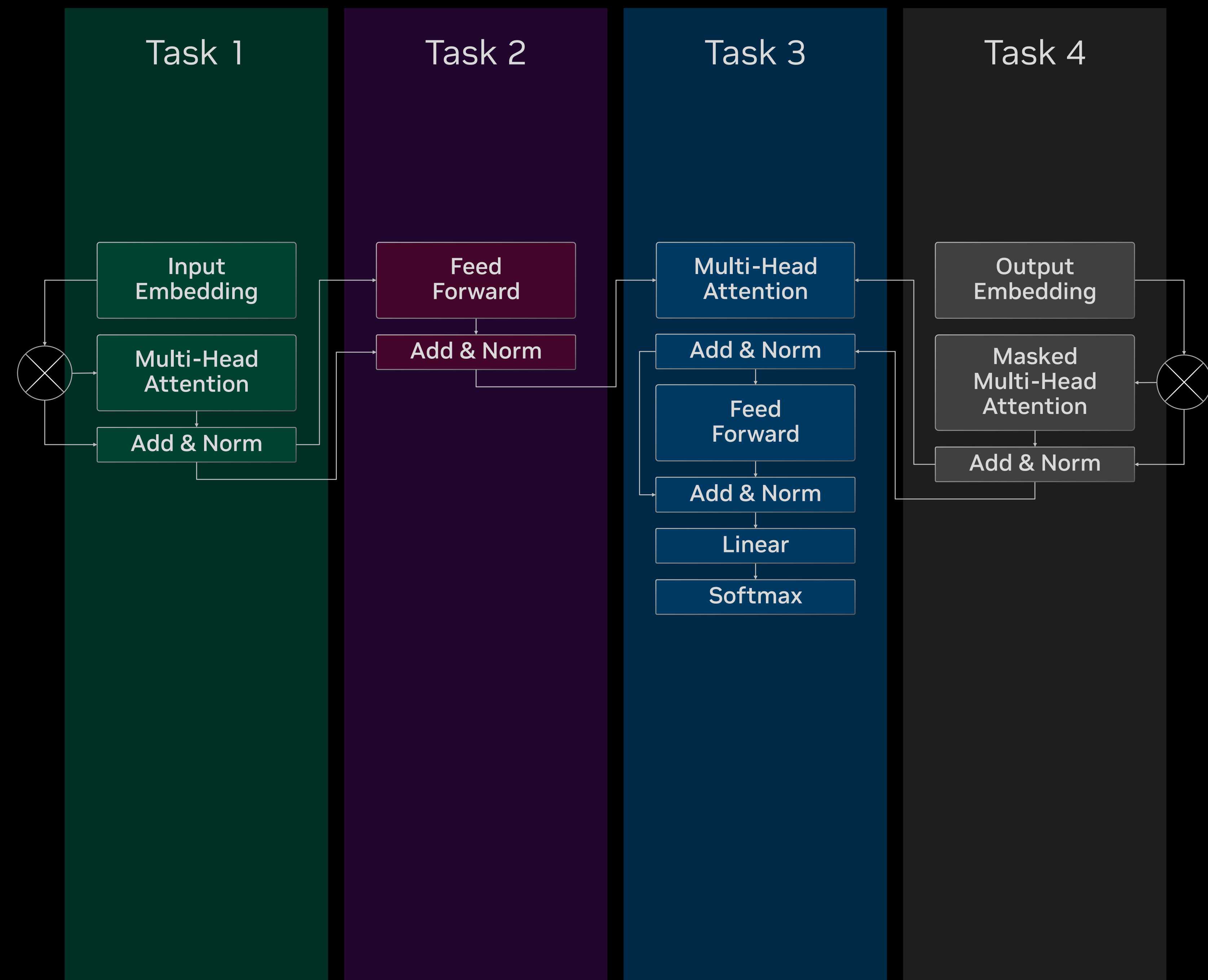
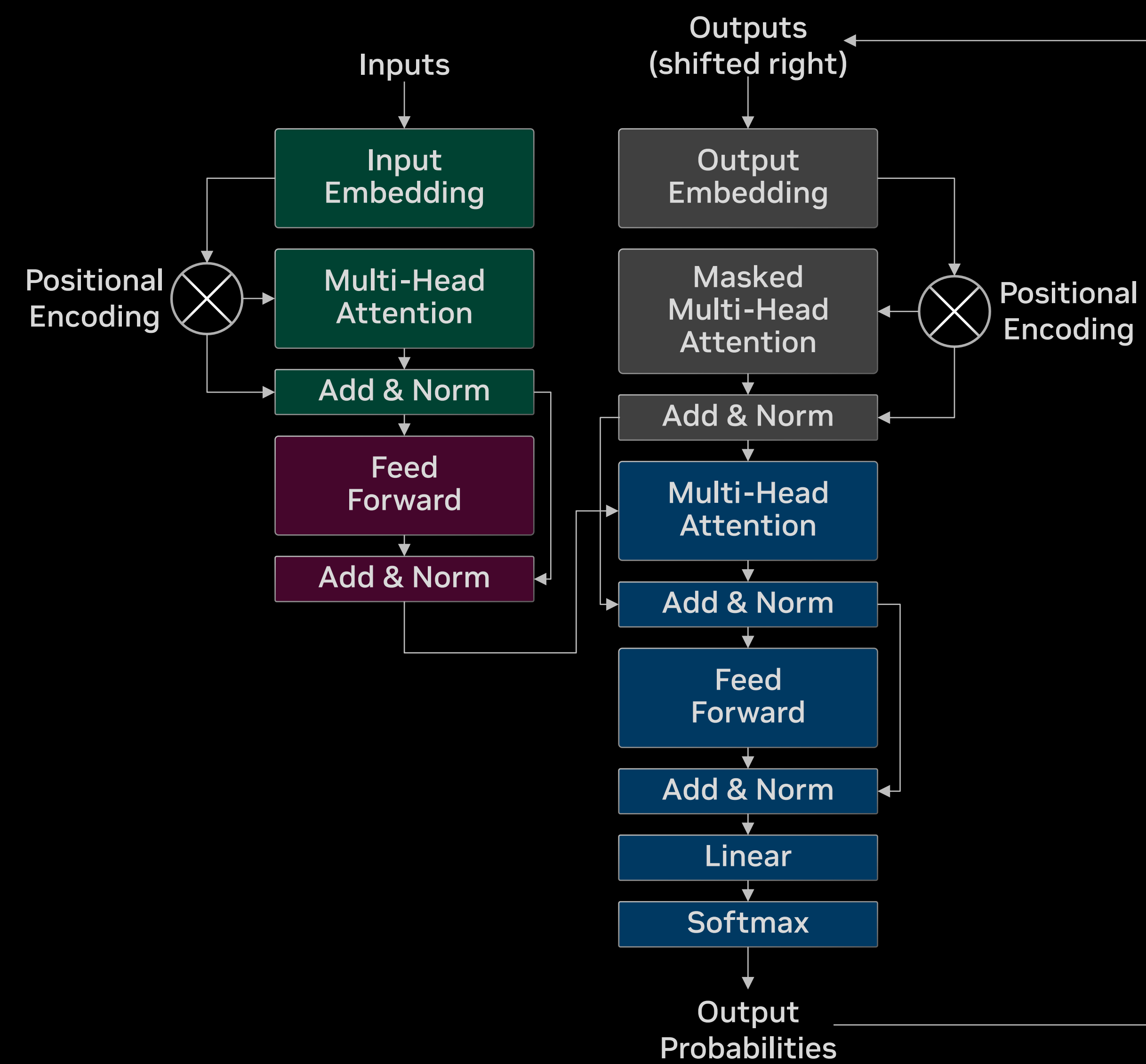


Producer/Consumer split: one dependency

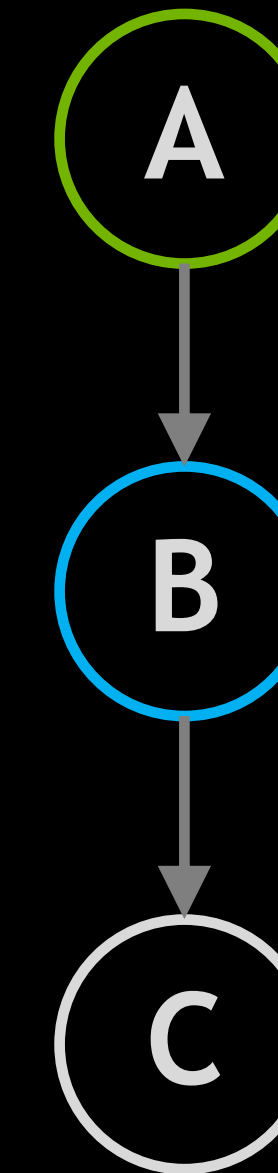
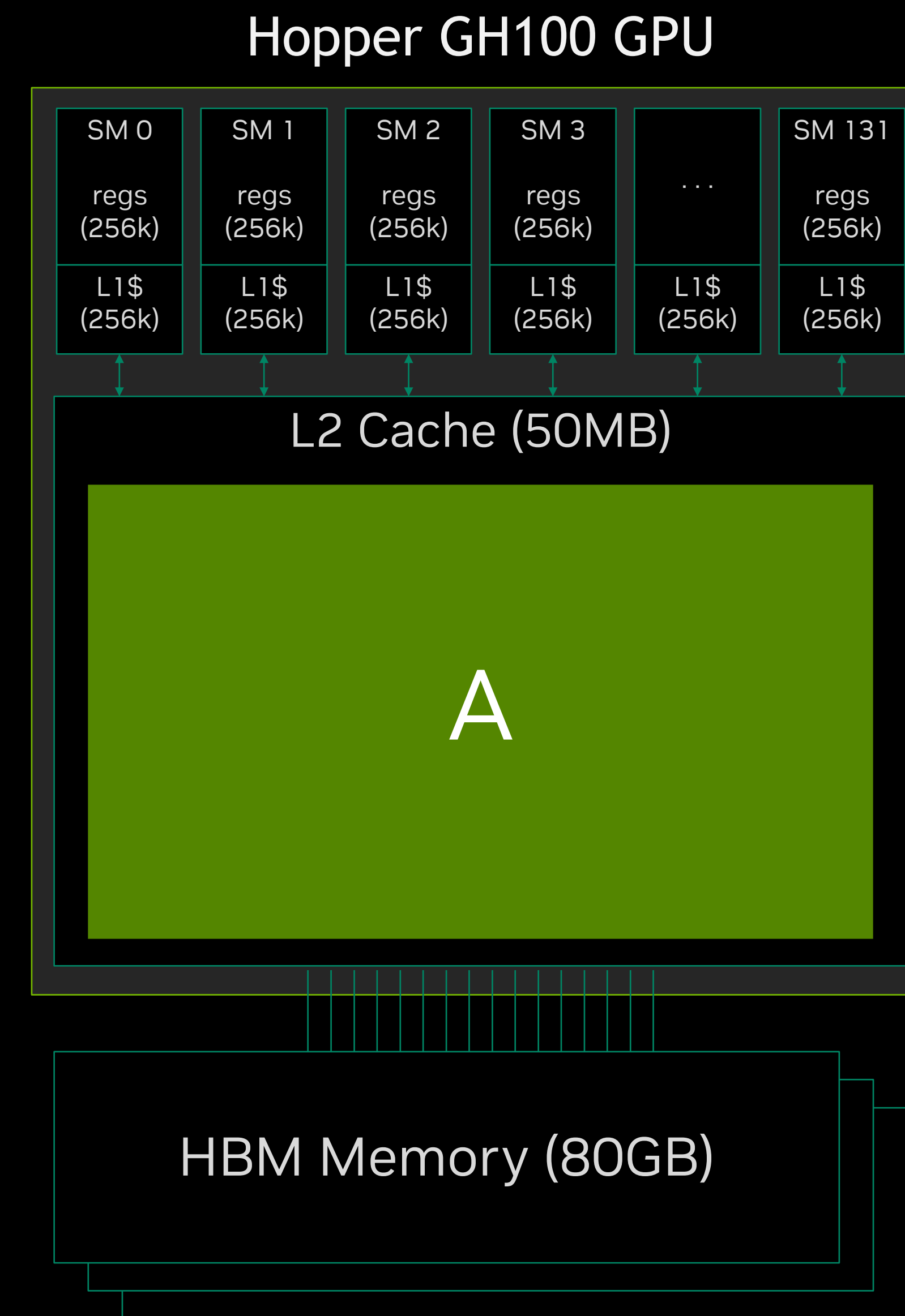


Naïve split: four dependencies in/out of one task

And If You Need To Split More, Do It Hierarchically

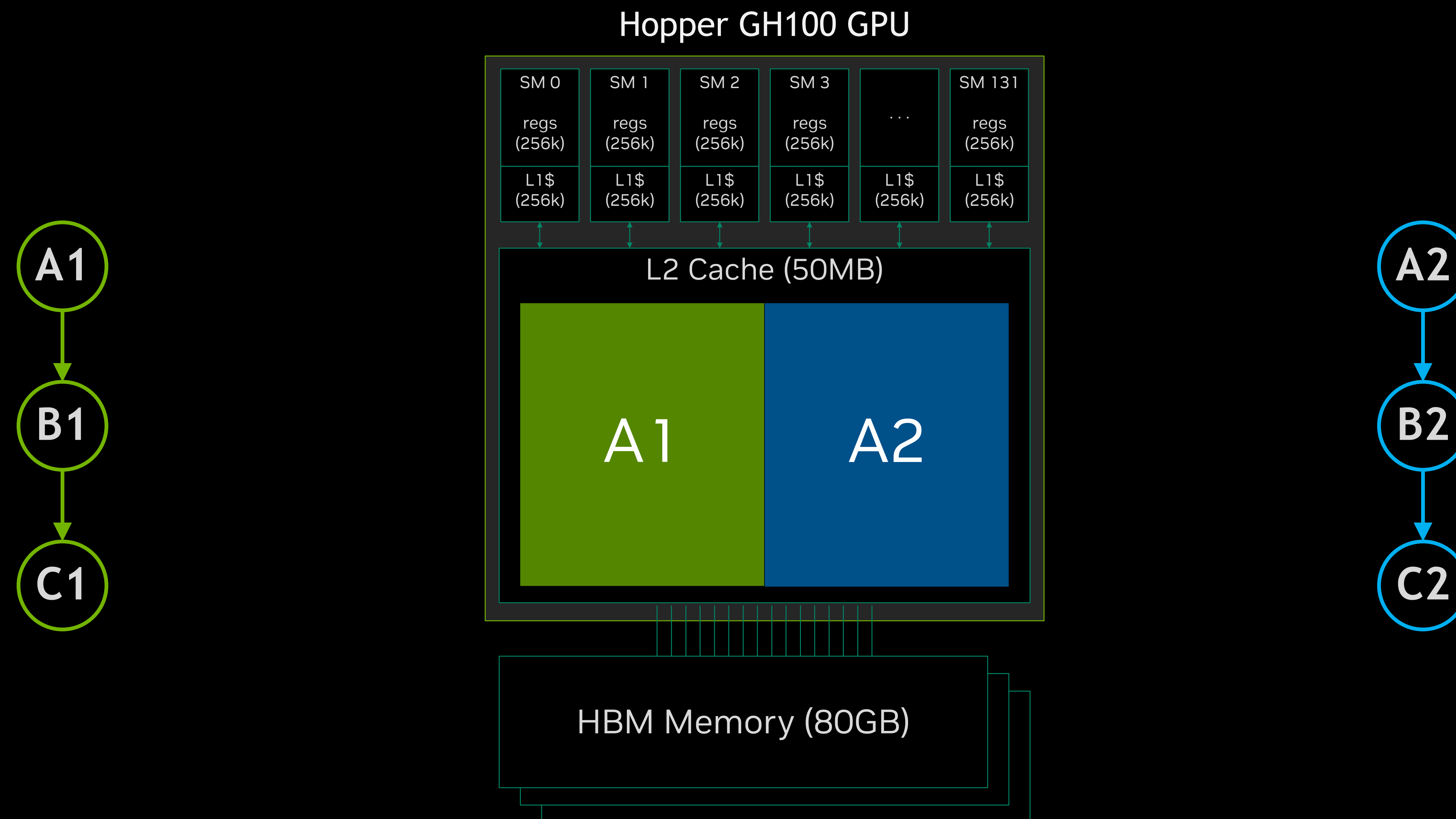


A Big Problem With Task Parallelism: Thrashing The Cache



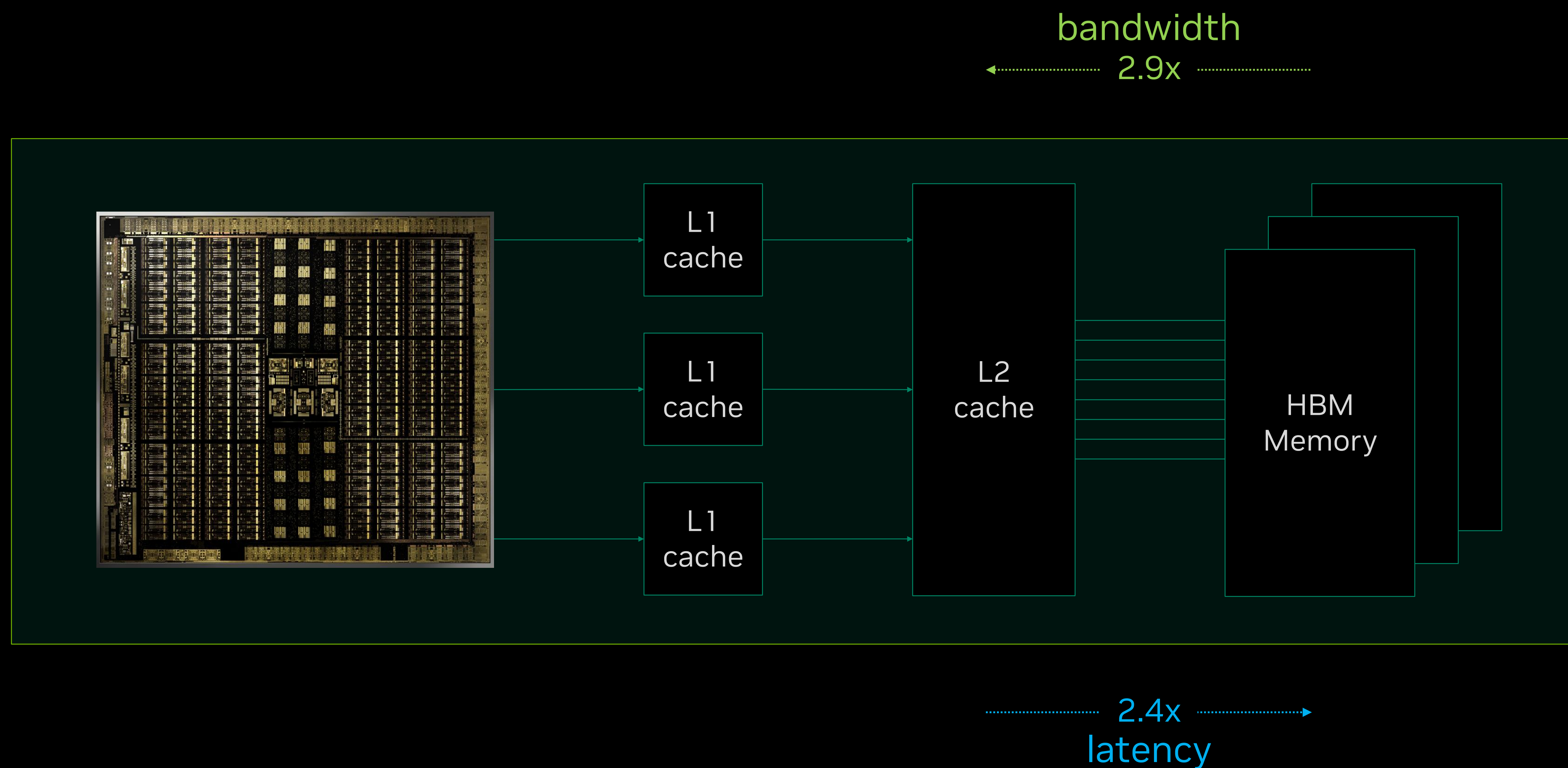
A Big Problem With Task Parallelism: Thrashing The Cache

Each task sees a proportionately smaller cache because by definition they are working on different data

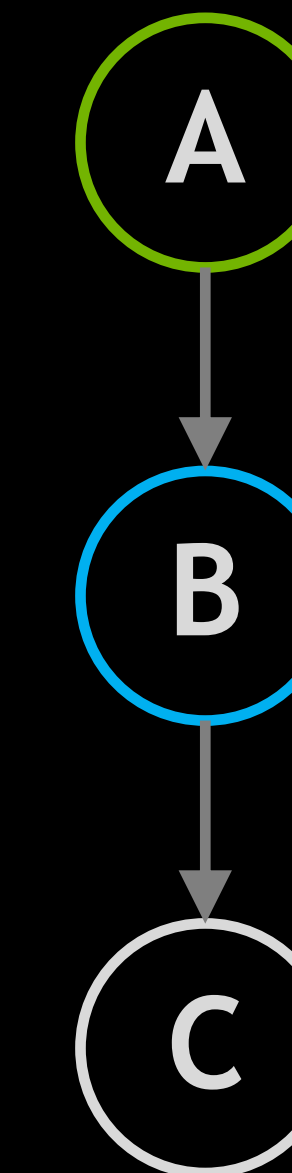
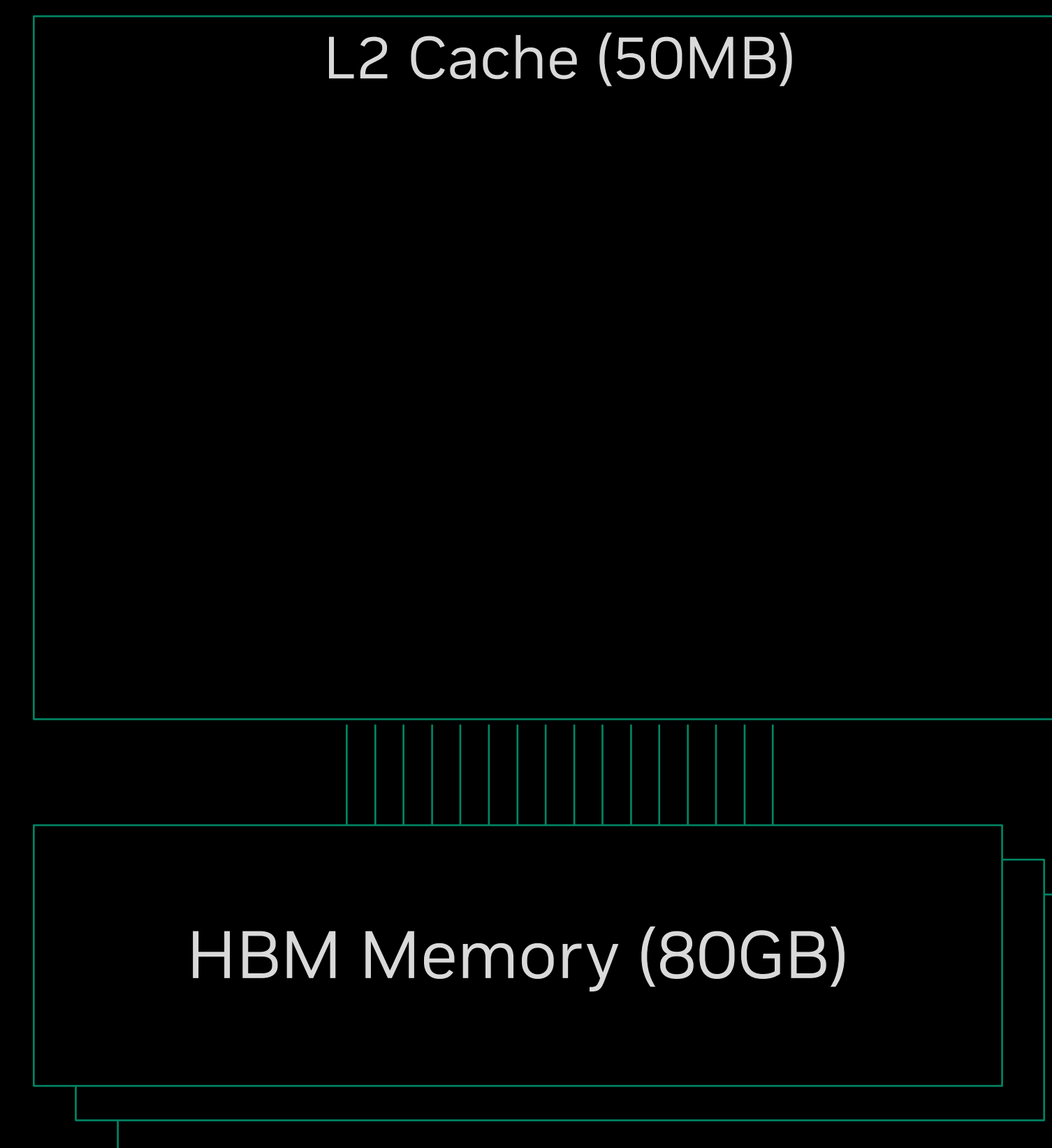
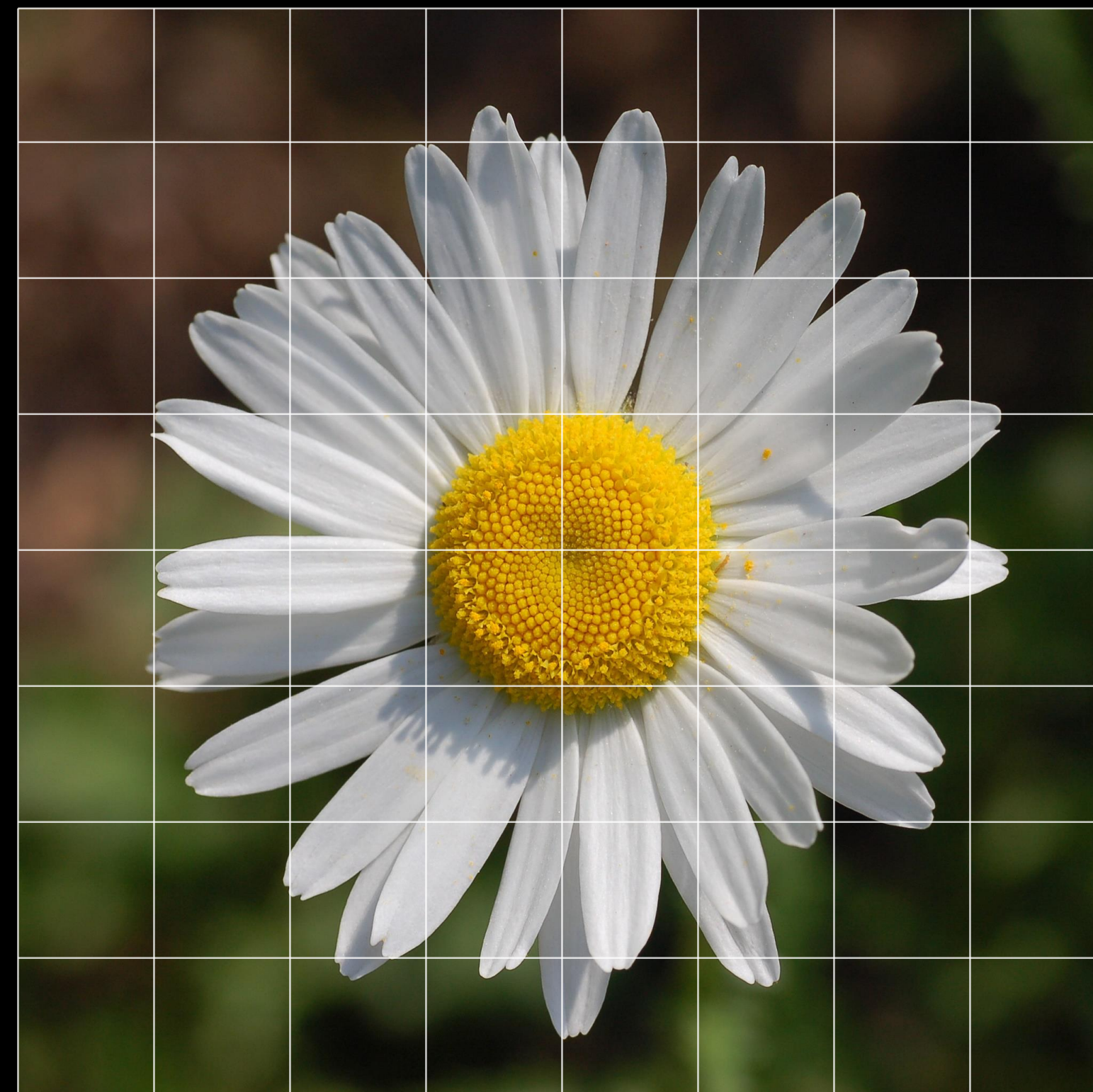


You really really want cached data

A high L2 hit rate can give as much as a 10x boost in performance

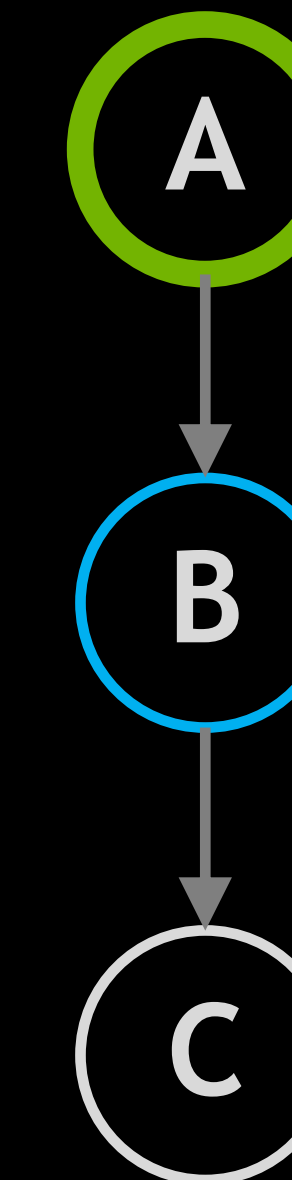
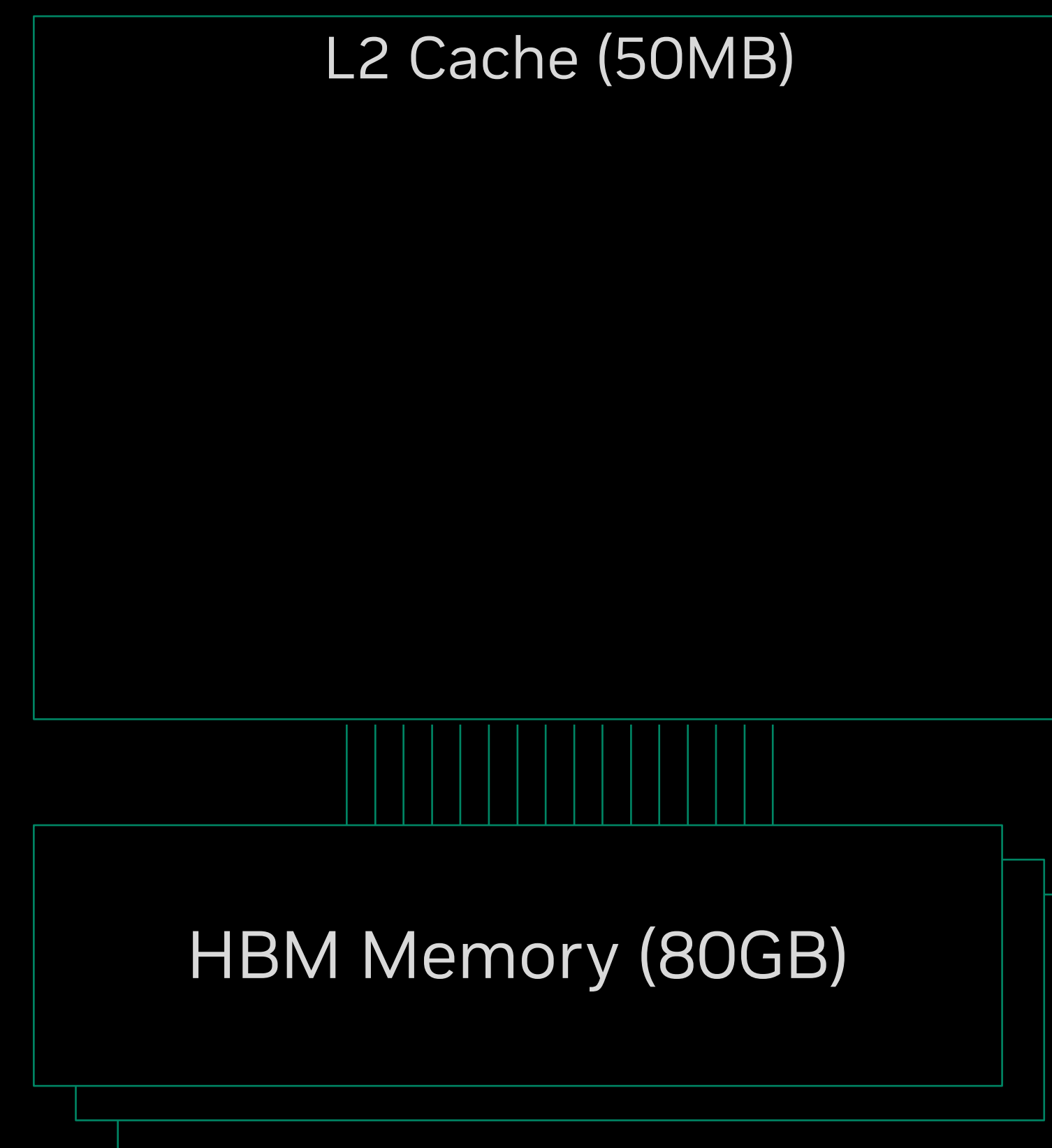
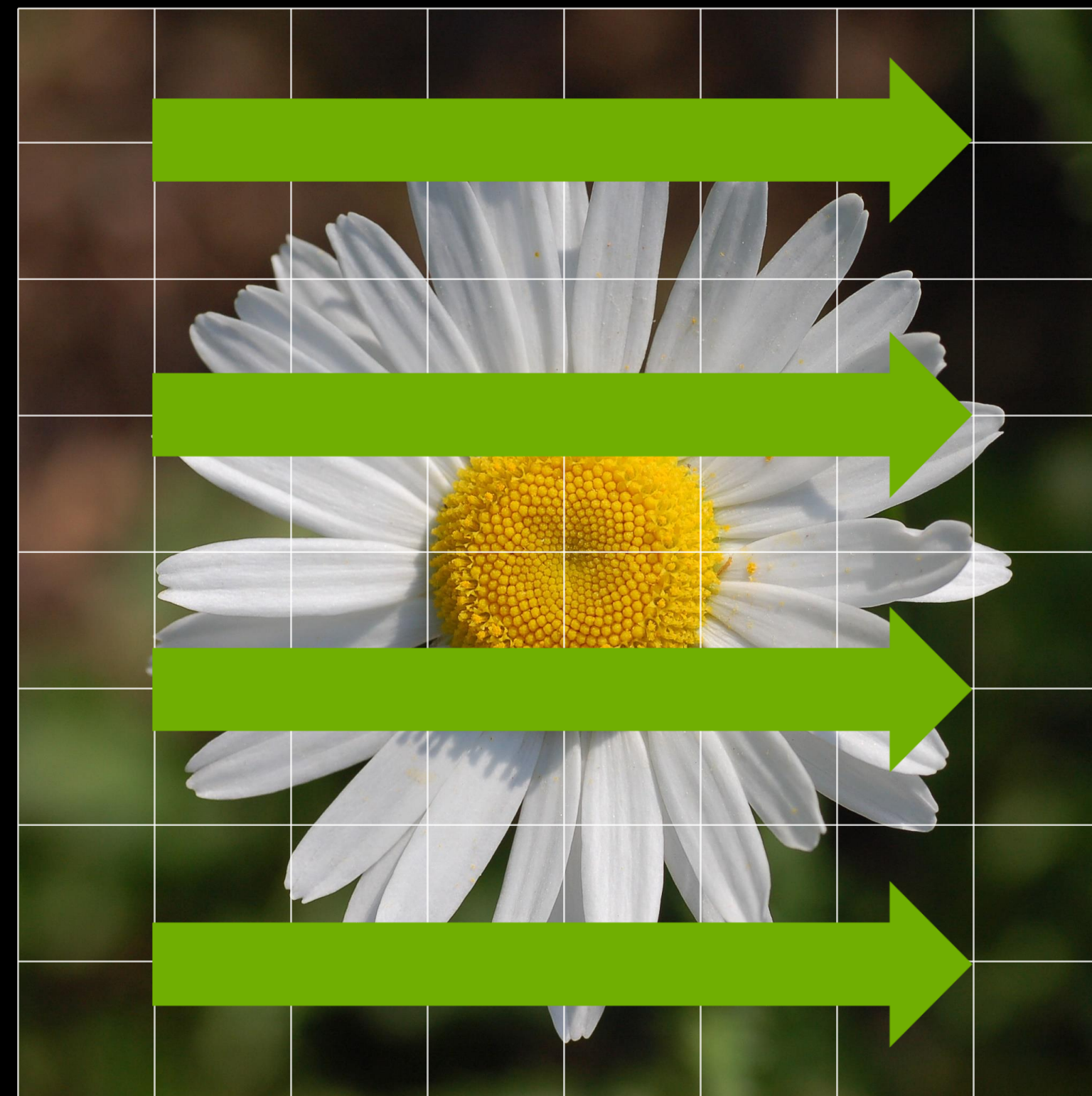


Aside: Stop running all your kernels row-major from the top left

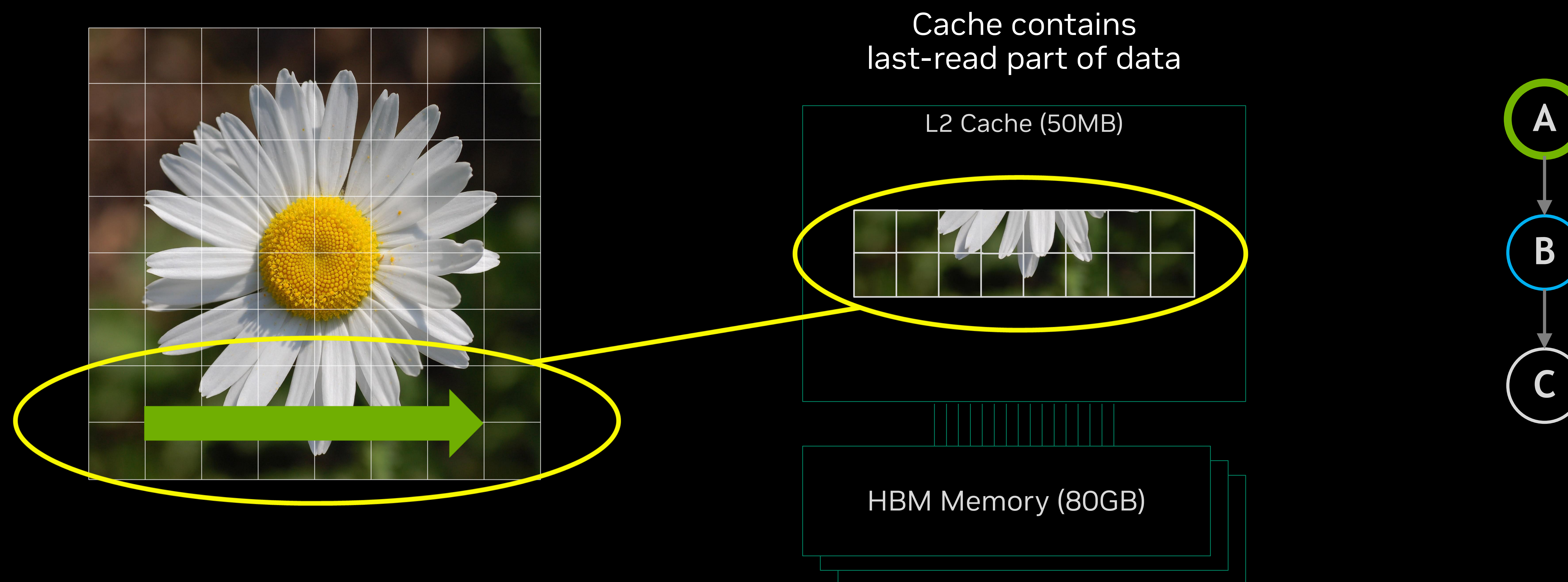


Aside: Stop running all your kernels row-major from the top left

A
starts
reading
here

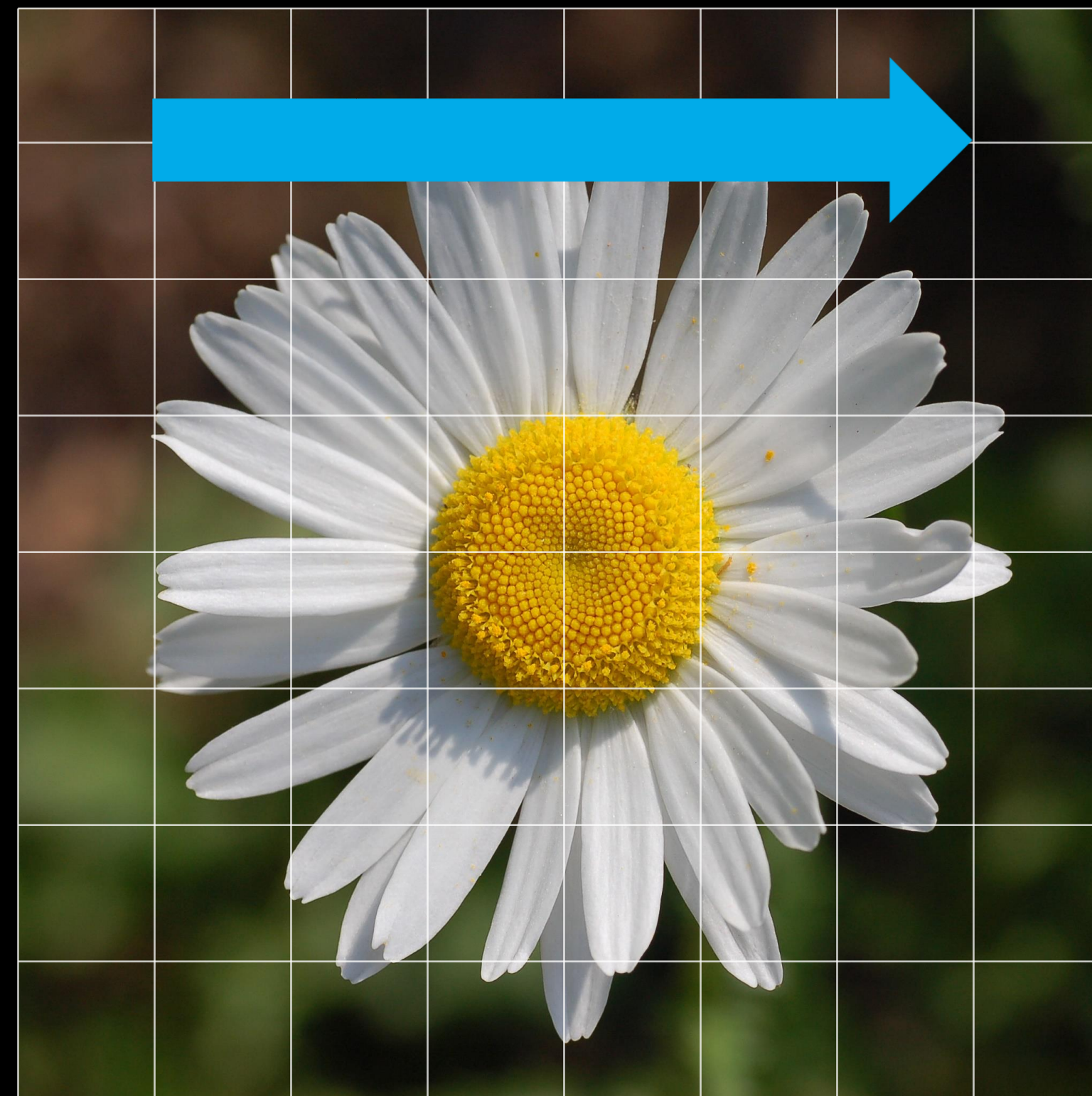


Aside: Stop running all your kernels row-major from the top left



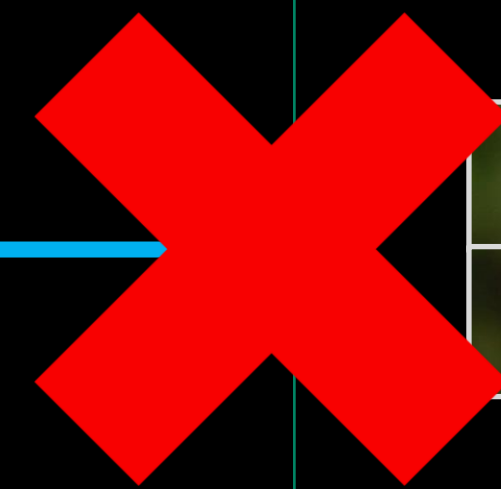
Aside: Stop running all your kernels row-major from the top left

B
starts
reading
here

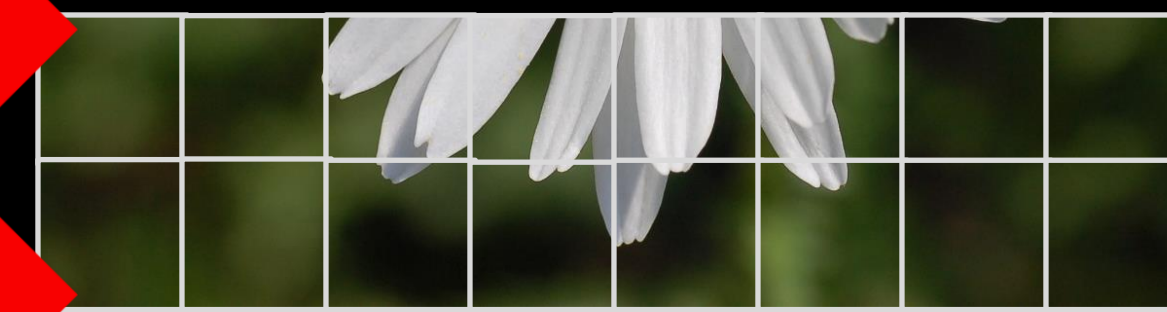


Finishes
here

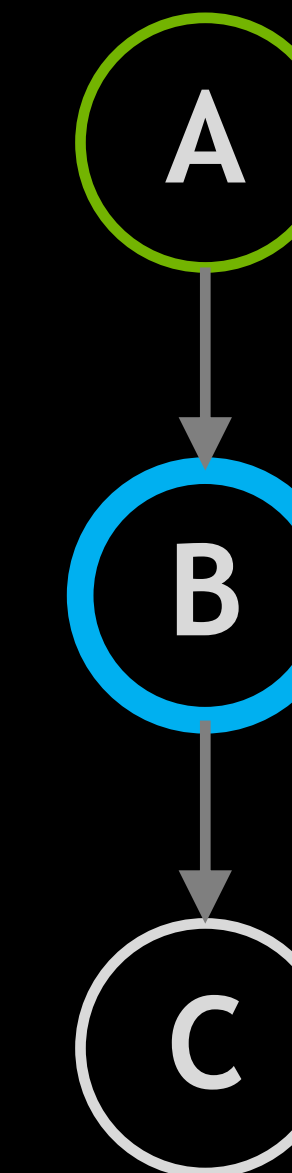
Then B **misses** in cache when
starting from the top



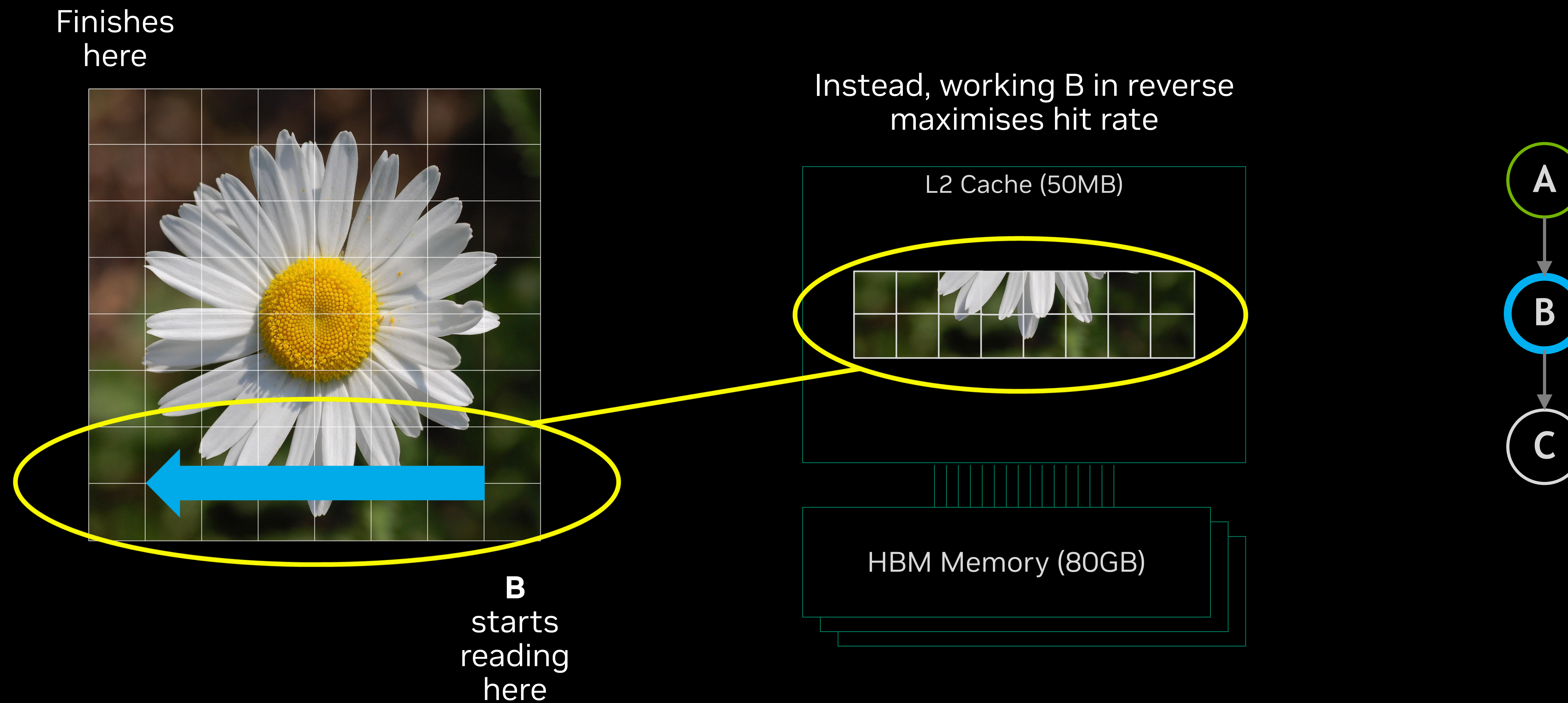
L2 Cache (50MB)



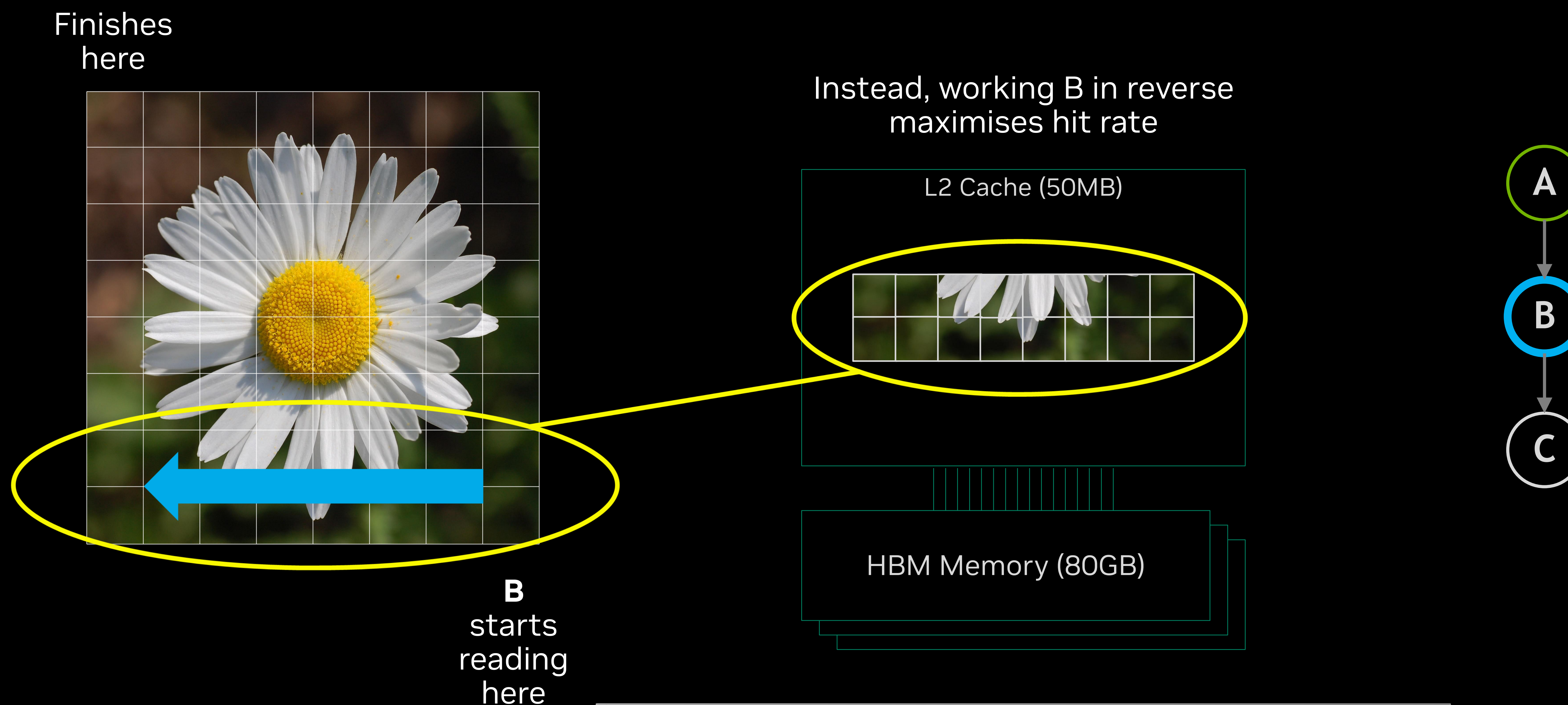
HBM Memory (80GB)



Aside: Stop running all your kernels row-major from the top left



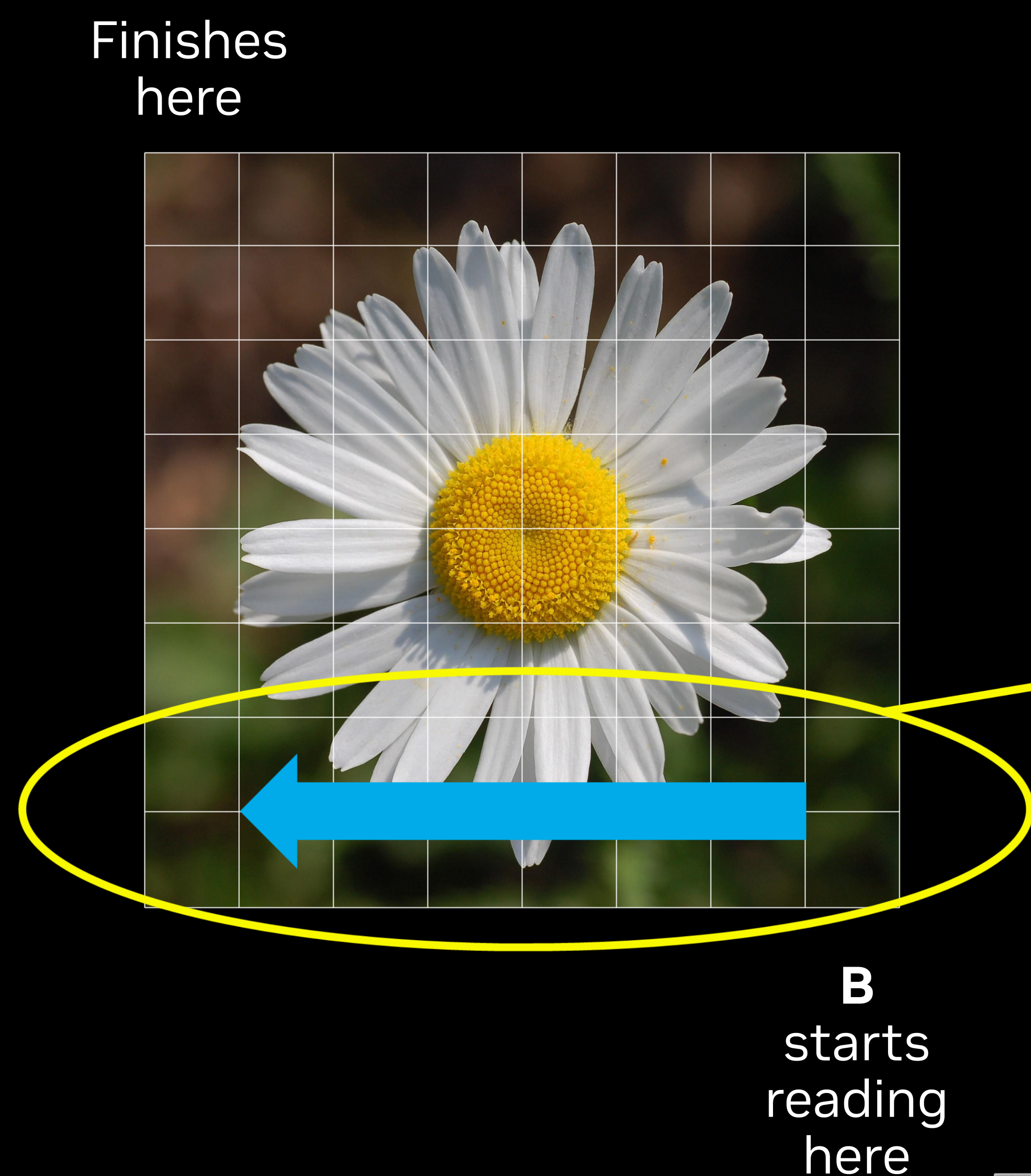
Aside: Stop running all your kernels row-major from the top left



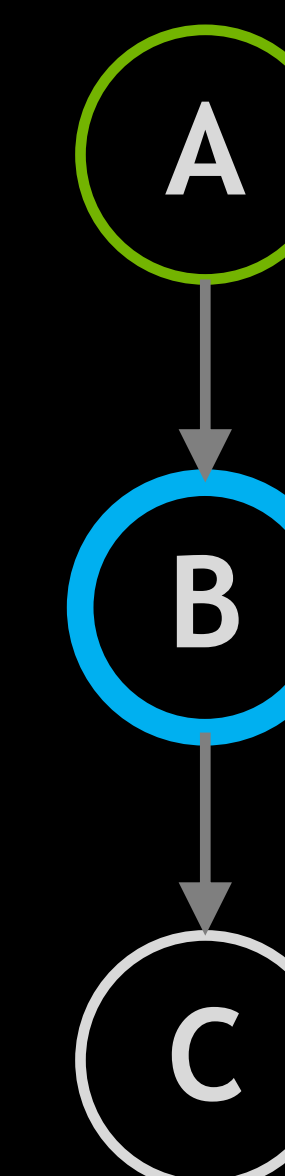
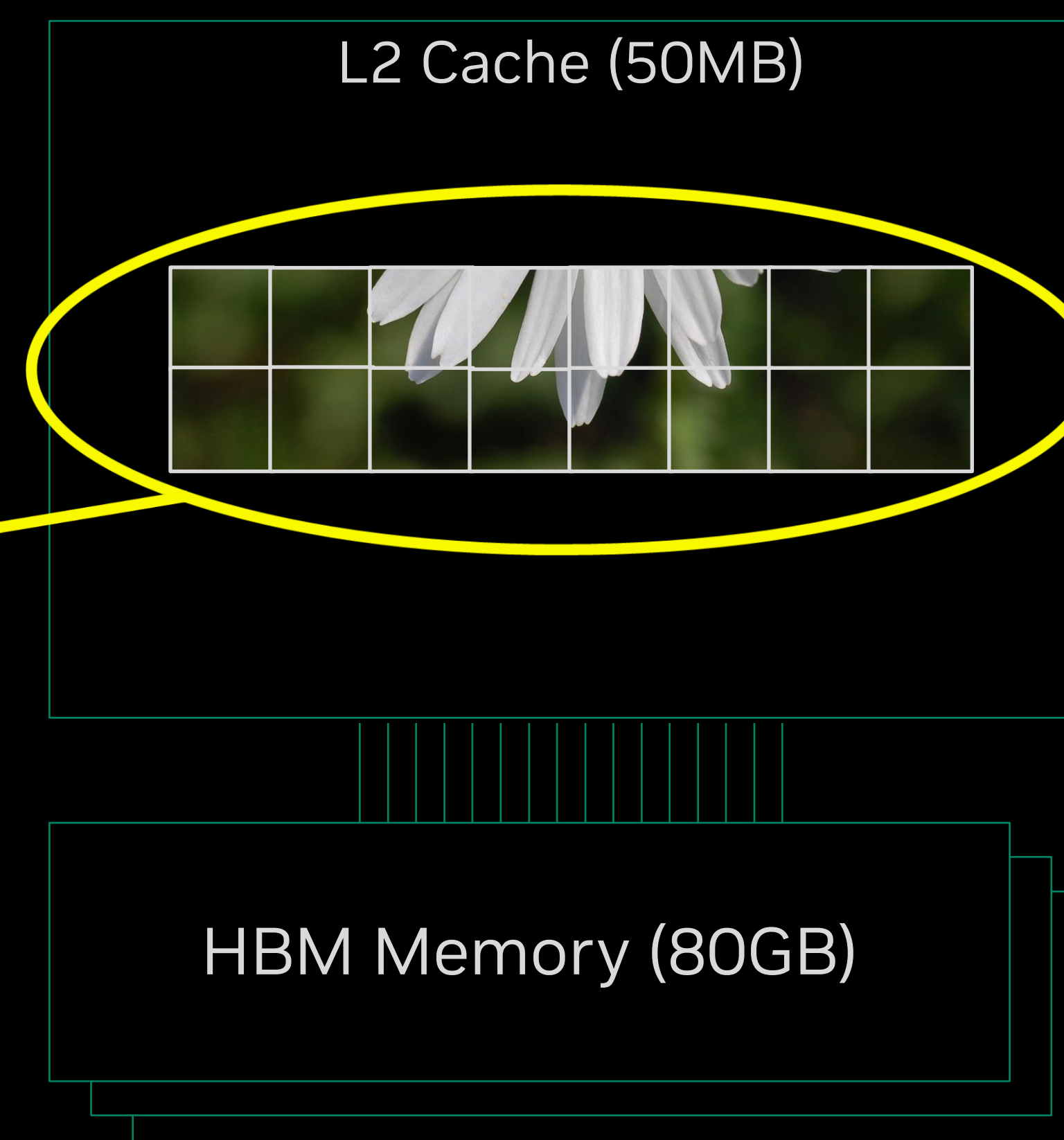
```
int data_index = threadIdx.x + blockIdx.x * blockDim.x;
```

Almost every program has code that looks like this...

Aside: Stop running all your kernels row-major from the top left



Instead, working B in reverse maximises hit rate



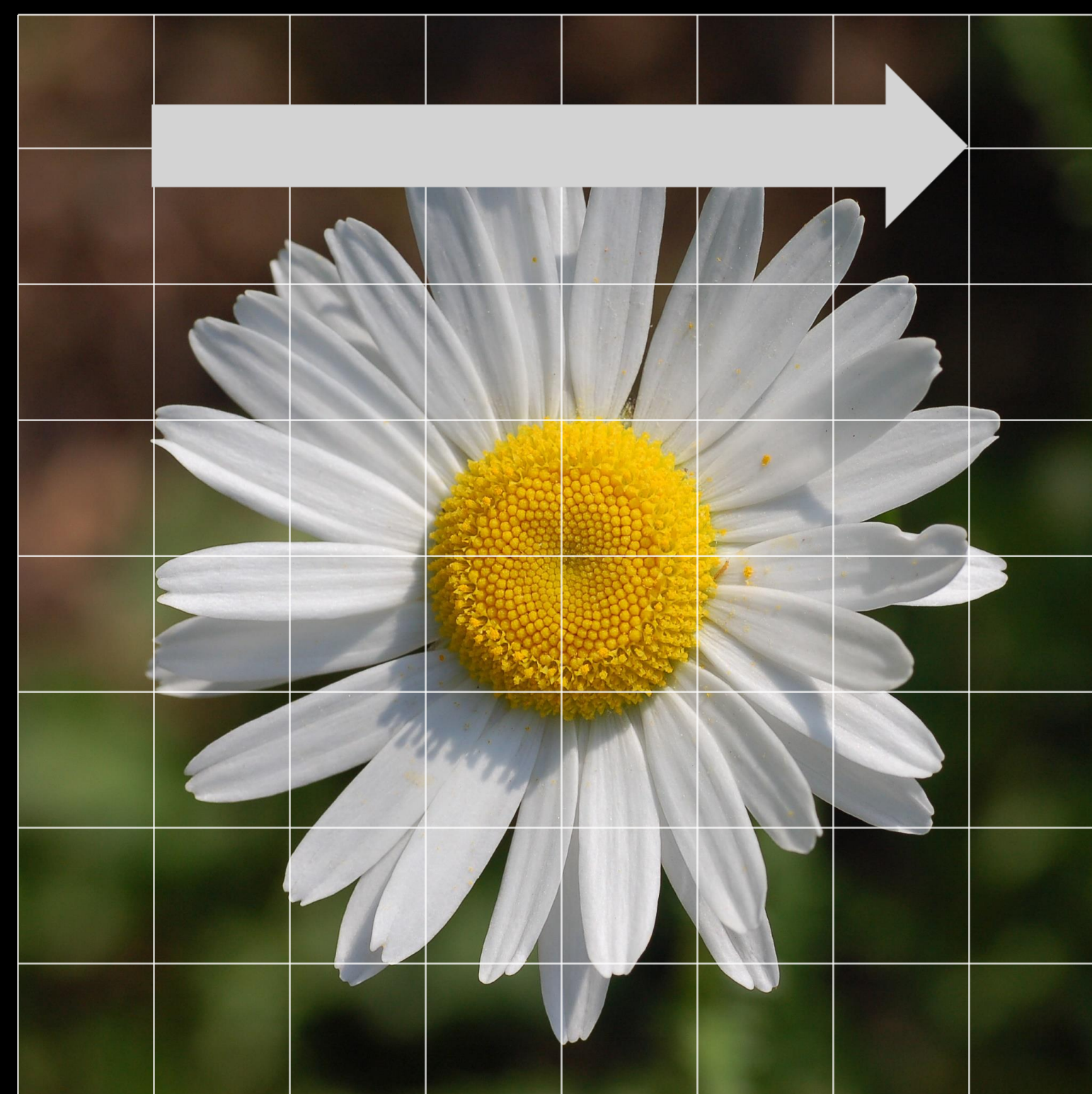
```
int blockId = gridDim.x - blockIdx.x - 1;  
int data_index = threadIdx.x + blockId * blockDim.x;
```

As all blocks execute independently, the program still runs correctly

So alternate block indexing between forwards $\leftrightarrow$ backwards

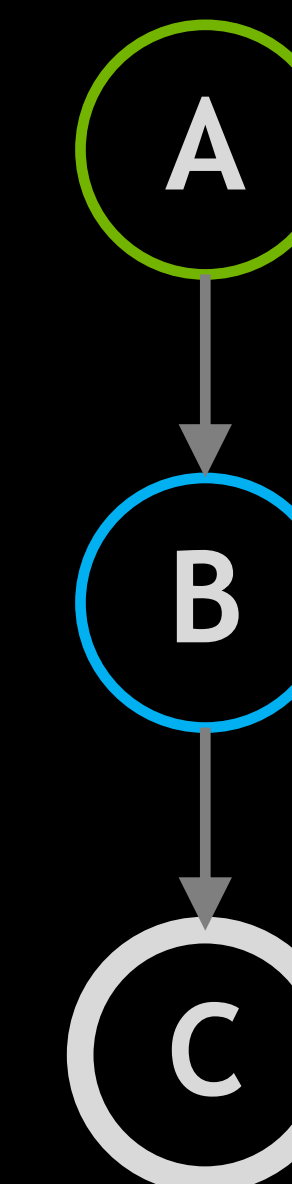
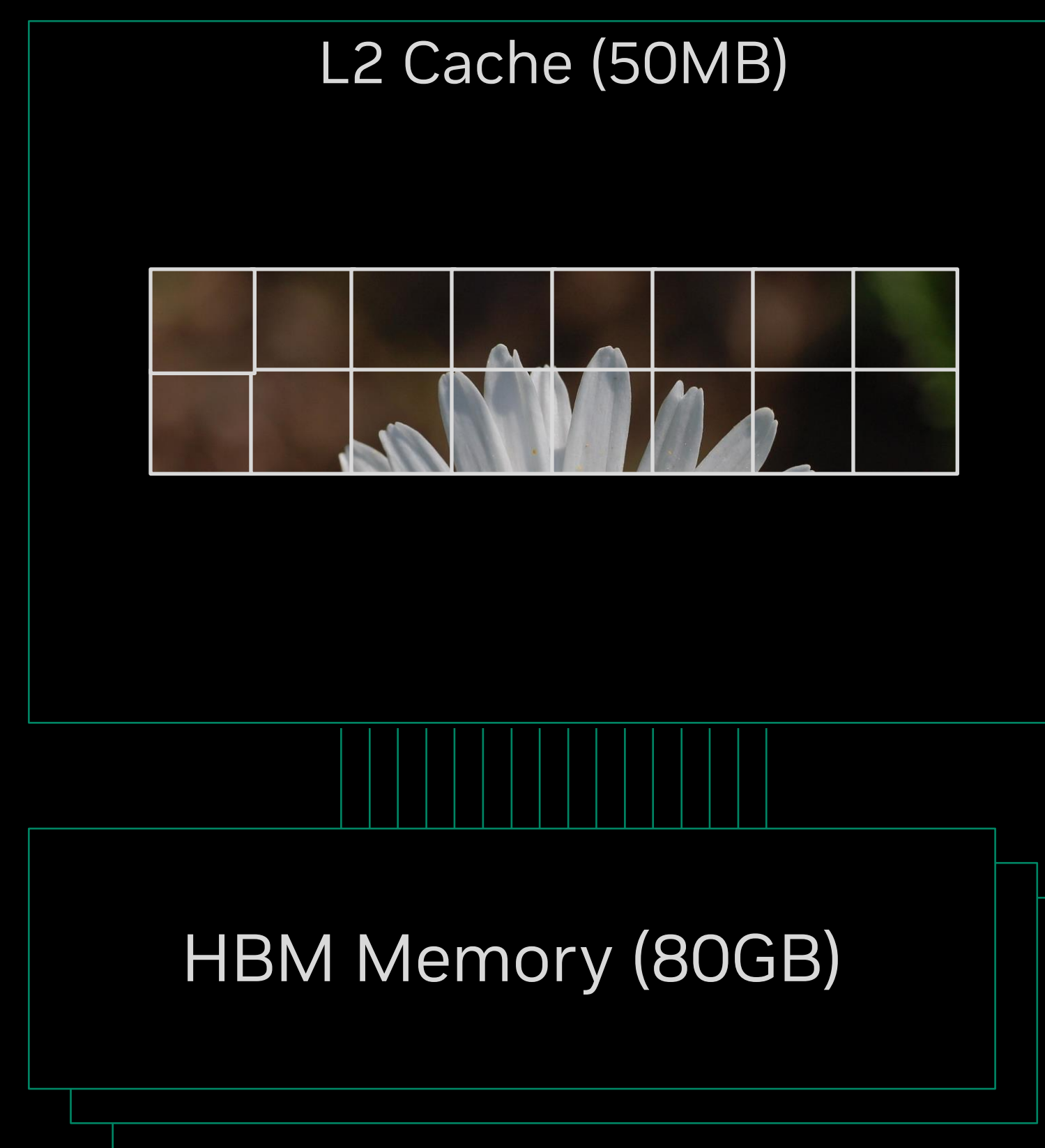
(obviously only if you have a row-major access pattern)

C
starts
reading
here

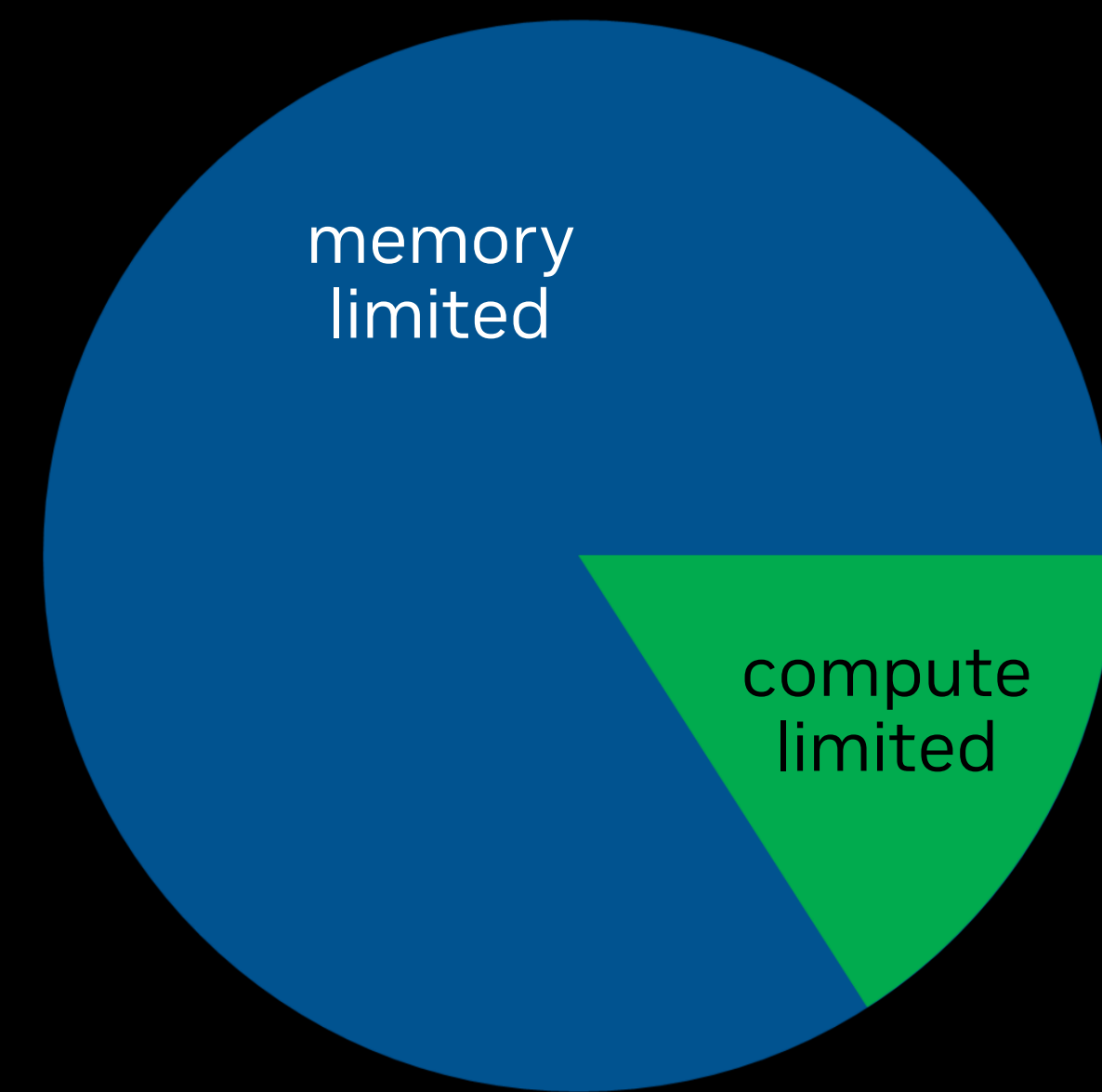


Finishes
here

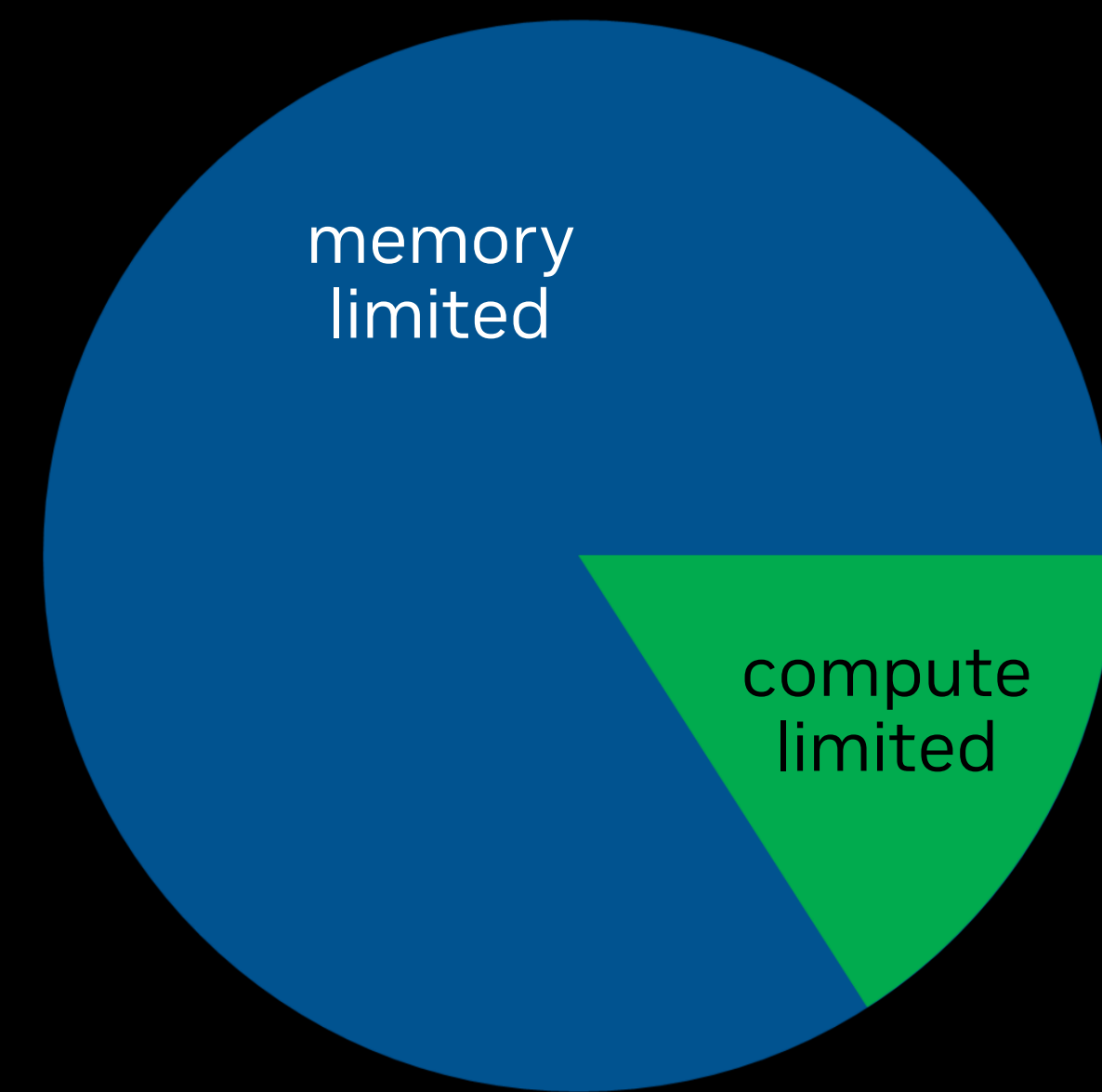
C then works from top and
hits the last accesses from B



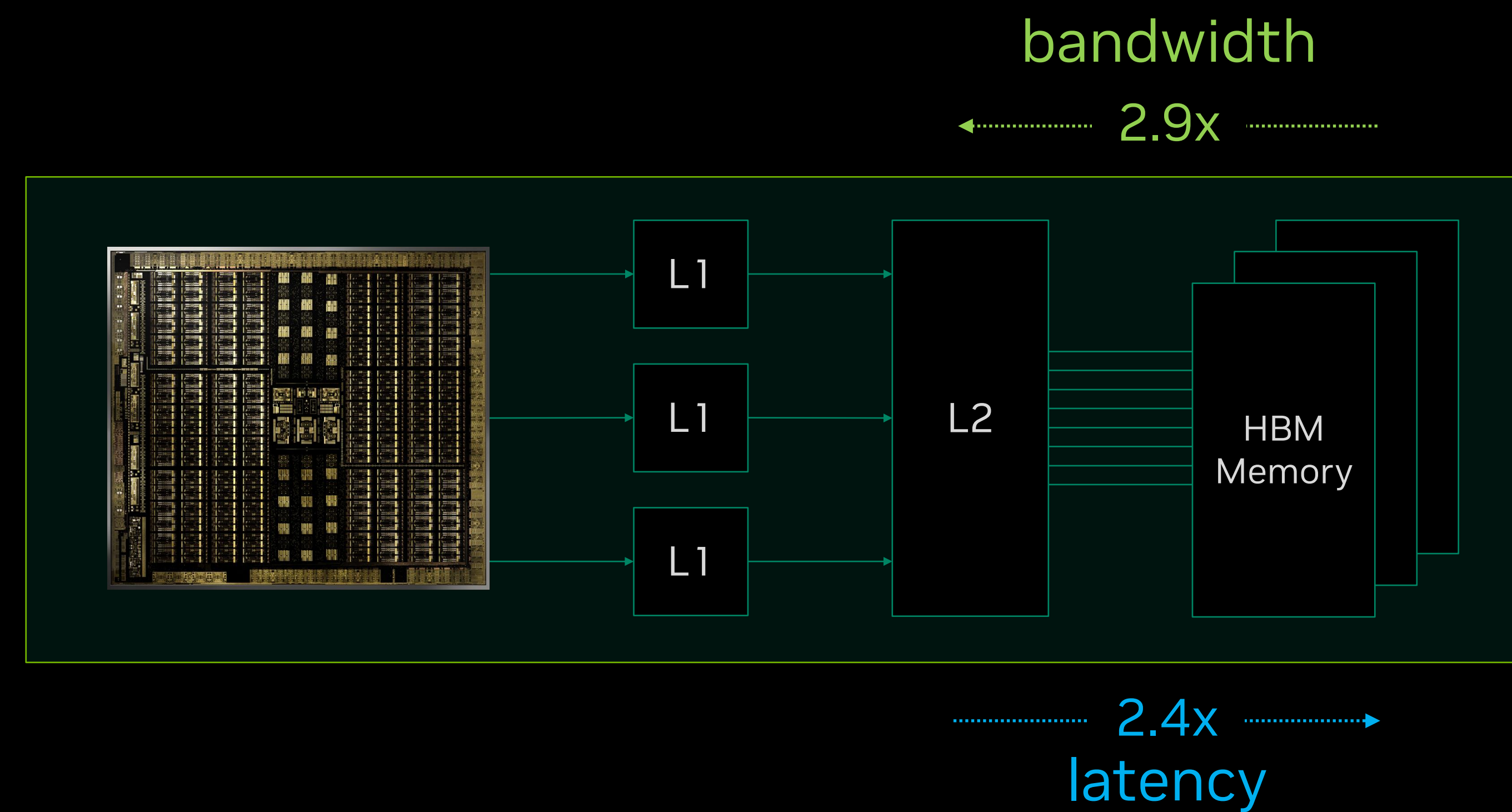
Which leads to the (possibly) obvious thought...



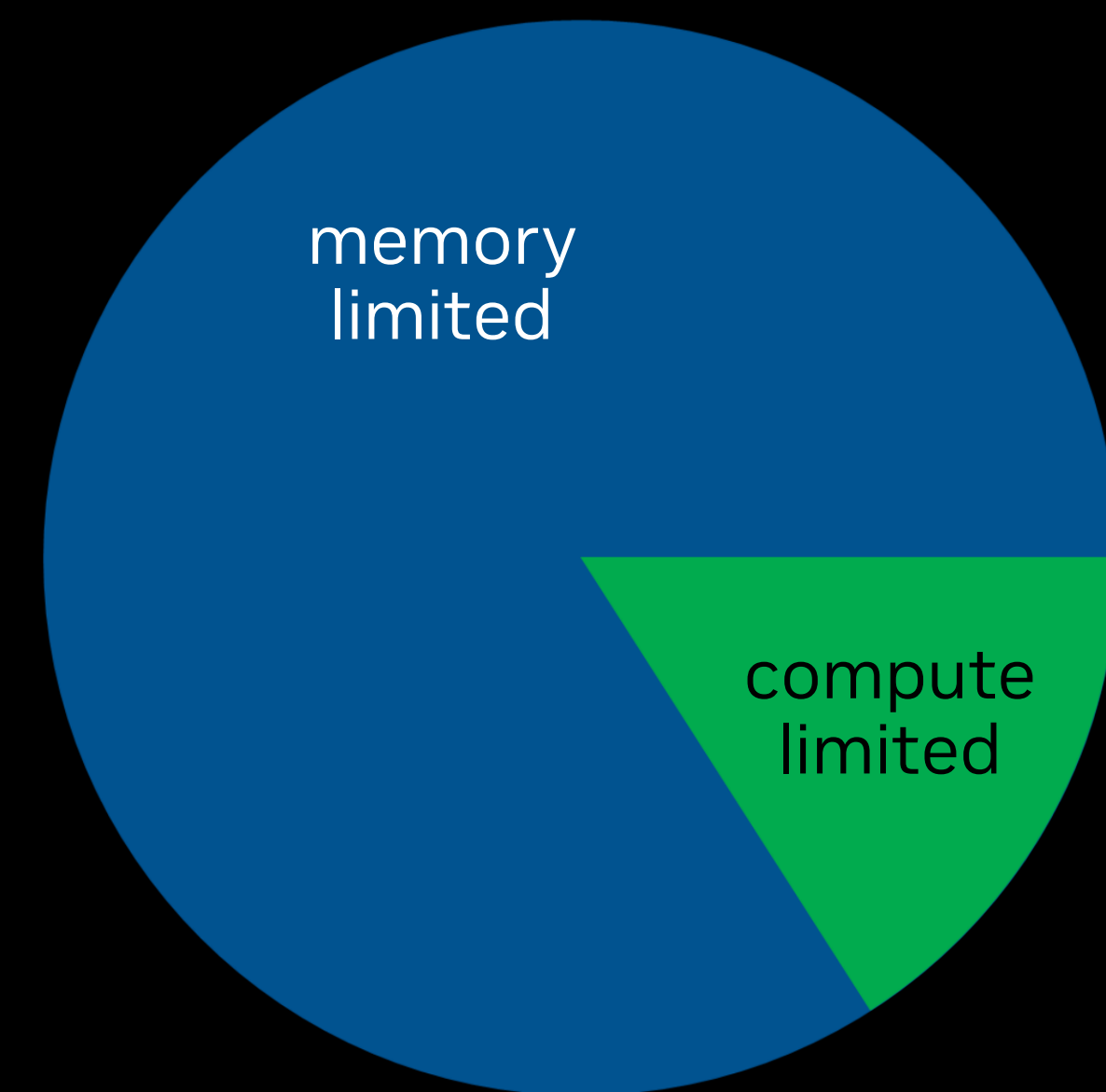
If most programs are
limited by memory
bandwidth ...



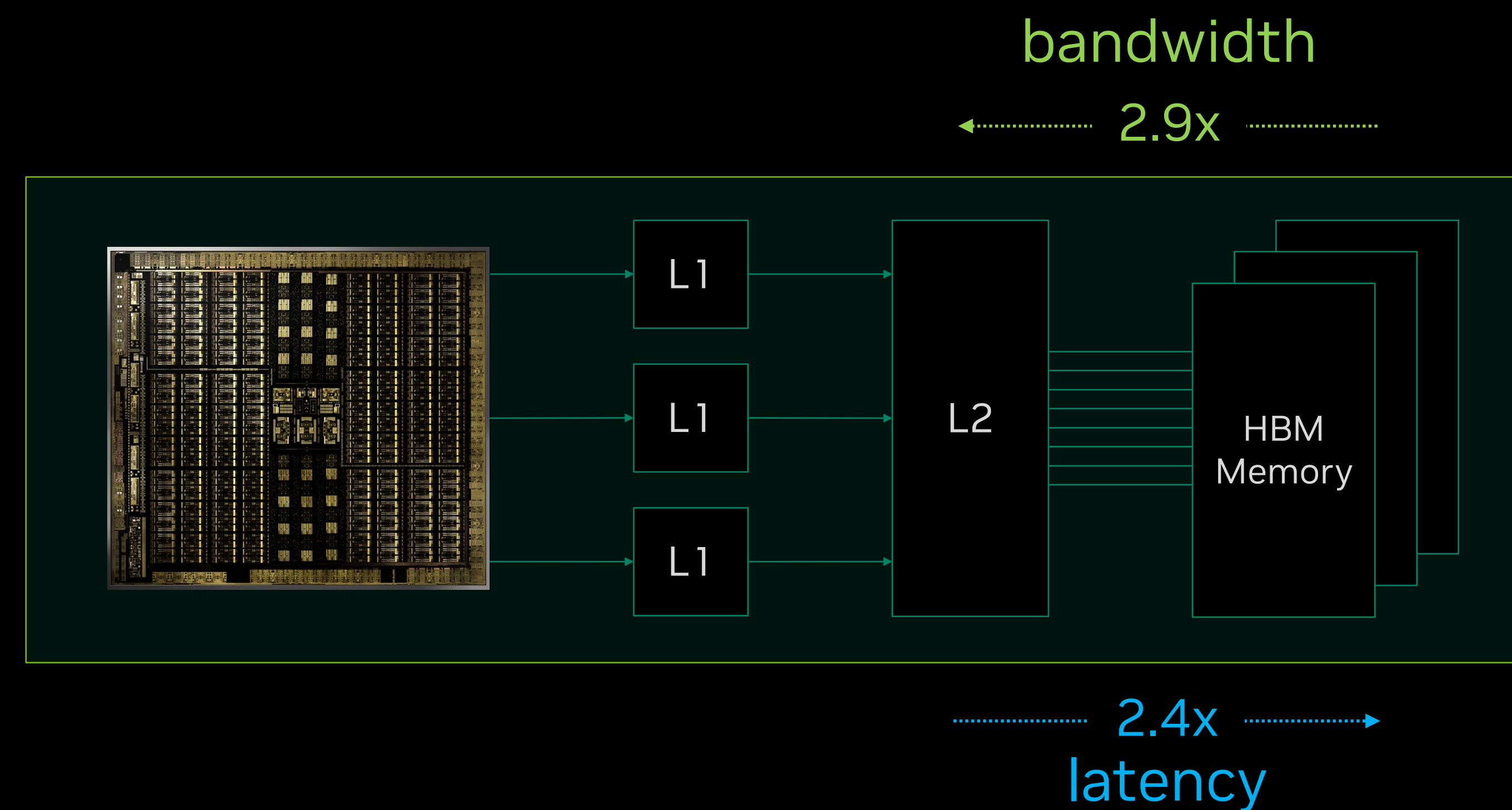
If most programs are limited by memory bandwidth ...



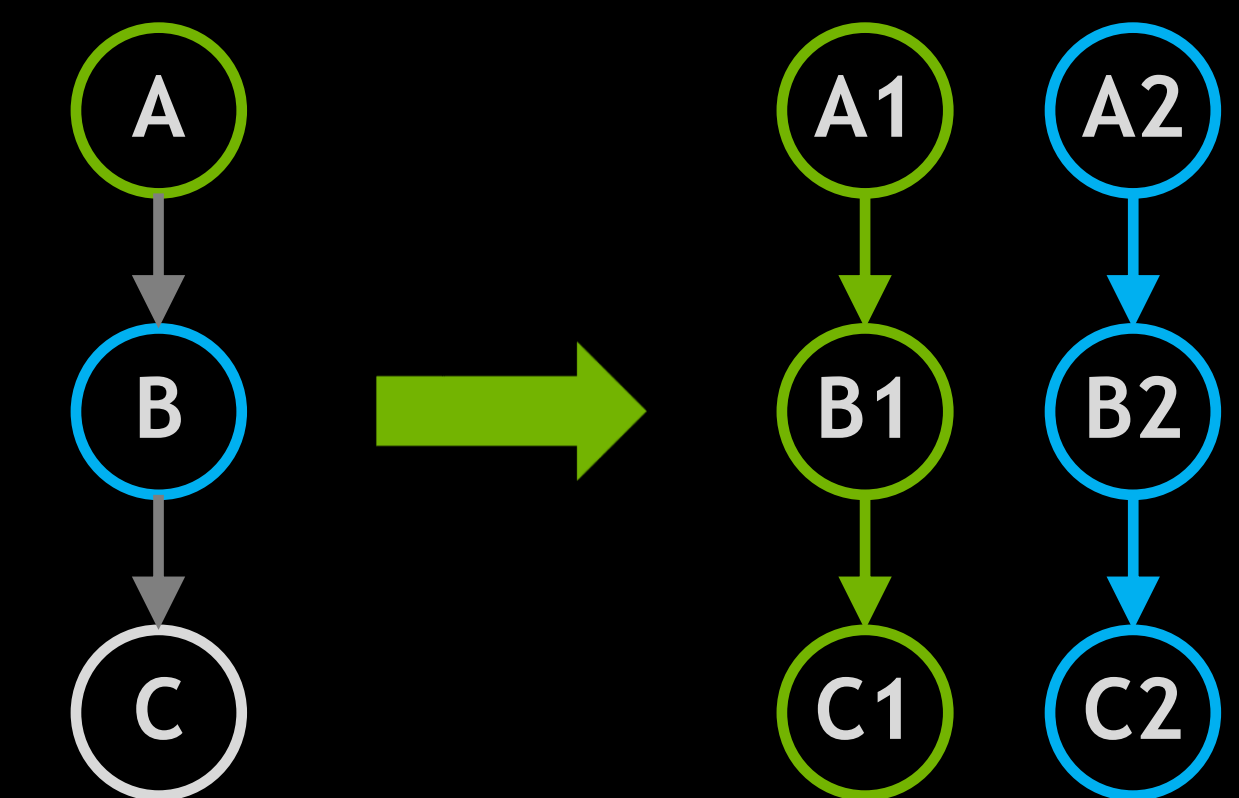
... and L2 cache can run as much as 10x faster than main memory ...



If most programs are limited by memory bandwidth ...



... and L2 cache can run as much as 10x faster than main memory ...

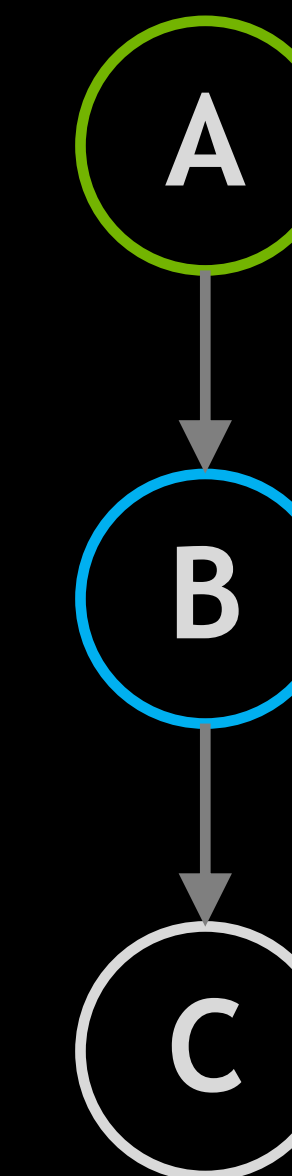
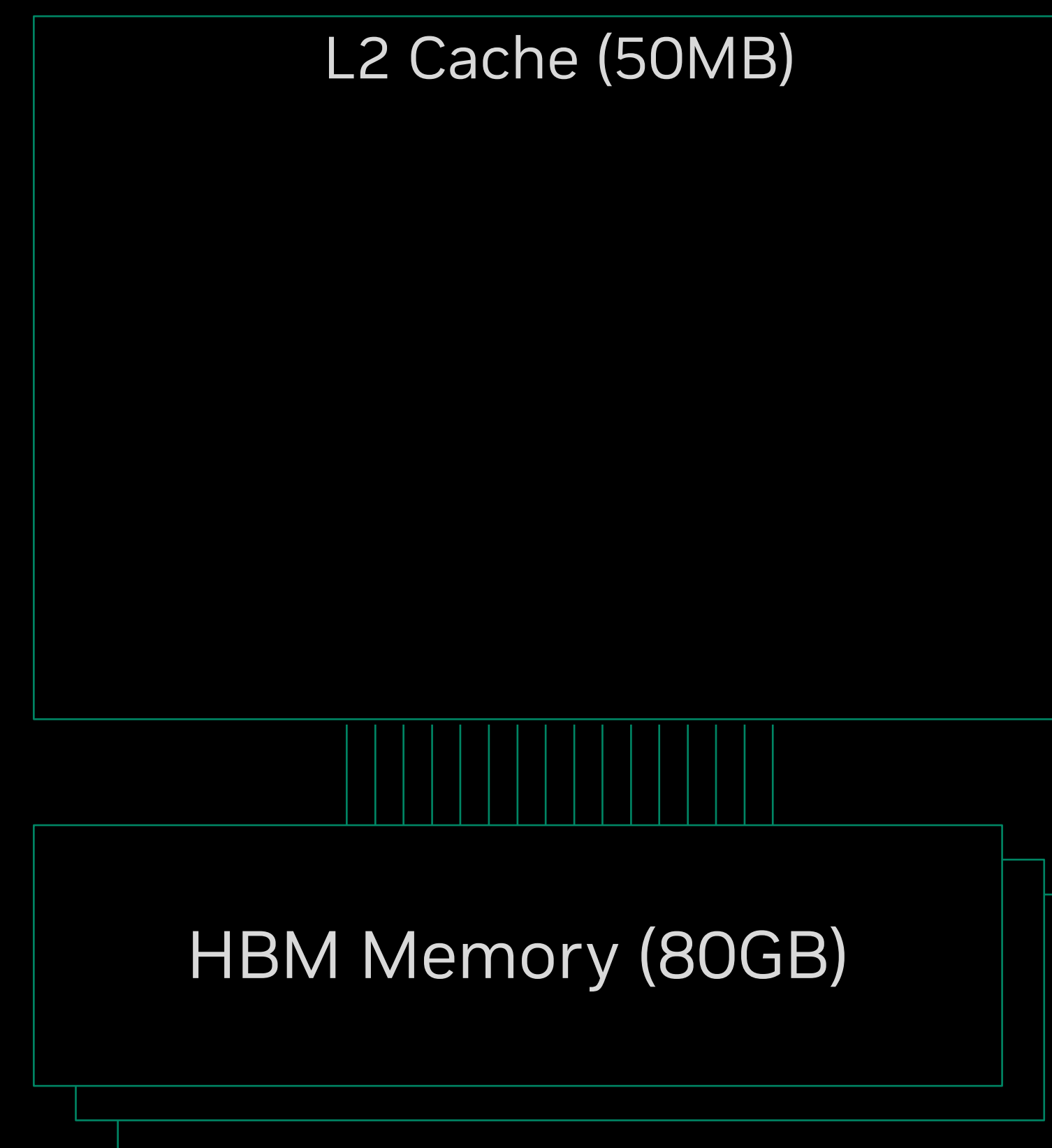
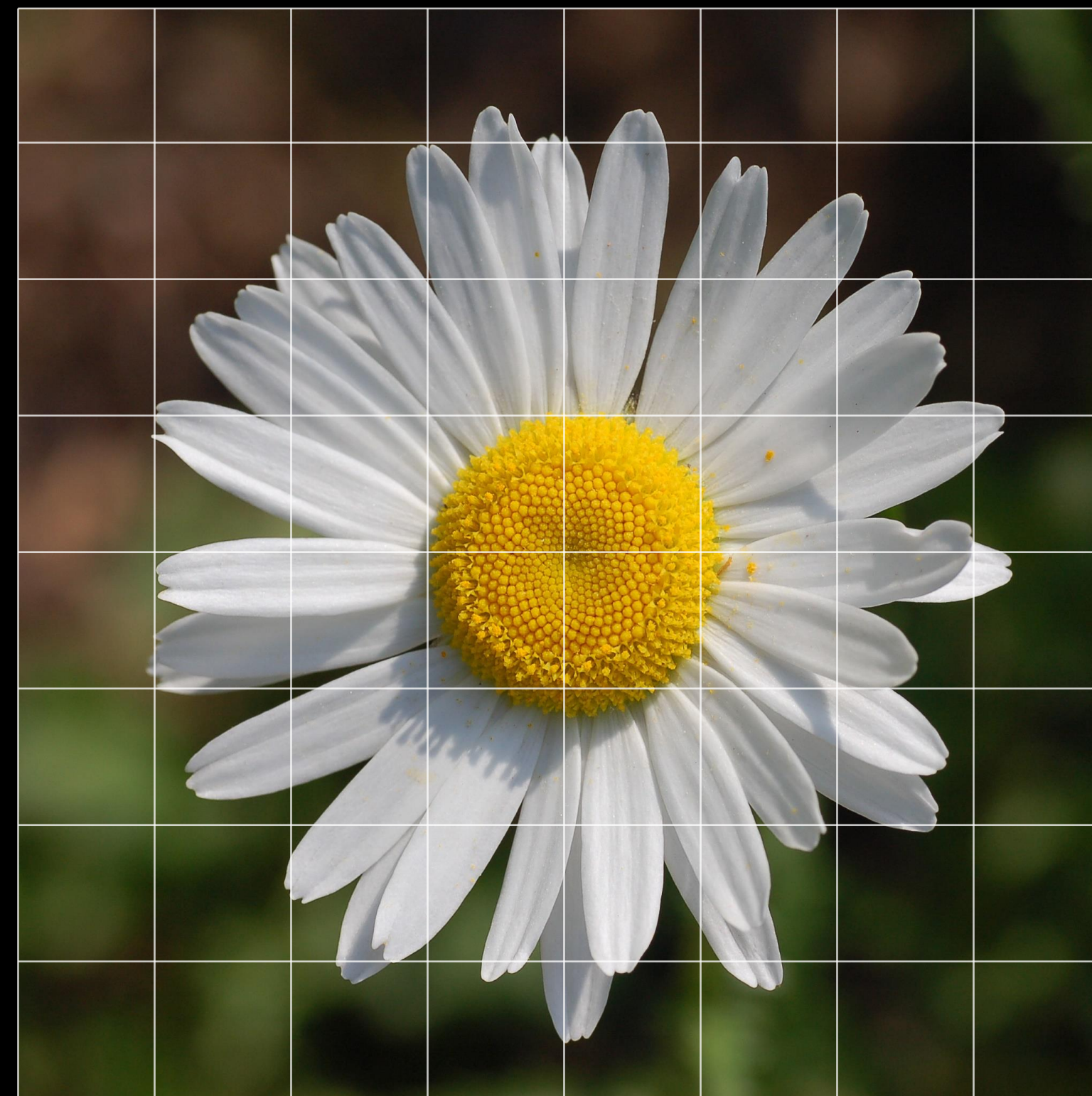


...and I know how to split my program into smaller tasks...

What if my tasks were made small enough to fit entirely in L2 cache?

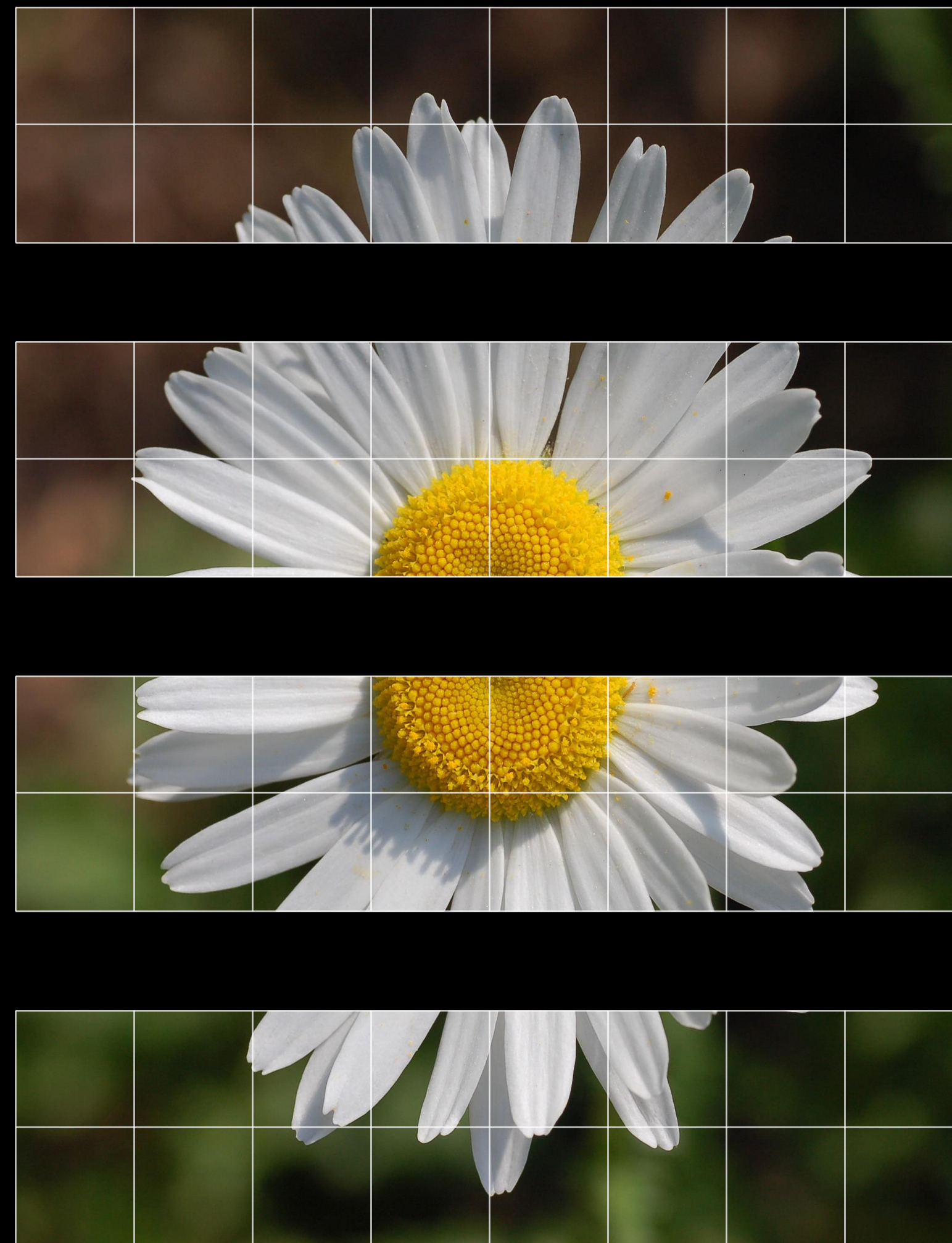
What If My Tasks Were Made Small Enough to Fit Entirely in L2 Cache?

Not all programs can do this, but more can than you might think...

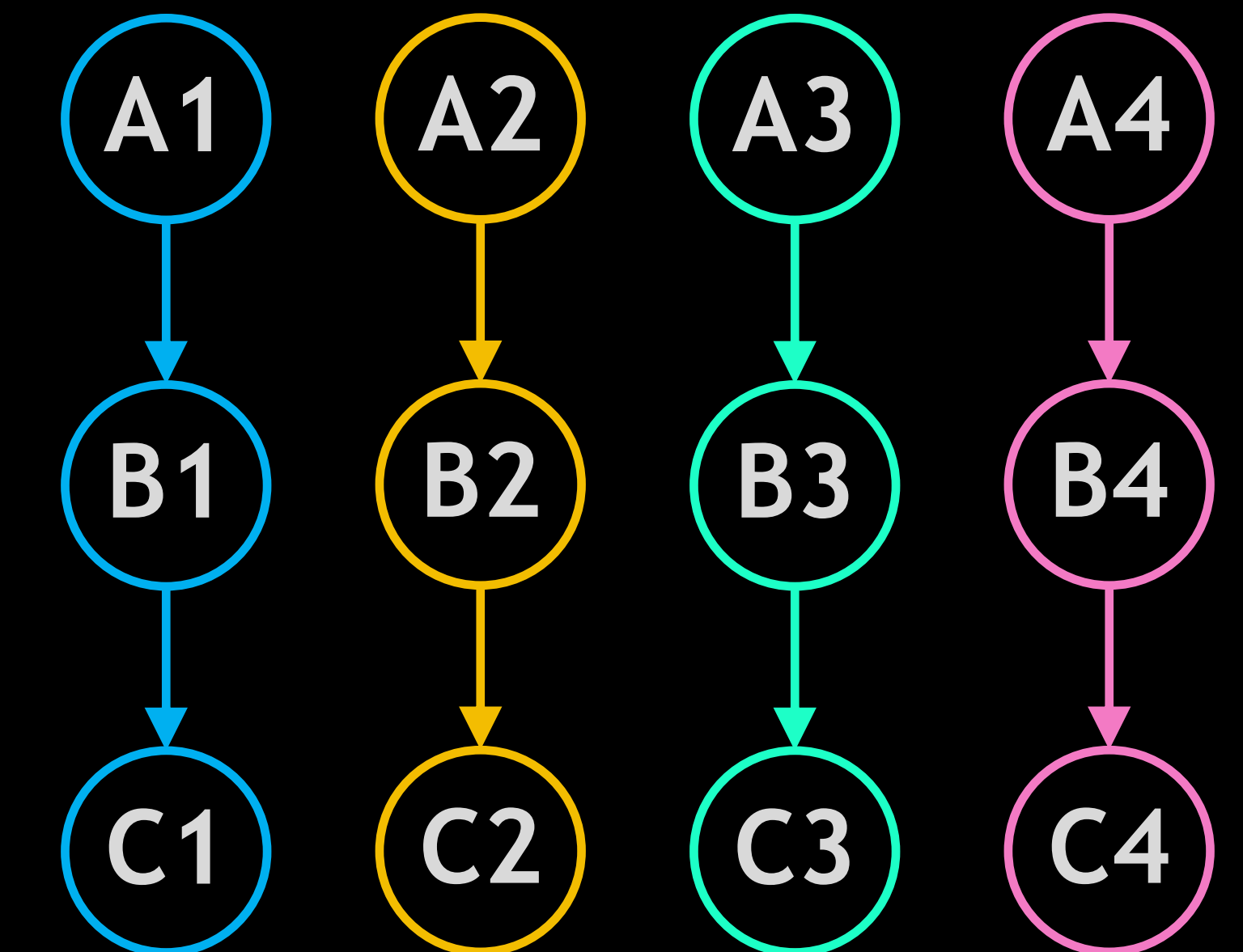
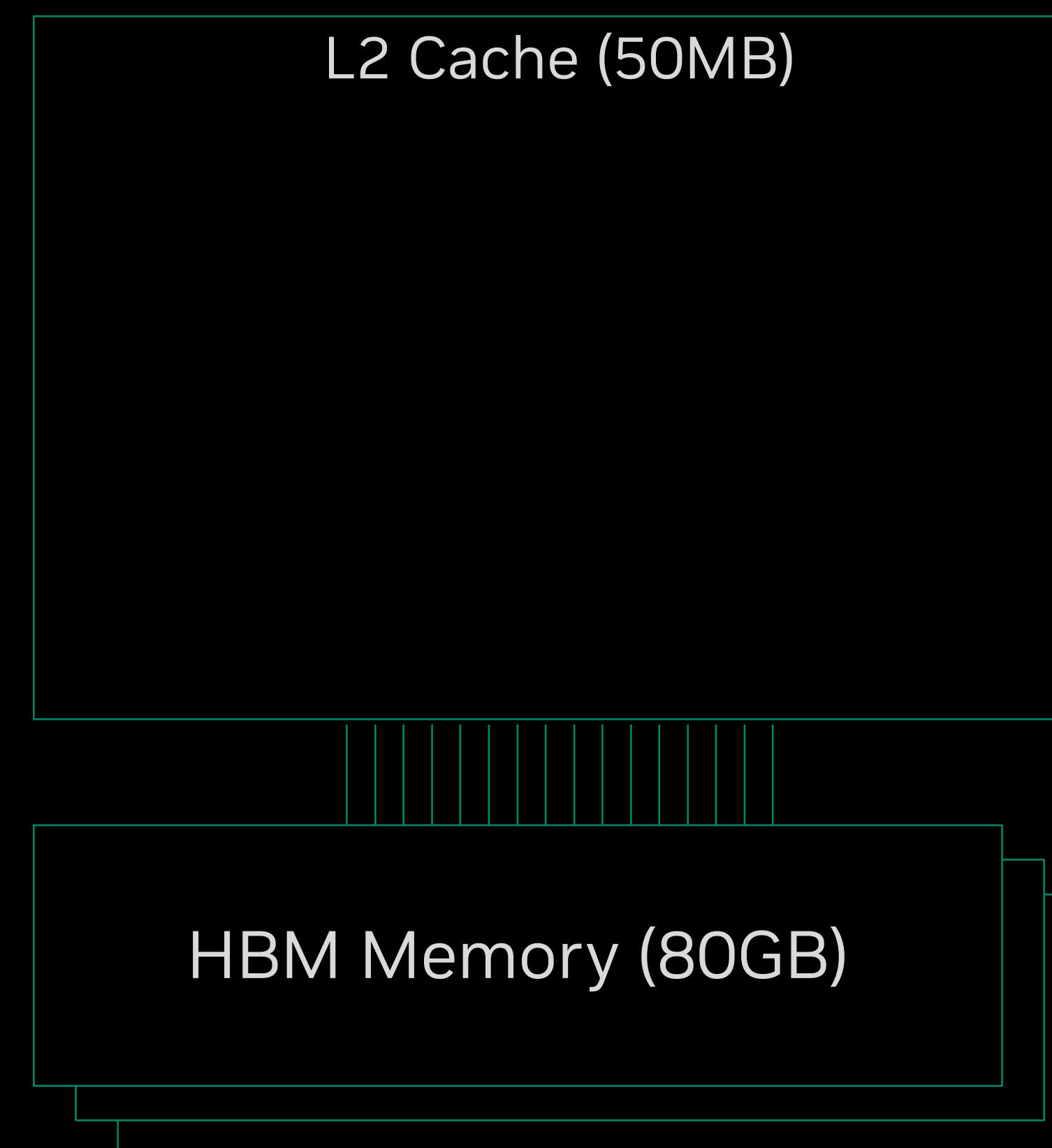


What If My Tasks Were Made Small Enough to Fit Entirely in L2 Cache?

Not all programs can do this, but more can than you might think...

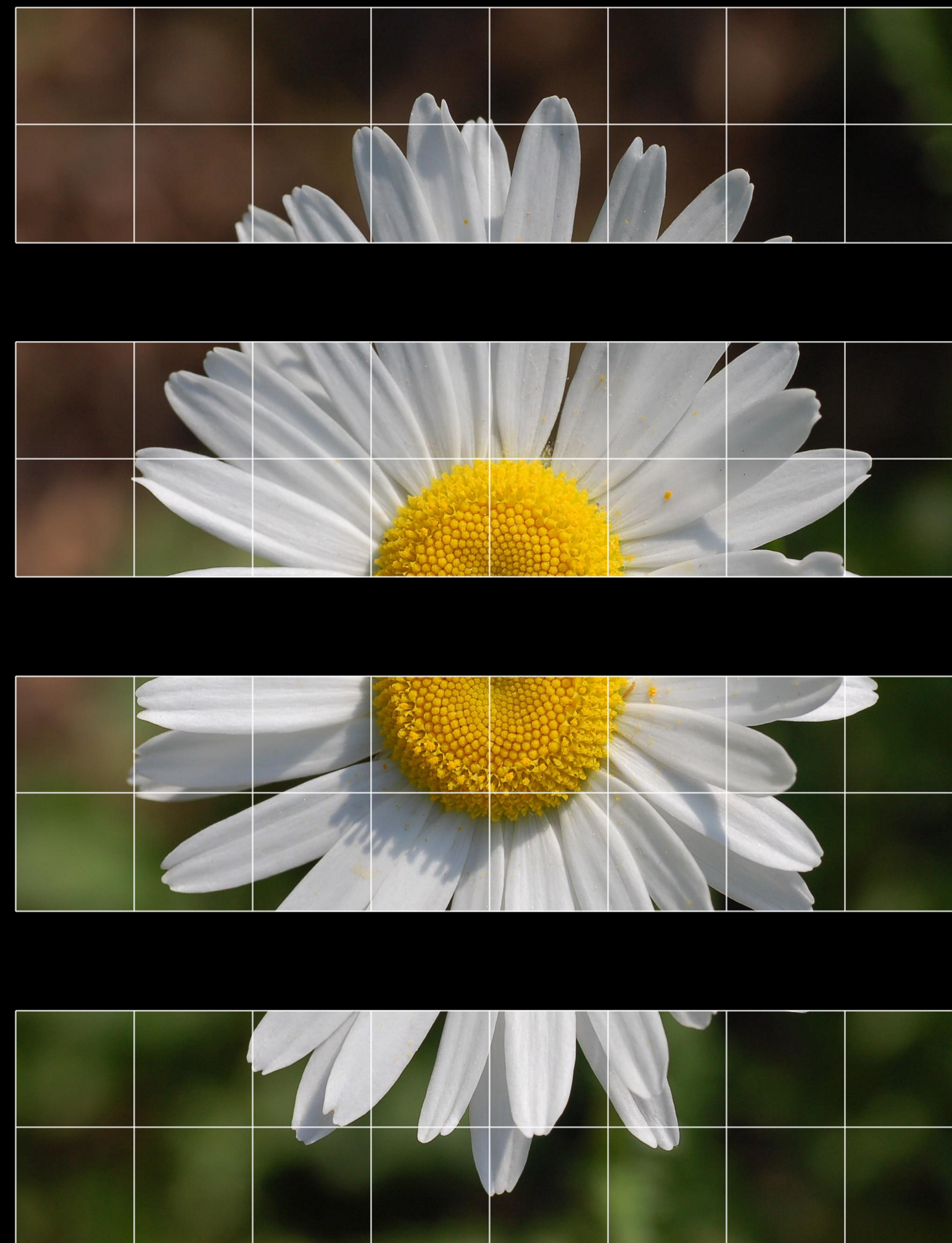


Split data into cache-sized chunks

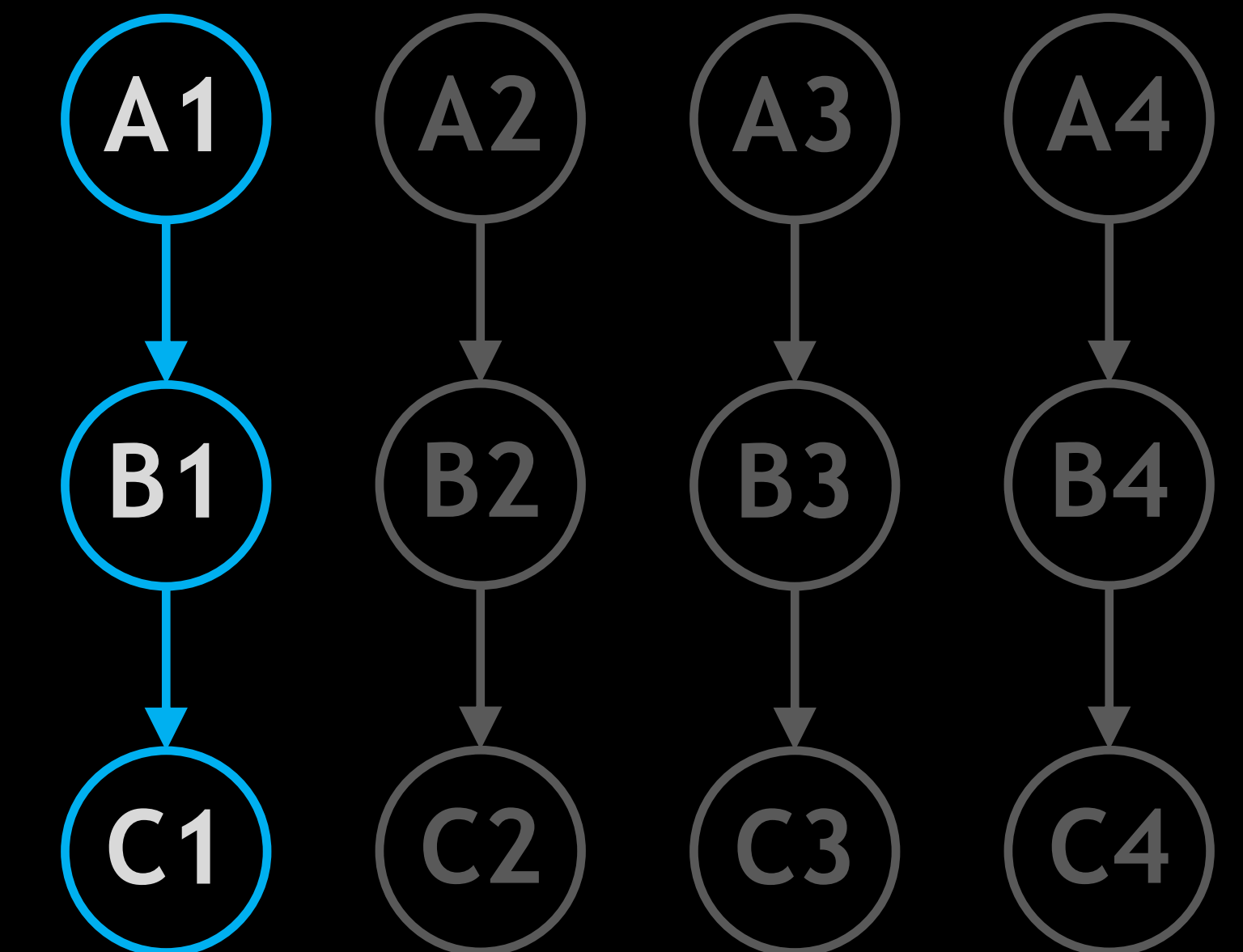
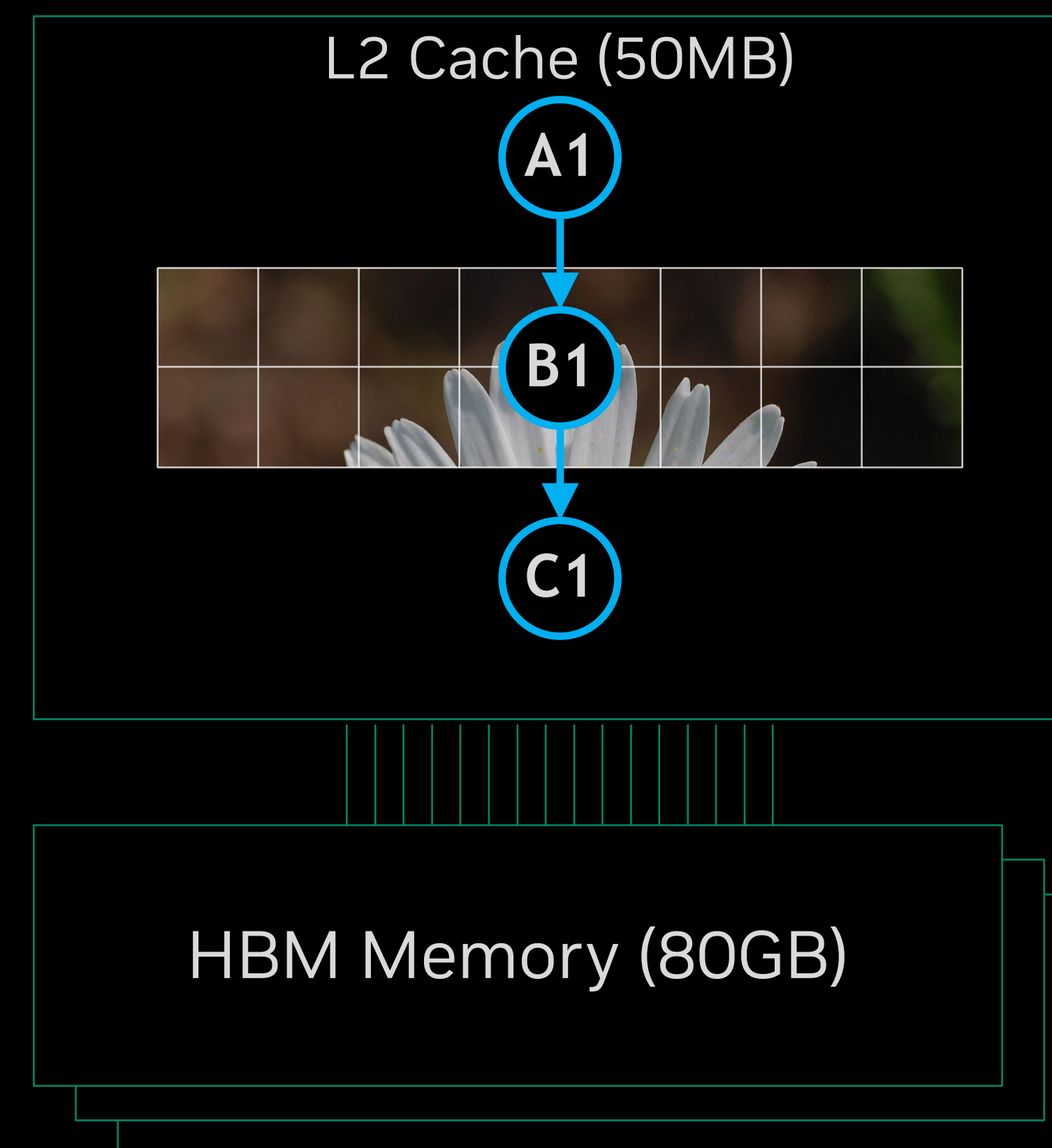


We'll have one task per chunk

To Keep Data in Cache We Run Each Task in Series, **NOT** in Parallel

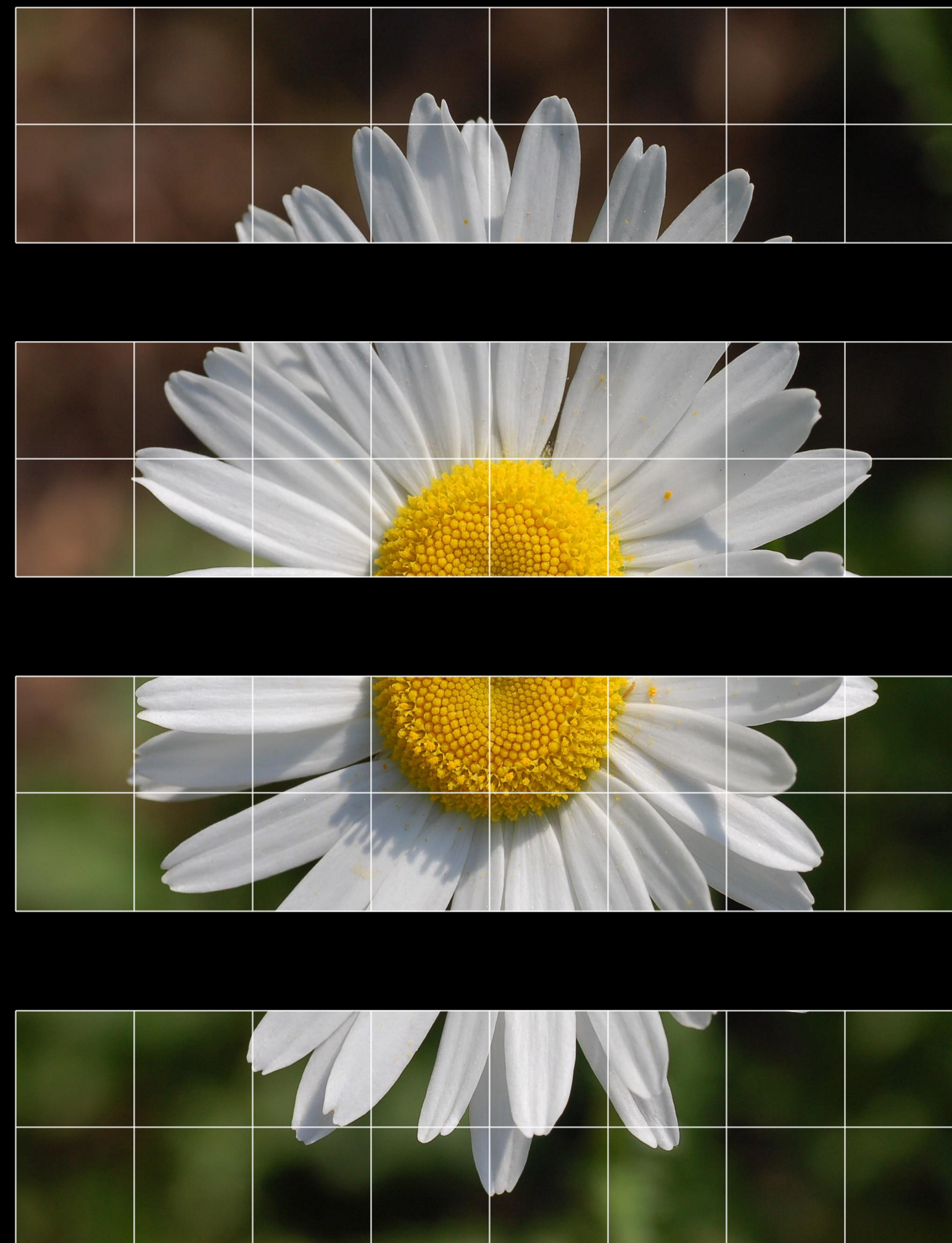


Split data into cache-sized chunks

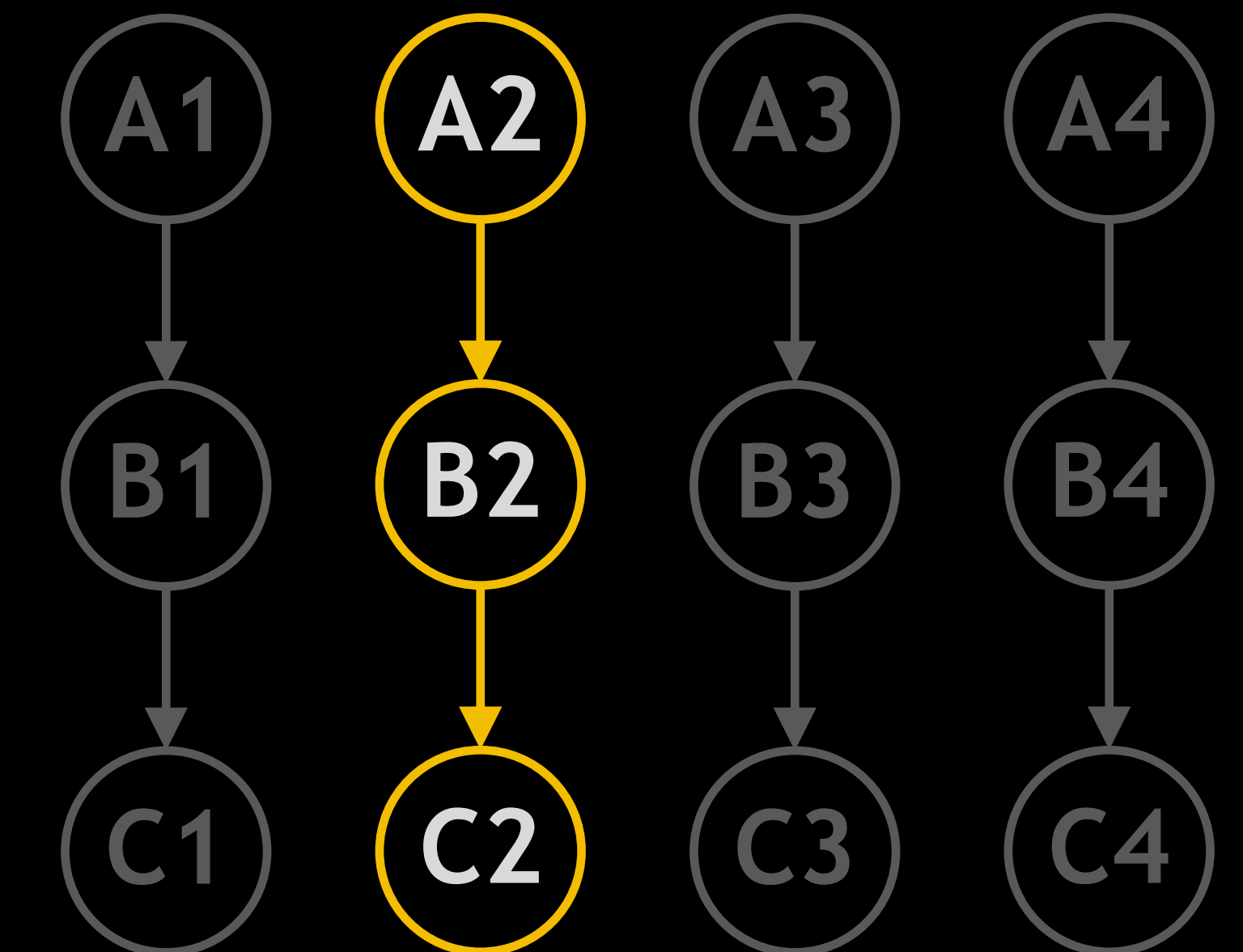
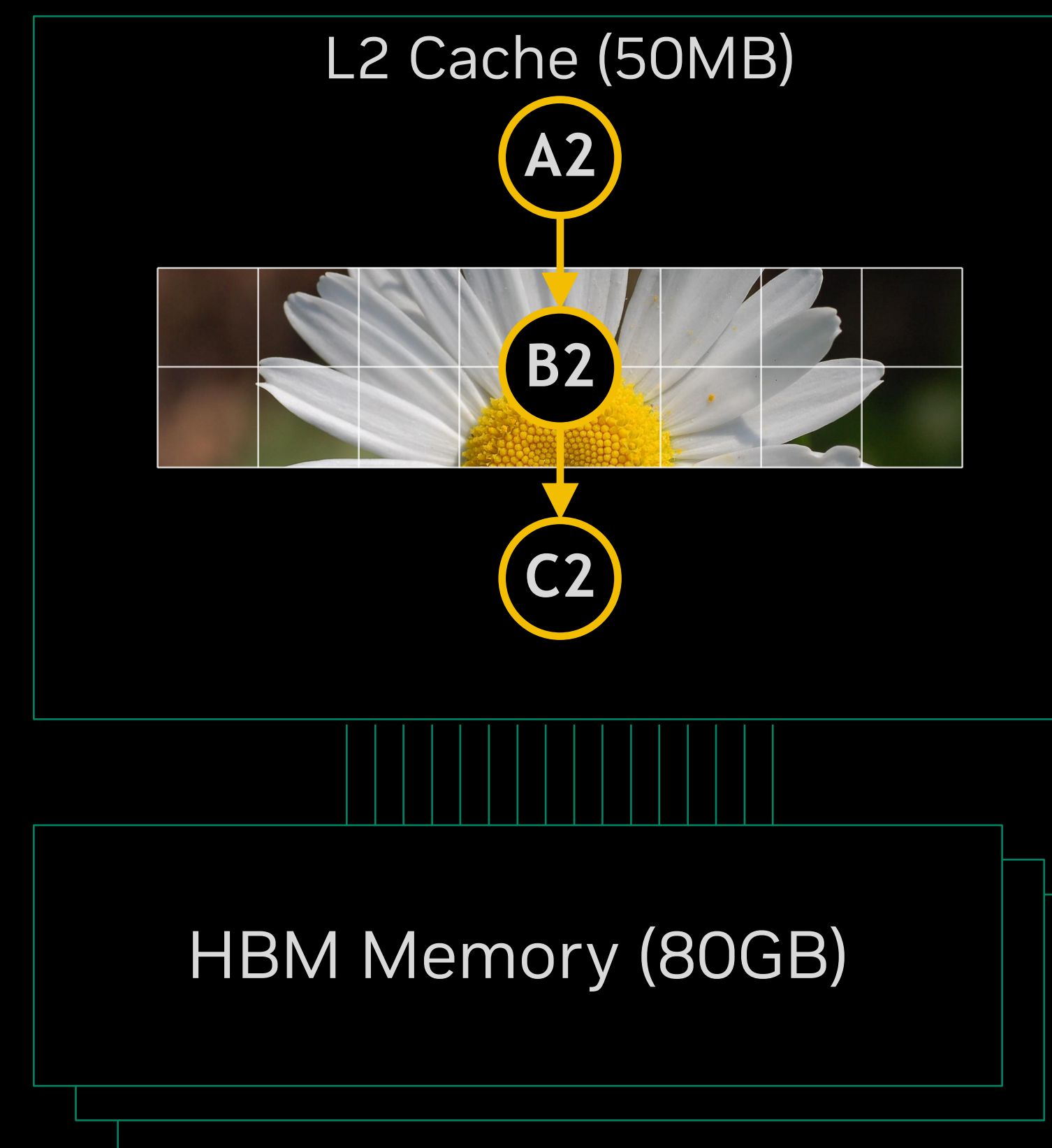


We'll have one task per chunk

To Keep Data in Cache We Run Each Task in Series, **NOT** in Parallel

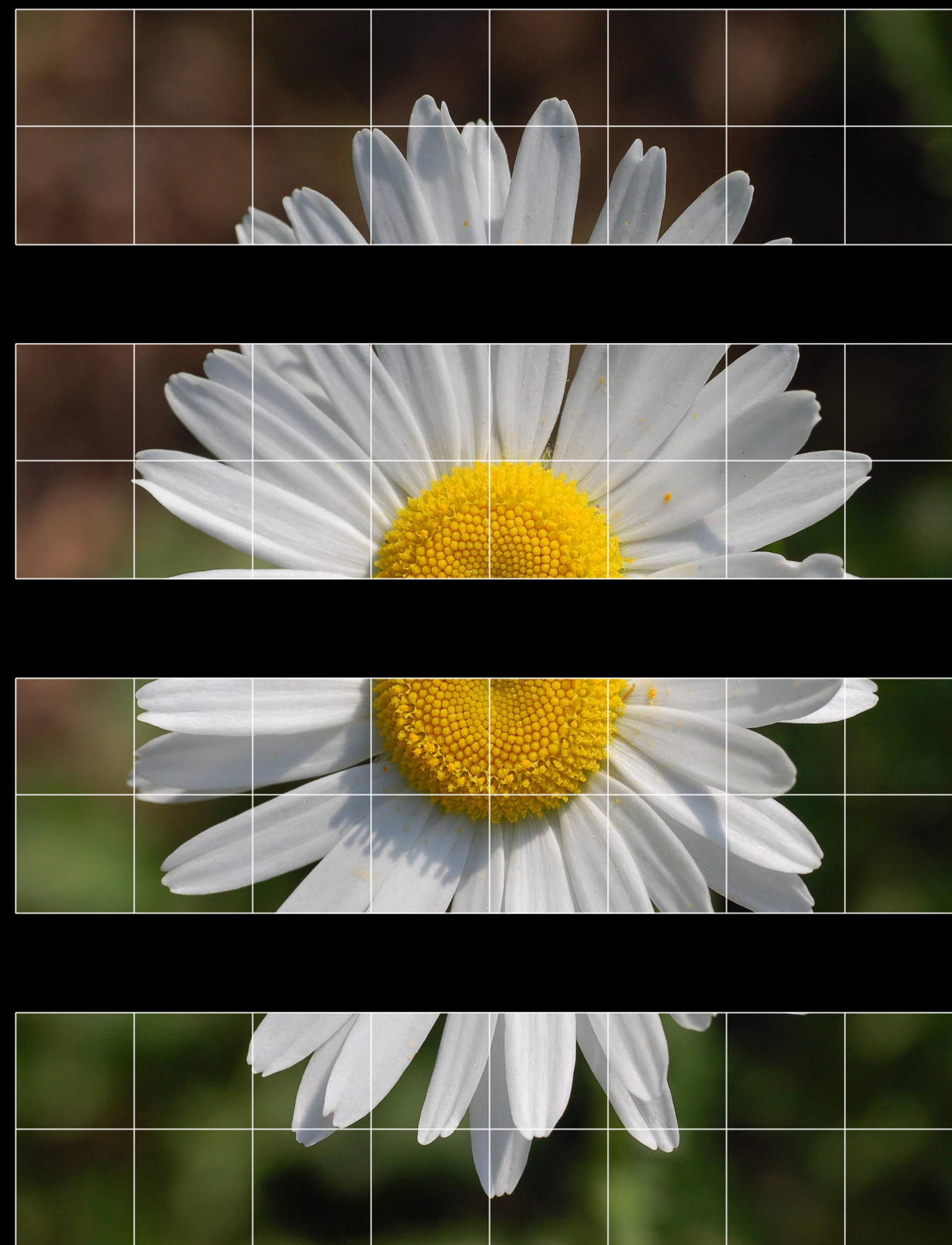


Split data into cache-sized chunks

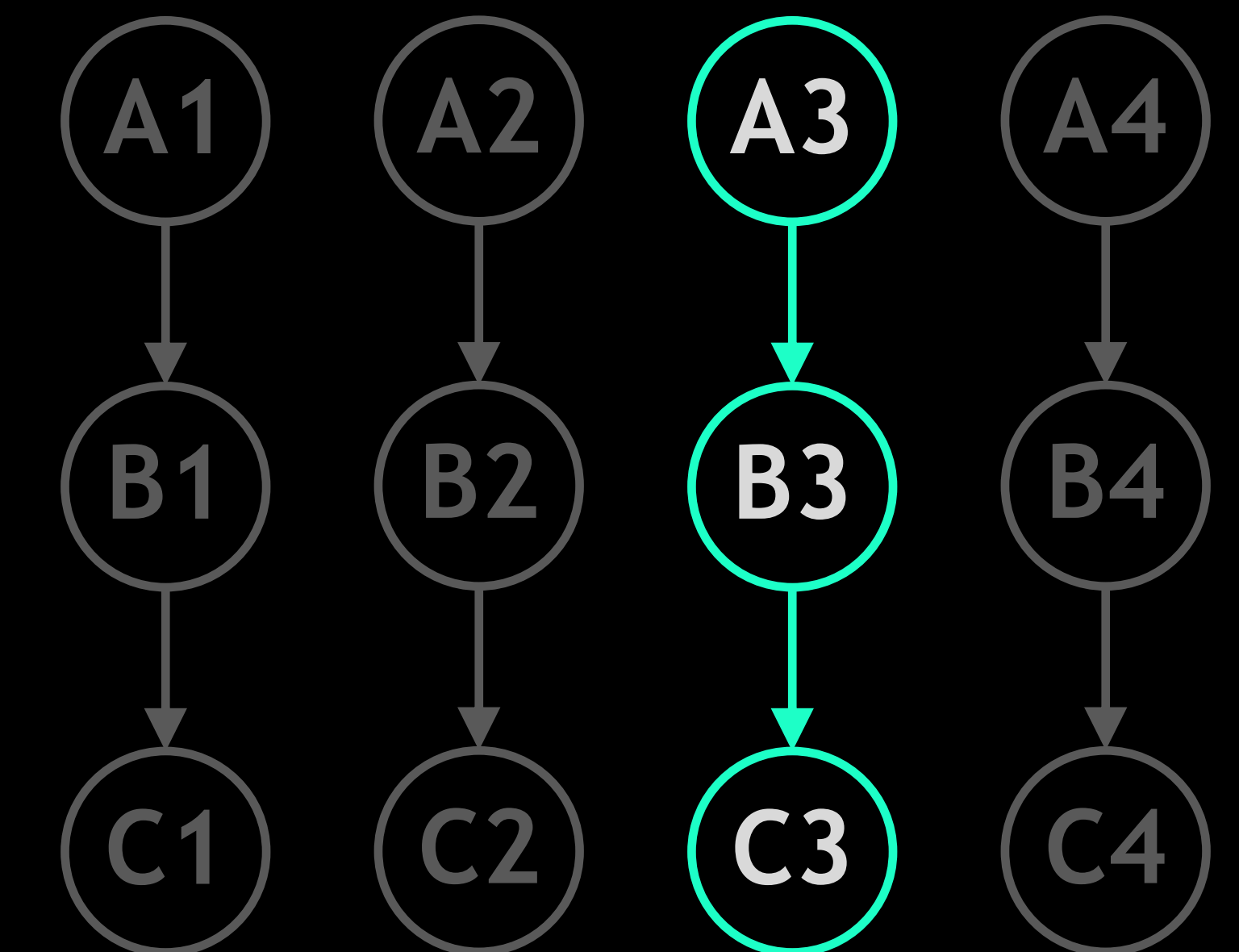
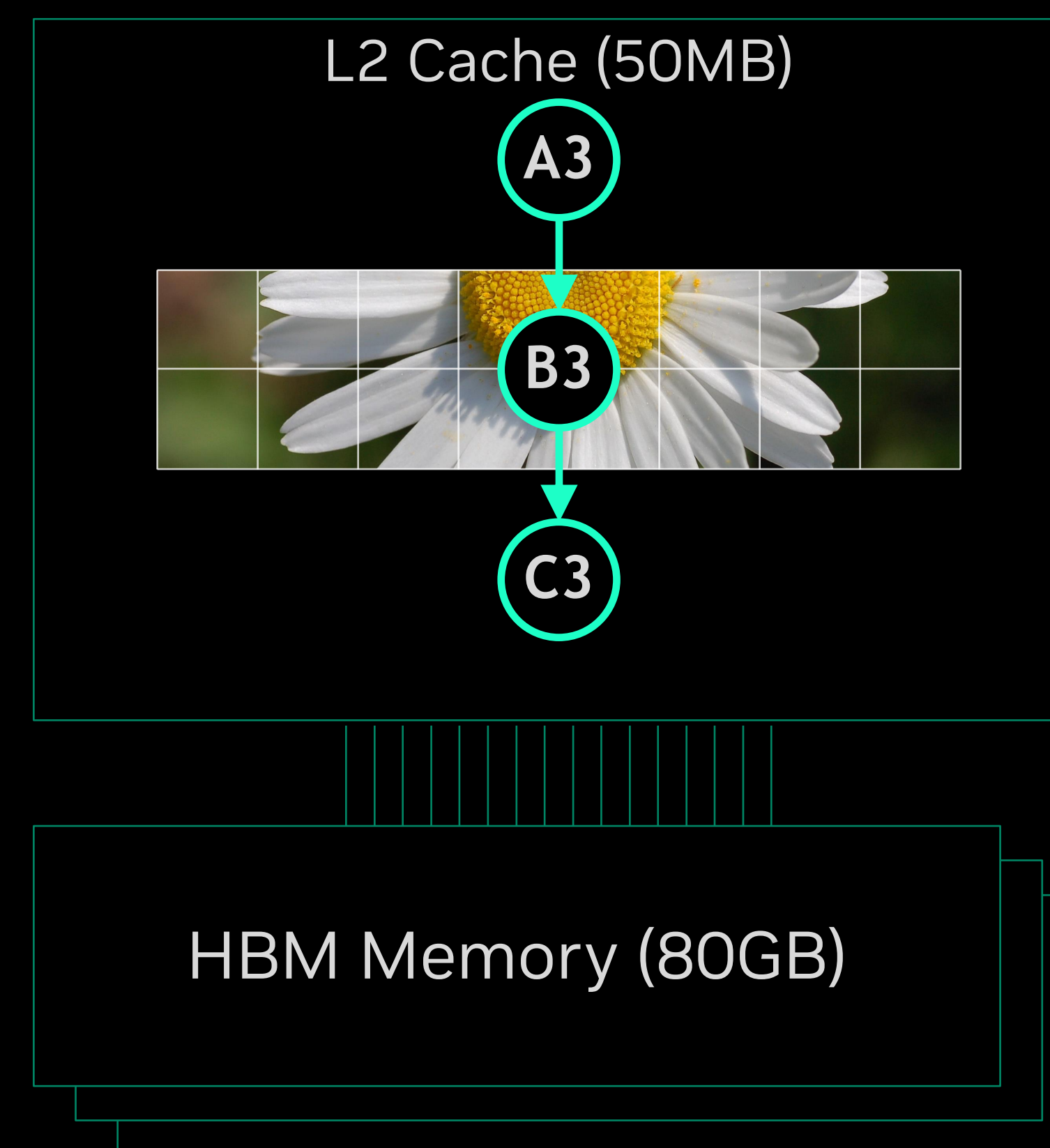


We'll have one task per chunk

To Keep Data in Cache We Run Each Task in Series, **NOT** in Parallel

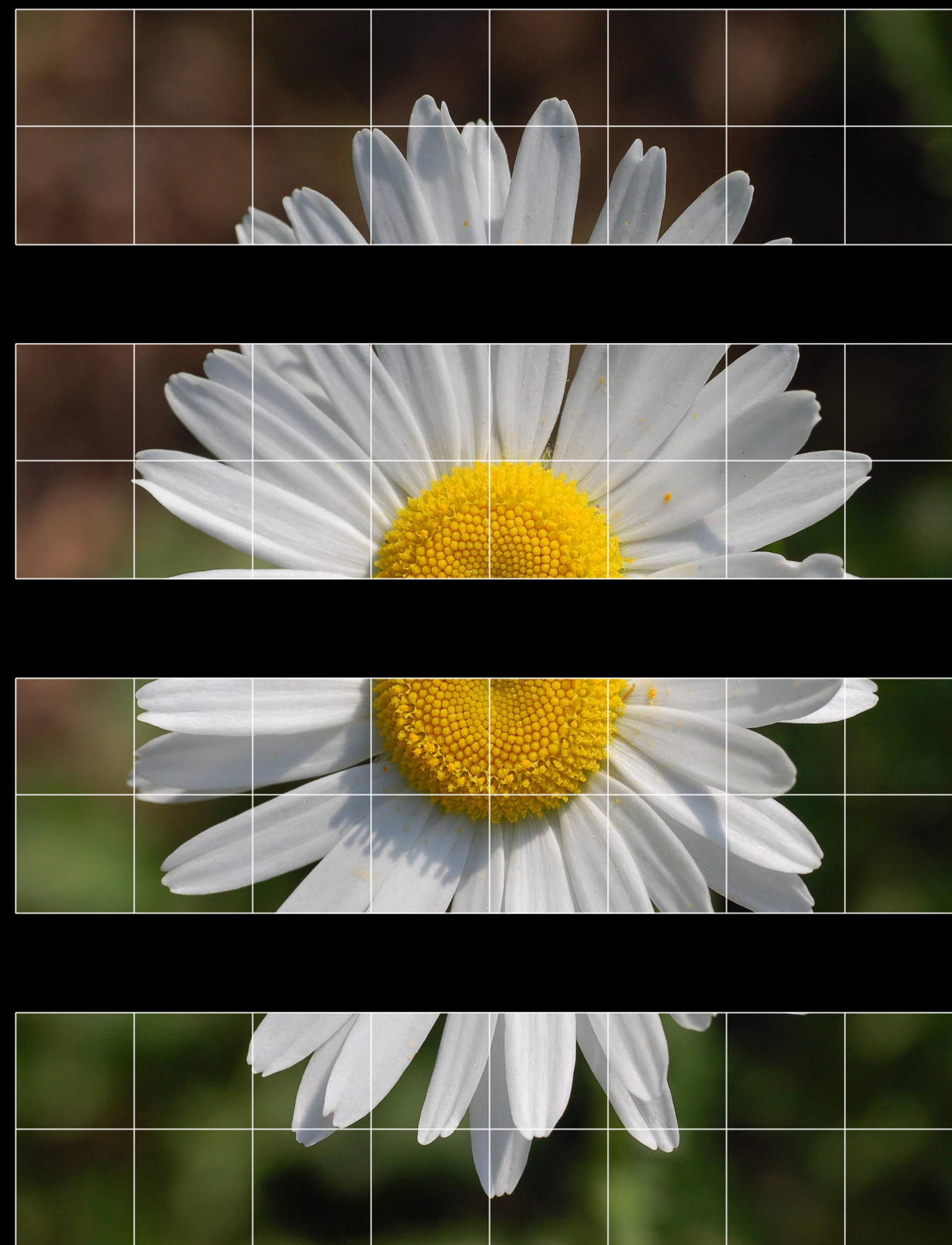


Split data into cache-sized chunks

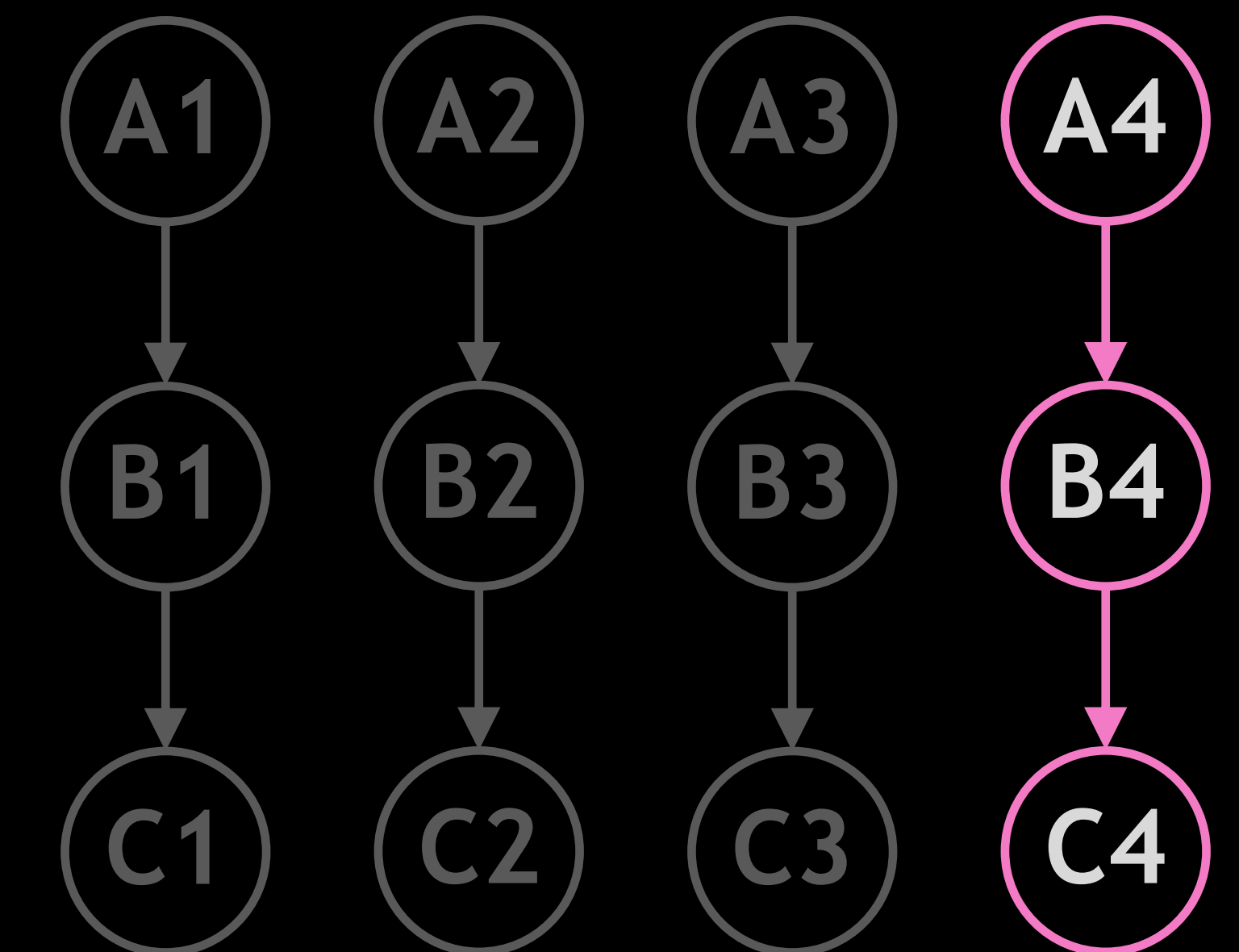
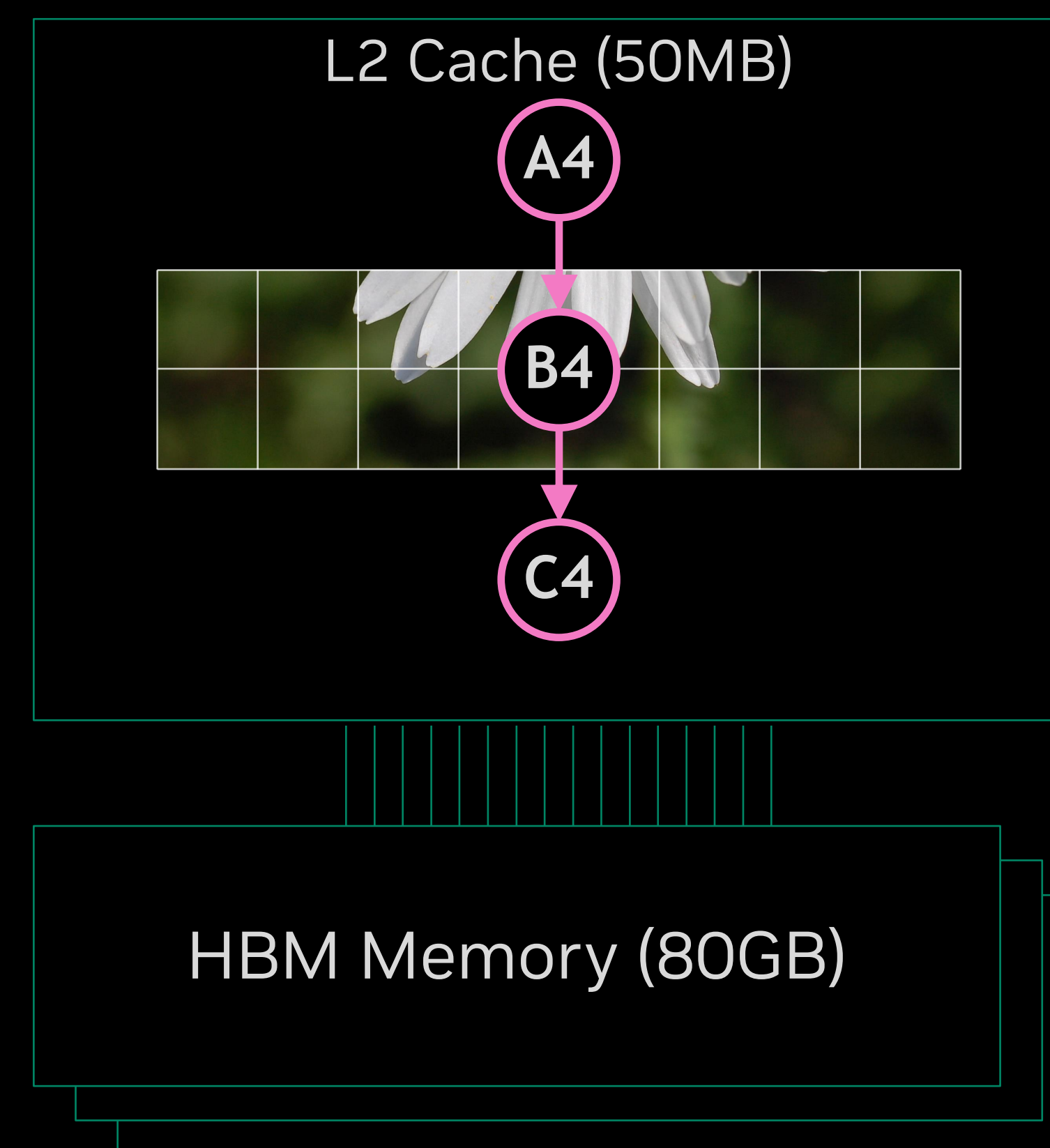


We'll have one task per chunk

To Keep Data in Cache We Run Each Task in Series, **NOT** in Parallel



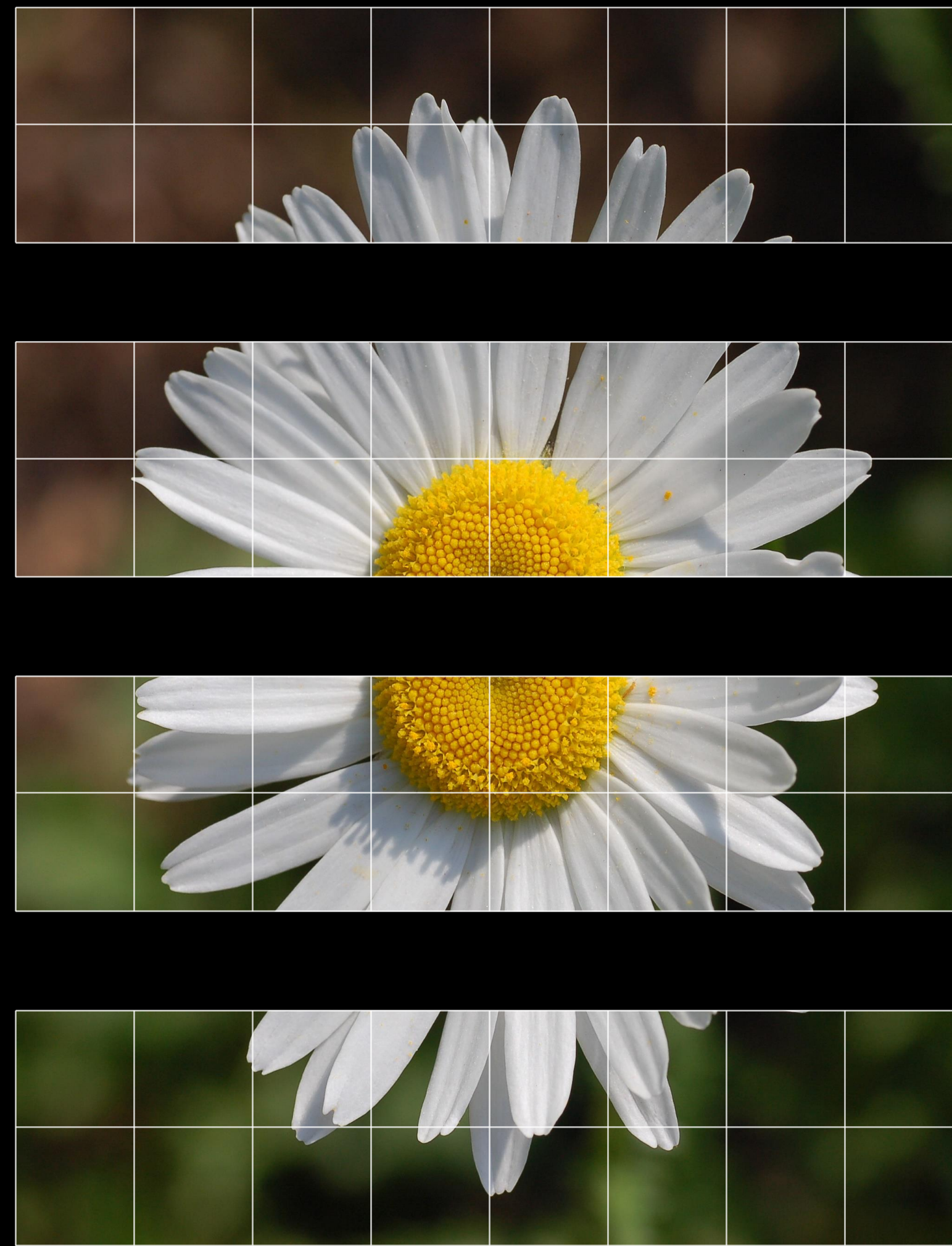
Split data into cache-sized chunks



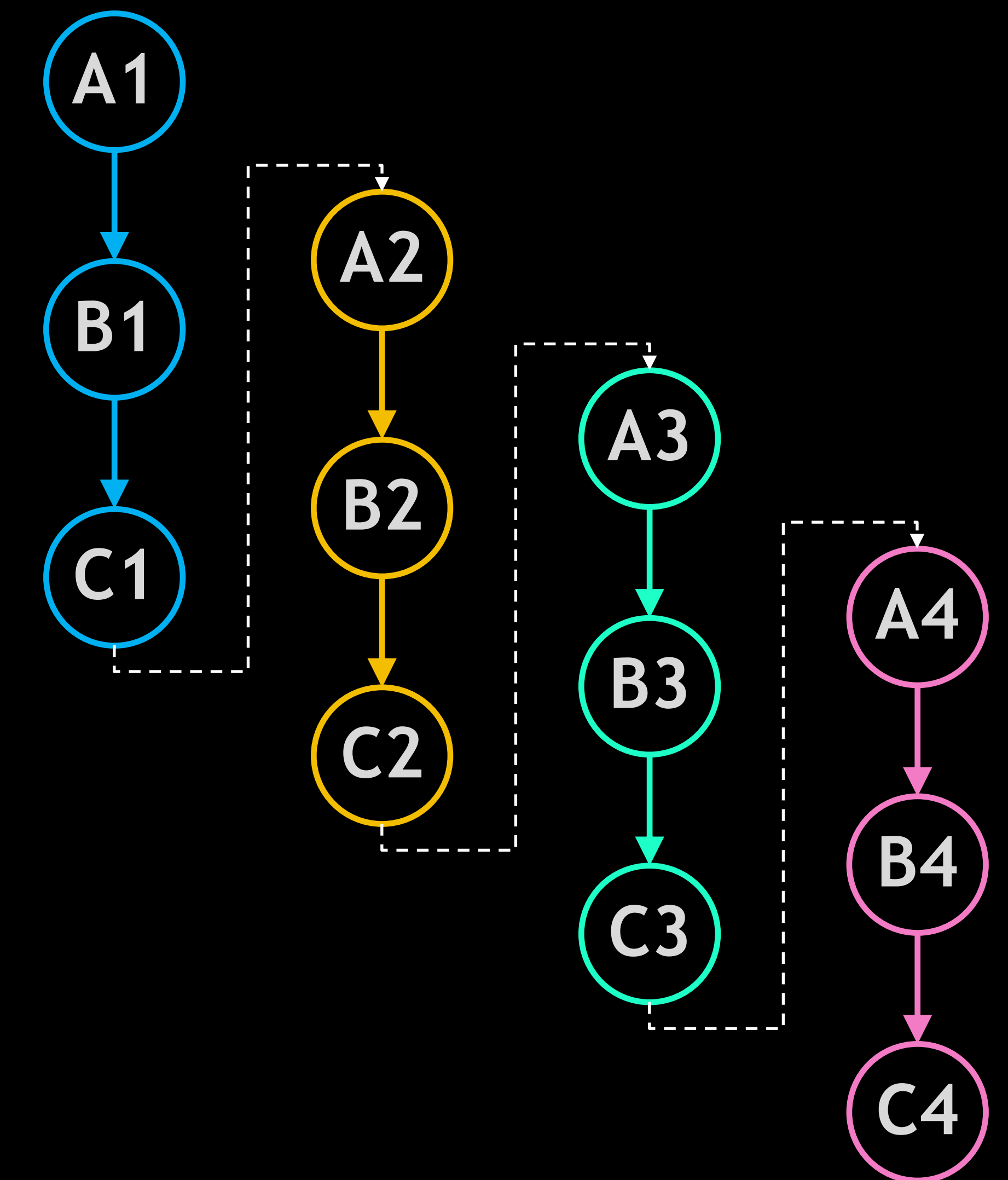
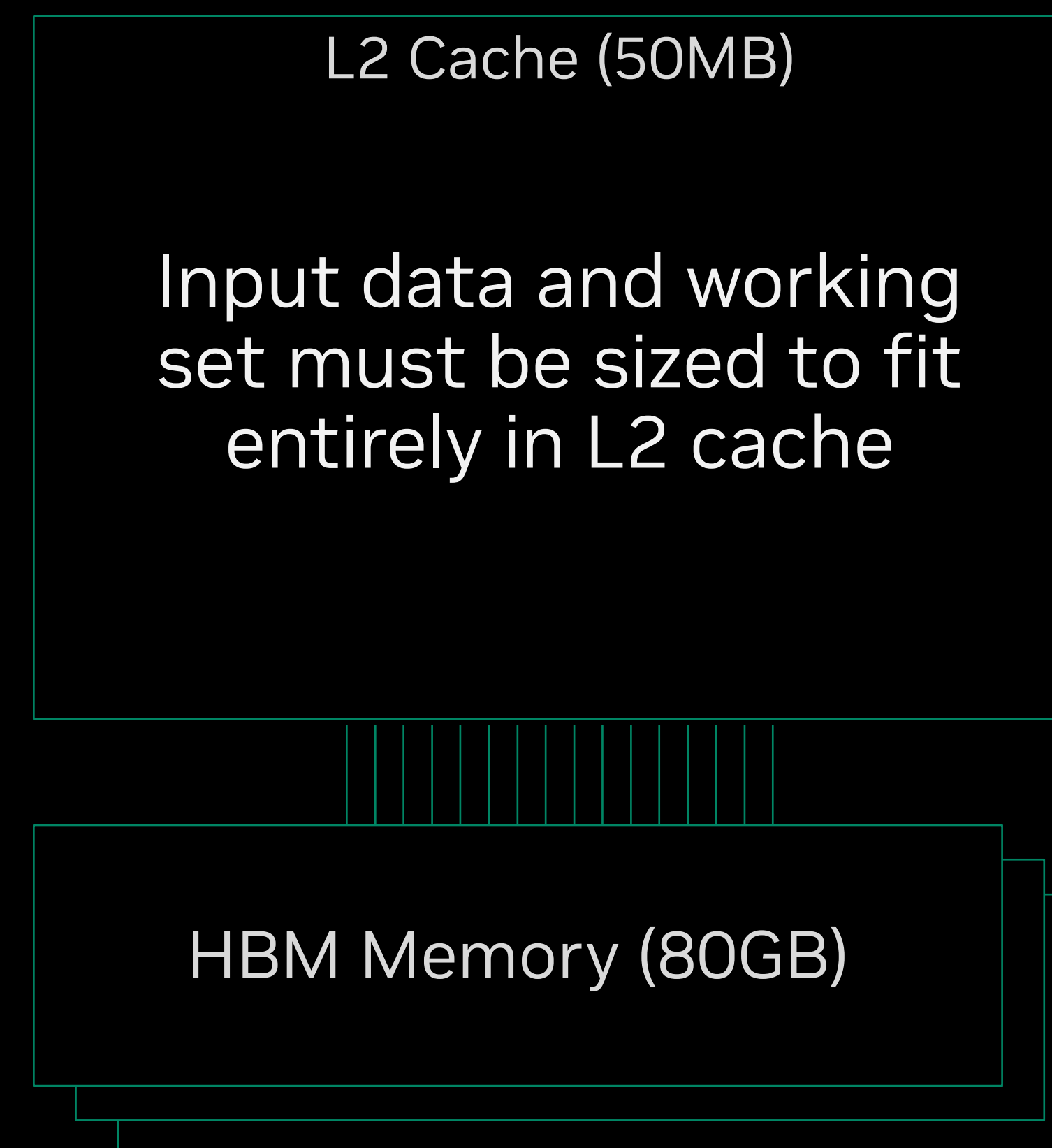
We'll have one task per chunk

This is Known As “Tiling” Your Execution in Cache

You'll really want to design your program for this up-front



Split data into cache-sized chunks

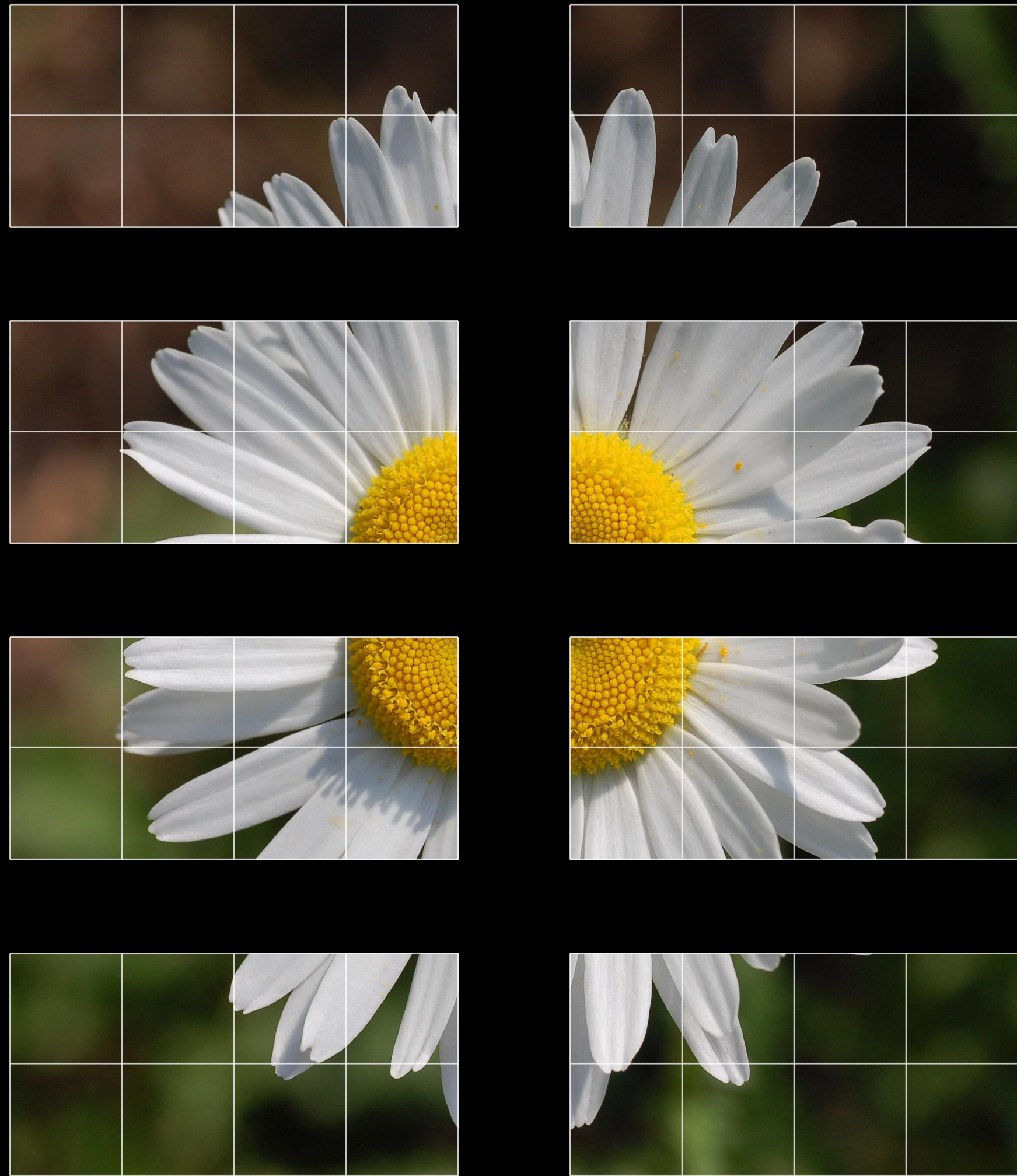


Run tasks in series

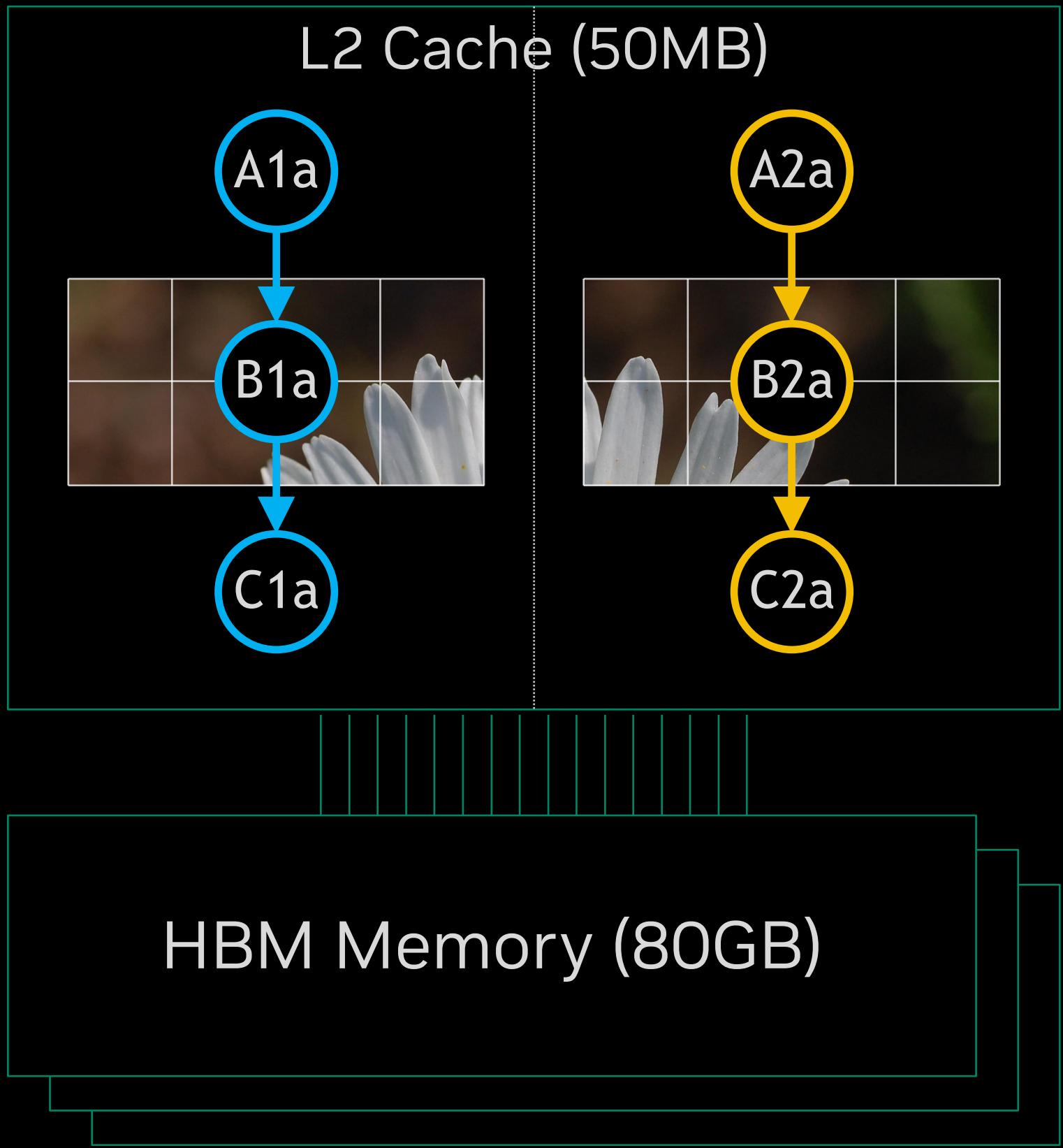
But now we've gone and created wave quantisation again...

So We Can Task-Parallelise Our Task-Based Cache Tiling

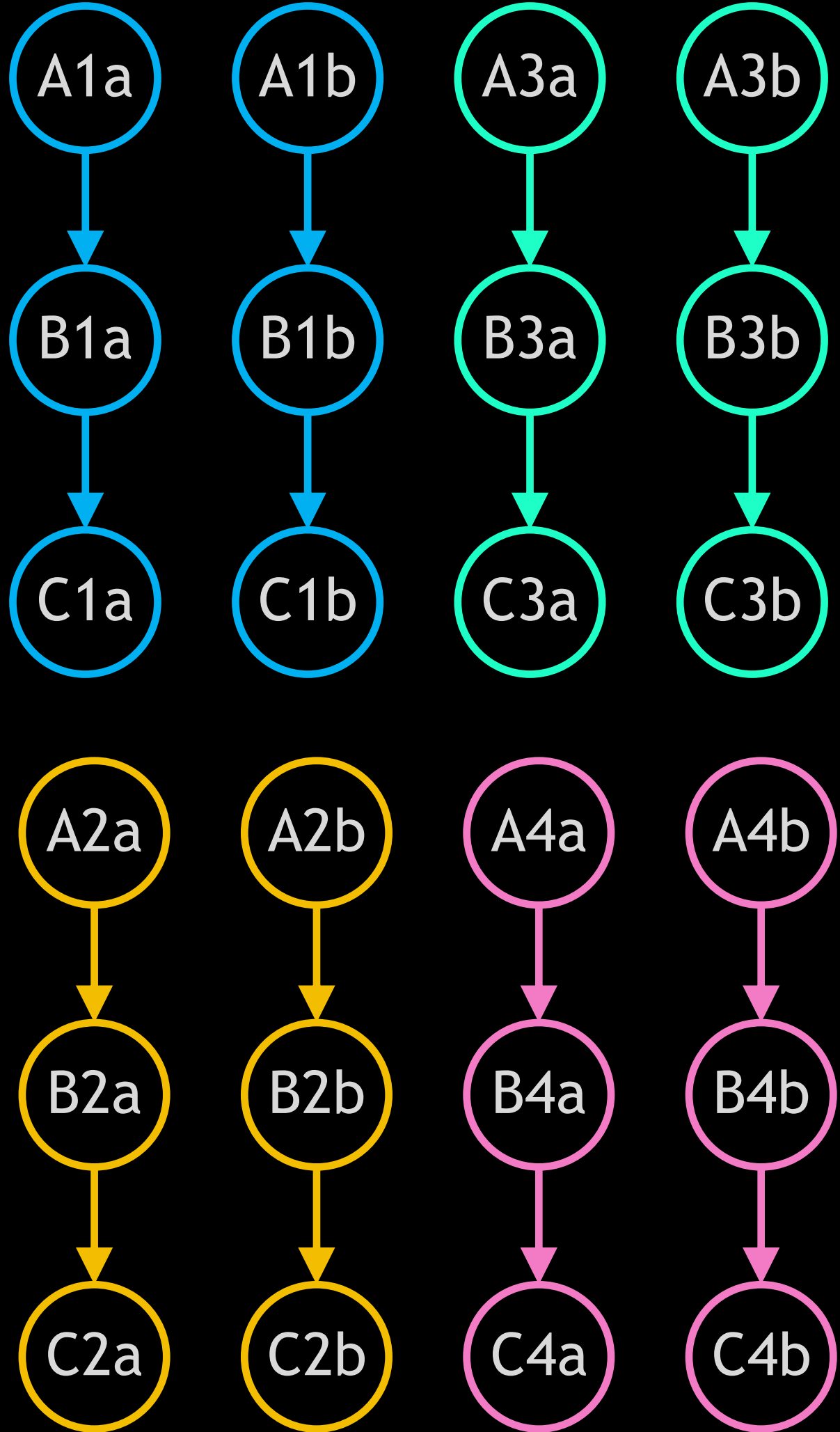
This can get silly pretty fast...



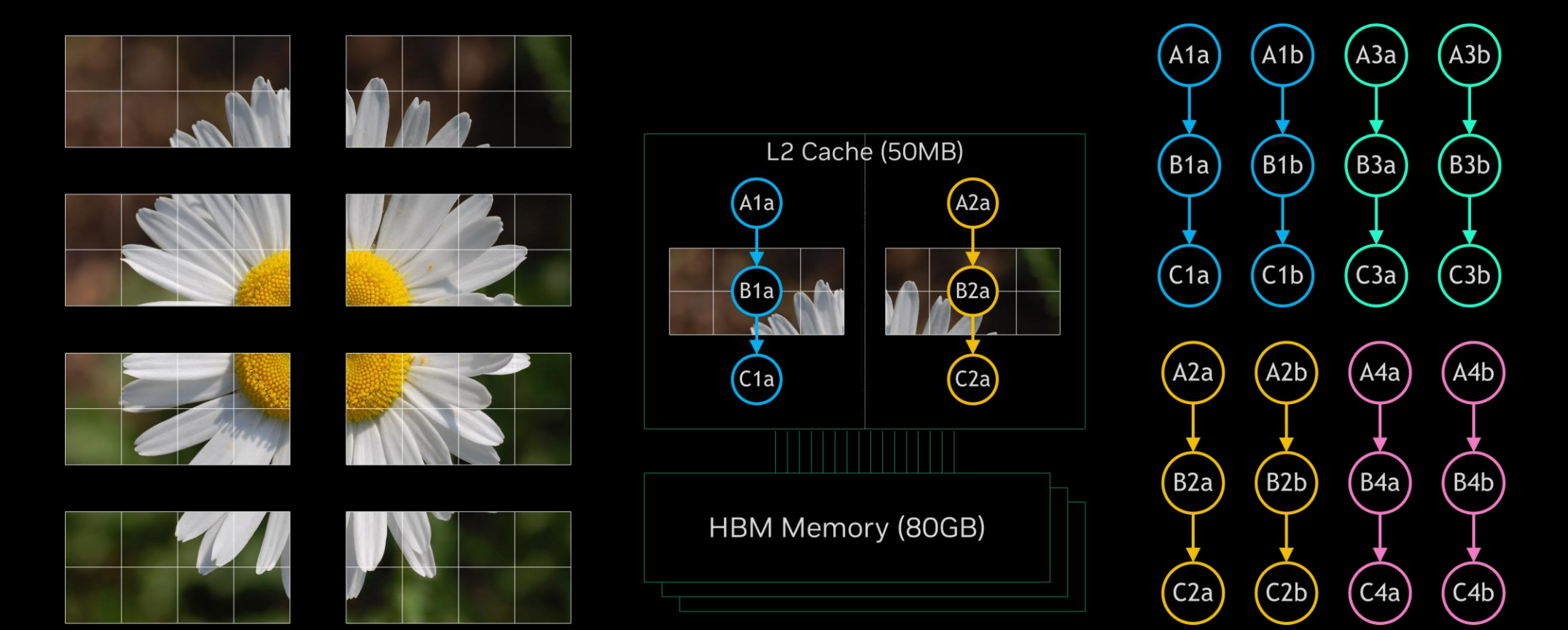
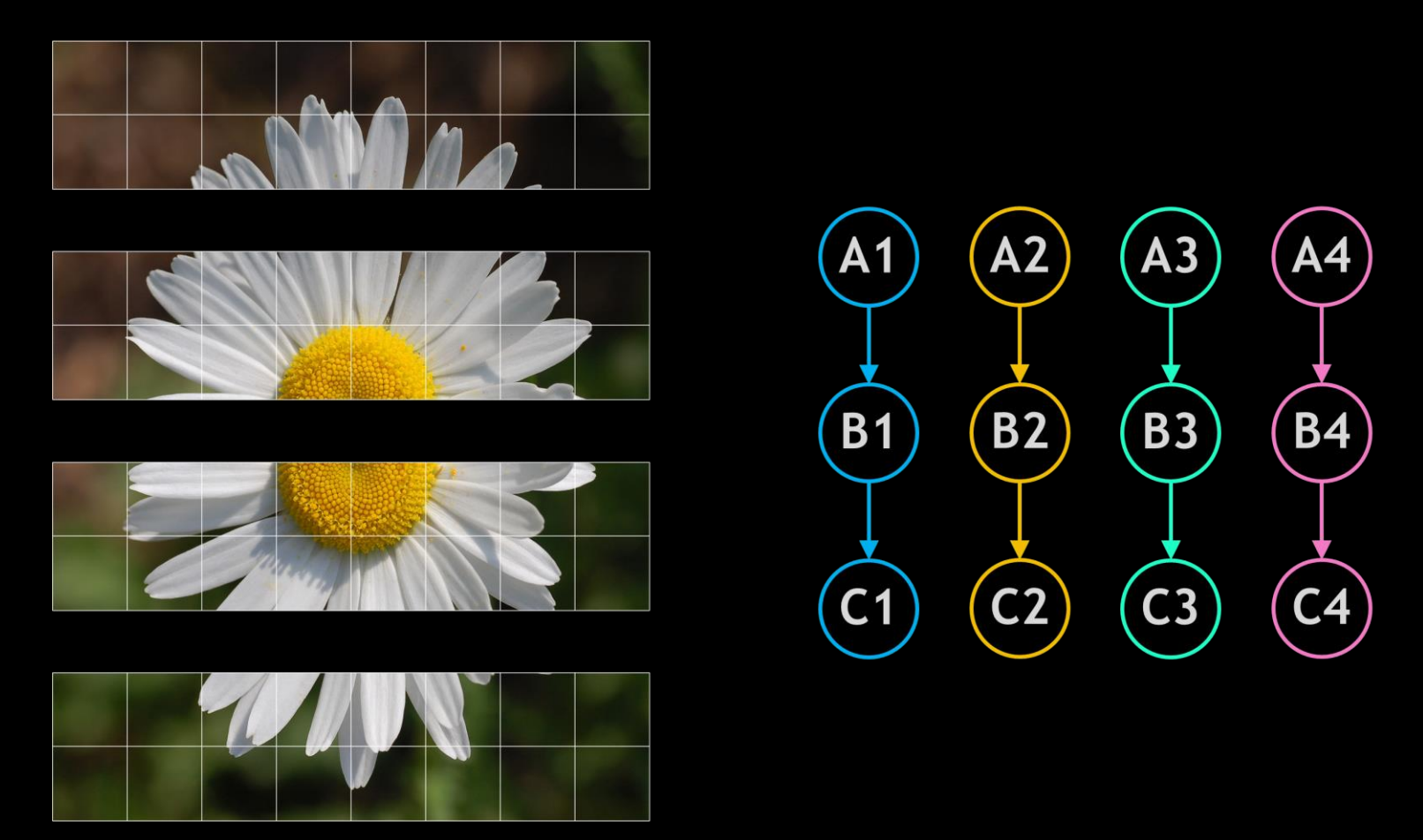
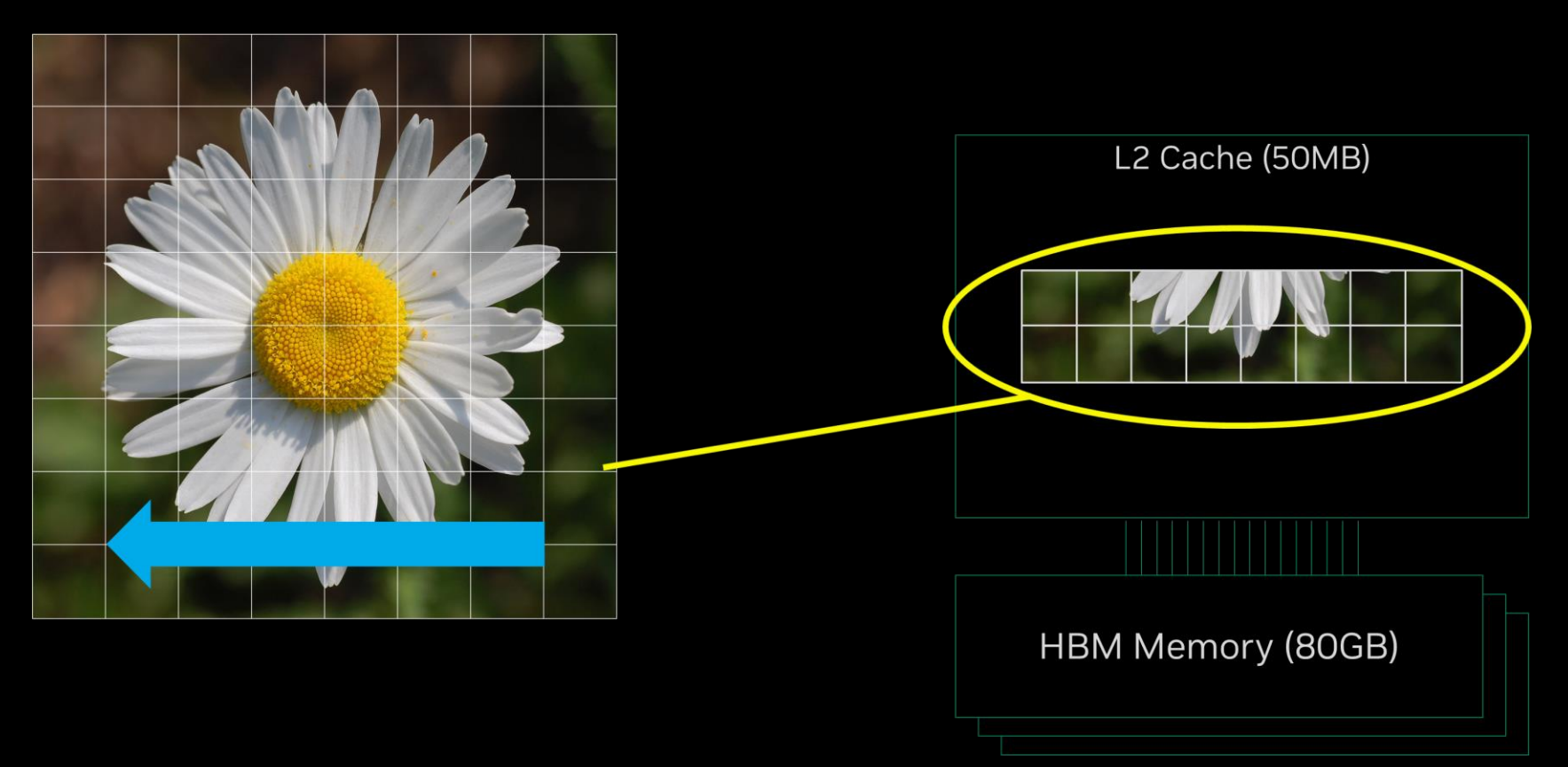
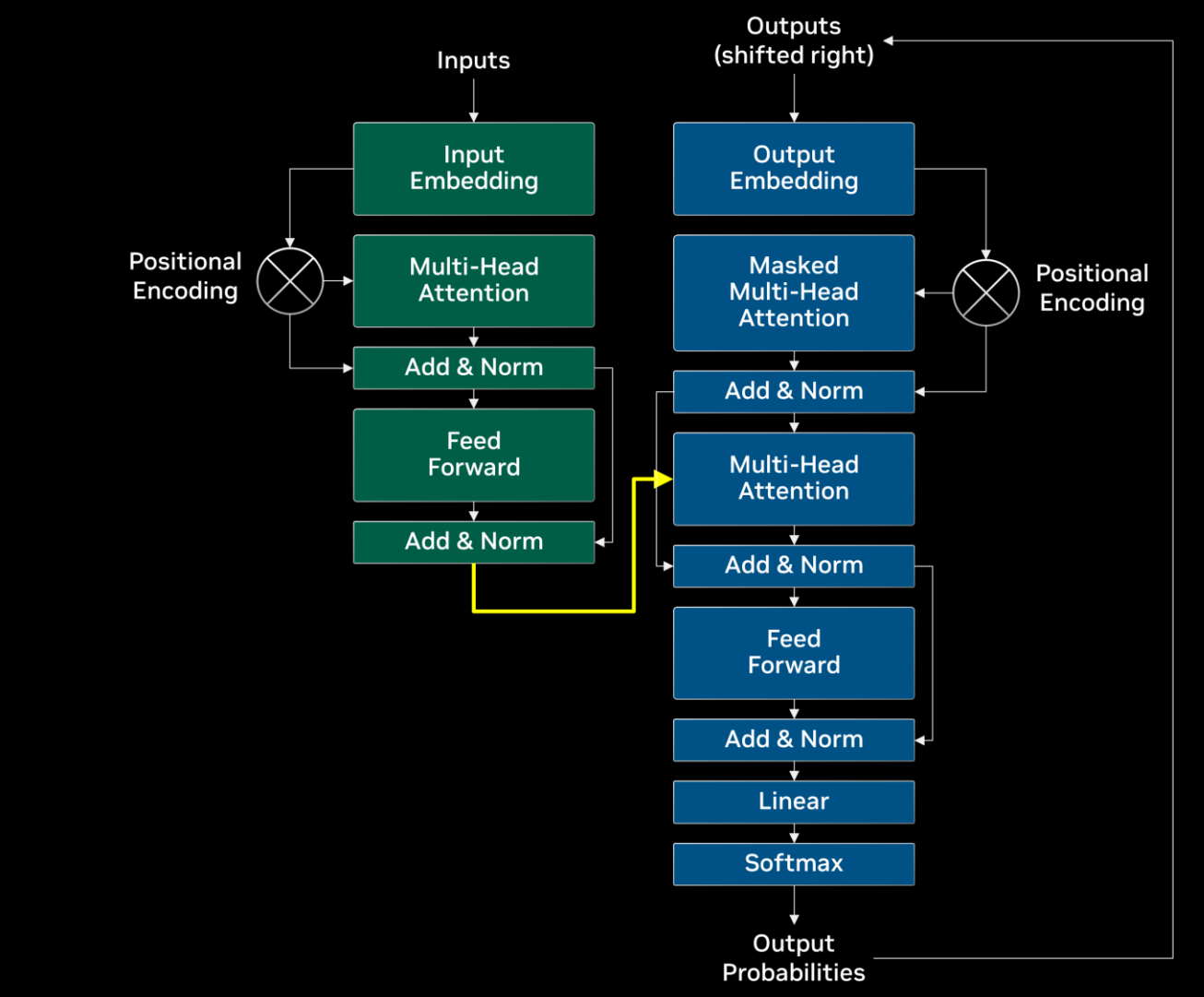
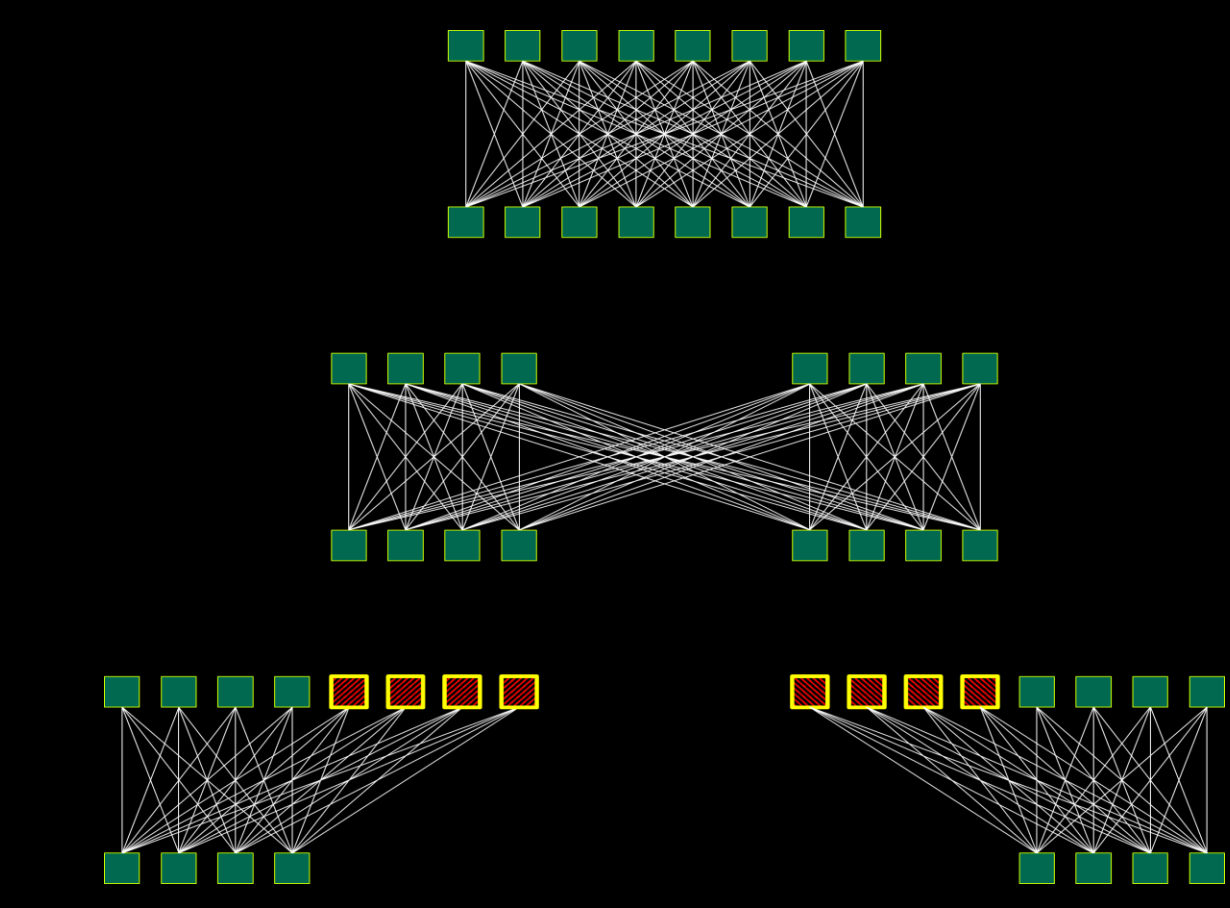
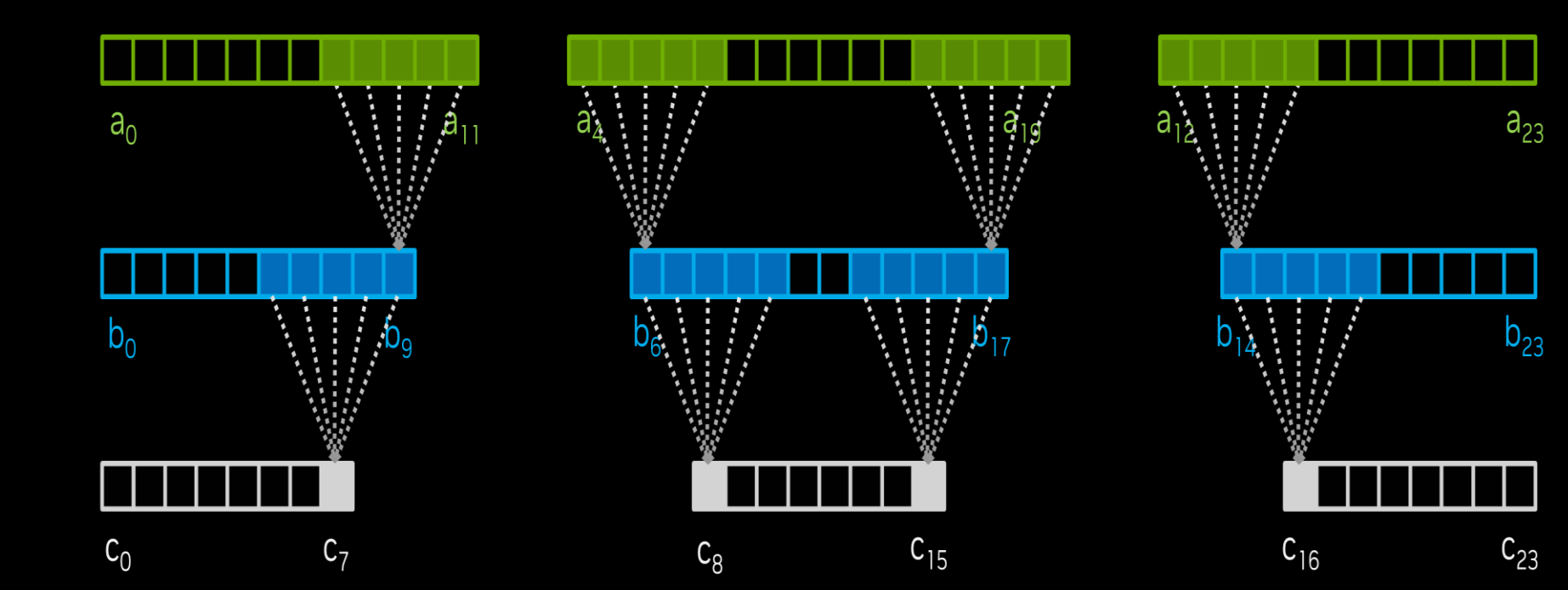
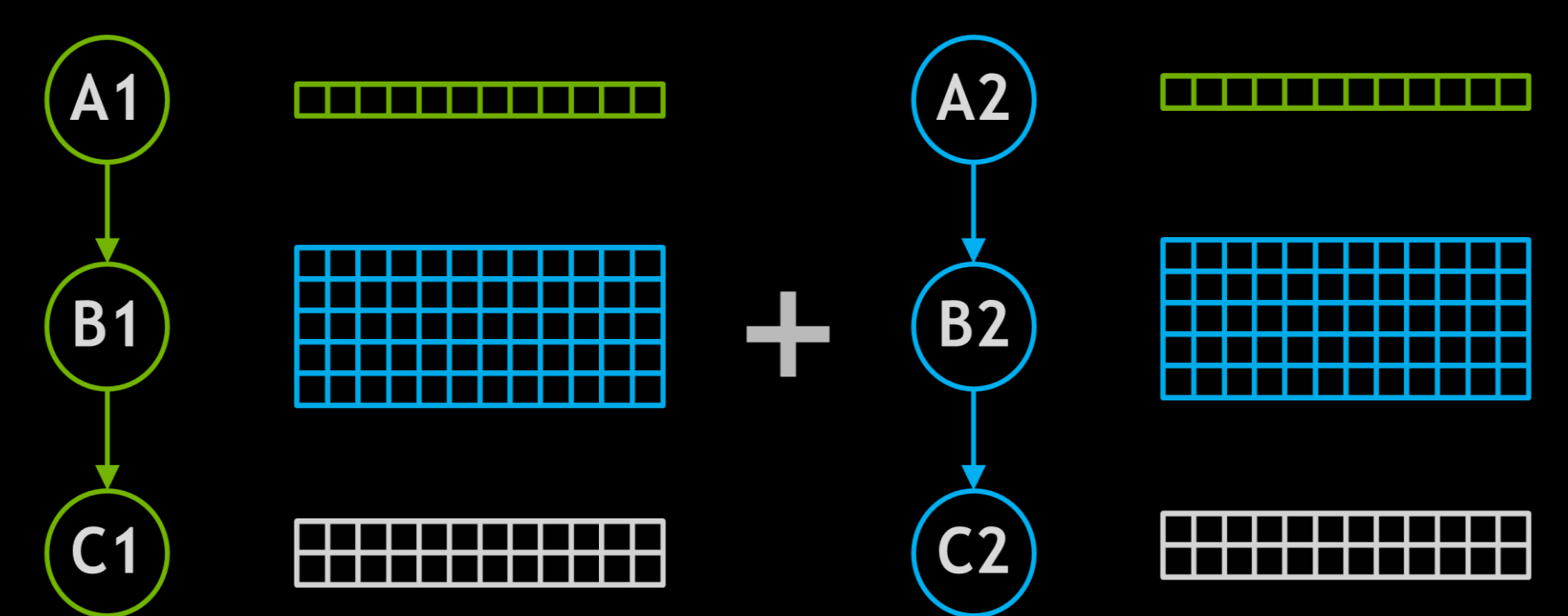
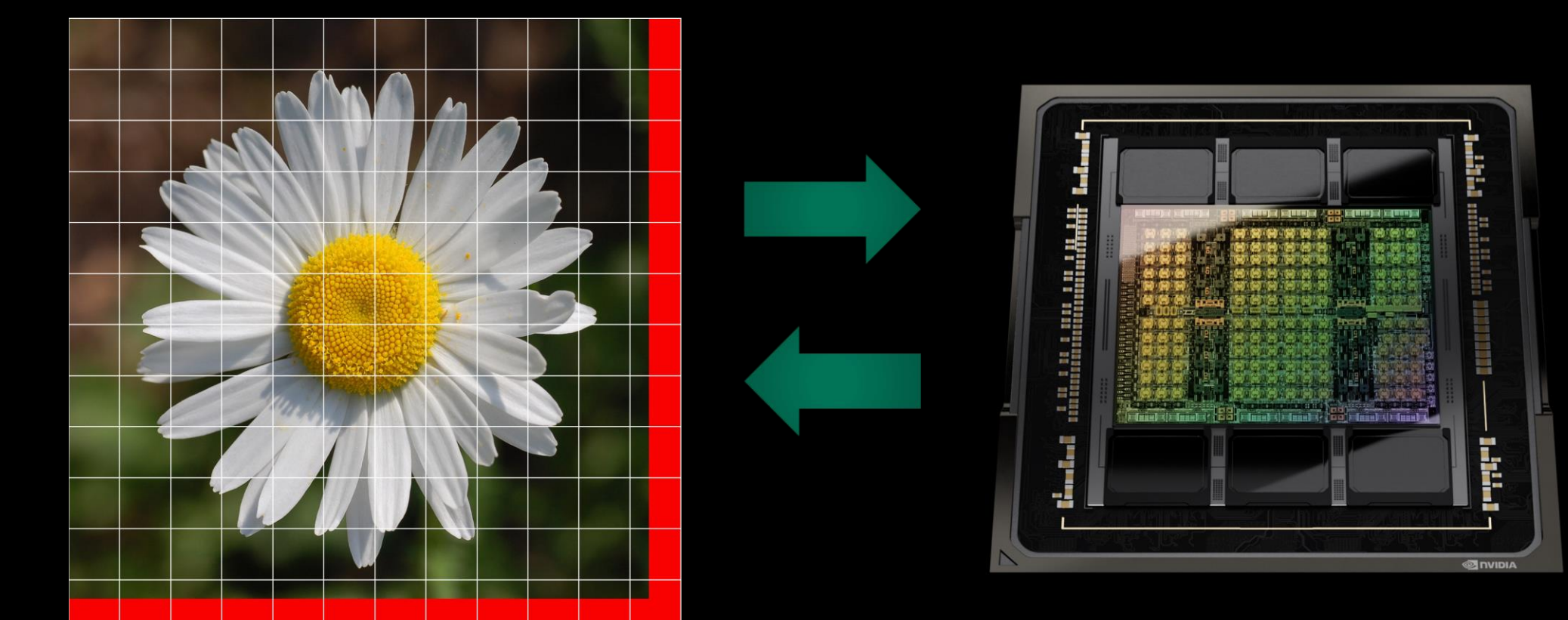
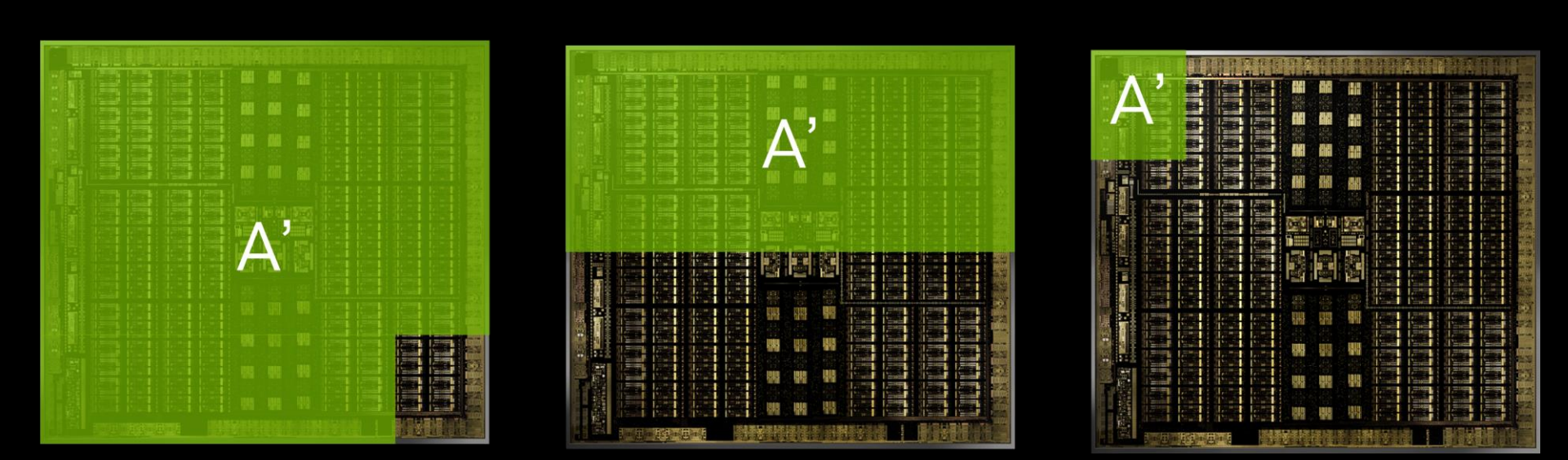
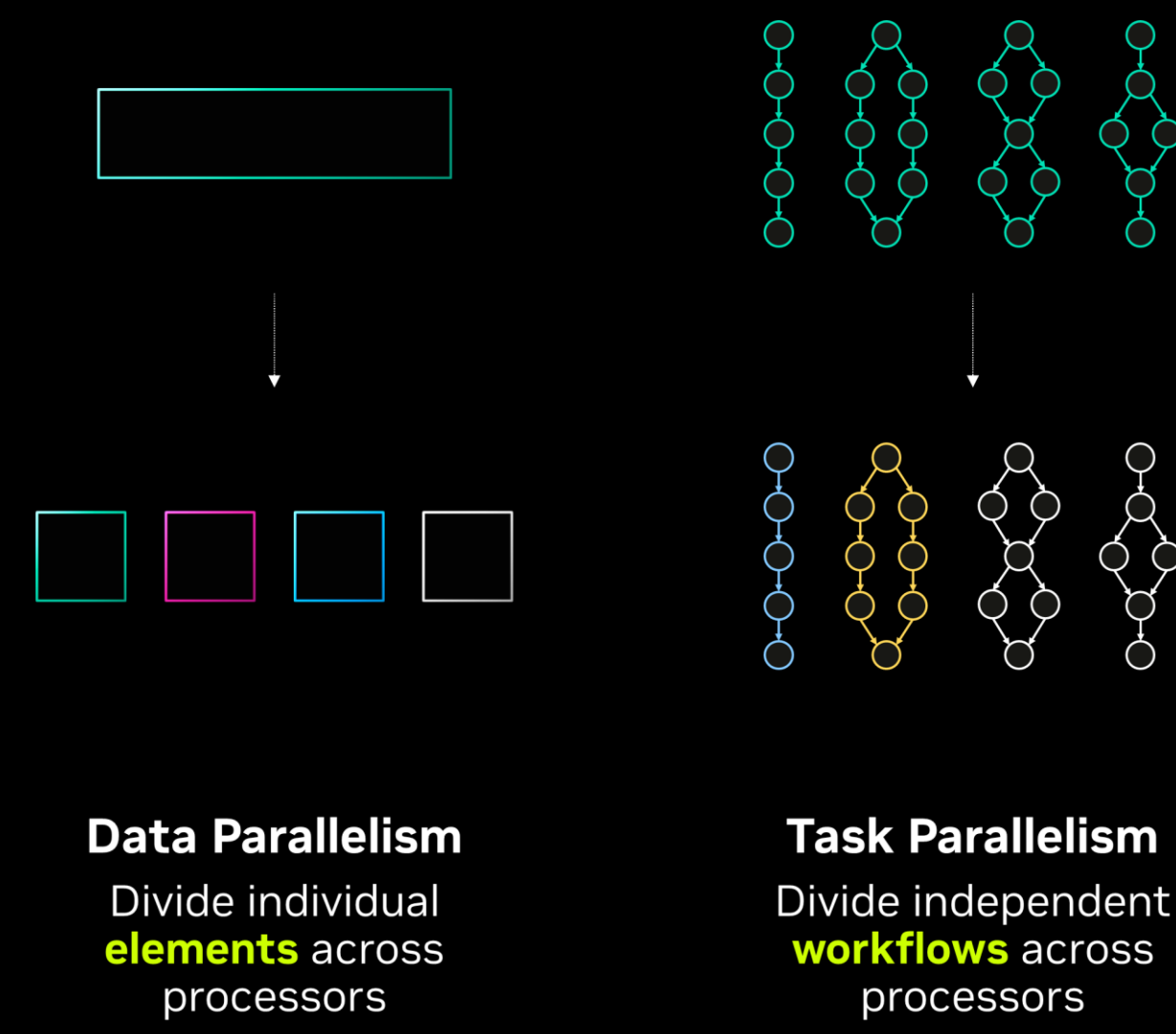
Split data into **half**-cache-sized chunks



So now my tasks fit in 25MB each



Now we need twice as many tasks





NVIDIA

*Think About
Writing*

How To ~~Write~~ A CUDA Program: **THE NINJA EDITION**

Stephen Jones, NVIDIA | GTC 2024